Probabilistic Data Science for Psychology

Dirk Ostwald

Table of contents

Welcome

Welcome to the working version of *Probabilistic Data Science for Psychology (PDSP)*, a textbook on data-scientific methodology at the Institute of Psychology at Otto von Guericke University Magdeburg.

Figure 1 Probabilistic Data Science for Psychology

Citation

Ostwald, D. (2024) Probabilistic Data Science for Psychology. 10.5281/zenodo.10730199

License

Part I

Mathematical foundations

As the language of scientific modeling, mathematics is also the language of probabilistic data science in psychology. In this role, mathematics mediates between intuitive scientific language and the variety of programming languages used to implement data-scientific analyses. Mathematical concept formation makes it possible to grasp concepts very precisely and in a generally understandable way, and to communicate them independently of the imprecision of everyday scientific language or the idiosyncrasies of different programming languages. Of course, mathematical language cannot assume this role detached from scientific-language intuition or practical application. In probabilistic data analysis, mathematics is therefore never an end in itself, but must always be understood as applied mathematics.

In this part, after some thoughts on the fundamental concepts of mathematics (Chapter 1), we present sets and functions (Chapter 2 and Chapter 4), the two pillars of modern mathematics, in compressed form. We supplement this presentation with some notational aspects in Chapter 3. Closely interwoven with the concept of a function are differential calculus and integral calculus. Here, too, the aim is not an exhaustive presentation, but in particular an explanation of some foundations that are central to the data-scientific concepts of optimization and to working with probability functions. We supplement these sections with some introductory thoughts on the analytical foundations of differential and integral calculus in Chapter 5. In dealing with the large data sets of contemporary science, matrix notation has proved useful. We first consider this subfield of linear algebra for single-column matrices and then for matrices in detail, again with the aim of providing notational and computational foundations for later sections.

Overall, this part therefore primarily serves as a brief introduction to foundations that will be taken up again in other parts, and in particular also as the introduction of a unified notation. Depending on need and interest, more in-depth self-study of the various contents is of course advisable. In the following literature notes, we provide an overview of sources and reading recommendations for the contents considered here.

Literature notes

Bärwolff (2017) and Arens et al. (2018) form the primary basis for many of the topics presented here and offer an excellent and comprehensive entry point into the material covered. At the international level, Spivak (2008) and Strang (2009) provide very readable introductions to differential calculus and linear algebra, respectively. A deeper understanding, especially of analytical concepts, is provided, for example, by Abbott (2015) and Chiossi (2021). Searle (1982) offers a compact presentation of those aspects of matrix calculus and linear algebra that are used mainly in probabilistic data analysis. Deisenroth et al. (2020) provides an overview of many of the topics treated here under the moniker of machine learning.

1 Language and logic

1.1 Mathematics is a language

Mathematics is the language of scientific model building. For example, the expression

$$F = ma \tag{1.1}$$

corresponds, in the sense of Newton's second axiom, to a theory of the motion of objects under the action of forces (Newton (1687)). Likewise, the expression

$$\max_{q(z)} \int q(z) \ln \left(\frac{p(y,z)}{q(z)} \right) dz \tag{1.2}$$

corresponds, in the sense of variational inference, to contemporary theory about how the brain works (Friston (2005)). Mathematical symbolism serves in particular to communicate scientific insights precisely and aims to describe complex matters exactly and efficiently. As in reflective engagement with any form of language, the question "What is this supposed to mean?" therefore always remains the guiding question when working with mathematical content and symbolism.

As a language system, mathematics has a number of special features. First, its contents are often abstract. This is because mathematics strives for the broadest possible general comprehensibility and applicability. Mathematical approaches to the phenomena of the world are interested in making insights as easy as possible to transfer to other contexts. To make this possible, mathematics tries to work as precisely and understandably as possible, that is, in terms of precise concepts. It proceeds in a strictly hierarchical way, so that concepts introduced later often presuppose a good understanding of the underlying concepts introduced earlier.

The precision of mathematical language implies a high density of information. It is therefore rather sober and leaves out what is superfluous, so that, in the best case, *everything* in a mathematical text is relevant for communicating an idea. As recipients of mathematical texts, we perceive their information density through the high cognitive energy required when reading such a text. This high energy expenditure calls in particular for calm and slowness when reading with the aim of genuine understanding. The following quotation may serve as a guiding principle for working with mathematical texts: "A mathematical text cannot be read like a novel; one has to work through it" (Unger (2000)). After reading a short mathematical text, one should always ask critically whether one has really understood what has been read or whether further sources should be consulted to clarify the matter. It is also helpful, in the spirit of Richard Feynman's famous quotation "What I cannot create, I do not understand", to make one's own notes and to reconstruct mathematical language systems oneself.

If one wants to gain access to the world of scientific model building, then it is helpful, when dealing with its mathematical expressions and symbolism, to use the same strategies as

when learning a foreign language. In addition to immersing oneself in the corresponding language space, that is, constant exposure to mathematical modes of expression, this certainly includes first memorizing concepts and translating texts into everyday language. A deep and secure understanding of mathematical model building then arises in particular through the application of mathematical ways of thinking in written and spoken form.

1.2 Basic building blocks

In the following, we introduce three basic building blocks of mathematical communication that will accompany us throughout: the concepts of a *definition*, a *theorem*, and a *proof*.

Definition

A *definition* is a cornerstone of a mathematical system that is neither justified nor deductively derived within that system. Definitions can only be evaluated according to their usefulness within a mathematical system. A definition is best memorized first and questioned only once one has understood its use in application or is not convinced by it. Some relaxation and calm are generally helpful when dealing with definitions that appear complex at first sight. To indicate that we define a symbol as something, we use the notation ":=". For example, the expression "$a := 2$" defines the symbol a as the number two. In this text, definitions always end with the symbol •.

Theorem

A *theorem* is a mathematical statement that can be recognized as correct (true) by means of a proof. That is, a theorem is always derived from definitions and/or other theorems. In this sense, theorems are the empirical results of mathematics. In applied, data-analytic mathematics, theorems are often helpful for calculations. It is therefore worth memorizing them, because they usually form the basis for data analysis and data interpretation. Equations often appear in theorems. These equations arise from the assumptions of the theorem. To mark equations, we use the equals sign "=". For example, in a given context, the expression "$a = 2$" says that, because of certain assumptions, the variable a has the value two. In this text, theorems always end with the symbol ∘.

Proof

A *proof* is a chain of argumentation that draws on known definitions and theorems to establish the correctness (truth) of a theorem. Short proofs often contribute to understanding a theorem; long proofs tend to do so less. Proofs are therefore, in particular, answers to the question why a mathematical statement holds ("Why is this so?"). Proofs are not memorized. When proofs are short, it is sensible to work through them, because they usually repeat content that is assumed to be known. When they are long, it is more sensible to skip them at first so as not to get lost in details and lose the main path through the mathematical structure. In this text, proofs always end with the symbol □.

In addition to definitions, theorems, and proofs, mathematical texts contain other typical building blocks, such as *axioms*, *lemmas*, *corollaries*, and *conjectures*. We will not use these terms and therefore give only a brief overview of them.

- *Axioms* are unproved fundamental assumptions in the sense that they serve as starting points for building mathematical systems. The transition between definitions and axioms is often fluid. Because we will not work at a particularly deep mathematical level, we prefer the term definition in almost all cases.
- A *lemma* is an "auxiliary theorem", that is, a mathematical statement that is proved but is not as important as a theorem. Because, on the one hand, we focus on important content and, on the other hand, we do not want to discriminate among mathematical statements, we avoid this term and instead use the term theorem throughout.
- A *corollary* is a mathematical statement that follows from a theorem by a simple proof. Because the "simplicity" of mathematical proofs is a relative property, we avoid this term and instead also use the term theorem throughout.
- *Conjectures* are mathematical statements for which it is unknown whether they are provable or refutable. Because we work in the area of applied mathematics, we will not encounter conjectures.

1.3 Propositional logic

Having now become acquainted with some basic building blocks of mathematical language systems, we turn to *propositional logic*, a simple system that allows relationships between mathematical statements to be established and formalized. Propositional logic plays a central role, for example, in the definition of set operations, in optimization conditions for functions, and in many proofs. In data-analytic applications, propositional logic is the basis of Boolean logic in programming. In mathematical psychology, propositional logic is the basis of the representational theory of measurement.

We begin with the definition of the concept of a mathematical *statement*.

Definition 1.1 (Statement)**.** A *statement* is a sentence to which the property *true* or *false* can be assigned unambiguously.

•

The adjective *true* can also be understood as *correct*. We abbreviate true by "t" and false by "f". In the field of real numbers, for example, the statement $1 + 1 = 2$ is true and the statement $1+1 = 3$ is false. Note that the binarity of the truth value of statements is a basic assumption of propositional logic and therefore to be understood formally and scientifically, not empirically. Truth values do not refer to definitions; definitions fix meanings.

A first way of working with statements is to negate them. This leads to the following definition.

Definition 1.2 (Negation)**.** Let A be a statement. Then the *negation of* A is the statement that is false when A is true and true when A is false. The negation of A is denoted by $\neg A$, read as "not A".

•

For example, the negation of the statement "The sun is shining" is the statement "The sun is not shining". The negation of the statement $1 + 1 = 2$ is the statement $1 + 1 \neq 2$, and the negation of the statement $x > 1$ is the statement $x \leq 1$. The definition of the negation of a statement A is represented in tabular form as follows:

Table 1.1 Truth table of negation

A	$\neg A$
t	f
f	t

Tables of this form are called *truth tables*. They are a popular tool in propositional logic, and we will use them often in the following.

If one wants to connect two statements logically, the concepts of *conjunction* and *disjunction* are available.

Definition 1.3 (Conjunction)**.** Let A and B be statements. Then the *conjunction of A and B* is the statement that is true if and only if both A and B are true. The conjunction of A and B is denoted by $A \wedge B$, read as "A and B".

•

The definition of conjunction implies the following truth table:

Table 1.2 Truth table of conjunction

A	B	$A \wedge B$
t	t	t
t	f	f
f	t	f
f	f	f

As an example, let A be the statement $2 \geq 1$ and let B be the statement $2 > 1$. Since both A and B are true, the statement $(2 \geq 1) \wedge (2 > 1)$ is also true. As another example, let A be the statement $1 \geq 1$ and let B be the statement $1 > 1$. Here, only A is true and B is false. Thus the statement $(1 \geq 1) \wedge (1 > 1)$ is false.

Definition 1.4 (Disjunction)**.** Let A and B be statements. Then the *disjunction of A and B* is the statement that is true if and only if at least one of the two statements A and B is true. The disjunction of A and B is denoted by $A \vee B$, read as "A or B".

•

The definition of disjunction implies the following truth table:

Table 1.3 Truth table of disjunction

A	B	$A \vee B$
t	t	t
t	f	t
f	t	t
f	f	f

$A \vee B$ is therefore also true when both A and B are true. The "or" considered here is thus more precisely an "and/or". For this reason, disjunction is also called a "non-exclusive or". As an example, let A be the statement $2 \geq 1$ and let B be the statement $2 > 1$. A is true and B is true. Thus the statement $(2 \geq 1) \vee (2 > 1)$ is true. Now again let A be the statement $1 \geq 1$ and let B be the statement $1 > 1$. Then A is true and B is false. Thus the statement $(1 \geq 1) \vee (1 > 1)$ is true.

One way of placing statements in a mechanical logical relationship is *implication*. It is defined as follows.

Definition 1.5 (Implication)**.** Let A and B be statements. Then the *implication*, denoted by $A \Rightarrow B$, is the statement that is false if and only if A is true and B is false. A is called the *assumption (premise)* and B the *conclusion* of the implication. $A \Rightarrow B$ is read as "from A follows B", "A implies B", or "if A, then B".

•

The definition of implication can be represented by the following truth table:

Table 1.4 Truth table of implication

A	B	$A \Rightarrow B$
t	t	t
t	f	f
f	t	t
f	f	t

An intuitive understanding of the definition of implication in the sense of the truth table above is obtained most readily by reading it as an attempt to capture and formalize the intuitive idea of a consequence in the context of propositional logic. To understand this, it is best to read the truth states in Table 1.4 in the order truth state of A, truth state of $A \Rightarrow B$, and finally truth state of B. Read in this way, the truth table shows that if A is true and $A \Rightarrow B$ is true, then B is true. Thus, if one constructs a true implication based on a true statement, for example by transforming equations, it follows that B is also true. If this is not possible, that is, if A is true and $A \Rightarrow B$ is always false, then B is also false. This is how statements may be refuted. Finally, one sees that if A is false and $A \Rightarrow B$ is true, then B can be either true or false. Only from a true premise does a true implication always yield a true conclusion. In particular, the definition of implication therefore satisfies the principle "anything follows from falsehood (ex falso sequitur quodlibet)". Thus one cannot infer anything meaningful from false statements by means of implication.

In the context of implication, the concepts of *sufficient* and *necessary* conditions arise: If $A \Rightarrow B$ is true, one says that "A is *sufficient* for B" and that "B is *necessary* for A". This terminology is explained in the context of implication as follows: If $A \Rightarrow B$ is true, then if A is true, B is also true. The truth of A is therefore enough for the truth of B. Thus A is sufficient for B. Furthermore, if $A \Rightarrow B$ is true, then if B is false, A is also false. The truth of B is therefore necessary for the truth of A.

Finally, to illustrate Table 1.4, we give a concrete everyday example. Let A be the statement "The bridge is damaged", let B be the statement "The bridge is closed", and let $A \Rightarrow B$ be the implication "If the bridge is damaged, then the bridge is closed". We discuss the four cases of Table 1.4.

- A true, B true, $A \Rightarrow B$ true. If A is true, then the bridge is damaged, and if B is also true, then the bridge is also closed. In this case, the implication "If the bridge is damaged, then the bridge is closed" is therefore obviously true.
- A true, B false, $A \Rightarrow B$ false. If A is true, then the bridge is damaged, and if B is false, then the bridge is not closed. In this case, the implication "If the bridge is damaged, then the bridge is closed" is therefore obviously false.
- A false, B true, $A \Rightarrow B$ true. If A is false, then the bridge is not damaged, and if B is true, then the bridge is closed. Since the bridge is not damaged, however, no conclusion can be drawn regarding the implication "If the bridge is damaged, then the bridge is closed"; in particular, one cannot show that it would be false. Thus one assumes that the statement "If the bridge is damaged, then the bridge is closed" is true.
- A false, B false, $A \Rightarrow B$ true. If A is false, then the bridge is not damaged, and if B is false, then the bridge is not closed. Since the bridge is not damaged, however, no conclusion can be drawn regarding the implication "If the bridge is damaged, then the bridge is closed"; in particular, one cannot show that it would be false. Thus one again assumes that the statement "If the bridge is damaged, then the bridge is closed" is true.

A very frequently occurring relation between two statements is their *equivalence*.

Definition 1.6 (Equivalence)**.** Let A and B be statements. The *equivalence of A and B* is the statement that is true if and only if A and B are both true or if A and B are both false. The equivalence of A and B is denoted by $A \Leftrightarrow B$ and read as "A if and only if B" or "A is equivalent to B".

•

The definition of equivalence implies the following truth table:

Table 1.5 Truth table of equivalence

A	B	$A \Leftrightarrow B$
t	t	t
t	f	f
f	t	f
f	f	t

The definition of the concept of *logical equivalence* allows, among other things, the equivalence of two statements to be established by means of implications.

Definition 1.7 (Logical equivalence)**.** Two statements are called *logically equivalent* if their truth tables are the same.

•

As examples of logical equivalences that are frequently used in proof arguments, we show the statements of the following theorem.

Theorem 1.1 (Logical equivalences)**.** *Let A and B be two statements. Then the following statements are logically equivalent:*

(1) $A \Leftrightarrow B$ *and* $(A \Rightarrow B) \wedge (B \Rightarrow A)$
(2) $A \Rightarrow B$ *and* $(\neg B) \Rightarrow (\neg A)$

∘

Proof. By the definition of logical equivalence, we have to show that the truth tables of the statements under consideration are the same. We show first (1), then (2).

(1) We recall the truth table of $A \Leftrightarrow B$:

A	B	$A \Leftrightarrow B$
t	t	t
t	f	f
f	t	f
f	f	t

We further consider the truth table of $(A \Rightarrow B) \wedge (B \Rightarrow A)$:

A	B	$A \Rightarrow B$	$B \Rightarrow A$	$(A \Rightarrow B) \wedge (B \Rightarrow A)$
t	t	t	t	t
t	f	f	t	f
f	t	t	f	f
f	f	t	t	t

Comparing the truth table of $A \Leftrightarrow B$ with the first two and the last column of the truth table of $(A \Rightarrow B) \wedge (B \Rightarrow A)$ shows their equality.

(2) We recall the truth table of $A \Rightarrow B$:

A	B	$A \Rightarrow B$
t	t	t
t	f	f
f	t	t
f	f	t

We further consider the truth table of $(\neg B) \Rightarrow (\neg A)$:

A	B	$\neg B$	$\neg A$	$(\neg B) \Rightarrow (\neg A)$
t	t	f	f	t
t	f	t	f	f
f	t	f	t	t
f	f	t	t	t

Comparing the truth table of $A \Rightarrow B$ with the first two and the last column of the truth table of $(\neg B) \Rightarrow (\neg A)$ shows their equality. □

The first statement of Theorem 1.1 says that the statement "A and B are equivalent" is logically equivalent to the statement "from A follows B and from B follows A". This is the basis for many so-called *direct proofs* using equivalence transformations. The second statement of Theorem 1.1 says that the statement "from A follows B" is logically equivalent to the statement "from not B follows not A". This is the basis for the technique of *indirect proof*. We consider these proof techniques more closely in the following section. First, we summarize the meanings of the symbols introduced in this section once more in the table below.

Table 1.10 Symbol overview. A and B are statements, that is, A and B are either true or false.

Symbol	Meaning	Note
$\neg A$	Not A	True when A is false, and vice versa
$A \wedge B$	A and B	True only when both A and B are true
$A \vee B$	A and/or B	True when at least one of the statements is true
$A \Rightarrow B$	From A follows B	B is necessary for A, A is sufficient for B
$A \Leftrightarrow B$	A is equivalent to B	Both $A \Rightarrow B$ and $B \Rightarrow A$ hold

1.4 Equivalence transformations

Mathematical problems often lead to the case in which information about an unknown variable is represented implicitly by means of an equation or an inequality. To represent the information about the corresponding variable explicitly, that is, to determine its value in the case of equations or to determine the range of numbers in which the value of the variable lies in the case of inequalities, one uses *equivalence transformations*. Here, we consider only the equivalence transformations of equations and inequalities with respect to real variables that are familiar from school mathematics. Equivalence transformations of equations and inequalities have the property that the truth value of the statement formulated by an equation or inequality does not change when the corresponding transformation is applied. Both sides of the equation or inequality are always transformed. For the application of a mathematical operation that transforms an equation or inequality statement A into an equation or inequality statement B to be an equivalence transformation of the form $A \Leftrightarrow B$, both $A \Rightarrow B$ and $B \Rightarrow A$ must hold, as is well known. This implies that equivalence transformations are reversible (invertible) operations.

For equations, admissible equivalence transformations include in particular

- adding a number to both sides of the equation,
- subtracting a number from both sides of the equation,
- multiplying both sides of the equation by a nonzero number,
- dividing both sides of the equation by a nonzero number, and

- applying an invertible function to both sides of the equation.

Example

Anticipating Section 4.3, we consider the statement

$$2\exp(x) - 2 = 0. \tag{1.3}$$

Then

$$\begin{aligned} & 2\exp(x) - 2 = 0 \\ \Leftrightarrow\ & 2\exp(x) - 2 + 2 = +2 \\ \Leftrightarrow\ & 2\exp(x) = 2 \\ \Leftrightarrow\ & \frac{1}{2} \cdot 2\exp(x) = \frac{1}{2} \cdot 2 \\ \Leftrightarrow\ & \exp(x) = 1 \\ \Leftrightarrow\ & \ln(\exp(x)) = \ln(1) \\ \Leftrightarrow\ & x = 0. \end{aligned} \tag{1.4}$$

In summary, therefore,

$$2\exp(x) - 2 = 0 \Leftrightarrow x = 0. \tag{1.5}$$

For inequalities, admissible equivalence transformations include in particular

- adding a number to both sides of the inequality,
- subtracting a number from both sides of the inequality,
- multiplying both sides of the inequality by a nonzero number, with multiplication by a negative number reversing the inequality sign,
- dividing both sides of the inequality by a nonzero number, with division by a negative number reversing the inequality sign, and
- applying an invertible monotonically increasing function to both sides of the inequality; for monotonically decreasing functions, the inequality sign is reversed.

Example

We consider the statement

$$-5x - 2 \geq 8. \tag{1.6}$$

Then

$$\begin{aligned} & -5x - 2 \geq 8 \\ \Leftrightarrow\ & -5x - 2 + 2 \geq 8 + 2 \\ \Leftrightarrow\ & -5x \geq 10 \\ \Leftrightarrow\ & -\frac{1}{5} \cdot 5x \geq \frac{1}{5} \cdot 10 \\ \Leftrightarrow\ & -x \geq 2 \\ \Leftrightarrow\ & x \leq -2. \end{aligned} \tag{1.7}$$

In summary, therefore,

$$-5x - 2 \geq 8 \Leftrightarrow x \leq -2. \tag{1.8}$$

1.5 Proof techniques

In this section, we outline three fundamental proof techniques: *direct* and *indirect proofs*, and *proof by contradiction*. In what follows, especially the first technique will be used repeatedly to justify theorems.

We have:

- *Direct proofs* use equivalence transformations to show $A \Rightarrow B$.
- *Indirect proofs* use the logical equivalence of $A \Rightarrow B$ and $(\neg B) \Rightarrow (\neg A)$.
- *Proofs by contradiction* show that $(\neg B) \wedge A$ is false.

Equipped with this, we now prove the following theorem by means of a direct proof, an indirect proof, and a proof by contradiction (cf. Arens et al. (2018)).

Theorem 1.2 (Squares of positive numbers)**.** *Let a and b be two positive numbers. Then $a^2 < b^2 \Rightarrow a < b$.*

∘

Proof. We first give a *direct proof*. Let $a^2 < b^2$ be the statement A and let $a < b$ be the statement B. Then

$$a^2 < b^2 \Leftrightarrow 0 < b^2 - a^2 \Leftrightarrow 0 < (b+a)(b-a) \Leftrightarrow 0 < (b-a) \Leftrightarrow a < b. \tag{1.9}$$

We now give an *indirect proof*. Let $a^2 \geq b^2$ be the statement $\neg A$. Furthermore, let $a \geq b$ be the statement $\neg B$. Then

$$a \geq b \Leftrightarrow a^2 \geq ab \wedge ab \geq b^2 \Leftrightarrow a^2 \geq b^2. \tag{1.10}$$

Finally, we give a *proof by contradiction*. To do so, we show that the assumption $(\neg B) \wedge A$ leads to a false statement. We have

$$a \geq b \wedge a^2 < b^2 \Leftrightarrow a^2 \geq ab \wedge a^2 < b^2 \Leftrightarrow ab \leq a^2 < b^2. \tag{1.11}$$

Furthermore,

$$a \geq b \wedge a^2 < b^2 \Leftrightarrow ab \geq b^2 \wedge a^2 < b^2 \Leftrightarrow a^2 < b^2 \leq ab. \tag{1.12}$$

Overall, this yields the false statement

$$ab \leq a^2 < b^2 \leq ab \Leftrightarrow ab < ab. \tag{1.13}$$

□

2 Sets

2.1 Basic definitions

Sets collect mathematical objects and form the foundation of modern mathematics. We begin with the following definition.

Definition 2.1 (Sets)**.** Following Cantor (1895), a *set* M is defined as "a collection into a whole of definite, distinct objects m of our intuition or of our thought (which are called the elements of the set)". We write

$$m \in M \text{ or } m \notin M \tag{2.1}$$

to express that m is an element or not an element of M, respectively.

•

There are at least the following ways of defining sets:

- Listing the elements in curly braces, for example $M := \{1, 2, 3\}$.
- Specifying the properties of the elements, for example $M := \{x \in \mathbb{N} | x < 4\}$.
- Equating the set with another uniquely defined set, for example $M := \mathbb{N}_3$.

The notation $\{x \in \mathbb{N} | x < 4\}$ is read as "$x \in \mathbb{N}$ such that $x < 4$", where the meaning of $\mathbb{N}$ will be explained below. It is important to recognize that sets are *unordered* mathematical objects, that is, the order in which the elements of a set are listed does not matter. For example, $\{1, 2, 3\}$, $\{1, 3, 2\}$, and $\{2, 3, 1\}$ denote the same set, namely the set of the first three natural numbers.

Basic relations between several sets are fixed in the next definition.

Definition 2.2 (Subsets and set equality)**.** Let M and N be two sets.

- A set M is called a *subset* of a set N if, for every element $m \in M$, we also have $m \in N$. If M is a subset of N, we write

  $$M \subseteq N \tag{2.2}$$

  and call M a *subset* of N and N a *superset* of M.
- A set M is called a *proper subset* of a set N if, for every element $m \in M$, we also have $m \in N$, but there is at least one element $n \in N$ for which $n \notin M$. If M is a proper subset of N, we write

  $$M \subset N. \tag{2.3}$$

- Two sets M and N are called *equal* if, for every element $m \in M$, we also have $m \in N$, and if, for every element $n \in N$, we also have $n \in M$. If the sets M and N are equal, we write
$$M = N. \tag{2.4}$$

•

Example

For example, consider the sets $M := \{1\}$, $N := \{1, 2\}$, and $O := \{1, 2\}$. Then, by the definitions above, $M \subset N$, because $1 \in M$ and $1 \in N$, but $2 \in N$ and $2 \notin M$. Furthermore, $N \subseteq O$, because $1 \in N$ and $1 \in O$, as well as $2 \in N$ and $2 \in O$, and there is no element of O that is not in N. Likewise, $O \subseteq N$, because $1 \in O$ and $1 \in N$, as well as $2 \in O$ and $2 \in N$, and there is no element of N that is not in O. Finally, we even have $N = O$, because, for every element $n \in N$, we also have $n \in O$, and at the same time, for every element $o \in O$, we also have $o \in N$. We represent these relations schematically by means of *Venn-Euler diagrams* in Figure 2.1.

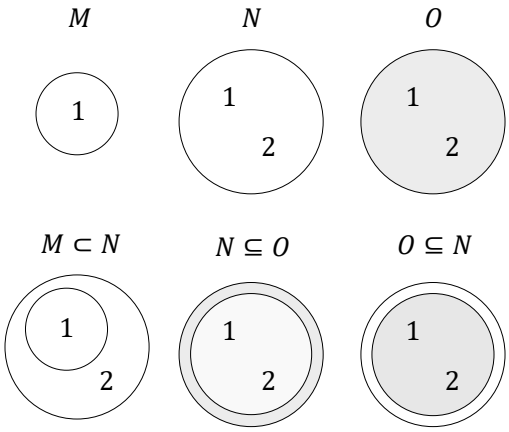


Figure 2.1 Venn-Euler diagrams of the subset properties of the sets $M := \{1\}$, $N := \{1, 2\}$, and $O := \{1, 2\}$.

An important property of a set is the number of elements it contains. This is called the *cardinality* of the set.

Definition 2.3 (Cardinality)**.** The number of elements of a set M is called its *cardinality* and is denoted by $|M|$.

•

A special set is the set without elements.

Definition 2.4. A set with cardinality zero is called the *empty set* and is denoted by $\emptyset$.

•

As examples, let $M := \{1, 2, 3\}$, $N = \{a, b, c, d\}$, and $O := \emptyset$. Then $|M| = 3$, $|N| = 4$, and $|O| = 0$.

For every set, one can consider the set of all subsets of this set. This leads to the important concept of the *power set*.

Definition 2.5 (Power set)**.** The set of all subsets of a set M is called the *power set of* M and is denoted by $\mathcal{P}(M)$.

•

Note that the empty subset of M and M itself are also always elements of $\mathcal{P}(M)$. We first consider four examples of the concept of a power set.

- Let $M_0 := \emptyset$ be the empty set. Then

$$\mathcal{P}(M_0) = \{\emptyset\}. \tag{2.5}$$

- Let M_1 be the one-element set $M_1 := \{a\}$. Then

$$\mathcal{P}(M_1) = \{\emptyset, \{a\}\}. \tag{2.6}$$

- Let $M_2 := \{a, b\}$. Then M_2 has both one-element and two-element subsets, and

$$\mathcal{P}(M_2) = \{\emptyset, \{a\}, \{b\}, \{a, b\}\}. \tag{2.7}$$

- Finally, let $M_3 := \{a, b, c\}$. Then M_3 has, among others, one-element, two-element, and three-element subsets, and

$$\mathcal{P}(M_3) = \{\emptyset, \{a\}, \{b\}, \{c\}, \{a, b\}, \{a, c\}, \{b, c\}, \{a, b, c\}\}. \tag{2.8}$$

Theorem 2.1 (Cardinality of the power set)**.** *Given a set M with cardinality $|M| = n$, and let $\mathcal{P}(M)$ be its power set. Then $|\mathcal{P}(M)| = 2^n$.*

∘

Proof. To prove the statement of the theorem, we associate each element P of the power set of M uniquely with a binary sequence of length n, where the entry at the i-th position represents whether the i-th element of M is an element of P or not. For example, let $M := \{m_1, m_2, m_3\}$ and $P := \{m_2, m_3\}$. Then P corresponds to the binary sequence 011. The empty set $P := \emptyset$ corresponds to the binary sequence 000, and the original set $P = M$ corresponds to the binary sequence 111. Thus the question is how many unique binary sequences of length n there are. Since each element of the sequence has two possible states, there are n factors $2 \cdot 2 \cdots 2$, that is, 2^n. □

In the examples above, we have the cases

- $|M_0| = 0 \Rightarrow |\mathcal{P}(M_0)| = 2^0 = 1$,
- $|M_1| = 1 \Rightarrow |\mathcal{P}(M_1)| = 2^1 = 2$,
- $|M_2| = 2 \Rightarrow |\mathcal{P}(M_2)| = 2^2 = 4$,
- $|M_3| = 3 \Rightarrow |\mathcal{P}(M_3)| = 2^3 = 8$,

as can be verified by counting the elements of the corresponding power sets.

2.2 Operations

Two sets can be combined with one another in different ways. The result of such a combination is another set. We call the combination of two sets a *set operation* and give the following definitions.

Definition 2.6 (Set operations)**.** Let M and N be two sets.

- The *union of M and N* is defined as the set

$$M \cup N := \{x | x \in M \vee x \in N\}, \tag{2.9}$$

  where $\vee$ is to be understood according to Definition 1.4 as a *non-exclusive or*, that is, as and/or.

- The *intersection of M and N* is defined as the set

$$M \cap N := \{x | x \in M \wedge x \in N\}. \tag{2.10}$$

  If M and N satisfy $M \cap N = \emptyset$, then M and N are called *disjoint*.

- The *difference of M and N* is defined as the set

$$M \setminus N := \{x | x \in M \wedge x \notin N\}. \tag{2.11}$$

  The difference of M and N is also called, especially when $N \subseteq M$, the *complement of N with respect to M* and is denoted by N^c. In this context, M is also called the *basic set* or the *universe*.

- The *symmetric difference of M and N* is defined as the set

$$M \Delta N := \{x | (x \in M \vee x \in N) \wedge x \notin M \cap N\}. \tag{2.12}$$

  The symmetric difference can therefore be understood as an *exclusive or*.

•

Example

As an example, consider the sets $M := \{1, 2, 3\}$ and $N := \{2, 3, 4, 5\}$. Then

- $M \cup N = \{1, 2, 3, 4, 5\}$, because $1 \in M$, $2 \in M$, $3 \in M$, $4 \in N$, and $5 \in N$.
- $M \cap N = \{2, 3\}$, because only for 2 and 3 do we have $2 \in M, 3 \in M$ and also $2 \in N, 3 \in N$. For 1, we only have $1 \in M$, and for 4 and 5, we only have $4 \in N$ and $5 \in N$.
- $M \setminus N = \{1\}$, because $1 \in M$, but $1 \notin N$, and $2 \in M$, but also $2 \in N$.
- $N \setminus M = \{4, 5\}$, because $4 \in N$ and $5 \in N$, but $4 \notin M$ and $5 \notin M$. This shows in particular that the difference of M and N is *not* symmetric, that is, it is *not* necessarily the case that $M \setminus N$ equals $N \setminus M$.
- $M \Delta N = \{1, 4, 5\}$, because $1 \in M$, but $1 \notin \{2, 3\}$, $2 \in M$, but $2 \in \{2, 3\}$, $3 \in M$, but $3 \in \{2, 3\}$, $4 \in N$, but $4 \notin \{2, 3\}$, and $5 \in N$, but $5 \notin \{2, 3\}$.

Figure 2.2 visualizes the set operations considered in this example.

Finally, we introduce the concept of a partition of a set.

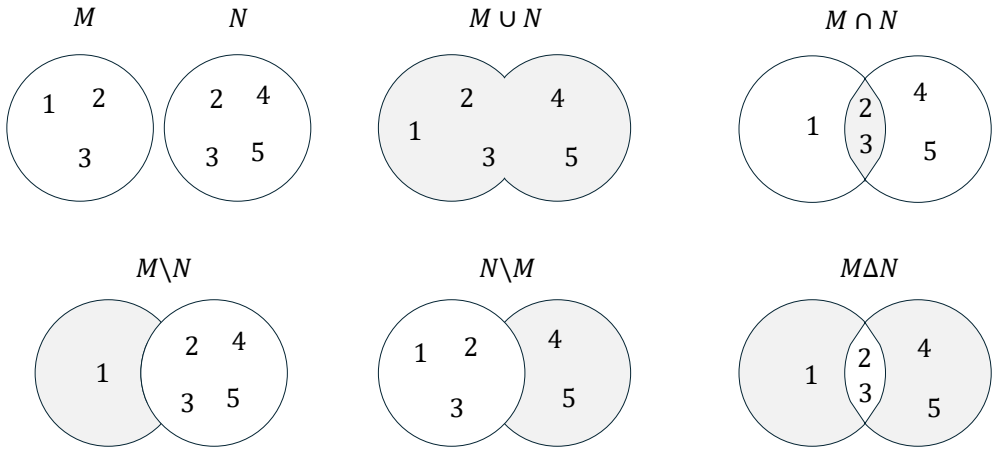


Figure 2.2 Venn-Euler diagrams of the set operations considered in the example for $M := \{1, 2, 3\}$ and $N := \{2, 3, 4, 5\}$. The gray shaded regions correspond to the resulting sets.

Definition 2.7 (Partition)**.** Let M be a set, and let $P := \{N_i\}$ be a set of sets N_i with $i = 1, ..., n$, such that

$$(M = \cup_{i=1}^{n} N_i) \wedge \left(N_i \cap N_j = \emptyset \text{ for } i, j = 1, ..., n \text{ and } i \neq j\right). \tag{2.13}$$

Then P is called a *partition of* M.

•

The partition of a set therefore corresponds to splitting the set into disjoint subsets. Partitions are generally not unique, that is, there are usually several ways to partition a given set.

As an example, consider the set $M := \{1, 2, 3, 4, 5, 6\}$. Then $P_1 := \{\{1\}, \{2, 3, 4, 5, 6\}\}$, $P_2 := \{\{1, 2, 3\}, \{4, 5, 6\}\}$, and $P_3 := \{\{1, 2\}, \{3, 4\}, \{5, 6\}\}$ are three possible partitions of M. Figure 2.3 visualizes the partitions considered in this example.

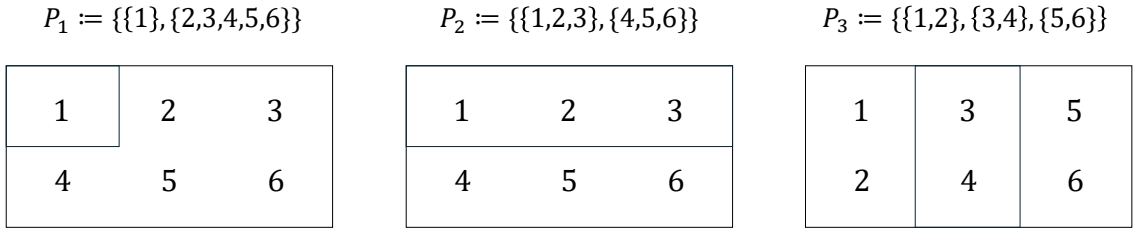


Figure 2.3 Diagrams of the partitions considered in the example for $M := \{1, 2, 3, 4, 5, 6\}$.

2.3 Special sets

Number sets

In natural science, one attempts to describe phenomena of the world that are intuitively identified as discrete or continuous by means of numbers. Depending on the type of

phenomenon, different number sets are suitable for this purpose. Mathematics provides, among others, the number sets given in the following definition.

Definition 2.8 (Number sets)**.** The following denote:

- $\mathbb{N} := \{1, 2, 3, ...\}$ the *natural numbers*,
- $\mathbb{N}_n := \{1, 2, 3, ..., n\}$ the *natural numbers of order* n,
- $\mathbb{N}^0 := \mathbb{N} \cup \{0\}$ the *natural numbers* and zero,
- $\mathbb{Z} := \{..., -3, -2, -1, 0, 1, 2, 3...\}$ the *integers*,
- $\mathbb{Q} := \{\frac{p}{q} | p \in \mathbb{Z}, q \in \mathbb{N}\}$ the *rational numbers*,
- $\mathbb{R}$ the *real numbers*, and
- $\mathbb{C} := \{a + ib | a, b \in \mathbb{R}, i := \sqrt{-1}\}$ the *complex numbers*.

•

The natural numbers and the integers are suitable for quantifying discrete phenomena. The rational numbers and especially the real numbers are suitable for quantifying continuous phenomena. $\mathbb{R}$ includes the rational numbers and the so-called *irrational numbers* $\mathbb{R} \setminus \mathbb{Q}$. Rational numbers are numbers that can be expressed as fractions of integers and natural numbers. These include all integers as well as negative and positive decimal numbers such as $-\frac{9}{10} = -0.9$, $\frac{1}{3} = 0.\bar{3}$, and $\frac{196}{100} = 1.96$. Irrational numbers are numbers that cannot be expressed as rational numbers. Examples of irrational numbers are *Euler's number* $e \approx 2.71$, the *circle constant* $\pi \approx 3.14$, and the square root of 2, $\sqrt{2} \approx 1.41$.

The real numbers contain the natural numbers, the integers, and the rational numbers as subsets. Thus there are very many real numbers. Indeed, Cantor (1874) proved that there are more real numbers than natural numbers, even though there are infinitely many real numbers and infinitely many natural numbers. This property of the real numbers is called their *uncountability*. We will deepen this concept in Chapter 4. In particular,

$$\mathbb{N} \subset \mathbb{Z} \subset \mathbb{Q} \subset \mathbb{R}. \tag{2.14}$$

Between any two real numbers there are infinitely many further real numbers. Positive infinity ∞ and negative infinity $-\infty$ are not numbers with which one could calculate in standard mathematics. They also do not belong to the number sets given in the definition above; thus $\infty \notin \mathbb{R}$ and $-\infty \notin \mathbb{R}$. Complex numbers are suitable for describing two-dimensional continuous phenomena. The values of the first dimension are represented in the real part a, and the values of the second dimension in the complex part b of a complex number. Complex numbers are used, for example, in the modeling of physical phenomena and in Fourier analysis.

Important subsets of the real numbers are the so-called *intervals*. We give the following definitions.

Intervals

Definition 2.9. Connected subsets of the real numbers are called *intervals*. For $a, b \in \mathbb{R}$, one distinguishes

- the *closed interval*

$$[a, b] := \{x \in \mathbb{R} | a \leq x \leq b\}, \tag{2.15}$$

- the *open interval*

$$]a, b[:= \{x \in \mathbb{R} | a < x < b\}, \tag{2.16}$$

- and the *half-open intervals*

$$]a, b] := \{x \in \mathbb{R} | a < x \leq b\} \text{ and } [a, b[:= \{x \in \mathbb{R} | a \leq x < b\}. \tag{2.17}$$

•

As examples, Figure 2.4 graphically represents the intervals $[1, 2]$, $]1, 2]$, $[1, 2[$, and $]1, 2[$. Here one imagines the real numbers as a continuous number line and must in each case note whether the left and right endpoints are part of the interval or not. As mentioned above, positive infinity (∞) and negative infinity $-\infty$ are not elements of $\mathbb{R}$. Thus one always writes $]-\infty, b]$ or $]-\infty, b[$ and $]a, \infty[$ or $[a, \infty[$, as well as $\mathbb{R} =]-\infty, \infty[$.

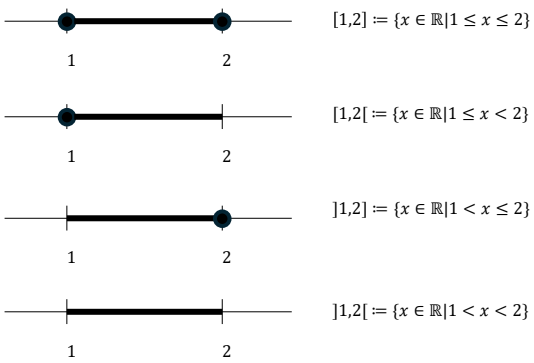


Figure 2.4 Representation of intervals on the number line. The black dot indicates that the corresponding number is part of the interval.

Cartesian products

Often one wants to quantitatively describe several independent properties of a phenomenon at the same time. For this purpose, the one-dimensional number sets defined above can be extended to multidimensional number sets by forming *Cartesian products*. The elements of Cartesian products are called *ordered tuples* or *vectors*.

Definition 2.10 (Cartesian products)**.** Let M and N be two sets. Then the *Cartesian product of the sets M and N* is the set of all ordered tuples (m, n) with $m \in M$ and $n \in N$, formally

$$M \times N := \{(m, n) | m \in M, n \in N\}. \tag{2.18}$$

The Cartesian product of a set M with itself is denoted by

$$M^2 := M \times M. \tag{2.19}$$

Furthermore, let $M_1, M_2, ..., M_n$ be sets. Then the *Cartesian product of the sets $M_1, ..., M_n$* is the set of all ordered n-tuples $(m_1, ..., m_n)$ with $m_i \in M_i$ for $i = 1, ..., n$, formally

$$\prod_{i=1}^{n} M_i := M_1 \times \cdots \times M_n := \{(m_1, ..., m_n) | m_i \in M_i \text{ for } i = 1, ..., n\}. \tag{2.20}$$

The n-fold Cartesian product of a set M with itself is denoted by

$$M^n := \prod_{i=1}^{n} M := \{(m_1, ..., m_n) | m_i \in M \text{ for } i = 1, ..., n\}. \tag{2.21}$$

•

In contrast to sets, the tuples introduced in Definition 2.10 are *ordered*. This means, for example, that for sets we have $\{1, 2\} = \{2, 1\}$, but for tuples we have $(1, 2) \neq (2, 1)$.

Example

Let $M := \{1, 2\}$ and $N := \{1, 2, 3\}$. Then the Cartesian product $M \times N$ is given by

$$M \times N := \{(1, 1), (1, 2), (1, 3), (2, 1), (2, 2), (2, 3)\} \tag{2.22}$$

and the Cartesian product $N \times M$ is given by

$$N \times M := \{(1, 1), (1, 2), (2, 1), (2, 2), (3, 1), (3, 2)\}. \tag{2.23}$$

The Cartesian product is therefore generally not commutative, that is, it is not necessarily the case that $M \times N = N \times M$. One may construct the sets $M \times N \neq N \times M$ in this example from Table 2.1 and Table 2.2.

Table 2.1 Cartesian product $M \times N$

(m, n)	$n = 1$	$n = 2$	$n = 3$
$m = 1$	$(1, 1)$	$(1, 2)$	$(1, 3)$
$m = 2$	$(2, 1)$	$(2, 2)$	$(2, 3)$

Table 2.2 Cartesian product $N \times M$

(n, m)	$m = 1$	$m = 2$
$n = 1$	$(1, 1)$	$(1, 2)$
$n = 2$	$(2, 1)$	$(2, 2)$
$n = 3$	$(3, 1)$	$(3, 2)$

$\mathbb{R}$ to the Power of n

As described above, the real numbers are particularly suitable for describing continuous phenomena. For the simultaneous description of several aspects of a continuous phenomenon, the *set of real tuples of n-th order*, or $\mathbb{R}$ to the power of n for short, is correspondingly useful.

Definition 2.11 (Set of real tuples of n-th order)**.** The n-fold Cartesian product of the real numbers with themselves is denoted by

$$\mathbb{R}^n := \prod_{i=1}^{n} \mathbb{R} := \{x := (x_1, ..., x_n) | x_i \in \mathbb{R}\} \tag{2.24}$$

and is read as "$\mathbb{R}$ to the power of n". We write the elements of $\mathbb{R}^n$ as columns

$$x := \begin{pmatrix} x_1 \\ \vdots \\ x_n \end{pmatrix} \tag{2.25}$$

and call them *n-dimensional vectors*. For distinction, we also call the elements of $\mathbb{R}^1 = \mathbb{R}$ *scalars*.

•

Examples

Familiar examples of $\mathbb{R}^n$ are $\mathbb{R}^1$ as the set of real numbers, $\mathbb{R}^2$ as the set of real tuples in the model of the two-dimensional plane, and $\mathbb{R}^3$ as the set of real triples in the model of three-dimensional space, as visualized in Figure 2.5.

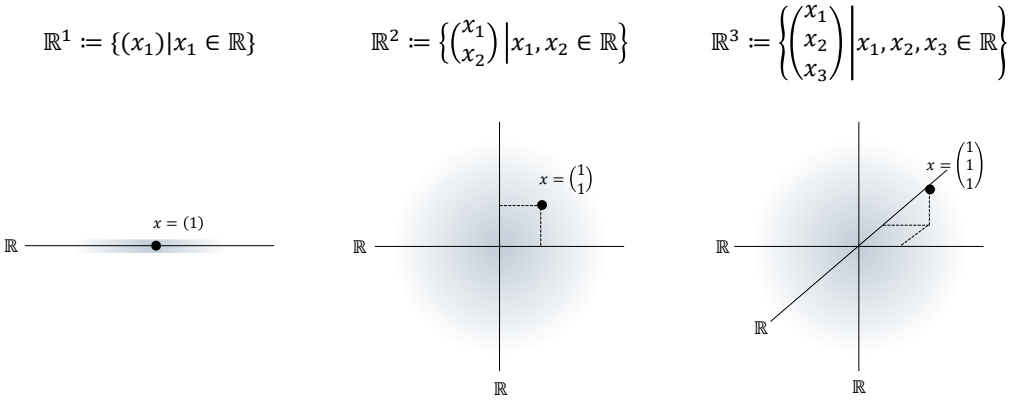


Figure 2.5 $\mathbb{R}^n$ for $n = 1$, $n = 2$, and $n = 3$. The black dots each represent an element of the corresponding set.

An example of an $x \in \mathbb{R}^4$ is

$$x = \begin{pmatrix} 0.16 \\ 1.76 \\ 0.23 \\ 7.11 \end{pmatrix}. \tag{2.26}$$

3 Sums, products, powers

This unit introduces notation for the elementary arithmetic operations.

3.1 Sums

Definition 3.1 (Summation symbol)**.** It denotes

$$\sum_{i=1}^{n} x_i = x_1 + x_2 + \cdots + x_n. \tag{3.1}$$

Here,

- Σ denotes the Greek letter *S*igma, mnemonically standing for *s*um,
- the subscript $i = 1$ denotes the *running index* and the *starting index*,
- the superscript n denotes the *terminal index*, and
- $x_1, x_2, ..., x_n$ denote the *summands*.

•

For meaningful use of the summation symbol, it is essential to specify the beginning and the end of the summation by means of the subscript and the superscript. The precise name of the running index, by contrast, is irrelevant for the value of the sum; thus,

$$\sum_{i=1}^{n} x_i = \sum_{j=1}^{n} x_j. \tag{3.2}$$

Sometimes the running index is also given as an element of an *index set*. If, for example, the index set $I := \{1, 5, 7\}$ is defined, then

$$\sum_{i \in I} x_i := x_1 + x_5 + x_7. \tag{3.3}$$

In the following, we briefly consider a few examples of how the summation symbol is used.

- *Summation of predefined summands.* Let $x_1 := 2$, $x_2 := 10$, and $x_3 := -4$. Then

$$\sum_{i=1}^{3} x_i = x_1 + x_2 + x_3 = 2 + 10 - 4 = 8. \tag{3.4}$$

- *Summation of weighted predefined summands.* Again let $x_1 := 2$, $x_2 := 10$, and $x_3 := -4$. In addition, let the *weighting coefficients* $a_1 := \frac{1}{2}$, $a_2 := \frac{1}{5}$, and $a_3 := 2$ be defined. Then

$$\sum_{i=1}^{3} a_i x_i = a_1 x_1 + a_2 x_2 + a_3 x_3 = \frac{1}{2} \cdot 2 + \frac{1}{5} \cdot 10 + 2 \cdot (-4) = 1 + 2 - 8 = -5. \quad (3.5)$$

  Expressions of the form $\sum_{i=1}^{n} a_i x_i$ are also called *linear combinations* of $x_1, ..., x_n$ with the *coefficients* or *weighting parameters* $a_1, ..., a_n$.
- *Summation of natural numbers.* We have

$$\sum_{i=1}^{5} i = 1 + 2 + 3 + 4 + 5 = 15. \quad (3.6)$$

- *Summation of even natural numbers.* We have

$$\sum_{i=1}^{5} 2i = 2 \cdot 1 + 2 \cdot 2 + 2 \cdot 3 + 2 \cdot 4 + 2 \cdot 5 = 2 + 4 + 6 + 8 + 10 = 30. \quad (3.7)$$

- *Summation of odd natural numbers.* We have

$$\sum_{i=1}^{5} (2i-1) = 2 \cdot 1 - 1 + 2 \cdot 2 - 1 + 2 \cdot 3 - 1 + 2 \cdot 4 - 1 + 2 \cdot 5 - 1 = 1 + 3 + 5 + 7 + 9 = 25. \quad (3.8)$$

Working with the summation symbol can often be simplified by applying the following calculation rules.

Theorem 3.1 (Calculation rules for sums)**.**

(1) Sums of identical summands

$$\sum_{i=1}^{n} x = nx \quad (3.9)$$

(2) Associativity for sums of equal length

$$\sum_{i=1}^{n} x_i + \sum_{i=1}^{n} y_i = \sum_{i=1}^{n} (x_i + y_i) \quad (3.10)$$

(3) Distributivity under multiplication by a constant

$$\sum_{i=1}^{n} a x_i = a \sum_{i=1}^{n} x_i \quad (3.11)$$

(4) Splitting sums for $1 < m < n$

$$\sum_{i=1}^{n} x_i = \sum_{i=1}^{m} x_i + \sum_{i=m+1}^{n} x_i \quad (3.12)$$

(5) Reindexing

$$\sum_{i=0}^{n} x_i = \sum_{j=m}^{n+m} x_{j-m} \quad (3.13)$$

∘

Proof. These calculation rules can be verified by writing out the sums and applying the rules of addition and multiplication. Here, we show associativity for sums of equal length and distributivity under multiplication by a constant as examples. For the former, we have

$$\begin{aligned}\sum_{i=1}^{n} x_i + \sum_{i=1}^{n} y_i &= x_1 + x_2 + \cdots + x_n + y_1 + y_2 + \cdots + y_n \\ &= x_1 + y_1 + x_2 + y_2 + \cdots + x_n + y_n \\ &= \sum_{i=1}^{n} (x_i + y_i).\end{aligned} \tag{3.14}$$

For the latter, we have

$$\begin{aligned}\sum_{i=1}^{n} a x_i &= a x_1 + a x_2 + \cdots + a x_n \\ &= a(x_1 + x_2 + \cdots + x_n) \\ &= a \sum_{i=1}^{n} x_i.\end{aligned} \tag{3.15}$$

□

Examples

As a first example of applying the calculation rules recorded in Theorem 3.1, we consider the evaluation of a *mean* (sometimes also called an *average*). Let $x_1, x_2, ..., x_n$ be real numbers. The mean of these numbers is the sum of $x_1, x_2, ..., x_n$ divided by the number of numbers n. By statement (3) of Theorem 3.1, it is irrelevant whether the numbers are first added up and the resulting sum is then divided by n, or whether the individual numbers are each divided by n and the corresponding results are then added up. More precisely, by applying Theorem 3.1 (3) with $a = 1/n$, we obtain

$$\frac{1}{n} \sum_{i=1}^{n} x_i = \sum_{i=1}^{n} \frac{x_i}{n}. \tag{3.16}$$

For example, the mean of $x_1 := 1, x_2 := 4, x_3 := 2$, and $x_4 := 1$ is given by

$$\frac{1}{4} \sum_{i=1}^{4} x_i = \frac{1}{4}(1 + 4 + 2 + 1) = \frac{8}{4} = 2 = \frac{8}{4} = \frac{1}{4} + \frac{4}{4} + \frac{2}{4} + \frac{1}{4} = \sum_{i=1}^{4} \frac{x_i}{4}. \tag{3.17}$$

As a second example, we consider the reindexing rule recorded in Theorem 3.1 (5). Let $n := 3$ and $m := 2$, and let $x_0 := 2$, $x_1 := 3$, $x_2 := 5$, and $x_3 := 10$. Then, evidently,

$$\begin{aligned}\sum_{i=0}^{3} x_i &= x_0 + x_1 + x_2 + x_3 \\ &= 2 + 3 + 5 + 10 \\ &= 20.\end{aligned} \tag{3.18}$$

But we also have

$$\begin{aligned}\sum_{j=m}^{n+m} x_{j-m} &= \sum_{j=2}^{3+2} x_{j-2} \\ &= \sum_{j=2}^{5} x_{j-2} \\ &= x_{2-2} + x_{3-2} + x_{4-2} + x_{5-2} \\ &= x_0 + x_1 + x_2 + x_3 \\ &= 2 + 3 + 5 + 10 \\ &= 20.\end{aligned} \tag{3.19}$$

Double Sums

In applications, one often encounters expressions that carry out several summations one after another. For each of these sums, the definitions and calculation rules listed above apply. The meaning of double sums is best made clear by writing them out from the inside to the outside, while noting that the running index of the outer sum remains constant during each iteration of the inner sum.

The following examples may illustrate this.

(1)

$$\begin{aligned}\sum_{i=1}^{2}\sum_{j=1}^{3}(i+j) &= \sum_{i=1}^{2}(i+1+i+2+i+3) \\ &= (1+1+1+2+1+3) + (2+1+2+2+2+3) \\ &= 9 + 12 \\ &= 21\end{aligned} \tag{3.20}$$

(2)

$$\begin{aligned}\sum_{i=1}^{2}\sum_{j=1}^{3}(x_i+y_j) &= \sum_{i=1}^{2}(x_i+y_1+x_i+y_2+x_i+y_3) \\ &= (x_1+y_1+x_1+y_2+x_1+y_3) + (x_2+y_1+x_2+y_2+x_2+y_3)\end{aligned} \tag{3.21}$$

(3)

$$\begin{aligned}\sum_{i=1}^{3}\sum_{j=1}^{2}(x_i y_j) &= \sum_{i=1}^{3}(x_i y_1 + x_i y_2) \\ &= (x_1 y_1 + x_1 y_2) + (x_2 y_1 + x_2 y_2) + (x_3 y_1 + x_3 y_2)\end{aligned} \tag{3.22}$$

3.2 Products

The product symbol provides notation for products that is analogous to the summation symbol.

Definition 3.2 (Product symbol)**.** It denotes

$$\prod_{i=1}^{n} x_i = x_1 \cdot x_2 \cdot \dots \cdot x_n. \tag{3.23}$$

Here,

- $\prod$ denotes the Greek letter *P*i, mnemonically standing for *p*roduct,
- the subscript $i = 1$ denotes the *running index* and the *starting index*,
- the superscript n denotes the *terminal index*, and
- $x_1, x_2, ..., x_n$ denote the *product terms*.

•

Analogously to the summation symbol, the product symbol is meaningful only with a subscript and a superscript specifying the running and terminal index. The precise name of the running index is again irrelevant; thus,

$$\prod_{i=1}^{n} x_i = \prod_{j=1}^{n} x_j. \tag{3.24}$$

Here, too, the running index is in rare cases given as an element of an index set. If, for example, the index set $J := \mathbb{N}_2^0$ is defined, then

$$\prod_{j \in J} x_j := x_0 \cdot x_1 \cdot x_2. \tag{3.25}$$

One example of the use of the product symbol is the definition of the *factorial* of a natural number n by

$$n! := \prod_{i=1}^{n} i. \tag{3.26}$$

For example,

$$3! := \prod_{i=1}^{3} i = 1 \cdot 2 \cdot 3 = 6. \tag{3.27}$$

There are also a number of calculation rules for products that often simplify working with them. We list some in the following theorem. In doing so, we make anticipatory use of the notation of *powers* defined in Section 3.3.

Theorem 3.2 (Calculation rules for products)**.**

(1) Products of identical factors

$$\prod_{i=1}^{n} x = x^n \tag{3.28}$$

(2) Raising constants to powers

$$\prod_{i=1}^{n} a x_i = a^n \prod_{i=1}^{n} x_i \tag{3.29}$$

(3) Splitting products for $1 < m < n$

$$\prod_{i=1}^{n} x_i = \prod_{i=1}^{m} x_i \prod_{j=m+1}^{n} x_j \tag{3.30}$$

(4) Product of products

$$\prod_{i=1}^{n} x_i y_i = \prod_{i=1}^{n} x_i \prod_{i=1}^{n} y_i \tag{3.31}$$

∘

3.3 Powers

Products of numbers with themselves can be abbreviated using power notation.

Definition 3.3 (Power)**.** For $a \in \mathbb{R}$ and $n \in \mathbb{N}^0$, the *n-th power of a* is defined by

$$a^0 := 1 \text{ and } a^{n+1} := a^n \cdot a. \tag{3.32}$$

Furthermore, for $a \in \mathbb{R} \setminus \{0\}$ and $n \in \mathbb{N}^0$, the *negative n-th power of a* is defined by

$$a^{-n} := (a^n)^{-1} := \frac{1}{a^n}. \tag{3.33}$$

a is called the *base*, and n is called the *exponent*.

•

The way a^{n+1} is defined by referring back to the power a^n in the above definition is called *recursive*. The definition $a^0 := 1$ is called the *initial case* of the recursion; it is what makes the recursive definition of a^{n+1} possible in the first place. The definition $a^{n+1} := a^n \cdot a$ is also called the *recursion step*. The following calculation rules simplify computations with powers.

Theorem 3.3 (Calculation rules for powers)**.** *For* $a, b \in \mathbb{R}$ *and* $n, m \in \mathbb{Z}$ *with* $a \neq 0$ *for negative exponents, the following calculation rules hold:*

$$a^n a^m = a^{n+m} \tag{3.34}$$
$$(a^n)^m = a^{nm} \tag{3.35}$$
$$(ab)^n = a^n b^n \tag{3.36}$$

∘

Instead of a proof, we consider the following three examples.

(1)

$$2^2 \cdot 2^3 = (2 \cdot 2) \cdot (2 \cdot 2 \cdot 2) = 2^5 = 2^{2+3} \tag{3.37}$$

(2)

$$(3^2)^3 = (3 \cdot 3)^3 = (3 \cdot 3) \cdot (3 \cdot 3) \cdot (3 \cdot 3) = 3^6 = 3^{2 \cdot 3} \tag{3.38}$$

(3)

$$(2 \cdot 4)^2 = (2 \cdot 4) \cdot (2 \cdot 4) = (2 \cdot 2) \cdot (4 \cdot 4) = 2^2 \cdot 4^2 \tag{3.39}$$

Closely related to powers is the definition of the n-th root:

Definition 3.4 (n-th root)**.** For $a \in \mathbb{R}_{\geq 0}$ and $n \in \mathbb{N}$, the *n-th root of a* is defined as the nonnegative real number r with

$$r^n = a. \tag{3.40}$$

•

When computing with roots, power notation for roots is often helpful because it allows the calculation rules for powers to be applied directly.

Theorem 3.4 (Power notation for the n-th root)**.** *Let $a \in \mathbb{R}_{\geq 0}$, let $n \in \mathbb{N}$, and let r be the n-th root of a. Then*

$$r = a^{\frac{1}{n}} \tag{3.41}$$

∘

Proof. We have

$$\left(a^{\frac{1}{n}}\right)^n = a^{\frac{1}{n}} \cdot a^{\frac{1}{n}} \cdot \dots \cdot a^{\frac{1}{n}} = a^{\sum_{i=1}^{n} \frac{1}{n}} = a^1 = a. \tag{3.42}$$

Thus, by Definition 3.4, $r = a^{\frac{1}{n}}$. □

Computing with square roots is greatly facilitated by the power notation

$$\sqrt{x} = x^{\frac{1}{2}} \tag{3.43}$$

For example,

$$\frac{2\pi}{\sqrt{2\pi}} = \frac{2\pi}{(2\pi)^{\frac{1}{2}}} = (2\pi)^1 \cdot (2\pi)^{-\frac{1}{2}} = (2\pi)^{1-\frac{1}{2}} = (2\pi)^{\frac{1}{2}} = \sqrt{2\pi}. \tag{3.44}$$

The following relation between square, root, and absolute value is used quite often.

Theorem 3.5 (Root and absolute value)**.** *Let $x \in \mathbb{R}$, and let*

$$|x| := \begin{cases} x & \text{for } x \geq 0 \\ -x & \text{for } x < 0 \end{cases} \tag{3.45}$$

denote the absolute value of x*. Then*

$$\sqrt{x^2} = |x|. \tag{3.46}$$

∘

Proof. We consider the cases $x \geq 0$ and $x < 0$. First, let $x \geq 0$. Then

$$x \geq 0 \Rightarrow x^2 \geq 0 \Rightarrow \sqrt{x^2} = x = |x|. \tag{3.47}$$

Now let $x < 0$. Then

$$x < 0 \Rightarrow x^2 > 0 \Rightarrow \sqrt{x^2} = -x = |x|. \tag{3.48}$$

□

4 Functions

Alongside sets, functions are the second foundational pillar of modern mathematics. In this unit, we define the concept of a function, introduce first properties of functions, and give an overview of several elementary functions.

4.1 Definition and properties

Definition 4.1 (Function)**.** A *function* or *mapping* f is an assignment rule that assigns exactly one element of a set Z to each element of a set D. D is called the *domain of* f, and Z is called the *codomain of* f. We write

$$f : D \to Z, x \mapsto f(x), \tag{4.1}$$

where $f : D \to Z$ is read as "the function f maps all elements of the set D uniquely to elements in Z" and $x \mapsto f(x)$ is read as "x, which is an element of D, is mapped by the function f to $f(x)$, where $f(x)$ is an element of Z." The arrow $\to$ denotes the mapping between the sets D and Z, while the arrow $\mapsto$ denotes the mapping between an element of D and an element of Z.

•

It is essential to distinguish between the *function* f as an assignment rule and a *value of the function* $f(x)$ as an element of Z. x is the *argument* of the function (the function's *input*), and $f(x)$ is the value that the function f takes for the argument x (the function's *output*). Usually, the definition of a function specifies, after $f(x)$, the *functional form of* f, that is, a rule for forming the value $f(x)$ from x. For example, in the following definition of a function

$$f : \mathbb{R} \to \mathbb{R}_{\geq 0}, x \mapsto f(x) := x^2 \tag{4.2}$$

the definition of a power is used. Note that functions are always *unique* in the sense that, whenever the function is applied, they always assign one and the same $f(x) \in Z$ to each $x \in D$.

Figure 4.1 visualizes several aspects of the definition of a function. Figure 4.1 A first illustrates the central terms of the definition. Figure 4.1 B shows a first example of a function in which the function values assigned to the arguments are determined not by a computational rule but by direct definition. Figure 4.1 C and Figure 4.1 D use the same pictorial language to emphasize two aspects of the definition by means of counterexamples: the drawing in Figure 4.1 C is not the representation of a function, because by Definition 4.1 a function assigns *each element* of a set D exactly one element of a codomain Z. The sketched assignment rule does not assign any element in Z to the element $2 \in D$, and therefore it is not a function. Similarly, according to Definition 4.1, a function assigns exactly one element of a codomain Z to each element of a set D. The assignment rule

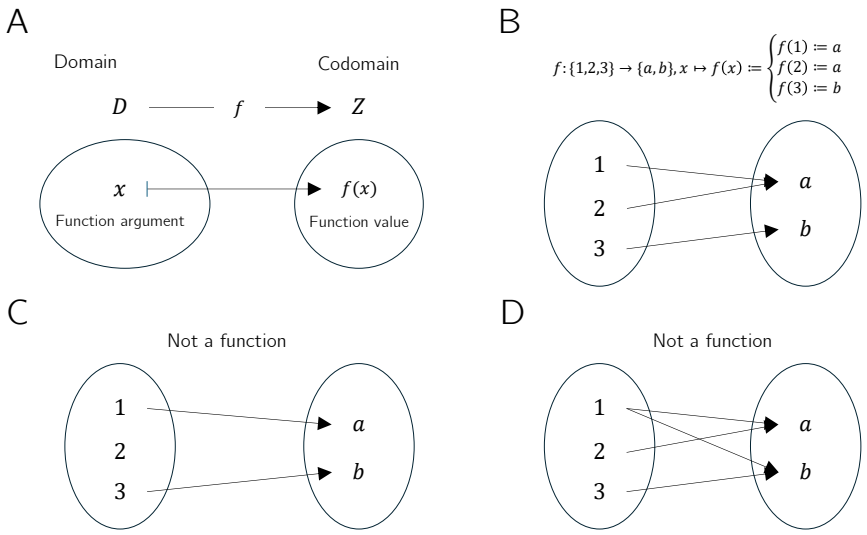


Figure 4.1 Aspects of the definition of a function

sketched in Figure 4.1 D assigns both the element $a \in Z$ and the element $b \in Z$ to $1 \in D$, and therefore it is incompatible with Definition 4.1.

Thus, functions put elements of sets into relation with one another. The sets of these elements receive special names.

Definition 4.2 (Image, range, preimage set, preimage)**.** Let $f : D \to Z, x \mapsto f(x)$ be a function and let $D' \subseteq D$ and $Z' \subseteq Z$. The set

$$f(D') := \{z \in Z | \text{there exists an } x \in D' \text{ with } z = f(x)\} \tag{4.3}$$

is called the *image of* D', and $f(D) \subseteq Z$ is called the *range* of f. Furthermore, the set

$$f^{-1}(Z') := \{x \in D | f(x) \in Z'\} \tag{4.4}$$

is called the *preimage set of* Z'. An $x \in D$ with $z = f(x) \in Z$ is called a *preimage of* z.

•

Note that the range $f(D)$ of f and the codomain Z of f need not be identical.

Example

To clarify the concepts introduced in Definition 4.2, we consider the function shown in Figure 4.2 A,

$$f : \{1,2,3,4,5\} \to \{a,b,c,d\}, x \mapsto f(x) := \begin{cases} f(1) := b \\ f(2) := d \\ f(3) := c \\ f(4) := c \\ f(5) := d \end{cases} . \tag{4.5}$$

According to Definition 4.2, an image is always defined with respect to a subset D' of the domain D. Thus, let $D' := \{2,3\} \subset D$, as shown in Figure 4.2 B. Then the image

of D' is the set of those $z \in Z$ for which there exists an $x \in D'$ such that $z = f(x)$. Here, the relevant $z \in Z$ for which such an $x \in D'$ exists are precisely $c, d \in Z$, because $f(2) = d$ and $f(3) = c$. Beyond these, there is no $z \in Z$ with $f(x) = z$ and $x \in \{2, 3\}$. By Definition 4.2, the range $f(D)$ is the subset of those $z \in Z$ for which there exists an $x \in D$ such that $z = f(x)$. This is true for all elements of Z except for $a \in Z$, because there is no $x \in D$ with $a = f(x)$. For the function considered here, the range of f is therefore given by $f(D) := \{b, c, d\}$.

Conversely, according to Definition 4.2, a preimage set is always defined with respect to a subset Z' of the codomain Z. Thus, let $Z' := \{c, d\} \subset Z$, as shown in Figure 4.2 C. Then the preimage set of Z' is the set of those $x \in D$ for which $f(x) \in Z'$. The elements of D whose function values under f are elements of Z' are precisely $\{2, 3, 4, 5\}$. By contrast, $1 \in D$ is not an element of this preimage set, because f maps $1 \in D$ to $b \in Z$ and $b \notin Z'$. Nevertheless, $1 \in D$ is of course a preimage of b.

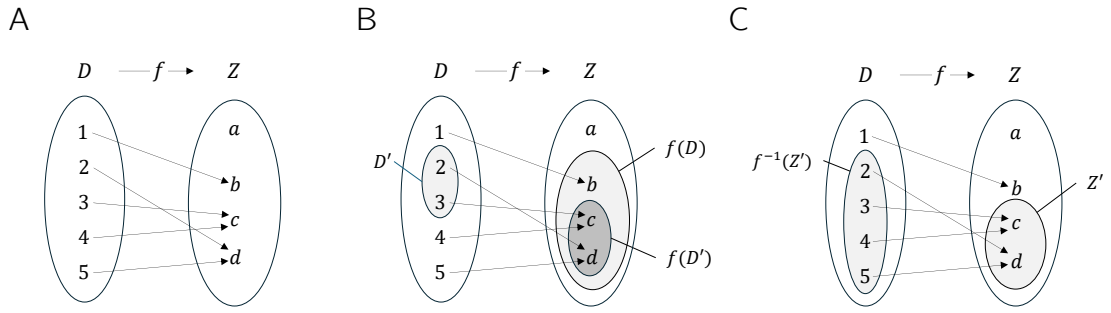


Figure 4.2 (A) Visualization of the function Equation 4.5. (B) Range and an image of f. (C) Preimage set of a set Z' under f

Basic properties of functions are named in the following definition.

Definition 4.3 (Injectivity, surjectivity, bijectivity)**.** Let $f : D \to Z, x \mapsto f(x)$ be a function. f is called *injective* if, for every image element $z \in f(D)$, there is exactly one preimage $x \in D$. Equivalently, f is injective if $x_1, x_2 \in D$ and $x_1 \neq x_2$ imply $f(x_1) \neq f(x_2)$. f is called *surjective* if $f(D) = Z$, that is, if every element of the codomain Z has a preimage in the domain D. Finally, f is called *bijective* if it is both injective and surjective. Bijective functions are also called *one-to-one correspondences* or *one-to-one mappings*.

•

Examples

We first illustrate Definition 4.3 using three (counter)examples in Figure 4.3. Figure 4.3 A shows the *non-injective* function

$$f : \{1, 2, 3\} \to \{a, b\}, x \mapsto f(x) := \begin{cases} f(1) := a \\ f(2) := a \\ f(3) := b \end{cases} . \tag{4.6}$$

The function is non-injective because the element a in the range of f has more than one preimage in the domain of f, namely the elements 1 and 2. Figure 4.3 B shows the

non-surjective function

$$g : \{1, 2, 3\} \to \{a, b, c, d\}, x \mapsto g(x) := \begin{cases} g(1) := a \\ g(2) := b \\ g(3) := d \end{cases} . \tag{4.7}$$

The function is non-surjective because the element c in the codomain of g has no preimage in the domain of g. Finally, Figure 4.3 C shows the bijective function

$$h : \{1, 2, 3\} \to \{a, b, c\}, x \mapsto h(x) := \begin{cases} h(1) := a \\ h(2) := b \\ h(3) := c \end{cases} . \tag{4.8}$$

For *every* element in the codomain of h there is *exactly one* preimage, so the function is injective and surjective and therefore bijective.

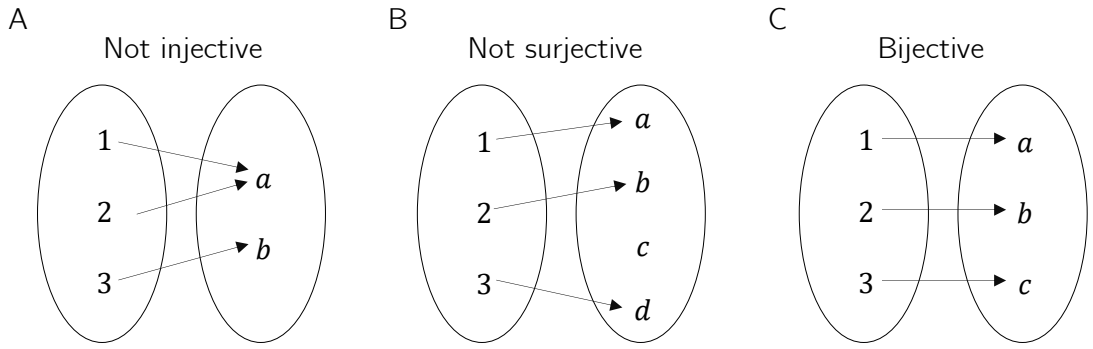


Figure 4.3 Injectivity, surjectivity, bijectivity.

As a further example, consider the function

$$f : \mathbb{R} \to \mathbb{R}, x \mapsto f(x) := x^2 \tag{4.9}$$

This function is not injective because, for example, with $x_1 = 2 \neq -2 = x_2$ we have $f(x_1) = 2^2 = 4 = (-2)^2 = f(x_2)$. Moreover, f is not surjective because, for example, $-1 \in \mathbb{R}$ has no preimage under f. However, if the domain of f is restricted to the non-negative real numbers, that is, if we define the function

$$\tilde{f} : [0, \infty[\to [0, \infty[, x \mapsto \tilde{f}(x) := x^2, \tag{4.10}$$

then, in contrast to f, $\tilde{f}$ is injective and surjective, hence bijective.

On the uncountability of the real numbers

At this point, we use the concepts of surjectivity and bijectivity of functions to deepen the idea of the *uncountability of the real numbers* a little. This idea says that there are different kinds of infinities, which is certainly not easy to grasp intuitively at first. Moreover, some aspects of probability theory are formulated in ways that accommodate the uncountability of the real numbers, so even a rough understanding of this property can help motivate some of the subtleties of probability theory. Specifically, we briefly sketch *Cantor's diagonal argument* (Cantor (1891), Gray (1994)) for the uncountability of the real numbers. We begin by recording what, against the background of bijective mappings, is meant by a *finite* and a *countably infinite* set.

Definition 4.4. A set M is called *finite* if there exists a bijective mapping of the form

$$f : \mathbb{N}_n \to M \tag{4.11}$$

onto the elements of M. Furthermore, a set M is called *countably infinite* if there exists a bijective mapping of the form

$$f : \mathbb{N} \to M \tag{4.12}$$

onto the elements of M.

•

In the case of a finite set, each element of the set can thus be assigned a natural number with maximum value $n < \infty$ in the sense of a bijective mapping. In the case of a countably infinite set, each element of the set can at least be assigned a natural number in the sense of a bijective mapping. The elements of these sets can therefore be counted.

Cantor (1891) showed that this is not true in general for the real numbers: no bijective mapping between the real numbers and the natural numbers can be constructed, and hence there must be more real numbers than natural numbers. More specifically, one can argue that even the interval $[0, 1]$ already contains more real numbers than there are natural numbers, that is, no surjective function from $\mathbb{N}$ to $[0, 1]$ can be constructed.

Cantor's argument has the following form. Suppose that $f : \mathbb{N} \to [0, 1]$ is an injective function represented in tabular form, for example as in Table 4.1.

Table 4.1 Mapping $f : \mathbb{N} \to [0, 1]$. The dots ⋯ and ⋮ indicate that the table continues indefinitely to the right and downward.

n	$f(n)$									
1	0.	**3**	1	4	1	5	9	2	6	5	⋯
2	0.	8	**9**	4	5	9	7	8	1	0	⋯
3	0.	9	6	**2**	3	2	1	5	8	7	⋯
4	0.	5	3	6	**8**	8	8	9	7	3	⋯
5	0.	7	4	3	8	**1**	8	0	9	7	⋯
⋮	⋮										

Now construct another number in $[0, 1]$ by adding 1 to each of the bold diagonal entries in Table 4.1 whenever the corresponding value is less than 9, and otherwise setting the value to 0. This is shown in Table 4.2.

Table 4.2 Construction of another real number in $[0, 1]$.

n	$f(n)$									
1	0.	**4**	1	4	1	5	9	2	6	5	⋯
2	0.	8	**0**	4	5	9	7	8	1	0	⋯
3	0.	9	6	**3**	3	2	1	5	8	7	⋯
4	0.	5	3	6	**9**	8	8	9	7	3	⋯
5	0.	7	4	3	8	**2**	8	0	9	7	⋯
⋮	⋮										

The number thus constructed,

$$0.40392... \tag{4.13}$$

cannot occur in Table 4.1, because it differs from $f(1)$ in its first decimal place, from $f(2)$ in its second decimal place, from $f(3)$ in its third decimal place, ..., from $f(n)$ in its nth decimal place, and so on indefinitely. Thus there is at least one number in $[0, 1]$ to which no natural number can be assigned. The injective mapping f is not surjective and therefore not bijective. Consequently, $[0, 1]$ is not countably infinite and is accordingly called *uncountable*.

4.2 Types of functions

By composition, further functions can be formed from given functions.

Definition 4.5 (Composition of functions)**.** Let $f : D \to Z$ and $g : Z \to S$ be two functions, where the range of f is assumed to agree with the domain of g. Then

$$g \circ f : D \to S, x \mapsto (g \circ f)(x) := g(f(x)) \tag{4.14}$$

defines a function called the *composition of f and g*.

•

The notation for composed functions takes a little getting used to. It is important to recognize that $g \circ f$ denotes the composed function and $(g \circ f)(x)$ denotes an element in the codomain of the composed function. Intuitively, when evaluating $(g \circ f)(x)$, one first applies the function f to x and then applies the function g to the element $f(x)$ of Z. This is captured by the functional form $g(f(x))$. For simplicity, the composition of two functions is often denoted by a single letter; for example, one writes $h := g \circ f$ with $h(x) = g(f(x))$. A mild source of confusion can arise when elements in the codomain of f are denoted by y, so that the notation $y = f(x)$ and $h(x) = g(y)$ is used. However, this notation is sometimes needed for notational simplicity. We visualize Definition 4.5 in Figure 4.4.

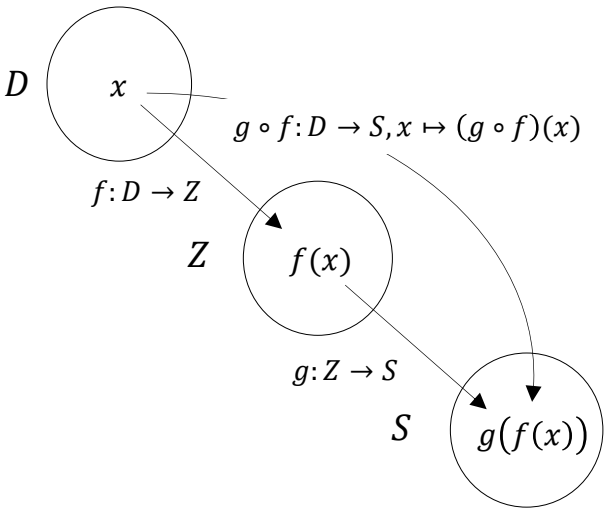


Figure 4.4 Composition of functions.

As an example of the composition of two functions, consider

$$f : \mathbb{R} \to \mathbb{R}, x \mapsto f(x) := -x^2 \tag{4.15}$$

and

$$g : \mathbb{R} \to \mathbb{R}, x \mapsto g(x) := \exp(x). \tag{4.16}$$

In this case, the composition of f and g is

$$g \circ f : \mathbb{R} \to \mathbb{R}, x \mapsto (g \circ f)(x) := g(f(x)) = \exp\left(-x^2\right). \tag{4.17}$$

A first application of function composition appears in the following definition.

Definition 4.6 (Inverse function)**.** Let $f : D \to Z, x \mapsto f(x)$ be a bijective function. Then the function f^{-1} with

$$f^{-1} \circ f : D \to D, x \mapsto (f^{-1} \circ f)(x) := f^{-1}(f(x)) = x \tag{4.18}$$

is called the *inverse function*, *inverse mapping*, or simply the *inverse of* f.

•

Inverse functions are always bijective. This follows because f is bijective and therefore each $x \in D$ is assigned exactly one $f(x) = z \in Z$. Hence, each $z \in Z$ is also assigned exactly one $x \in D$, namely $f^{-1}(f(x)) = x$. Intuitively, the inverse function of f undoes the effect of f on an element x. We visualize Definition 4.6 in Figure 4.5 A.

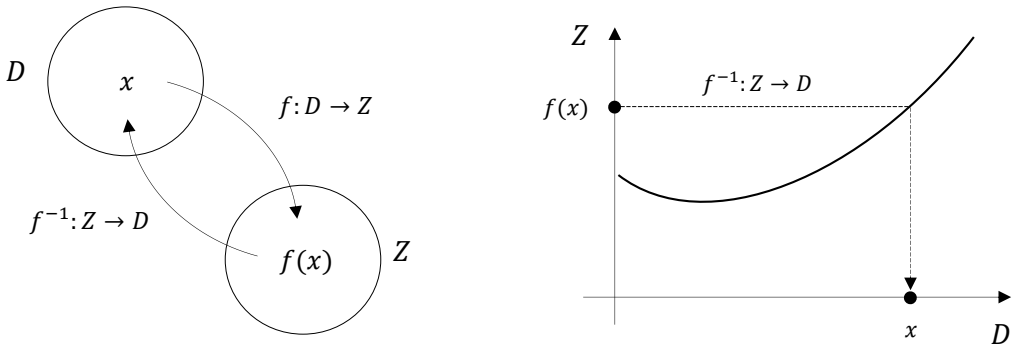


Figure 4.5 Inverse function.

If one considers the graph of a function in a Cartesian coordinate system, then applying a function to a value on the x-axis leads to a value on the y-axis. Applying the inverse function correspondingly leads from a value on the y-axis to a value on the x-axis. We visualize this in Figure 4.5 B. For example, consider the function

$$f : \mathbb{R} \to \mathbb{R}, x \mapsto f(x) := 2x =: y. \tag{4.19}$$

Then the inverse function of f is given by

$$f^{-1} : \mathbb{R} \to \mathbb{R}, y \mapsto f^{-1}(y) := \frac{1}{2}y, \tag{4.20}$$

because for every $x \in \mathbb{R}$,

$$(f^{-1} \circ f)(x) := f^{-1}(f(x)) = f^{-1}(2x) = \frac{1}{2} \cdot 2x = x. \tag{4.21}$$

An important class of functions consists of *linear maps*.

Definition 4.7 (Linear map)**.** A map $f : D \to Z, x \mapsto f(x)$ is called a *linear map* if, for $x, y \in D$ and a scalar c,

$$f(x + y) = f(x) + f(y) \qquad \text{(additivity)}$$

and

$$f(cx) = cf(x) \qquad \text{(homogeneity)}$$

hold. A map for which the above properties do not hold is called a *nonlinear map*.

•

Linear maps are often known as "straight lines." The general definition of linear maps is not completely congruent with this intuition. In particular, linear maps are only those functions that map zero to zero. We show this in the following theorem.

Theorem 4.1 (Linear map of zero)**.** *Let $f : D \to Z$ be a linear map. Then*

$$f(0) = 0. \tag{4.22}$$

∘

Proof. First note that by additivity of f,

$$f(0) = f(0 + 0) = f(0) + f(0). \tag{4.23}$$

Adding $-f(0)$ to both sides of the above equation gives

$$f(0) - f(0) = f(0) + f(0) - f(0) \Leftrightarrow f(0) = 0 \tag{4.24}$$

and this proves the claim. □

We illustrate the concept of a linear map with two examples.

- For $a \in \mathbb{R}$, the map
$$f : \mathbb{R} \to \mathbb{R}, x \mapsto f(x) := ax \tag{4.25}$$
is a linear map because
$$f(x+y) = a(x+y) = ax+ay = f(x)+f(y) \text{ and } f(cx) = acx = cax = cf(x). \tag{4.26}$$
- For $a, b \in \mathbb{R}$, by contrast, the map
$$f : \mathbb{R} \to \mathbb{R}, x \mapsto f(x) := ax + b \tag{4.27}$$
is nonlinear because, for example, for $a := b := 1$,
$$f(x + y) = 1(x + y) + 1 = x + y + 1 \neq x + 1 + y + 1 = f(x) + f(y). \tag{4.28}$$

A map of the form $f(x) := ax + b$ is called a *linear-affine map* or *linear-affine function*. Somewhat imprecisely, functions of the form $f(x) := ax + b$ are sometimes also called *linear functions*.

Besides the types of functions discussed so far, there are many further classes of functions. In the following definition, we classify functions by the dimensionality of their domains and codomains. This type of function classification is often helpful for gaining an initial overview of a mathematical model.

Definition 4.8 (Function types)**.** We distinguish

- *univariate real-valued functions* of the form

$$f : \mathbb{R} \to \mathbb{R}, x \mapsto f(x), \tag{4.29}$$

- *multivariate real-valued functions* of the form

$$f : \mathbb{R}^n \to \mathbb{R}, x \mapsto f(x) = f(x_1, ..., x_n), \tag{4.30}$$

- and *multivariate vector-valued functions* of the form

$$f : \mathbb{R}^n \to \mathbb{R}^m, x \mapsto f(x) = \begin{pmatrix} f_1(x_1, ..., x_n) \\ \vdots \\ f_m(x_1, ..., x_n) \end{pmatrix}, \tag{4.31}$$

  where $f_i, i = 1, ..., m$ are called the *components* or *component functions* of f.

•

In physics, multivariate real-valued functions are called *scalar fields*, and multivariate vector-valued functions are called *vector fields*. In some applications, *matrix-variate matrix-valued functions* also occur.

4.3 Elementary functions

By *elementary functions* we mean a small set of univariate real-valued functions that frequently appear as building blocks of more complex functions. These are the *polynomial functions*, the *exponential function*, the *logarithm function*, and the *gamma function*. In the following, we give the definitions of these functions and state their main properties as theorems, and then present their graphs. For proofs of the properties introduced here, we refer to the advanced literature.

Definition 4.9 (Polynomial functions)**.** A function of the form

$$f : \mathbb{R} \to \mathbb{R}, x \mapsto f(x) := \sum_{i=0}^{k} a_i x^i = a_0 + a_1 x^1 + a_2 x^2 + \cdots + a_k x^k \tag{4.32}$$

is called a *polynomial function* of degree k with coefficients $a_0, a_1, ..., a_k \in \mathbb{R}$. Typical polynomial functions are listed in Table 4.3.

Table 4.3 Polynomial functions.

Name	Form	Coefficients
Constant function	$f(x) = a$	$a_0 := a$, $a_i := 0, i > 0$
Identity function	$f(x) = x$	$a_0 := 0$, $a_1 := 1$, $a_i := 0, i > 1$
Linear-affine function	$f(x) = ax + b$	$a_0 := b$, $a_1 := a$, $a_i := 0, i > 1$
Square function	$f(x) = x^2$	$a_0 := 0$, $a_1 := 0$, $a_2 := 1$, $a_i := 0, i > 2$

•

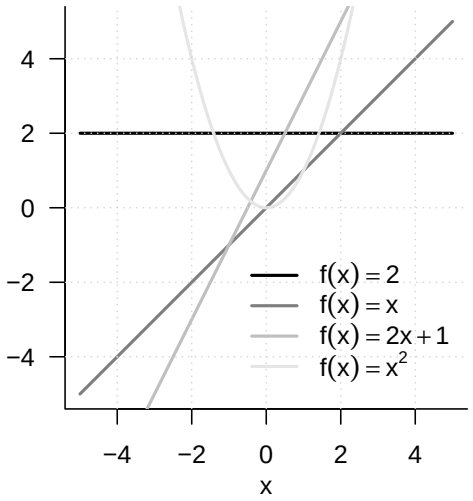


Figure 4.6 Selected polynomial functions

Figure 4.6 shows the graphs of the polynomial functions listed in Definition 4.9.

An important pair of functions consists of the exponential function and the logarithm function.

Theorem 4.2 (Exponential function and its properties)**.** *The* exponential function *is defined as*

$$\exp : \mathbb{R} \to \mathbb{R}, x \mapsto \exp(x) := e^x := \sum_{n=0}^{\infty} \frac{x^n}{n!} = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \frac{x^4}{4!} + \cdots \tag{4.33}$$

Among others, the exponential function has the properties listed in Table 4.4.

Table 4.4 Properties of the exponential function.

Property	*Meaning*
Range	$x \in]-\infty, 0[\Rightarrow \exp(x) \in]0, 1[$
	$x \in]0, \infty[\quad \Rightarrow \exp(x) \in]1, \infty[$
Monotonicity	$x < y \Rightarrow \exp(x) < \exp(y)$
Special values	$\exp(0) = 1$ *and* $\exp(1) = e$
Summation property	$\exp(x + y) = \exp(x)\exp(y)$
Subtraction property	$\exp(x - y) = \frac{\exp(x)}{\exp(y)}$

∘

In particular, the exponential function only takes positive values and intersects the y-axis at $x = 0$. The number $\exp(1) := e \approx 2.71...$ is called *Euler's number*. Finally, the special values of the exponential function imply

$$\exp(x)\exp(-x) = \exp(x - x) = \exp(0) = 1. \tag{4.34}$$

Theorem 4.3 (Logarithm function and its properties)**.** *The* logarithm function *is defined as the inverse function of the exponential function,*

$$\ln :]0, \infty[\to \mathbb{R}, x \mapsto \ln(x) \text{ with } \ln(\exp(x)) = x \text{ for all } x \in \mathbb{R}. \tag{4.35}$$

Among others, the logarithm function has the properties listed in Table 4.5.

Table 4.5 Properties of the logarithm function.

Property	*Meaning*
Range	$x \in]0, 1[\Rightarrow \ln(x) \in]-\infty, 0[$
	$x \in]1, \infty[\Rightarrow \ln(x) \in]0, \infty[$
Monotonicity	$x < y \Rightarrow \ln(x) < \ln(y)$
Special values	$\ln(1) = 0$ *and* $\ln(e) = 1$
Product property	$\ln(xy) = \ln(x) + \ln(y)$
Power property	$\ln(x^c) = c \ln(x)$
Division property	$\ln\left(\frac{1}{x}\right) = -\ln(x)$

∘

In contrast to the exponential function, the logarithm function takes both negative and positive values. The logarithm function intersects the x-axis at $x = 1$. The product property and the power property are central when calculating with the logarithm function. Intuitively, they can be remembered as: "The logarithm function turns products into sums and powers into products." The graphs of the exponential and logarithm functions are shown in Figure 4.7.

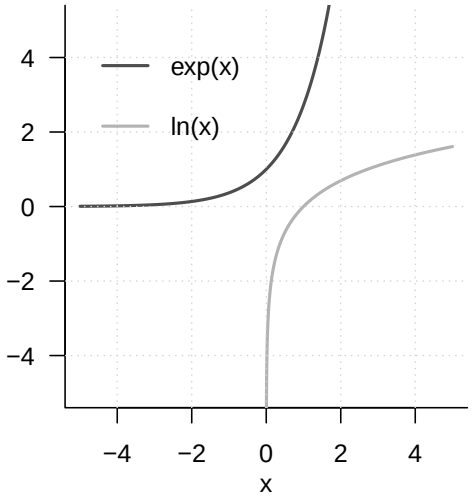


Figure 4.7 Exponential function and logarithm function

A frequent companion in probability theory is the *gamma function*.

Definition 4.10 (Gamma function)**.** The *gamma function* is defined by

$$\Gamma : \mathbb{R} \to \mathbb{R}, x \mapsto \Gamma(x) := \int_0^\infty \xi^{x-1} \exp(-\xi)\, d\xi \tag{4.36}$$

Among others, the gamma function has the properties listed in Table 4.6.

Table 4.6 Properties of the gamma function.

Property	Meaning
Special values	$\Gamma(1) = 1$
	$\Gamma\left(\frac{1}{2}\right) = \sqrt{\pi}$
	$\Gamma(n) = (n-1)!$ for $n \in \mathbb{N}$
Recursion property	For $x > 0$, $\Gamma(x+1) = x\Gamma(x)$

•

A section of the graph of the gamma function is shown in Figure 4.8.

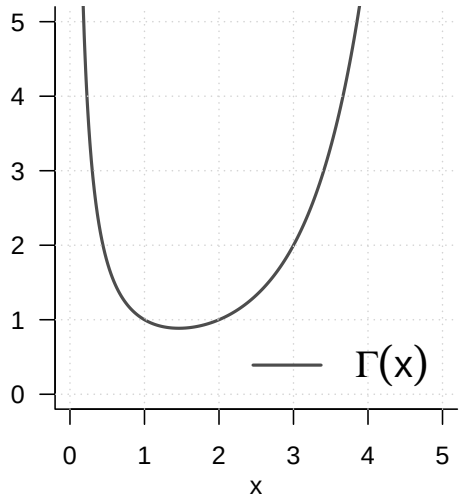


Figure 4.8 Gamma function

5 Sequences, limits, continuity

The topics treated in this chapter are not central to probabilistic data science but form basic building blocks of real analysis. Because modern probability theory is closely intertwined with analytic approaches, however, they help us understand certain results of probability theory. One example of such a result is the *central limit theorem*. The central limit theorem, in turn, forms the basis for the widely used normal-distribution assumption in probabilistic data science. The foundations considered here therefore indirectly increase our understanding of data-scientific principles. In brief, the central limit theorem is a statement about the *limit function* of a *sequence of functions*, more precisely about a sequence of random variables. Knowledge of the nature of *sequences*, *sequences of functions*, and their *limits* therefore allows an informed entry into the study of the central limit theorem. Furthermore, the topics treated in this chapter provide at least a first entry point for understanding continuity and smoothness of functions, which are important basic concepts in nonlinear optimization, for example when determining parameter estimators of probabilistic models.

5.1 Sequences

We begin with the definition of the concept of a *real sequence*.

Definition 5.1 (Real sequence)**.** A *real sequence* is a function of the form

$$f : \mathbb{N} \to \mathbb{R}, n \mapsto f(n). \tag{5.1}$$

The function values $f(n)$ of a real sequence are usually denoted by x_n and called *sequence terms*. Common notations for sequences are

$$(x_1, x_2, ...) \text{ or } (x_n)_{n=1}^{\infty} \text{ or } (x_n)_{n\in\mathbb{N}} \text{ or } (x_n). \tag{5.2}$$

•

Note that a real sequence, because there are infinitely many natural numbers, always has infinitely many sequence terms. This should be kept in mind especially when using the notation $(x_1, x_2, ...)$. We consider two standard examples of real sequences.

Examples of Real Sequences

(1) Real sequences of the form

$$f : \mathbb{N} \to \mathbb{R}, n \mapsto f(n) := \left(\frac{1}{n}\right)^{\frac{p}{q}} \text{ with } p, q \in \mathbb{N} \tag{5.3}$$

are called *harmonic sequences*. For $p := q := 1$, a harmonic sequence has the sequence-term form

$$\left(\frac{1}{1}, \frac{1}{2}, \frac{1}{3}, ...\right). \tag{5.4}$$

(2) Real sequences of the form

$$f : \mathbb{N} \to \mathbb{R}, n \mapsto f(n) := q^n \text{ with } q \in]-1, 1[\tag{5.5}$$

are called *geometric sequences*. For $q := \frac{1}{2}$, a geometric sequence has the sequence-term form

$$\begin{aligned}\left(\left(\frac{1}{2}\right)^1, \left(\frac{1}{2}\right)^2, \left(\frac{1}{2}\right)^3, ...\right) &= \left(\frac{1^1}{2^1}, \frac{1^2}{2^2}, \frac{1^3}{2^3}, ...\right) \\ &= \left(\frac{1}{2}, \frac{1}{4}, \frac{1}{8}, ...\right).\end{aligned} \tag{5.6}$$

In addition to real sequences, that is, sequences of real numbers, one can also consider sequences of other mathematical objects. An important type of sequence is the *sequence of functions*.

Definition 5.2 (Sequence of functions)**.** Let ϕ be a set of univariate real-valued functions with domain $D \subseteq \mathbb{R}$. Then a sequence of functions is a function of the form

$$F : \mathbb{N} \to \phi, n \mapsto F(n). \tag{5.7}$$

The function values $F(n)$ of a sequence of functions are usually denoted by f_n and called *sequence terms*. Common notations for sequences of functions are

$$(f_1, f_2, ...) \text{ or } (f_n)_{n=1}^{\infty} \text{ or } (f_n)_{n\in\mathbb{N}} \text{ or } (f_n). \tag{5.8}$$

•

The definition of a sequence of functions is evidently analogous to the definition of a real sequence. The difference between a real sequence and a sequence of functions is that the sequence terms of a real sequence are real numbers, whereas the sequence terms of a sequence of functions are univariate real-valued functions. Here, too, we discuss two standard examples.

Examples of Sequences of Functions

(1) We consider the set ϕ of univariate real-valued functions of the form

$$\phi := \{f_n | f_n : [0, 1] \to \mathbb{R}, x \mapsto f_n(x) := x^n \text{ for } n \in \mathbb{N}\}. \tag{5.9}$$

Then

$$F : \mathbb{N} \to \phi, n \mapsto F(n) := f_n \tag{5.10}$$

defines a sequence of functions. For the function values of the sequence terms of F, we have

$$f_1(x) := x^1, f_2(x) := x^2, f_3(x) := x^3, ... \tag{5.11}$$

(2) We consider the set ϕ of univariate real-valued functions of the form

$$\phi := \{f_n | f_n : [-a, a] \to \mathbb{R}, x \mapsto f_n(x) := \sum_{k=0}^{n} \frac{x^k}{k!} \text{ for } n \in \mathbb{N}\}. \tag{5.12}$$

Then

$$F : \mathbb{N} \to \phi, n \mapsto F(n) := f_n \tag{5.13}$$

defines a sequence of functions. For the function values of the sequence terms of F, we have

$$f_1(x) := \sum_{k=0}^{1} \frac{x^k}{k!}, f_2(x) := \sum_{k=0}^{2} \frac{x^k}{k!}, f_3(x) := \sum_{k=0}^{3} \frac{x^k}{k!}, ... \tag{5.14}$$

5.2 Limits

When considering the terms of a sequence, one can ask which values a sequence might take when the sequence index n becomes very large, that is, tends to infinity. If, in this case, the sequence terms take very similar values (and do not themselves become infinitely large), one is led to the concept of a *limit* for real sequences and a *limit function* for sequences of functions.

Definition 5.3 (Limit of a real sequence)**.** $x \in \mathbb{R}$ is called the limit of a real sequence $(x_n)_{n=1}^{\infty}$ if, for every $\epsilon > 0$, there exists an $m \in \mathbb{N}$ such that

$$|x_n - x| < \epsilon \text{ for all } n \geq m. \tag{5.15}$$

A sequence that has a limit is called a *convergent sequence*; a sequence that has no limit is called a *divergent sequence*. To express that $x \in \mathbb{R}$ is the limit of the sequence $(x_n)_{n=1}^{\infty}$, one also writes

$$\lim_{n\to\infty} x_n = x \text{ or } x_n \to x \text{ for } n \to \infty \text{ or } x_n \xrightarrow{n\to\infty} x. \tag{5.16}$$

•

According to Definition 5.3, the limit of a sequence may exist, but it need not. For example, the sequence

$$f : \mathbb{N} \to \mathbb{R}, n \mapsto f(n) := n \tag{5.17}$$

has no limit, because here both n and $f(n)$ become infinitely large. For a convergent sequence, that is, a sequence whose limit exists, Definition 5.3 says that, for an arbitrarily small but positive ϵ, an $m \in \mathbb{N}$ can be specified such that, for all sequence terms x_n with $n \geq m$, the distance from the limit in the positive or negative direction is smaller than ϵ. Thus the sequence terms with $n \geq m$ lie arbitrarily close to the limit x. It may often be the case that a smaller ϵ requires a larger m.

Examples

The examples of real sequences considered above, by contrast, have limits.

(1) For the generalized harmonic sequences, with $p, q \in \mathbb{N}$,

$$\lim_{n\to\infty} \left(\frac{1}{n}\right)^{\frac{p}{q}} = 0. \tag{5.18}$$

(2) For the geometric sequences, with $q \in]-1, 1[$,

$$\lim_{n\to\infty} q^n = 0. \tag{5.19}$$

The harmonic and geometric sequences are therefore also called *null sequences*. For general proofs of Equation 5.18 and Equation 5.19, we refer to the advanced literature. In fact, these proofs are not trivial and touch on the basic assumptions about the nature of the real numbers. In Figure 5.1, we visualize the first ten sequence terms as well as the limits of the harmonic sequence for $p := q := 1$ and of the geometric sequence for $q := 1/2$. The sequence terms shown as points evidently lie increasingly closer to the limit shown by a gray line as n increases.

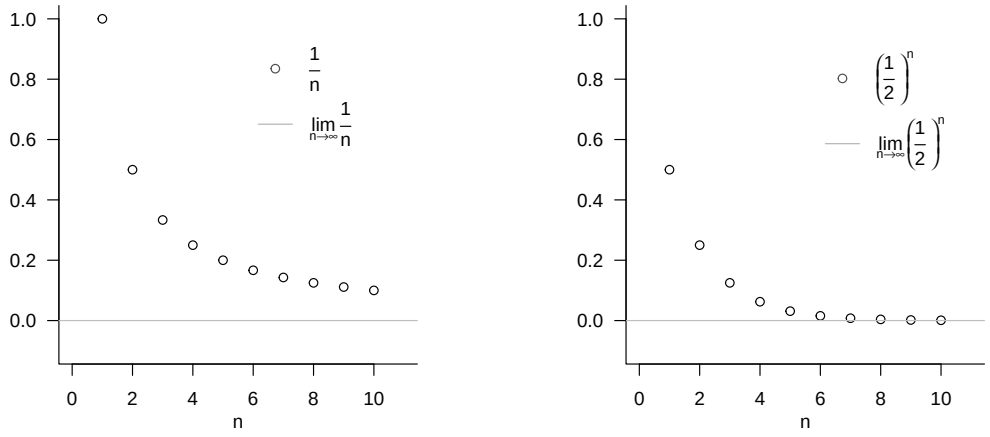


Figure 5.1 Examples of limits of real sequences.

For the special case of the harmonic sequence with $p = q = 1$, we briefly present the argument for the limit

$$\lim_{n\to\infty} \frac{1}{n} = 0. \tag{5.20}$$

According to Definition 5.3, Equation 5.20 holds if and only if, for an arbitrary $\epsilon > 0$ (and in particular for an arbitrarily small one), an $m \in \mathbb{N}$ can be specified such that, for all $n \geq m$,

$$\left|\frac{1}{n} - 0\right| < \epsilon \Leftrightarrow \frac{1}{n} < \epsilon.$$

Thus, using the ceiling function $\lceil\cdot\rceil$, if for an arbitrary $\epsilon > 0$ we set

$$m := \left\lceil \frac{1}{\epsilon} \right\rceil + 1,$$

for example, for $\epsilon := 0.01$ correspondingly $m := \lceil \frac{1}{0.01} \rceil + 1 = 101$, then for all $n \geq m$ we have $\frac{1}{n} < \epsilon$. Since $\epsilon > 0$ was chosen arbitrarily, this construction is possible for all $\epsilon > 0$.

For sequences of functions, one possible extension of the concepts of convergence and limit is the following.

Definition 5.4 (Pointwise convergence and limit function of a sequence of functions)**.** Let $F = (f_n)_{n\in\mathbb{N}}$ be a sequence of functions of univariate real-valued functions with domain D. F is called *pointwise convergent* if the real sequence $(f_n(x))_{n\in\mathbb{N}}$ is a convergent sequence for every $x \in D$, that is, if it has a limit. The function that assigns to each $x \in D$ this limit of $(f_n(x))_{n\in\mathbb{N}}$ is then called the *limit function of the sequence of functions* F and has the form

$$f : D \to \mathbb{R}, x \mapsto f(x) := \lim_{n\to\infty} f_n(x). \tag{5.21}$$

•

Note that the limits of convergent real sequences are real numbers, whereas the limit functions of pointwise convergent sequences of functions are functions. In addition to

pointwise convergence of sequences of functions, there is the more powerful concept of *uniform convergence* of sequences of functions, for which we refer to the advanced literature. As examples, we consider the limit functions of the sequences of functions discussed above; for proofs, we again refer to the advanced literature.

Examples

(1) We consider the sequence of functions

$$F : \mathbb{N} \to \phi, n \mapsto F(n) \tag{5.22}$$

with

$$\phi := \{f_n | f_n : [0,1] \to \mathbb{R}, x \mapsto f_n(x) := x^n \text{ for } n \in \mathbb{N}\}. \tag{5.23}$$

Then F is pointwise convergent with limit function

$$f : [0,1] \to \mathbb{R}, x \mapsto f(x) := \begin{cases} 0, & \text{for } x \in [0,1[\\ 1, & \text{for } x = 1 \end{cases} \tag{5.24}$$

because $f_n(x) := x^n$ is a geometric sequence, and thus a null sequence, for $x \in [0,1[$, while $f_n(x) := x^n$ is a constant sequence for $x = 1$, all of whose sequence terms have distance 0 from 1. The sequence of functions F therefore converges to a function that is equal to zero on the entire interval $[0,1]$, except at the point 1. This function evidently has a jump.

(2) We consider the sequence of functions

$$F : \mathbb{N} \to \phi, n \mapsto F(n) \tag{5.25}$$

with

$$\phi := \{f_n | f_n : [-a,a] \to \mathbb{R}, x \mapsto f_n(x) := \sum_{k=0}^{n} \frac{x^k}{k!} \text{ for } n \in \mathbb{N}\}. \tag{5.26}$$

Then F is pointwise convergent with limit function

$$f : [-a,a] \to \mathbb{R}, x \mapsto f(x) := \sum_{k=0}^{\infty} \frac{x^k}{k!} =: \exp(x). \tag{5.27}$$

The sequence of functions F therefore converges to the exponential function on $[-a,a]$. Conversely, the exponential function is defined precisely by

$$\exp(x) := \sum_{k=0}^{\infty} \frac{x^k}{k!}. \tag{5.28}$$

5.3 Continuity

In this section, we approach the concept of the *continuity* of a function. Intuitively, a function is continuous if it has no jumps or, equivalently, if small changes in its arguments always lead only to small changes in its function values (and therefore precisely to no jumps). To define continuity, we first need the concept of the *limit of a function*.

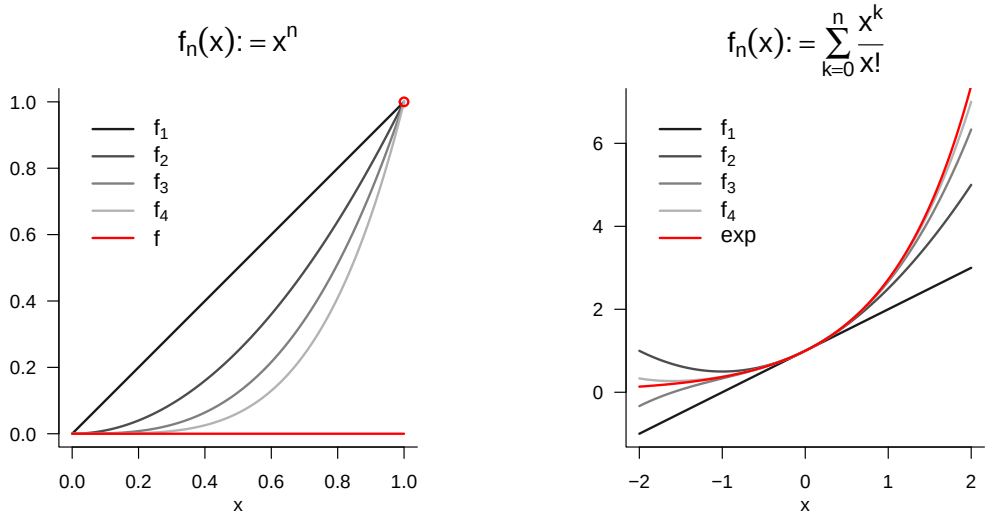


Figure 5.2 Examples of limits of sequences of functions.

Definition 5.5 (Limit of a function)**.**

For $D \subseteq \mathbb{R}$ and $Z \subseteq \mathbb{R}$, let $f : D \to Z, x \mapsto f(x)$ be a function, and let $a, b \in \mathbb{R}$. b is called the *limit of the function f as x tends to a* if

(1) there exists a real sequence $(x_n)_{n=1}^{\infty}$ with sequence terms in D and limit a, that is, $\lim_{n\to\infty} x_n = a$, and

(2) for every such sequence, b is the limit of the sequence of function values $f(x_n)$ of the sequence terms of $(x_n)_{n=1}^{\infty}$, that is, $\lim_{n\to\infty} f(x_n) = b$.

If b is the limit of the function f as x tends to a, one also writes $\lim_{x\to a} f(x) = b$.

•

In Figure 5.3, we visualize the limit of the exponential function at $a = 1$ by representing sequence terms $x_n \to 1$ as points parallel to the x-axis and the corresponding sequence terms $f(x_n)$ on the function graph. Evidently, $\lim_{x\to 1} \exp(x) = e \approx 2.71$.

We can now define the concept of continuity of a function.

Definition 5.6 (Continuity of a function)**.** A function $f : D \to Z$ with $D \subseteq \mathbb{R}$ and $Z \subseteq \mathbb{R}$ is called *continuous at $a \in D$* if

$$\lim_{x\to a} f(x) = f(a). \tag{5.29}$$

If f is continuous at every $x \in D$, then f *is continuous on D*.

•

Note that, for a function continuous at a, it follows that

$$\lim_{x\to a} f(x) = f\left(\lim_{x\to a} x\right). \tag{5.30}$$

For continuous functions, taking limits and evaluating the function can therefore be interchanged.

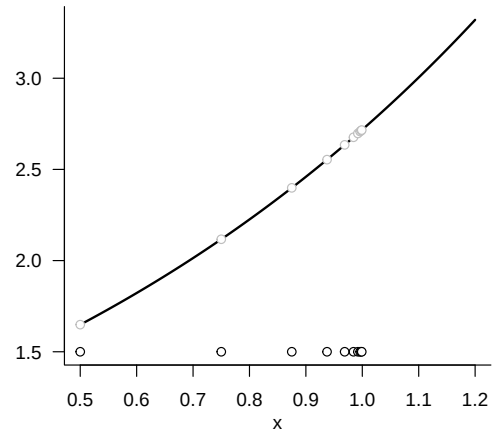


Figure 5.3 Example of the limit of a function. The sequence of x-values is defined here as $x_n := 1-(1/2)^n$ for $n = 1, 2, \ldots$.

6 Differential calculus

Differential calculus is concerned with the change of functions. On the one hand, it provides the basis for the mathematical modelling of dynamic systems by means of *differential equations*, that is, the description of functions in terms of their rates of change. On the other hand, differential calculus provides the basis of *optimization*, that is, of determining extrema of functions. In Section 6.1 we first introduce the concept of the derivative and elementary calculation rules associated with it. In Section 6.2 we then turn to the question of how derivatives can be used to determine extrema of functions.

6.1 Definitions and calculation rules

We begin with the following definition.

Definition 6.1 (Differentiability and derivative)**.** Let $I \subseteq \mathbb{R}$ be an interval and let

$$f : I \to \mathbb{R}, x \mapsto f(x) \tag{6.1}$$

be a univariate real-valued function. f is called *differentiable* at $a \in I$ if the limit

$$f'(a) := \lim_{h \to 0} \frac{f(a+h) - f(a)}{h} \tag{6.2}$$

exists. $f'(a)$ is then called the *derivative of f at the point a*. If f is differentiable for all $x \in I$, then f is called *differentiable*, and the function

$$f' : I \to \mathbb{R}, x \mapsto f'(x) \tag{6.3}$$

is called the *derivative of f*.

•

For $h > 0$, the expression

$$\frac{f(a+h) - f(a)}{h} \tag{6.4}$$

is called the *Newton difference quotient*. As shown in Figure 6.1, the Newton difference quotient measures the change $f(a+h)-f(a)$ of f on the y-axis per distance h on the x-axis. If, for example, $f(a)$ and $f(a+h)$ represent the position of an object at a time a and at a later time $a+h$, then $f(a+h)-f(a)$ is the distance travelled by this object in the time h, that is, its average velocity over the time interval h. For $h \to 0$, the Newton difference quotient then measures the instantaneous rate of change of f at a, in this example the velocity of the object at time a.

From a mathematical point of view, it is important to distinguish between the symbols $f'(a)$ and f' in the definition of the derivative. As usual, $f'(a)$ denotes the value of a

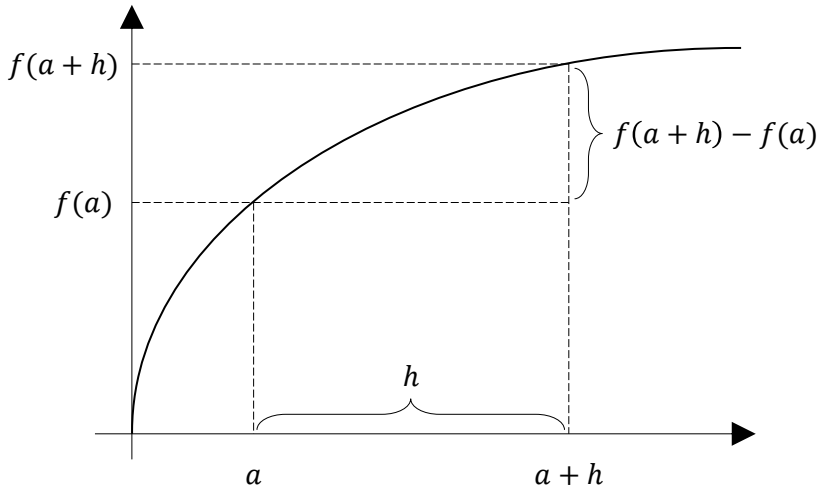


Figure 6.1 Elements of the Newton difference quotient.

function, that is, a number. In contrast, f' denotes a function, namely the function whose values are given by $f'(a)$ for all $a \in \mathbb{R}$.

Several historically established notations for derivatives exist in the literature; all of them represent the same concept of the derivative.

Definition 6.2 (Notation for derivatives of univariate real-valued functions)**.** Let f be a univariate real-valued function. Equivalent notations for the derivative of f and the derivative of f at a point x are

- the *Lagrange notation* f' and $f'(x)$,
- the *Leibniz notation* $\frac{df}{dx}$ and $\frac{df(x)}{dx}$,
- the *Newton notation* $\dot{f}$ and $\dot{f}(x)$,
- the *Euler notation* Df and $Df(x)$.

•

In the following, for univariate real-valued functions we will mainly use the Lagrange notation f' and $f'(x)$ as labels. In calculations, we also use an adapted form of Leibniz notation and understand the expression $\frac{d}{dx}f(x)$ as the instruction to compute the derivative of f. Newton notation is used mainly when the function argument represents time and is then usually denoted by t for *time*. Accordingly, $\dot{f}(t)$ denotes the rate of change of f at time t. Euler notation is particularly useful in the context of multivariate real-valued or vector-valued functions.

We assume a number of derivatives of elementary functions to be known; these are summarized in Table 6.1. For proofs, we refer to the advanced literature.

Table 6.1 Derivatives of elementary functions

Name	Definition	Derivative
Polynomial function	$f(x) := \sum_{i=0}^{k} a_i x^i$	$f'(x) = \sum_{i=1}^{k} i a_i x^{i-1}$
Constant function	$f(x) := a$	$f'(x) = 0$
Identity function	$f(x) := x$	$f'(x) = 1$
Linear-affine function	$f(x) := ax + b$	$f'(x) = a$
Square function	$f(x) := x^2$	$f'(x) = 2x$
Exponential function	$f(x) := \exp(x)$	$f'(x) = \exp(x)$
Logarithm function	$f(x) := \ln(x)$	$f'(x) = \frac{1}{x}$

In Figure 6.2 we visualize the identity function, a linear function, and the square function together with their respective derivatives. In Figure 6.3 we visualize the exponential and logarithm functions together with their respective derivatives.

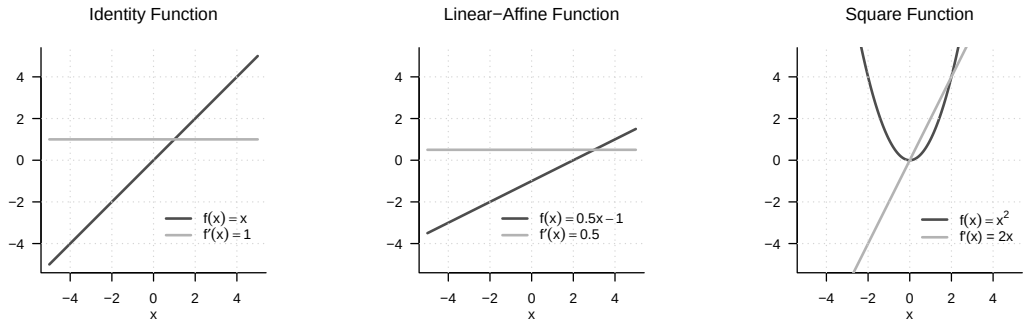


Figure 6.2 Derivatives of three elementary functions. The derivative of the identity function is the constant function with value 1, the derivative of the linear-affine function considered here is the constant function with value 0.5, and the derivative of the square function is the linear-affine function with $a = 2$ and $b = 0$.

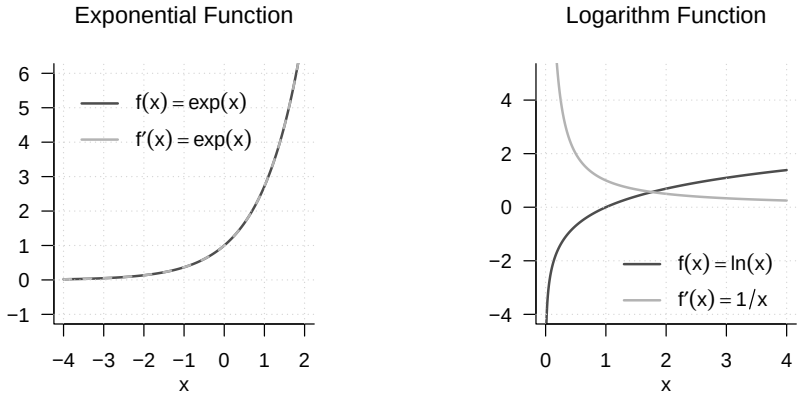


Figure 6.3 Derivatives of the exponential function and the logarithm function. The derivative of the exponential function is the exponential function itself; the derivative of the logarithm function has function values $1/x$.

Based on the definition of the derivative of a univariate real-valued function, further derivatives of such a function can be defined easily.

Definition 6.3 (Higher derivatives)**.** Let f be a univariate real-valued function and let

$$f^{(1)} := f' \tag{6.5}$$

be the derivative of f. The kth derivative of f is defined recursively by

$$f^{(k)} := \left(f^{(k-1)}\right)' \text{ for } k > 1, \tag{6.6}$$

assuming that $f^{(k-1)}$ is differentiable. In particular, the *second derivative of* f is defined as the derivative of f', that is,

$$f'' := (f')'. \tag{6.7}$$

•

In analogy to the above, in calculations we also write $\frac{d^2}{dx^2}f(x)$ for the instruction to determine the second derivative of a function f. The zeroth derivative $f^{(0)}$ of f is f itself. By tradition and for simplicity, for $k < 4$ one usually writes f', f'', and f''' according to Lagrange notation instead of $f^{(1)}, f^{(2)}$, and $f^{(3)}$.

Example

Let

$$f : \mathbb{R} \to \mathbb{R}, x \mapsto f(x) := x^2. \tag{6.8}$$

Then

$$f^{(1)}(x) = f'(x) = \frac{d}{dx}\left(x^2\right) = 2x. \tag{6.9}$$

Furthermore,

$$\begin{aligned} f^{(2)}(x) &= \left(f^{(2-1)}\right)'(x) = \left(f^{(1)}\right)'(x) = \frac{d}{dx}(2x) = 2, \\ f^{(3)}(x) &= \left(f^{(3-1)}\right)'(x) = \left(f^{(2)}\right)'(x) = \frac{d}{dx}(2) = 0, \\ f^{(4)}(x) &= \left(f^{(4-1)}\right)'(x) = \left(f^{(3)}\right)'(x) = \frac{d}{dx}(0) = 0. \end{aligned} \tag{6.10}$$

Thus, throughout this calculation, one computes only first derivatives.

A number of calculation rules are helpful for determining the derivative of a function because they allow the derivative of a function to be derived from the derivatives of its subfunctions. For proofs of the calculation rules introduced in the following theorem, we refer to the advanced literature.

Theorem 6.1 (Calculation rules for derivatives)**.** *For $i = 1, ..., n$, let g_i be real-valued univariate differentiable functions. Then the following calculation rules hold:*

(1) Sum rule

$$\text{For } f(x) := \sum_{i=1}^{n} g_i(x),\ f'(x) = \sum_{i=1}^{n} g_i'(x). \tag{6.11}$$

(2) Product rule

$$\text{For } f(x) := g_1(x)g_2(x),\ f'(x) = g_1'(x)g_2(x) + g_1(x)g_2'(x). \tag{6.12}$$

(3) Quotient rule

$$For\ f(x) := \frac{g_1(x)}{g_2(x)},\ f'(x) = \frac{g_1'(x)g_2(x) - g_1(x)g_2'(x)}{g_2^2(x)}. \tag{6.13}$$

(4) Chain rule

$$For\ f(x) := g_1(g_2(x)),\ f'(x) = g_1'(g_2(x))g_2'(x). \tag{6.14}$$

∘

Examples

(1) Sum rule

Let

$$f : \mathbb{R} \to \mathbb{R}, x \mapsto f(x) := 4x^3 + 3x^2. \tag{6.15}$$

Then f has the form

$$f(x) = \sum_{i=1}^{2} g_i(x) = g_1(x) + g_2(x) \text{ with } g_1(x) := 4x^3 \text{ and } g_2(x) := 3x^2. \tag{6.16}$$

Moreover,

$$g_1'(x) = \frac{d}{dx}\left(4x^3\right) = 12x^2 \text{ and } g_2'(x) = \frac{d}{dx}\left(3x^2\right) = 6x. \tag{6.17}$$

Thus, by the sum rule,

$$f'(x) = \sum_{i=1}^{2} g_i'(x) = g_1'(x) + g_2'(x) = 12x^2 + 6x. \tag{6.18}$$

(2) Product rule

Let

$$f : \mathbb{R} \to \mathbb{R}, x \mapsto f(x) := x^2 \sin(x). \tag{6.19}$$

Then f has the form

$$f(x) = g_1(x)g_2(x) \text{ with } g_1(x) := x^2 \text{ and } g_2(x) := \sin(x). \tag{6.20}$$

Moreover,

$$g_1'(x) = \frac{d}{dx}\left(x^2\right) = 2x \text{ and } g_2'(x) = \frac{d}{dx}\left(\sin x\right) = \cos(x). \tag{6.21}$$

Thus, by the product rule,

$$f'(x) = g_1'(x)g_2(x) + g_1(x)g_2'(x) = 2x\sin(x) + x^2\cos(x). \tag{6.22}$$

(3) Quotient rule

Let

$$f : \mathbb{R} \to \mathbb{R}, x \mapsto f(x) := \frac{x^2}{x+1}. \tag{6.23}$$

Then f has the form

$$f(x) = \frac{g_1(x)}{g_2(x)} \text{ with } g_1(x) := x^2 \text{ and } g_2(x) := x + 1. \tag{6.24}$$

Moreover,

$$g_1'(x) = \frac{d}{dx}\left(x^2\right) = 2x \text{ and } g_2'(x) = \frac{d}{dx}\left(1 + x\right) = 1. \tag{6.25}$$

Thus, by the quotient rule,

$$f'(x) = \frac{g_1'(x)g_2(x) - g_1(x)g_2'(x)}{g_2^2(x)} = \frac{2x(x+1) - x^2}{(x+1)^2} = \frac{2x^2 + 2x - x^2}{(x+1)^2} = \frac{x^2 + 2x}{(x+1)^2}. \tag{6.26}$$

(4) Chain rule

Let

$$f : \mathbb{R} \to \mathbb{R}, x \mapsto f(x) := \exp\left(-x^2\right). \tag{6.27}$$

Then f has the form

$$f(x) = g_1(g_2(x)) \text{ with } g_1(x) := \exp(x) \text{ and } g_2(x) := -x^2. \tag{6.28}$$

Moreover,

$$g_1'(x) = \frac{d}{dx}\left(\exp(x)\right) = \exp(x) \text{ and } g_2'(x) = \frac{d}{dx}\left(-x^2\right) = -2x. \tag{6.29}$$

Thus, by the chain rule,

$$f'(x) = g_1'(g_2(x))g_2'(x) = \exp\left(-x^2\right)(-2x) = -2x\exp\left(-x^2\right). \tag{6.30}$$

One can therefore remember the derivative resulting from the chain rule as: "The derivative of the outer function at the value of the inner function times the derivative of the inner function."

6.2 Analytic optimization

An important application of differential calculus is the determination of extrema of functions. At its core, this concerns the question for which values in its domain a function assumes a maximum or a minimum. For simple functions this is possible analytically. The general procedure is often also known under the keyword "curve sketching". In applications, an analytic approach to optimizing functions is usually not possible, and numerical algorithms are used to determine extrema. Understanding these algorithms, however, presupposes an understanding of the principles of analytic optimization. In this section, we give an introduction to analytic optimization of univariate real-valued functions. We begin by making precise the concepts of maxima and minima of univariate real-valued functions mentioned above.

Definition 6.4 (Extreme points and extreme values)**.** Let $U \subseteq \mathbb{R}$ and let $f : U \to \mathbb{R}$ be a univariate real-valued function. f has at the point $x_0 \in U$

- a *local minimum* if there is an interval $I :=]a, b[$ with $x_0 \in]a, b[$ and

$$f(x_0) \leq f(x) \text{ for all } x \in I \cap U, \tag{6.31}$$

- a *global minimum* if

$$f(x_0) \leq f(x) \text{ for all } x \in U, \tag{6.32}$$

- a *local maximum* if there is an interval $I :=]a, b[$ with $x_0 \in]a, b[$ and

$$f(x_0) \geq f(x) \text{ for all } x \in I \cap U, \tag{6.33}$$

- a *global maximum* if

$$f(x_0) \geq f(x) \text{ for all } x \in U. \tag{6.34}$$

The value $x_0 \in U$ of the domain of f is called, respectively, a *local* or *global minimizer* or *maximizer*, and the function value $f(x_0) \in \mathbb{R}$ is called, respectively, a *local* or *global minimum* or *maximum*. In general, the value $x_0 \in U$ is called an *extreme point*, and the function value $f(x_0) \in \mathbb{R}$ is called an *extreme value*.

•

Extreme points of functions are often denoted by

$$\underset{x \in I \cap U}{\operatorname{argmin}} f(x) \text{ or } \underset{x \in I \cap U}{\operatorname{argmax}} f(x), \tag{6.35}$$

and extreme values of functions are often denoted by

$$\min_{x \in I \cap U} f(x) \text{ or } \max_{x \in I \cap U} f(x). \tag{6.36}$$

Figure 6.4 visualizes the different types of extreme values according to Definition 6.4.

Analytic optimization of univariate real-valued functions is based on the so-called *necessary* and *sufficient conditions for extrema*. The former makes a statement about the behavior of the first derivative of a function at an extreme point; the latter makes a statement about the behavior of a function at a point that satisfies certain requirements on its first and second derivatives.

Theorem 6.2 (Necessary condition for extrema)**.** *Let f be a univariate real-valued function that is differentiable in a neighborhood of x_0. If x_0 is an interior extreme point of f, then*

$$x_0 \textit{ is an interior extreme point of } f \Rightarrow f'(x_0) = 0. \tag{6.37}$$

∘

If x_0 is an interior extreme point of f, then the first derivative of f at x_0 is therefore equal to zero. Instead of giving a proof, let us reason that, for example, at a local maximizer x_0 of f, f increases to the left of x_0 and decreases to the right of x_0. At x_0, however, f neither increases nor decreases, so it is plausible that $f'(x_0) = 0$.

Theorem 6.3 (Sufficient conditions for local extrema)**.** *Let f be a twice differentiable univariate real-valued function.*

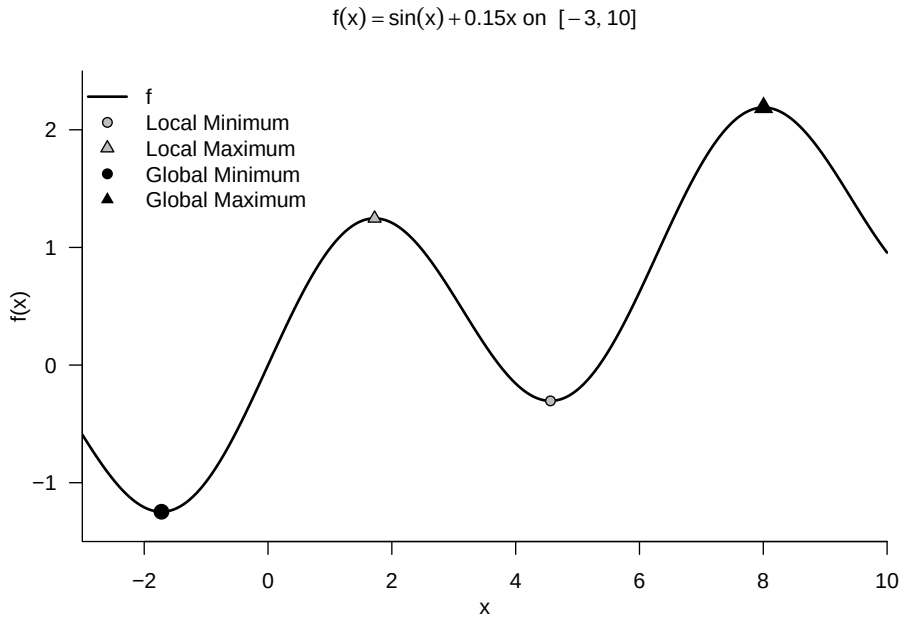


Figure 6.4 Extreme points of a function.

- *If, for* $x_0 \in U \subseteq \mathbb{R}$,
$$f'(x_0) = 0 \text{ and } f''(x_0) > 0 \tag{6.38}$$
holds, then f *has a local minimum at the point* x_0.
- *If, for* $x_0 \in U \subseteq \mathbb{R}$,
$$f'(x_0) = 0 \text{ and } f''(x_0) < 0 \tag{6.39}$$
holds, then f *has a local maximum at the point* x_0.

∘

Again we omit a proof and illustrate the condition with the example shown in Figure 6.5. Here, clearly, $x_0 = 1$ is a local minimizer of $f(x) = (x-1)^2$. We can see that to the left of x_0, f decreases, and to the right of x_0, f increases. At x_0, f neither increases nor decreases, so $f'(x_0) = 0$. Moreover, to the left and right of x_0 and at x_0, the change f'' of f' is positive: to the left of x_0, the negativity of f' weakens to 0, and to the right of x_0, the positivity of f' increases.

In particular, the sufficient conditions for local extrema suggest the following *standard procedure* for determining local extreme points.

Theorem 6.4 (Standard procedure of analytic optimization)**.** *Let* f *be a univariate real-valued function. Local extreme points of* f *can be identified by the following* standard procedure of analytic optimization*:*

1. *Compute the first and second derivatives of* f.
2. *Determine zeros* x^* *of* f' *by solving* $f'(x^*) = 0$ *for* x^*. *The zeros of* f' *are then candidates for extreme points of* f.

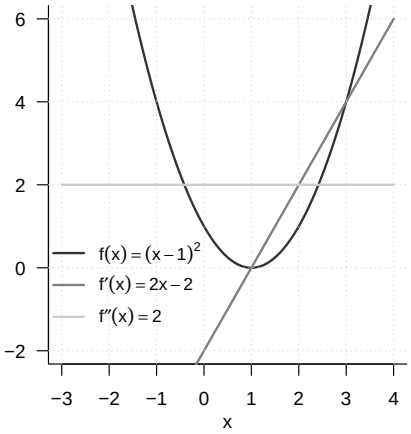


Figure 6.5 Analytic optimization of $f(x) := (x-1)^2$.

3. *Evaluate $f''(x^*)$: If $f''(x^*) > 0$, then x^* is a local minimizer of f; if $f''(x^*) < 0$, then x^* is a local maximizer of f; if $f''(x^*) = 0$, then this criterion does not allow a decision.*

∘

Example

Instead of giving a proof, we consider the function

$$f : \mathbb{R} \to \mathbb{R}, x \mapsto f(x) := (x-1)^2 \tag{6.40}$$

as an example. The first derivative of f from Figure 6.5 is obtained by the chain rule as

$$f'(x) = \frac{d}{dx}\left((x-1)^2\right) = 2(x-1) \cdot \frac{d}{dx}(x-1) = 2x - 2. \tag{6.41}$$

The second derivative of f is

$$f''(x) = \frac{d}{dx}f'(x) = \frac{d}{dx}(2x-2) = 2 > 0 \text{ for all } x \in \mathbb{R}. \tag{6.42}$$

Solving $f'(x^*) = 0$ for x^* yields

$$f'(x^*) = 0 \Leftrightarrow 2x^* - 2 = 0 \Leftrightarrow 2x^* = 2 \Leftrightarrow x^* = 1. \tag{6.43}$$

Consequently, $x^* = 1$ is a minimizer of f with associated minimum value $f(1) = 0$.

6.3 Differential calculus of multivariate real-valued functions

We first recall the concept of a multivariate real-valued function.

Definition 6.5 (Multivariate real-valued function)**.** A function of the form

$$f : \mathbb{R}^n \to \mathbb{R}, x \mapsto f(x) = f(x_1, ..., x_n) \tag{6.44}$$

is called a *multivariate real-valued* function.

•

The arguments of multivariate real-valued functions are thus real n-tuples of the form $x := (x_1, ..., x_n)$, whereas their function values are real numbers. An example of a multivariate real-valued function with $n := 2$ is

$$f : \mathbb{R}^2 \to \mathbb{R}, x \mapsto f(x) := x_1^2 + x_2^2. \tag{6.45}$$

We visualize this function in Figure 6.6. The right-hand panel shows a representation by means of so-called *isocontours*, that is, lines in the domain of the function for which the function assumes identical values. The corresponding values are marked for selected isocontours in the figure.

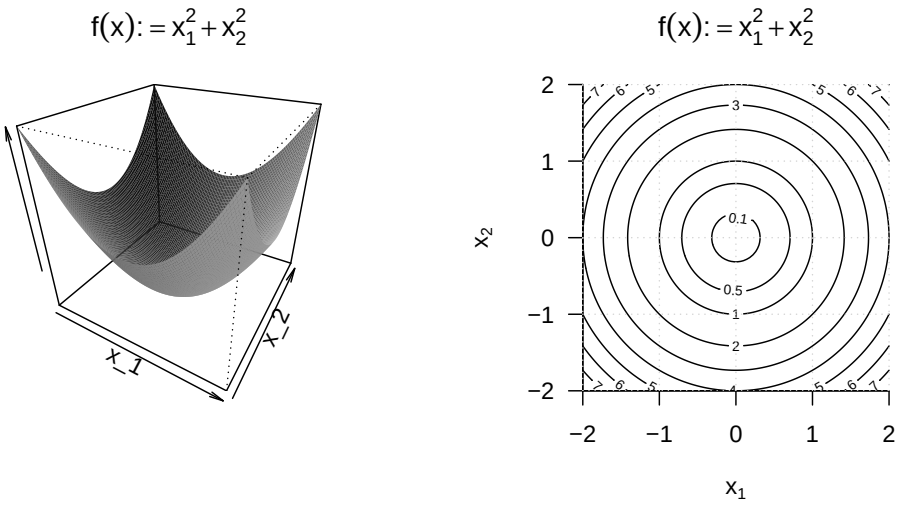


Figure 6.6 Visualizations of a bivariate function. The left-hand panel visualizes the function values as a surface above the two-dimensional domain of the function. Although this representation has a certain three-dimensional character, it is a bivariate function with a correspondingly two-dimensional domain. The right-hand panel visualizes lines with equal function values in the two-dimensional domain of the function.

We now begin to extend the concepts of differentiability and the derivative of univariate real-valued functions to the case of multivariate real-valued functions. To this end, we first introduce the concepts of *partial differentiability* and the *partial derivative*.

Definition 6.6 (Partial differentiability and partial derivative)**.** Let $D \subseteq \mathbb{R}^n$ be a set and let

$$f : D \to \mathbb{R}, x \mapsto f(x) \tag{6.46}$$

be a multivariate real-valued function. f is called *partially differentiable with respect to* x_i at $a \in D$ if the limit

$$\frac{\partial}{\partial x_i} f(a) := \lim_{h \to 0} \frac{f(a + he_i) - f(a)}{h} \tag{6.47}$$

exists. $\frac{\partial}{\partial x_i} f(a)$ is then called the *partial derivative of f with respect to x_i at the point a*. If f is partially differentiable with respect to x_i for all $x \in D$, then f is called *partially differentiable with respect to x_i*, and the function

$$\frac{\partial}{\partial x_i} f : D \to \mathbb{R}, x \mapsto \frac{\partial}{\partial x_i} f(x) \tag{6.48}$$

is called the *partial derivative of f with respect to x_i*. f is called *partially differentiable at $x \in D$* if f is partially differentiable with respect to x_i at $x \in D$ for all $i = 1, ..., n$, and f is called *partially differentiable* if f is partially differentiable with respect to x_i at all $x \in D$ for all $i = 1, ..., n$.

•

In Definition 6.6, $e_i \in \mathbb{R}^n$ denotes the ith canonical unit vector, for which $(e_i)_j = 1$ for $i = j$ and $(e_i)_j = 0$ for $i \neq j$ with $j = 1, ..., n$. In analogy and generalization to the Newton difference quotient, the difference quotient occurring here,

$$\frac{f(x + he_i) - f(x)}{h}, \tag{6.49}$$

therefore measures the change $f(x + he_i) - f(x)$ of f per distance h in the direction e_i. We visualize the components of this quotient for the case of a bivariate function in Figure 6.7.

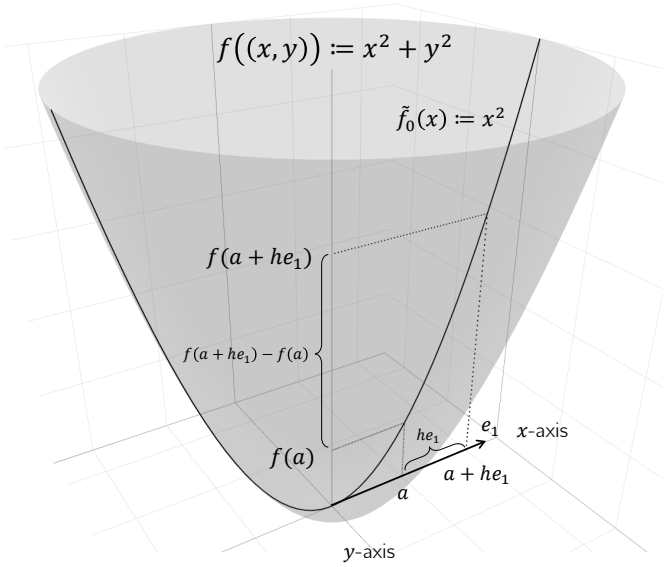


Figure 6.7 Components of the partial Newton difference quotient.

For $h \to 0$, the difference quotient correspondingly measures the *rate of change* of f at x in the direction e_i. As in the discussion of derivatives, $\frac{\partial}{\partial x_i} f(x)$ is a number, whereas $\frac{\partial}{\partial x_i} f$ is a function. In practice, one computes $\frac{\partial}{\partial x_i} f$ as the (ordinary) derivative

$$\frac{d}{dx_i} \tilde{f}_{x_1,...x_{i-1},x_{i+1},...,x_n}(x_i) \tag{6.50}$$

of the univariate real-valued function

$$\tilde{f} : \mathbb{R} \to \mathbb{R}, x_i \mapsto \tilde{f}_{x_1,...x_{i-1},x_{i+1},...,x_n}(x_i) := f(x_1, ..., x_i, ..., x_n). \tag{6.51}$$

Thus, for the ith partial derivative, all x_j with $j \neq i$ are regarded as constants, and one is led back to the familiar computation of derivatives of univariate real-valued functions. We illustrate the procedure for computing partial derivatives with a first example.

Example (1)

We consider the function

$$f : \mathbb{R}^2 \to \mathbb{R}, x \mapsto f(x) := x_1^2 + x_2^2. \tag{6.52}$$

Because the domain of this function is two-dimensional, two partial derivatives can be computed:

$$\frac{\partial}{\partial x_1} f : \mathbb{R}^2 \to \mathbb{R}, x \mapsto \frac{\partial}{\partial x_1} f(x) \text{ and } \frac{\partial}{\partial x_2} f : \mathbb{R}^2 \to \mathbb{R}, x \mapsto \frac{\partial}{\partial x_2} f(x). \tag{6.53}$$

To compute the first of these partial derivatives, one considers the function

$$f_{x_2} : \mathbb{R} \to \mathbb{R}, x_1 \mapsto f_{x_2}(x_1) := x_1^2 + x_2^2, \tag{6.54}$$

where x_2 takes the role of a constant. To make explicit that x_2 is not an argument of the function, while the function still depends on x_2, we have used the subscript notation $f_{x_2}(x_1)$. To compute the partial derivative, we now compute the (ordinary) derivative of f_{x_2},

$$f'_{x_2}(x_1) = 2x_1. \tag{6.55}$$

Thus,

$$\frac{\partial}{\partial x_1} f : \mathbb{R}^2 \to \mathbb{R}, x \mapsto \frac{\partial}{\partial x_1} f(x) = \frac{\partial}{\partial x_1}(x_1^2 + x_2^2) = f'_{x_2}(x_1) = 2x_1. \tag{6.56}$$

Analogously, with the corresponding formulation of f_{x_1},

$$\frac{\partial}{\partial x_2} f : \mathbb{R}^2 \to \mathbb{R}, x \mapsto \frac{\partial}{\partial x_2} f(x) = \frac{\partial}{\partial x_2}(x_1^2 + x_2^2) = f'_{x_1}(x_2) = 2x_2. \tag{6.57}$$

As for the derivative of a univariate real-valued function, it is also possible for a multivariate real-valued function to define a higher derivative recursively.

Definition 6.7 (Second partial derivatives)**.** Let $f : \mathbb{R}^n \to \mathbb{R}$ be a multivariate real-valued function, and let $\frac{\partial}{\partial x_i} f$ be the partial derivative of f with respect to x_i. Then the second partial derivative of f with respect to x_i and x_j is defined as

$$\frac{\partial^2}{\partial x_j \partial x_i} f(x) := \frac{\partial}{\partial x_j} \left(\frac{\partial}{\partial x_i} f \right). \tag{6.58}$$

•

Note that for each partial derivative $\frac{\partial}{\partial x_i} f$ for $i = 1, ..., n$, there are in total n second partial derivatives $\frac{\partial^2}{\partial x_j \partial x_i} f$ for $j = 1, ..., n$. The resulting n^2 second partial derivatives, however, are not all distinct. This is an essential statement of *Schwarz's theorem*.

Theorem 6.5 (Schwarz's theorem)**.** *Let $f : \mathbb{R}^n \to \mathbb{R}$ be a partially differentiable multivariate real-valued function. Then*

$$\frac{\partial^2}{\partial x_j \partial x_i} f(x) = \frac{\partial^2}{\partial x_i \partial x_j} f(x) \text{ for all } 1 \leq i, j \leq n. \tag{6.59}$$

∘

For a proof, we refer to the advanced literature. Schwarz's theorem states in particular that, when forming second partial derivatives, the order of partial differentiation is irrelevant. In this way, the theorem facilitates the calculation of second partial derivatives and also helps detect analytic errors when computing them. We illustrate this by continuing the example above.

Example (1)

We want to compute the second-order partial derivatives of the function

$$f : \mathbb{R}^2 \to \mathbb{R}, x \mapsto f(x) := x_1^2 + x_2^2 \tag{6.60}$$

With the results for the first-order partial derivatives of this function, we obtain

$$\begin{aligned}
\frac{\partial^2}{\partial x_1 \partial x_1} f(x) &= \frac{\partial}{\partial x_1}\left(\frac{\partial}{\partial x_1} f(x)\right) = \frac{\partial}{\partial x_1}(2x_1) = 2 \\
\frac{\partial^2}{\partial x_1 \partial x_2} f(x) &= \frac{\partial}{\partial x_1}\left(\frac{\partial}{\partial x_2} f(x)\right) = \frac{\partial}{\partial x_1}(2x_2) = 0 \\
\frac{\partial^2}{\partial x_2 \partial x_1} f(x) &= \frac{\partial}{\partial x_2}\left(\frac{\partial}{\partial x_1} f(x)\right) = \frac{\partial}{\partial x_2}(2x_1) = 0 \\
\frac{\partial^2}{\partial x_2 \partial x_2} f(x) &= \frac{\partial}{\partial x_2}\left(\frac{\partial}{\partial x_2} f(x)\right) = \frac{\partial}{\partial x_2}(2x_2) = 2.
\end{aligned} \tag{6.61}$$

Obviously,

$$\frac{\partial^2}{\partial x_1 x_2} f(x) = \frac{\partial^2}{\partial x_2 x_1} f(x). \tag{6.62}$$

Example (2)

As a further example, we want to compute the first- and second-order partial derivatives of the function

$$f : \mathbb{R}^3 \to \mathbb{R}, x \mapsto f(x) := x_1^2 + x_1 x_2 + x_2 \sqrt{x_3} \tag{6.63}$$

With the calculation rules for derivatives, the first-order partial derivatives are

$$\begin{aligned}
\frac{\partial}{\partial x_1} f(x) &= \frac{\partial}{\partial x_1}\left(x_1^2 + x_1 x_2 + x_2 \sqrt{x_3}\right) = 2x_1 + x_2, \\
\frac{\partial}{\partial x_2} f(x) &= \frac{\partial}{\partial x_2}\left(x_1^2 + x_1 x_2 + x_2 \sqrt{x_3}\right) = x_1 + \sqrt{x_3}, \\
\frac{\partial}{\partial x_3} f(x) &= \frac{\partial}{\partial x_3}\left(x_1^2 + x_1 x_2 + x_2 \sqrt{x_3}\right) = \frac{x_2}{2\sqrt{x_3}}.
\end{aligned} \tag{6.64}$$

For the second partial derivatives with respect to x_1, we obtain

$$\begin{aligned}
\frac{\partial^2}{\partial x_1 \partial x_1} f(x) &= \frac{\partial}{\partial x_1}\left(\frac{\partial}{\partial x_1} f(x)\right) = \frac{\partial}{\partial x_1}\left(2x_1 + x_2\right) = 2, \\
\frac{\partial^2}{\partial x_2 \partial x_1} f(x) &= \frac{\partial}{\partial x_2}\left(\frac{\partial}{\partial x_1} f(x)\right) = \frac{\partial}{\partial x_2}\left(2x_1 + x_2\right) = 1, \\
\frac{\partial^2}{\partial x_3 \partial x_1} f(x) &= \frac{\partial}{\partial x_3}\left(\frac{\partial}{\partial x_1} f(x)\right) = \frac{\partial}{\partial x_3}\left(2x_1 + x_2\right) = 0.
\end{aligned} \tag{6.65}$$

For the second partial derivatives with respect to x_2, we obtain

$$\begin{aligned}
\frac{\partial^2}{\partial x_1 \partial x_2} f(x) &= \frac{\partial}{\partial x_1}\left(\frac{\partial}{\partial x_2} f(x)\right) = \frac{\partial}{\partial x_1}\left(x_1 + \sqrt{x_3}\right) = 1, \\
\frac{\partial^2}{\partial x_2 \partial x_2} f(x) &= \frac{\partial}{\partial x_2}\left(\frac{\partial}{\partial x_2} f(x)\right) = \frac{\partial}{\partial x_2}\left(x_1 + \sqrt{x_3}\right) = 0, \\
\frac{\partial^2}{\partial x_3 \partial x_2} f(x) &= \frac{\partial}{\partial x_3}\left(\frac{\partial}{\partial x_2} f(x)\right) = \frac{\partial}{\partial x_3}\left(x_1 + \sqrt{x_3}\right) = \frac{1}{2\sqrt{x_3}}.
\end{aligned} \tag{6.66}$$

For the second partial derivatives with respect to x_3, we obtain

$$\begin{aligned}
\frac{\partial^2}{\partial x_1 \partial x_3} f(x) &= \frac{\partial}{\partial x_1}\left(\frac{\partial}{\partial x_3} f(x)\right) = \frac{\partial}{\partial x_1}\left(\frac{x_2}{2\sqrt{x_3}}\right) = 0, \\
\frac{\partial^2}{\partial x_2 \partial x_3} f(x) &= \frac{\partial}{\partial x_2}\left(\frac{\partial}{\partial x_3} f(x)\right) = \frac{\partial}{\partial x_2}\left(\frac{x_2}{2\sqrt{x_3}}\right) = \frac{1}{2\sqrt{x_3}}, \\
\frac{\partial^2}{\partial x_3 \partial x_3} f(x) &= \frac{\partial}{\partial x_3}\left(\frac{\partial}{\partial x_3} f(x)\right) = \frac{\partial}{\partial x_3}\left(x_2 \frac{1}{2} x_3^{-\frac{1}{2}}\right) = -\frac{1}{4} x_2 x_3^{-\frac{3}{2}}.
\end{aligned} \tag{6.67}$$

Furthermore, one sees that the order of partial differentiation is irrelevant, because

$$\begin{aligned}
\frac{\partial^2}{\partial x_1 \partial x_2} f(x) &= \frac{\partial^2}{\partial x_2 \partial x_1} f(x) = 1, \\
\frac{\partial^2}{\partial x_1 \partial x_3} f(x) &= \frac{\partial^2}{\partial x_3 \partial x_1} f(x) = 0, \\
\frac{\partial^2}{\partial x_2 \partial x_3} f(x) &= \frac{\partial^2}{\partial x_3 \partial x_2} f(x) = \frac{1}{2\sqrt{x_3}}.
\end{aligned} \tag{6.68}$$

As seen above, for a multivariate real-valued function $f : \mathbb{R}^n \to \mathbb{R}$ there are in total n first partial derivatives and n^2 second partial derivatives. These are collected in the *gradient* and the *Hessian matrix* of a multivariate real-valued function.

Definition 6.8 (Gradient)**.** Let $f : \mathbb{R}^n \to \mathbb{R}$ be a multivariate real-valued function. The *gradient* $\nabla f(x)$ of f at the point $x \in \mathbb{R}^n$ is defined as

$$\nabla f(x) := \begin{pmatrix} \frac{\partial}{\partial x_1} f(x) \\ \frac{\partial}{\partial x_2} f(x) \\ \vdots \\ \frac{\partial}{\partial x_n} f(x) \end{pmatrix} \in \mathbb{R}^n. \tag{6.69}$$

•

Note that gradients are multivariate vector-valued functions of the form

$$\nabla f : \mathbb{R}^n \to \mathbb{R}^n, x \mapsto \nabla f(x). \tag{6.70}$$

For $n = 1$, $\nabla f(x) = f'(x)$. An important property of the gradient is that $-\nabla f(x)$ indicates the direction of steepest descent of f in $\mathbb{R}^n$. This insight is not trivial, however, and will be deepened at a later point. As examples, we consider the gradients of the functions analyzed above.

Example (1)

For the function $f : \mathbb{R}^2 \to \mathbb{R}$ considered in Equation 6.60,

$$\nabla f(x) := \begin{pmatrix} \frac{\partial}{\partial x_1} f(x) \\ \frac{\partial}{\partial x_2} f(x) \end{pmatrix} = \begin{pmatrix} 2x_1 \\ 2x_2 \end{pmatrix} \in \mathbb{R}^2. \tag{6.71}$$

In Figure 6.8 we visualize color-coded selected values of this gradient for $(0.7, 0.7)^T$, $(-0.3, 0.1)^T$, $(-0.5, -0.4)^T$, and $(0.1, -1.0)^T$.

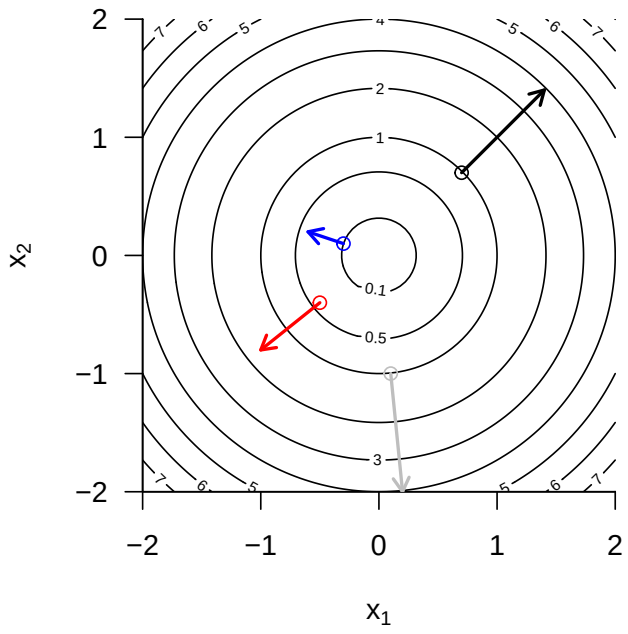


Figure 6.8 Example gradient values of the bivariate function $f(x) = x_1^2 + x_2^2$.

Example (2)

For the function $f : \mathbb{R}^3 \to \mathbb{R}$ considered in Equation 6.63,

$$\nabla f(x) := \begin{pmatrix} \frac{\partial}{\partial x_1} f(x) \\ \frac{\partial}{\partial x_2} f(x) \\ \frac{\partial}{\partial x_3} f(x) \end{pmatrix} = \begin{pmatrix} 2x_1 + x_2 \\ x_1 + \sqrt{x_3} \\ \frac{x_2}{2\sqrt{x_3}} \end{pmatrix} \in \mathbb{R}^3. \tag{6.72}$$

Finally, we turn to collecting the second partial derivatives of a multivariate real-valued function in the *Hessian matrix*.

Definition 6.9 (Hessian matrix)**.** Let $f : \mathbb{R}^n \to \mathbb{R}$ be a multivariate real-valued function. The *Hessian matrix* $\nabla^2 f(x)$ of f at the point $x \in \mathbb{R}^n$ is defined as

$$\nabla^2 f(x) := \begin{pmatrix} \frac{\partial^2}{\partial x_1 \partial x_1} f(x) & \frac{\partial^2}{\partial x_1 \partial x_2} f(x) & \cdots & \frac{\partial^2}{\partial x_1 \partial x_n} f(x) \\ \frac{\partial^2}{\partial x_2 \partial x_1} f(x) & \frac{\partial^2}{\partial x_2 \partial x_2} f(x) & \cdots & \frac{\partial^2}{\partial x_2 \partial x_n} f(x) \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2}{\partial x_n \partial x_1} f(x) & \frac{\partial^2}{\partial x_n \partial x_2} f(x) & \cdots & \frac{\partial^2}{\partial x_n \partial x_n} f(x) \end{pmatrix} \in \mathbb{R}^{n \times n}. \tag{6.73}$$

•

Note that Hessian matrices are multivariate matrix-valued mappings of the form

$$\nabla^2 f : \mathbb{R}^n \to \mathbb{R}^{n \times n}, x \mapsto \nabla^2 f(x). \tag{6.74}$$

For $n = 1$, $\nabla^2 f(x) = f''(x)$. Furthermore, from

$$\frac{\partial^2}{\partial x_i \partial x_j} f(x) = \frac{\partial^2}{\partial x_j \partial x_i} f(x) \text{ for } 1 \leq i, j \leq n \tag{6.75}$$

it follows that the Hessian matrix is symmetric, that is,

$$\left(\nabla^2 f(x)\right)^T = \nabla^2 f(x). \tag{6.76}$$

Example (1)

For the function $f : \mathbb{R}^2 \to \mathbb{R}$ considered in Equation 6.60,

$$\nabla^2 f(x) := \begin{pmatrix} \frac{\partial^2}{\partial x_1 x_1} f(x) & \frac{\partial^2}{\partial x_1 x_2} f(x) \\ \frac{\partial^2}{\partial x_2 x_1} f(x) & \frac{\partial^2}{\partial x_2 x_2} f(x) \end{pmatrix} = \begin{pmatrix} 2 & 0 \\ 0 & 2 \end{pmatrix} \in \mathbb{R}^{2 \times 2}. \tag{6.77}$$

The Hessian matrix of this function is therefore a constant function that does not depend on x.

Example (2)

For the function $f : \mathbb{R}^3 \to \mathbb{R}$ considered in Equation 6.63,

$$\nabla^2 f(x) := \begin{pmatrix} \frac{\partial^2}{\partial x_1 x_1} f(x) & \frac{\partial^2}{\partial x_1 x_2} f(x) & \frac{\partial^2}{\partial x_1 x_3} f(x) \\ \frac{\partial^2}{\partial x_2 x_1} f(x) & \frac{\partial^2}{\partial x_2 x_2} f(x) & \frac{\partial^2}{\partial x_2 x_3} f(x) \\ \frac{\partial^2}{\partial x_3 x_1} f(x) & \frac{\partial^2}{\partial x_3 x_2} f(x) & \frac{\partial^2}{\partial x_3 x_3} f(x) \end{pmatrix} = \begin{pmatrix} 2 & 1 & 0 \\ 1 & 0 & \frac{1}{2\sqrt{x_3}} \\ 0 & \frac{1}{2\sqrt{x_3}} & -\frac{1}{4} x_2 x_3^{-3/2} \end{pmatrix}. \tag{6.78}$$

In contrast to Example (1), the Hessian matrix of the function considered here is not a constant function, and its value depends on the value of the function argument $x \in \mathbb{R}^3$.

7 Integral calculus

This chapter gives an overview of central concepts of integral calculus. The main focus throughout is on clarifying terminology, mathematical symbolism, and the intuition conveyed by it, rather than on the concrete computation of integrals.

7.1 Indefinite integrals

We begin with the definition of the indefinite integral and the concept of an antiderivative.

Definition 7.1 (Indefinite integral and antiderivative)**.** Let $I \subseteq \mathbb{R}$ be an interval and let $f : I \to \mathbb{R}$ be a univariate real-valued function. Then a differentiable function $F : I \to \mathbb{R}$ with the property

$$F' = f \tag{7.1}$$

is called an *antiderivative of* f. If F is an antiderivative of f, then

$$\int f(x)\,dx := F + c \text{ with } c \in \mathbb{R} \tag{7.2}$$

is called the *indefinite integral of the function* f. The indefinite integral of a function thus denotes the set of all antiderivatives of a function.

•

The above definition states that the derivative of an antiderivative of a function f is precisely f. In addition, the indefinite integral of a function f is the set of *all* antiderivatives of f obtained by adding different constants $c \in \mathbb{R}$. Such a constant $c \in \mathbb{R}$ is also called an *integration constant*; of course, $\frac{d}{dx}c = 0$. The symbol $\int f(x)\,dx$ is defined as $F + c$. In this expression, $f(x)$ is called the *integrand*. $\int$ and dx have no meaning of their own; they are merely symbols.

For the elementary functions introduced in the previous sections, the antiderivatives listed in Table 7.1 result. This can be verified by differentiating the respective antiderivative with the calculation rules of differential calculus. The indefinite integrals of these elementary functions then follow directly from these antiderivatives by adding an integration constant.

Table 7.1 Antiderivatives of elementary functions

Name	Definition	Antiderivative
Polynomial function	$f(x) := \sum_{i=0}^{k} a_i x^i$	$F(x) = \sum_{i=0}^{k} \frac{a_i}{i+1} x^{i+1}$
Constant function	$f(x) := a$	$F(x) = ax$
Identity function	$f(x) := x$	$F(x) = \frac{1}{2}x^2$

Table 7.1 Antiderivatives of elementary functions

Name	Definition	Antiderivative
Linear-affine function	$f(x) := ax + b$	$F(x) = \frac{1}{2}ax^2 + bx$
Square function	$f(x) := x^2$	$F(x) = \frac{1}{3}x^3$
Exponential function	$f(x) := \exp(x)$	$F(x) = \exp(x)$
Logarithm function	$f(x) := \ln(x)$	$F(x) = x \ln x - x$

The calculation rules collected in the following theorem are often helpful for determining antiderivatives of functions that are composed of functions with known antiderivatives.

Theorem 7.1 (Calculation rules for antiderivatives)**.** *Let f and g be univariate real-valued functions that possess antiderivatives, and let g be invertible. Then the following calculation rules hold for determining antiderivatives.*

(1) Sum rule

$$\int af(x) + bg(x)\,dx = a\int f(x)\,dx + b\int g(x)\,dx \text{ for } a, b \in \mathbb{R} \tag{7.3}$$

(2) Integration by parts

$$\int f'(x)g(x)\,dx = f(x)g(x) - \int f(x)g'(x)\,dx \tag{7.4}$$

(3) Substitution rule

$$\int f(g(x))g'(x)\,dx = \int f(t)\,dt \text{ with } t = g(x) \tag{7.5}$$

∘

Proof. For a proof of the sum rule, we refer to the advanced literature. The calculation rule for integration by parts follows by integrating the product rule of differentiation. Recall that

$$(f(x)g(x))' = f'(x)g(x) + f(x)g'(x). \tag{7.6}$$

Integrating both sides of the equation and taking into account the sum rule for antiderivatives then yields

$$\begin{aligned} \textstyle\int (f(x)g(x))'\,dx &= \textstyle\int f'(x)g(x) + f(x)g'(x)\,dx \\ \Leftrightarrow f(x)g(x) &= \textstyle\int f'(x)g(x)\,dx + \int f(x)g'(x)\,dx \\ \Leftrightarrow \textstyle\int f'(x)g(x)\,dx &= f(x)g(x) - \textstyle\int f(x)g'(x)\,dx. \end{aligned} \tag{7.7}$$

For $F' = f$, the substitution rule follows by applying the chain rule of differentiation to the composite function $F(g)$. Specifically, first

$$(F(g(x)))' = F'(g(x))g'(x) = f(g(x))g'(x). \tag{7.8}$$

Integrating both sides of the equation

$$(F(g(x)))' = f(g(x))g'(x) \tag{7.9}$$

then yields

$$\begin{aligned} \textstyle\int (F(g(x)))'\,dx &= \textstyle\int f(g(x))g'(x)\,dx \\ \Leftrightarrow F(g(x)) + c &= \textstyle\int f(g(x))g'(x)\,dx \\ \Leftrightarrow \textstyle\int f(g(x))g'(x)\,dx &= \textstyle\int f(t)\,dt \text{ with } t := g(x). \end{aligned} \tag{7.10}$$

Here, the right-hand side of the last equation above is to be understood as $F(g(x)) + c$, that is, as the antiderivative of f evaluated at the point $t := g(x)$. The dt is not to be replaced by $dg(x)$; it is purely notational in nature. □

Indefinite integrals occupy a central place in the solution of differential equations. More immediate, however, is the use of indefinite integrals in the context of evaluating *definite integrals*, as introduced in the next section.

7.2 Definite integrals

Intuitively, a definite integral corresponds to the signed area between the graph of a function f and the x-axis, restricted to an interval $[a, b]$ (cf. Figure 7.1). *Signed* means that areas between the x-axis and positive values of f contribute positively to the area, whereas areas between the x-axis and negative values of f contribute negatively. For example, the value of the definite integral shown in Figure 7.1 A is 0.68, the value of the definite integral shown in Figure 7.1 B is 0.95 (the shaded area is clearly larger than in Figure 7.1 A), and the value of the definite integral shown in Figure 7.1 C is 0 (the shaded positive and negative areas cancel exactly). The latter example also suggests interpreting the integral as the average value of a function f over an interval $[a, b]$.

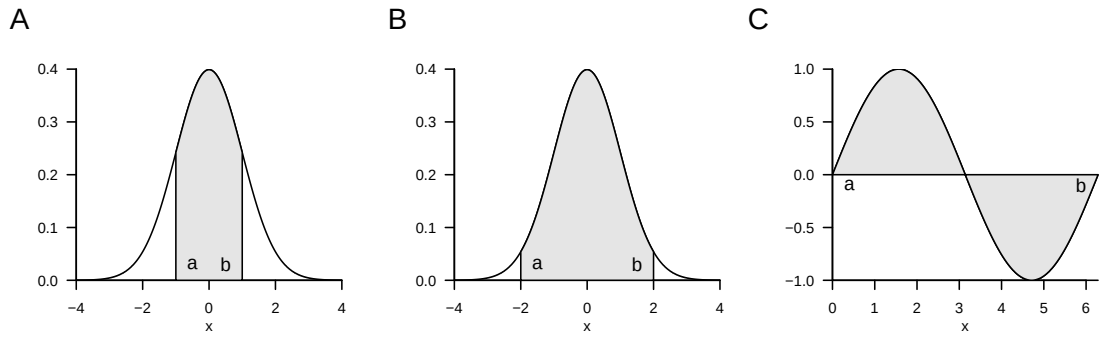


Figure 7.1 Examples of definite integrals

To introduce the concept of the *definite integral* in the sense of the *Riemann integral*, we first need to do some preliminary work. We begin by introducing a term for the subdivision of an interval into smaller sections.

Definition 7.2 (Partition of an interval and mesh)**.** Let $[a, b] \subset \mathbb{R}$ be an interval and let $x_0, x_1, x_2, ..., x_n \in [a, b]$ be a set of points with

$$a =: x_0 < x_1 < x_2 < \cdots < x_n := b \tag{7.11}$$

and

$$\Delta x_i := x_i - x_{i-1} \text{ for } i = 1, ..., n. \tag{7.12}$$

Then the set

$$Z := \{[x_0, x_1], [x_1, x_2], ..., [x_{n-1}, x_n]\} \tag{7.13}$$

of subintervals of $[a, b]$ defined by $x_0, x_1, x_2, ..., x_n$ is called a *partition of* $[a, b]$. Furthermore,

$$Z_{\max} := \max_{i \in \mathbb{N}_n} \Delta x_i, \tag{7.14}$$

that is, the largest of the subinterval lengths Δx_i, is called the *mesh of* Z.

•

Intuitively, Δx_i is the width of the rectangles in Figure 7.2, as we will see in what follows. With the concepts of the partition of an interval, we can now introduce the concept of *Riemann sums*.

Definition 7.3 (Riemann sums)**.** Let $f : [a, b] \to \mathbb{R}$ be a bounded function on $[a, b]$, that is, $|f(x)| < c$ for $0 < c < \infty$ and all $x \in [a, b]$. Let Z be a partition of $[a, b]$ with subinterval lengths Δx_i for $i = 1, ..., n$. Furthermore, let ξ_i for $i = 1, ..., n$ be an arbitrary point in the subinterval $[x_{i-1}, x_i]$ of the partition Z. Then

$$R(Z) := \sum_{i=1}^{n} f(\xi_i)\Delta x_i \tag{7.15}$$

is called the *Riemann sum of f on $[a, b]$ with respect to the partition Z.*

•

If, for example, one chooses the maximum of f in each subinterval in the Riemann sum, the so-called *upper Riemann sum* results,

$$R_o(Z) := \sum_{i=1}^{n} \left(\max_{[x_{i-1}, x_i]} f(\xi_i) \right) \cdot \Delta x_i. \tag{7.16}$$

If, by contrast, one chooses the minimum of f in each subinterval, the so-called *lower Riemann sum* results,

$$R_u(Z) := \sum_{i=1}^{n} \left(\min_{[x_{i-1}, x_i]} f(\xi_i) \right) \cdot \Delta x_i. \tag{7.17}$$

Figure 7.2 illustrates the definition of these Riemann sums: the dark gray rectangles each have area $\Delta x_i \cdot \min_{[x_{i-1}, x_i]} f(\xi)$ and thus form the summands in the lower Riemann sum

$$R_u(Z) := \sum_{i=1}^{4} \left(\min_{[x_{i-1}, x_i]} f(\xi_i) \right) \cdot \Delta x_i. \tag{7.18}$$

The vertical combination of dark gray and light gray rectangles each has area $\Delta x_i \cdot \max_{[x_{i-1}, x_i]} f(\xi)$ and thus forms the summands in the upper Riemann sum

$$R_o(Z) := \sum_{i=1}^{4} \left(\max_{[x_{i-1}, x_i]} f(\xi_i) \right) \cdot \Delta x_i. \tag{7.19}$$

Now imagine letting Δx_i tend to zero for all $i = 1, ..., n$, thereby making the mesh of the partition Z smaller and smaller. The values of $\min_{[x_{i-1}, x_i]} f(\xi_i)$ and $\max_{[x_{i-1}, x_i]} f(\xi_i)$, and hence also the values of $R_u(Z)$ and $R_o(Z)$, will then approach each other more and more. This limiting process is used in the definition of the Riemann integral.

Definition 7.4 (Definite riemann integral)**.** Let $f : [a, b] \to \mathbb{R}$ be a bounded real-valued function on $[a, b]$. Furthermore, for Z_k with $k = 1, 2, 3...$, let there be a sequence of partitions of $[a, b]$ with corresponding mesh $Z_{\max,k}$. If, for every sequence of partitions $Z_1, Z_2, ...$ with $|Z_{\max,k}| \to 0$ as $k \to \infty$ and for arbitrarily chosen points ξ_{ki} with $i = 1, ..., n$ in the subinterval $[x_{k,i-1}, x_{k,i}]$ of the partition Z_k, the sequence of corresponding Riemann sums $R(Z_1), R(Z_2), ...$ tends to the same limit, then f is called *integrable* on $[a, b]$. The

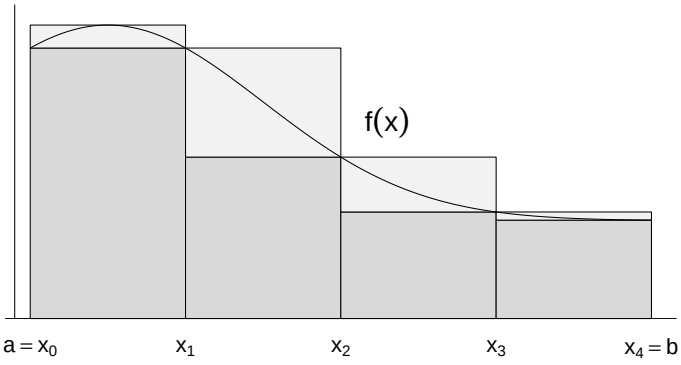


Figure 7.2 Riemann sums

corresponding limit of the sequence of Riemann sums is called the *definite Riemann integral* and denoted by

$$\int_a^b f(x)\,dx := \lim_{k\to\infty} R(Z_k) \text{ for } |Z_{\max,k}| \to 0. \tag{7.20}$$

In this context, the values a and b are called the *lower* and *upper* limits of integration, respectively, $f(x)$ is called the *integrand*, and x is called the *variable of integration*.

•

The Riemann integrability of a function and the value of a definite Riemann integral are thus defined in terms of a limiting process. However, the theory of Riemann integrals can be extended by the fundamental theorems of differential and integral calculus, so that the concrete computation of a definite integral rarely requires forming partitions and determining a limit. For simplicity, in what follows we omit the designation *Riemann* and simply speak of *definite integrals*.

A first step toward simplifying the computation of definite integrals is to record the following calculation rules, for whose proof we refer to the advanced literature.

Theorem 7.2 (Calculation rules for definite integrals)**.** *Let f and g be integrable functions on $[a, b]$. Then the following calculation rules hold.*

(1) Linearity. For $c_1, c_2 \in \mathbb{R}$,

$$\int_a^b (c_1 f(x) + c_2 g(x))\,dx = c_1 \int_a^b f(x)\,dx + c_2 \int_a^b g(x)\,dx. \tag{7.21}$$

(2) Additivity. For $a < c < b$,

$$\int_a^b f(x)\,dx = \int_a^c f(x)\,dx + \int_c^b f(x)\,dx. \tag{7.22}$$

(3) Change of sign when reversing the limits of integration

$$\int_a^b f(x)\,dx = -\int_b^a f(x)\,dx. \tag{7.23}$$

(4) Independence from the choice of the variable of integration

$$\int_a^b f(x)\,dx = \int_a^b f(y)\,dy. \tag{7.24}$$

(5) Independence of the integral from the type of interval

$$\int_a^b f(x)\,dx = \int_{]a,b[} f(x)\,dx = \int_{[a,b[} f(x)\,dx = \int_{]a,b]} f(x)\,dx = \int_{[a,b]} f(x)\,dx, \tag{7.25}$$

where $\int_I$ denotes the definite integral of f on the interval $I \subseteq \mathbb{R}$.

∘

Note that the linearity of the definite integral is analogous to associativity for sums of equal length and distributivity when multiplying by a constant (cf. Theorem 3.1),

$$\sum_{i=1}^{n} (c_1 x_i + c_2 y_i) = c_1 \sum_{i=1}^{n} x_i + c_2 \sum_{i=1}^{n} y_i,$$

that the additivity of definite integrals is analogous to splitting sums, and that the independence of the value of an integral from the choice of the variable of integration has its counterpart in the independence of the value of a sum from its summation index. A graphical representation of the additivity calculation rule for integrals is shown in Figure 7.3. The sum of the areas given by the definite integrals $\int_a^c f(x)\,dx$ and $\int_c^b f(x)\,dx$ with $a < c < b$ is the area of $\int_a^b f(x)\,dx$.

With the help of additivity and the zero-interval integral

$$\int_a^a f(x)\,dx = 0 \tag{7.26}$$

the change of sign when reversing the limits of integration, which will be important below, can be justified. To this end, as in Figure 7.3, let

$$\int_a^b f(x)\,dx = \int_a^c f(x)\,dx + \int_c^b f(x)\,dx. \text{ and } \int_b^a f(x)\,dx = \int_b^c f(x)\,dx + \int_c^a f(x)\,dx. \tag{7.27}$$

Adding both equations gives

$$\begin{aligned} \int_a^b f(x)\,dx + \int_b^a f(x)\,dx &= \int_a^c f(x)\,dx + \int_c^b f(x)\,dx + \int_b^c f(x)\,dx + \int_c^a f(x)\,dx \\ \Leftrightarrow \int_a^a f(x)\,dx &= \int_a^b f(x)\,dx + \int_b^a f(x)\,dx \\ \Leftrightarrow 0 &= \int_a^b f(x)\,dx + \int_b^a f(x)\,dx \\ \Leftrightarrow \int_a^b f(x)\,dx &= -\int_b^a f(x)\,dx. \end{aligned} \tag{7.28}$$

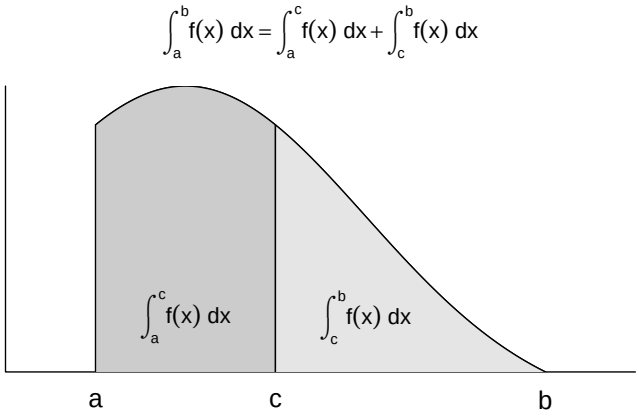


Figure 7.3 Additivity of definite integrals

The *fundamental theorems of differential and integral calculus* finally make it possible to compute definite integrals of a function f directly with the help of the antiderivative F of f. For the proof of the first fundamental theorem of differential and integral calculus, we need the *mean value theorem of integral calculus*, which we state here without proof and illustrate in Figure 7.4.

Theorem 7.3 (Mean value theorem of integral calculus)**.** *For a continuous function* $f : [a, b] \to \mathbb{R}$*, there exists a* $\xi \in]a, b[$ *such that*

$$\int_a^b f(x)\,dx = f(\xi)(b-a) \tag{7.29}$$

∘

The mean value theorem of integral calculus guarantees the existence of a $\xi \in [a, b]$ such that the definite integral $\int_a^b f(x)\,dx$ equals the product of the "rectangle height" $f(\xi)$ and the "rectangle width" $(b-a)$. In Figure 7.4, this ξ lies exactly halfway between a and b. That the resulting gray rectangle area equals $\int_a^b f(x)\,dx$ follows from the visually at least plausible fact that the areas between $f(x)$ and $f(\xi)$ on the interval $[a, \xi]$ and between $f(\xi)$ and $f(x)$ on the interval $[\xi, b]$ have the same magnitude, but the former has a negative sign. In general, however, the mean value theorem of integral calculus only guarantees the existence of a $\xi \in [a, b]$ with the discussed property; it does not provide a formula for determining ξ.

With this preliminary work, we can now formulate and prove the first fundamental theorem of differential and integral calculus.

Theorem 7.4 (First fundamental theorem of calculus)**.** *If* $f : I \to \mathbb{R}$ *is a continuous function on the interval* $I \subset \mathbb{R}$*, then the function*

$$F : I \to \mathbb{R}, x \mapsto F(x) := \int_a^x f(t)\,dt \text{ with } x, a \in I \tag{7.30}$$

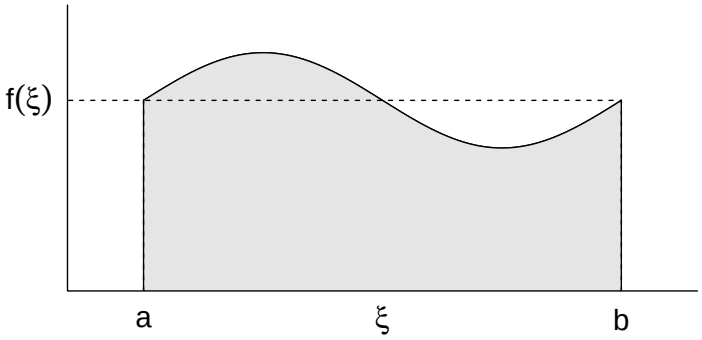


Figure 7.4 On the mean value theorem of integral calculus

is an antiderivative of f.

∘

Proof. We consider the difference quotient

$$\frac{1}{h}(F(x+h) - F(x)) \tag{7.31}$$

With the definition $F(x) := \int_a^x f(t)\,dt$ and the additivity of the definite integral, it follows that

$$\frac{1}{h}(F(x+h) - F(x)) = \frac{1}{h}\left(\int_a^{x+h} f(t)\,dt - \int_a^x f(t)\,dt\right) = \frac{1}{h}\int_x^{x+h} f(t)\,dt \tag{7.32}$$

By the mean value theorem of integral calculus, there is therefore a $\xi \in]x, x+h[$ such that

$$\frac{1}{h}(F(x+h) - F(x)) = f(\xi) \tag{7.33}$$

Taking the limit then yields

$$\lim_{h\to 0}\frac{1}{h}(F(x+h) - F(x)) = \lim_{h\to 0} f(\xi) \text{ for } \xi \in]x, x+h[\Leftrightarrow F'(x) = f(x). \tag{7.34}$$

□

The second fundamental theorem of differential and integral calculus states how to compute a definite integral with the help of the antiderivative.

Theorem 7.5 (Second fundamental theorem of calculus)**.** *If F is an antiderivative of a continuous function $f : I \to \mathbb{R}$ on an interval I, then, for $a, b \in I$ with $a \leq b$,*

$$\int_a^b f(x)\,dx = F(b) - F(a) =: F(x)|_a^b. \tag{7.35}$$

∘

Proof. Using the calculation rules for definite integrals and the first fundamental theorem of differential and integral calculus, we obtain

$$F(b) - F(a) = \int_{\alpha}^{b} f(t)\,dt - \int_{\alpha}^{a} f(t)\,dt = \int_{a}^{\alpha} f(t)\,dt + \int_{\alpha}^{b} f(t)\,dt = \int_{a}^{b} f(x)\,dx. \tag{7.36}$$

□

We want to apply the second fundamental theorem of differential and integral calculus in three examples (cf. Figure 7.5).

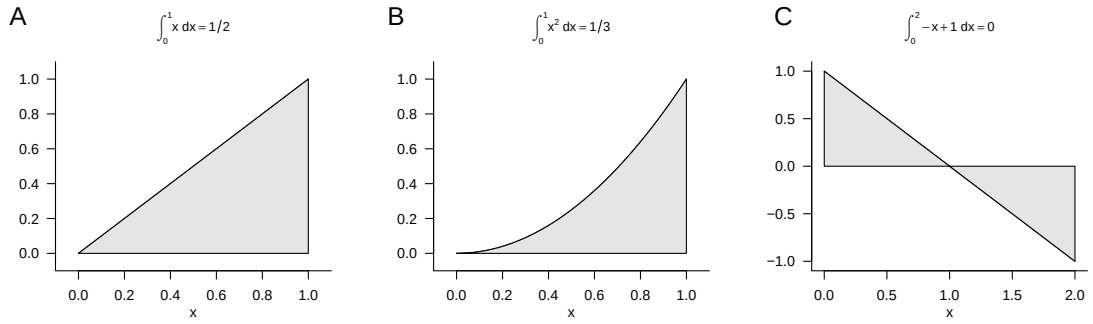


Figure 7.5 Examples of the second fundamental theorem of differential and integral calculus

Example (1)

We consider the identity function

$$f : \mathbb{R} \to \mathbb{R}, x \mapsto f(x) := x \tag{7.37}$$

and want to compute the definite integral of this function on the interval $[0, 1]$, that is,

$$\int_{0}^{1} f(x)\,dx = \int_{0}^{1} x\,dx. \tag{7.38}$$

To do so, recall that an antiderivative of f is given by

$$F : \mathbb{R} \to \mathbb{R}, x \mapsto F(x) := \frac{1}{2}x^2 \tag{7.39}$$

because

$$F'(x) = \frac{d}{dx}\left(\frac{1}{2}x^2\right) = 2 \cdot \frac{1}{2}x^{2-1} = x. \tag{7.40}$$

Substitution into the second fundamental theorem of differential and integral calculus then gives

$$\int_{0}^{1} x\,dx = \frac{1}{2}1^2 - \frac{1}{2}0^2 = \frac{1}{2}. \tag{7.41}$$

This result agrees with the intuition suggested by the gray area in Figure 7.5 A.

Example (2)

Next, we consider the square function

$$f : \mathbb{R} \to \mathbb{R}, x \mapsto f(x) := x^2 \tag{7.42}$$

and want to compute the definite integral of this function on the interval $[0, 1]$, that is,

$$\int_0^1 f(x)\,dx = \int_0^1 x^2\,dx. \tag{7.43}$$

To do so, recall that an antiderivative of f is given by

$$F : \mathbb{R} \to \mathbb{R}, x \mapsto F(x) := \frac{1}{3}x^3 \tag{7.44}$$

because

$$F'(x) = \frac{d}{dx}\left(\frac{1}{3}x^3\right) = 3 \cdot \frac{1}{3}x^{3-1} = x^2. \tag{7.45}$$

Substitution into the second fundamental theorem of differential and integral calculus then gives

$$\int_0^1 x^2\,dx = \frac{1}{3}1^3 - \frac{1}{3}0^3 = \frac{1}{3}. \tag{7.46}$$

This result agrees with the intuition that follows from comparing the gray areas in Figure 7.5 A and Figure 7.5 B.

Example (3)

Finally, we consider the linear-affine function

$$f : \mathbb{R} \to \mathbb{R}, x \mapsto f(x) := -x + 1 \tag{7.47}$$

and want to compute the definite integral of this function on the interval $[0, 2]$, that is,

$$\int_0^2 f(x)\,dx = \int_0^2 -x + 1\,dx. \tag{7.48}$$

To do so, recall that an antiderivative of the linear function with $a = -1$ and $b = 1$ (cf. Table 7.1) is given by

$$F : \mathbb{R} \to \mathbb{R}, x \mapsto F(x) := -\frac{1}{2}x^2 + x \tag{7.49}$$

because

$$F'(x) = \frac{d}{dx}\left(-\frac{1}{2}x^2 + x\right) = -2 \cdot \frac{1}{2}x^{2-1} + 1 \cdot x^{1-1} = -x + 1. \tag{7.50}$$

Substitution into the second fundamental theorem of differential and integral calculus then gives

$$\int_0^2 -x + 1\,dx = \left(-\frac{1}{2}2^2 + 2\right) - \left(-\frac{1}{2}0^2 + 0\right) = -2 + 2 - 0 = 0. \tag{7.51}$$

This result agrees with the intuition that the positive and negative gray areas in Figure 7.5 C cancel each other.

7.3 Improper integrals

Improper integrals are definite integrals in which at least one limit of integration is not a real number, but rather $-\infty$ or ∞. We illuminate the nature of improper integrals with the following definition and an example.

Definition 7.5 (Improper integrals)**.** Let $f : \mathbb{R} \to \mathbb{R}$ be a univariate real-valued function. With the definitions

$$\int_{-\infty}^{b} f(x)\,dx := \lim_{a\to-\infty}\int_{a}^{b} f(x)\,dx \text{ and } \int_{a}^{\infty} f(x)\,dx := \lim_{b\to\infty}\int_{a}^{b} f(x)\,dx \tag{7.52}$$

and the additivity of integrals

$$\int_{-\infty}^{\infty} f(x)\,dx = \int_{-\infty}^{b} f(x)\,dx + \int_{b}^{\infty} f(x)\,dx \tag{7.53}$$

the concept of the definite integral is extended to the unbounded intervals of integration $]-\infty, b]$, $[a, \infty[$, and $]-\infty, \infty[$. Integrals with unbounded intervals of integration are called *improper integrals*. If the corresponding limits exist, one says that the improper integrals *converge*.

•

As an example, we consider the improper integral of the function

$$f : \mathbb{R} \to \mathbb{R}, x \mapsto f(x) := \frac{1}{x^2} \tag{7.54}$$

on the interval $[1, \infty[$, that is,

$$\int_{1}^{\infty} \frac{1}{x^2}\,dx. \tag{7.55}$$

According to the conventions in the definition of improper integrals,

$$\int_{1}^{\infty} \frac{1}{x^2}\,dx = \lim_{b\to\infty}\int_{1}^{b} \frac{1}{x^2}\,dx. \tag{7.56}$$

With the antiderivative $F(x) = -x^{-1}$ of $f(x) = x^{-2}$, the definite integral in the above equation becomes

$$\int_{1}^{b} \frac{1}{x^2}\,dx = F(b) - F(1) = -\frac{1}{b} - \left(-\frac{1}{1}\right) = -\frac{1}{b} + 1. \tag{7.57}$$

Thus,

$$\int_{1}^{\infty} \frac{1}{x^2}\,dx = \lim_{b\to\infty}\int_{1}^{b} \frac{1}{x^2}\,dx = \lim_{b\to\infty}\left(-\frac{1}{b} + 1\right) = -\lim_{b\to\infty}\frac{1}{b} + \lim_{b\to\infty} 1 = 0 + 1 = 1. \tag{7.58}$$

7.4 Multidimensional integrals

So far, we have only considered integrals of univariate real-valued functions. The concept of the integral can also be extended to multivariate real-valued functions. In that case, however, the integration domain of the function is not necessarily as easy to describe as an interval. In particular, arbitrarily shaped integration domains are already possible for bivariate functions, for example. Here, we now want to consider the simplest case of a *hyperrectangle*. In this case, we can define multidimensional definite integrals as follows.

Definition 7.6 (Multidimensional definite integrals on hyperrectangles)**.** Let $f : \mathbb{R}^n \to \mathbb{R}$ be a multivariate real-valued function. Then integrals of the form

$$\int\limits_{[a_1,b_1]\times\cdots\times[a_n,b_n]} f(x)\,dx = \int_{a_1}^{b_1} \cdots \int_{a_n}^{b_n} f(x_1, ..., x_n)\,dx_1 ... dx_n \tag{7.59}$$

are called *multidimensional definite integrals on hyperrectangles*. Furthermore, integrals of the form

$$\int_{\mathbb{R}^n} f(x)\,dx = \int_{-\infty}^{\infty} \cdots \int_{-\infty}^{\infty} f(x_1, ..., x_n)\,dx_1 ... dx_n \tag{7.60}$$

are called *multidimensional improper integrals*.

•

As an example of Definition 7.6, we consider the two-dimensional definite integral of the function

$$f : \mathbb{R}^2 \to \mathbb{R}, (x_1, x_2) \mapsto f(x_1, x_2) := x_1^2 + 4x_2 \tag{7.61}$$

on the rectangle $[0, 1] \times [0, 1]$. Fubini's theorem from the theory of multidimensional integrals states that multidimensional integrals can be evaluated in any coordinate order. Thus, for example,

$$\int_{a_1}^{b_1} \left(\int_{a_2}^{b_2} f(x_1, x_2)\,dx_2 \right) dx_1 = \int_{a_2}^{b_2} \left(\int_{a_1}^{b_1} f(x_1, x_2)\,dx_1 \right) dx_2. \tag{7.62}$$

In this sense, for the example we consider

$$\int_0^1 \int_0^1 x_1^2 + 4x_2\,dx_1\,dx_2 = \int_0^1 \left(\int_0^1 x_1^2 + 4x_2\,dx_1 \right) dx_2 \tag{7.63}$$

and therefore first the inner integral. Here, x_2 takes on the role of a constant. An antiderivative of $g(x_1) := x_1^2 + 4x_2$ is $G(x_1) = \frac{1}{3}x_1^3 + 4x_2x_1$, which can be verified by differentiating G. Thus, for the inner integral,

$$\begin{aligned} \int_0^1 x_1^2 + 4x_2\,dx_1 &= G(1) - G(0) \\ &= \frac{1}{3} \cdot 1^3 + 4x_2 \cdot 1 - \frac{1}{3} \cdot 0^3 - 4x_2 \cdot 0 \\ &= \frac{1}{3} + 4x_2. \end{aligned} \tag{7.64}$$

Considering the outer integral

$$\int_0^1 4x_2 + \frac{1}{3}\,dx_2$$

then gives, with the antiderivative

$$H(x_2) = 2x_2^2 + \frac{1}{3}x_2 \tag{7.65}$$

of

$$h(x_2) := 4x_2 + \frac{1}{3}, \tag{7.66}$$

that

$$\begin{aligned}\int_0^1 \int_0^1 x_1^2 + 4x_2 \, dx_1 \, dx_2 &= \int_0^1 4x_2 + \frac{1}{3} \, dx_2 \\ &= H(1) - H(0) \\ &= 2 \cdot 1^2 + \frac{1}{3} \cdot 1 - 2 \cdot 0^2 - \frac{1}{3} \cdot 0 \\ &= \frac{7}{3}.\end{aligned} \tag{7.67}$$

8 Vectors

In scientific modeling, one often considers phenomena characterized by several quantitative features. For example, the position of an object in three-dimensional space is determined by three coordinates with respect to the three axes of a Cartesian coordinate system. Similarly, a person's health state may be characterized by three measured values, for example a self-report score, a biomarker, and an expert rating. For modeling and analyzing such multidimensional quantitative phenomena, mathematics provides a versatile tool in the form of the real vector space. In this chapter, we first introduce the concept of a real vector space and the basic arithmetic of vectors (Section 8.1). A vector space structure that is strongly guided by three-dimensional spatial intuition is then provided by the Euclidean vector space (Section 8.2). With vector arithmetic, all vectors of a vector space can be formed from a small collection of distinguished vectors. We discuss the concepts underlying this principle in Section 8.3 and Section 8.4.

8.1 Real vector space

We begin with the general definition of a vector space, which specifies basic rules for computing with vectors.

Definition 8.1 (Vector space)**.** Let V be a nonempty set and let S be a set of scalars. Furthermore, let a mapping

$$+ : V \times V \to V, (v_1, v_2) \mapsto +(v_1, v_2) =: v_1 + v_2, \tag{8.1}$$

called *vector addition*, be defined. Finally, let a mapping

$$\cdot : S \times V \to V, (s, v) \mapsto \cdot(s, v) =: sv, \tag{8.2}$$

called *scalar multiplication*, be defined. Then the tuple $(V, S, +, \cdot)$ is called a *vector space* if and only if, for arbitrary elements $v, w, u \in V$ and $a, b \in S$, the following conditions hold:

(1) *Commutativity of vector addition.*

$$v + w = w + v.$$

(2) *Associativity of vector addition.*

$$(v + w) + u = v + (w + u)$$

(3) *Existence of an identity element for vector addition.*

$$\text{There exists a vector } 0 \in V \text{ with } v + 0 = 0 + v = v.$$

(4) *Existence of inverse elements for vector addition.*

$$\text{For all vectors } v \in V \text{ there exists a vector } -v \in V \text{ with } v + (-v) = 0.$$

(5) *Existence of an identity element for scalar multiplication.*

$$\text{There exists a scalar } 1 \in S \text{ with } 1 \cdot v = v.$$

(6) *Associativity of scalar multiplication.*

$$a \cdot (b \cdot v) = (a \cdot b) \cdot v.$$

(7) *Distributivity with respect to vector addition.*

$$a \cdot (v + w) = a \cdot v + a \cdot w.$$

(8) *Distributivity with respect to scalar addition.*

$$(a + b) \cdot v = a \cdot v + b \cdot v.$$

•

It is striking that Definition 8.1 specifies how one should compute with vectors, but does not state what a vector actually is beyond being an element of a set. This is due to the fact that there are many different mathematical objects for which vector space structures can be defined. Examples include the set of real m-tuples, the set of matrices, the set of polynomials, the set of solutions of a linear system of equations, the set of real sequences, the set of continuous functions, and many others.

Here we are initially interested only in the vector space of the set of real m-tuples. We recall that we denote the real m-tuples by

$$\mathbb{R}^m := \left\{ \begin{pmatrix} x_1 \\ \vdots \\ x_m \end{pmatrix} | x_i \in \mathbb{R} \text{ for all } 1 \leq i \leq m \right\} \tag{8.3}$$

and pronounce $\mathbb{R}^m$ as "$\mathbb{R}$ to the power of m". The elements $x \in \mathbb{R}^m$ are called *real vectors* or simply *vectors*. We now want to use the set $\mathbb{R}^m$ as the basis for the definition of a vector space. To this end, we first define vector addition for elements of $\mathbb{R}^m$ and scalar multiplication for elements of $\mathbb{R}$ and $\mathbb{R}^m$.

Definition 8.2 (Vector addition and scalar multiplication in $\mathbb{R}^m$)**.** For all $x, y \in \mathbb{R}^m$ and $a \in \mathbb{R}$, let *vector addition* be defined by

$$+ : \mathbb{R}^m \times \mathbb{R}^m \to \mathbb{R}^m, (x, y) \mapsto x + y = \begin{pmatrix} x_1 \\ \vdots \\ x_m \end{pmatrix} + \begin{pmatrix} y_1 \\ \vdots \\ y_m \end{pmatrix} := \begin{pmatrix} x_1 + y_1 \\ \vdots \\ x_m + y_m \end{pmatrix} \tag{8.4}$$

and let *scalar multiplication* be defined by

$$\cdot : \mathbb{R} \times \mathbb{R}^m \to \mathbb{R}^m, (a, x) \mapsto ax = a \begin{pmatrix} x_1 \\ \vdots \\ x_m \end{pmatrix} := \begin{pmatrix} ax_1 \\ \vdots \\ ax_m \end{pmatrix} \tag{8.5}$$

•

This yields the following result.

Theorem 8.1 (Real vector space)**.** $(\mathbb{R}^m, \mathbb{R}, +, \cdot)$ *with the arithmetic rules of addition and multiplication in* $\mathbb{R}$ *is a vector space.*

∘

For a proof, which we omit here, one has to verify conditions (1) to (8) from Definition 8.1 for the set considered here and the forms of vector addition and scalar multiplication specified here. These follow easily from the arithmetic rules of addition and multiplication in $\mathbb{R}$ and from the fact that vector addition and scalar multiplication for elements of $\mathbb{R}^m$ were defined componentwise in Definition 8.2. We thus define the concept of the *real vector space*.

Definition 8.3 (Real vector space)**.** For $\mathbb{R}^m$, let $+$ and $\cdot$ be the vector addition and scalar multiplication defined in Definition 8.2. Then, based on Theorem 8.1, we call the vector space $(\mathbb{R}^m, \mathbb{R}, +, \cdot)$ the *real vector space*.

•

On the basis of Definition 8.3, we now illustrate computing with real vectors by means of a few examples.

Examples

(1) For

$$x := \begin{pmatrix} 1 \\ 2 \\ 3 \\ 4 \end{pmatrix} \in \mathbb{R}^4 \text{ and } y := \begin{pmatrix} 2 \\ 1 \\ 0 \\ 1 \end{pmatrix} \in \mathbb{R}^4$$

we have

$$x + y = \begin{pmatrix} 1 \\ 2 \\ 3 \\ 4 \end{pmatrix} + \begin{pmatrix} 2 \\ 1 \\ 0 \\ 1 \end{pmatrix} = \begin{pmatrix} 1+2 \\ 2+1 \\ 3+0 \\ 4+1 \end{pmatrix} = \begin{pmatrix} 3 \\ 3 \\ 3 \\ 5 \end{pmatrix} \in \mathbb{R}^4.$$

In **R**, one implements this example as follows.

```
x = matrix(c(1,2,3,4), nrow = 4)      # vector definition
y = matrix(c(2,1,0,1), nrow = 4)      # vector definition
x + y                                 # vector addition
```

```
     [,1]
[1,]    3
[2,]    3
[3,]    3
[4,]    5
```

(2) For

$$x := \begin{pmatrix} 2 \\ 3 \end{pmatrix} \in \mathbb{R}^2 \text{ and } y := \begin{pmatrix} 1 \\ 3 \end{pmatrix} \in \mathbb{R}^2$$

we have

$$x - y = \begin{pmatrix} 2 \\ 3 \end{pmatrix} - \begin{pmatrix} 1 \\ 3 \end{pmatrix} = \begin{pmatrix} 2-1 \\ 3-3 \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \end{pmatrix} \in \mathbb{R}^2.$$

In **R**, one implements this example as follows.

```
x = matrix(c(2,3), nrow = 2)          # vector definition
y = matrix(c(1,3), nrow = 2)          # vector definition
x - y                                 # vector subtraction
```

```
     [,1]
[1,]    1
[2,]    0
```

(3) For

$$x := \begin{pmatrix} 2 \\ 1 \\ 3 \end{pmatrix} \in \mathbb{R}^3 \text{ and } a := 3 \in \mathbb{R}$$

we have

$$ax = 3 \begin{pmatrix} 2 \\ 1 \\ 3 \end{pmatrix} = \begin{pmatrix} 3 \cdot 2 \\ 3 \cdot 1 \\ 3 \cdot 3 \end{pmatrix} = \begin{pmatrix} 6 \\ 3 \\ 9 \end{pmatrix} \in \mathbb{R}^3.$$

In **R**, one implements this example as follows.

```
x = matrix(c(2,1,3), nrow = 3)        # vector definition
a = 3                                 # scalar definition
a*x                                   # scalar multiplication
```

```
     [,1]
[1,]    6
[2,]    3
[3,]    9
```

For $m \in \{1, 2, 3\}$, real vectors and computations with them can be visualized. For $m > 3$, for example when more than three quantitative features of a person's health state are available, as is regularly the case in applications, this is not possible. Nevertheless, visual intuition for $m \leq 3$ may facilitate an initial understanding of vector spaces. We focus here on the case $m := 2$. In this case, the real vectors under consideration lie in the two-dimensional plane and are usually visualized as points or arrows (Figure 8.1).

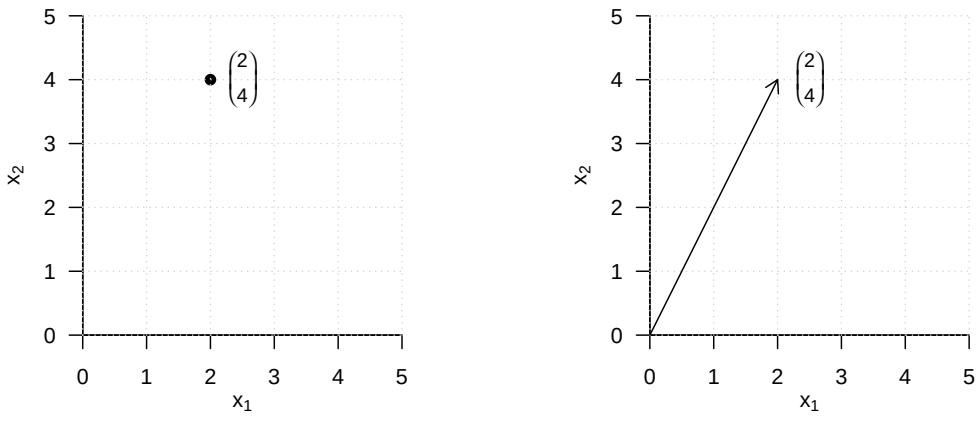


Figure 8.1 Visualization of vectors in $\mathbb{R}^2$

Figure 8.2 visualizes the vector addition

$$\begin{pmatrix} 1 \\ 2 \end{pmatrix} + \begin{pmatrix} 3 \\ 1 \end{pmatrix} = \begin{pmatrix} 4 \\ 3 \end{pmatrix}. \tag{8.6}$$

The sum vector corresponds to the diagonal of the parallelogram spanned by the two summands.

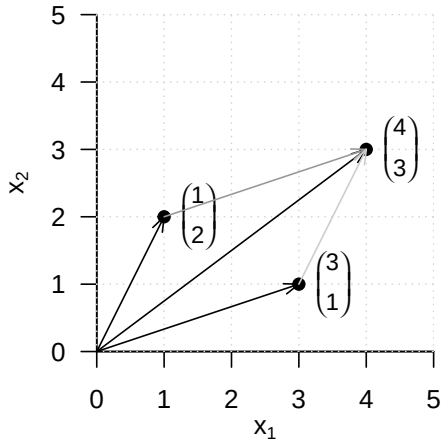


Figure 8.2 Vector addition in $\mathbb{R}^2$

Figure 8.3 visualizes the vector subtraction

$$\begin{pmatrix} 1 \\ 2 \end{pmatrix} - \begin{pmatrix} 3 \\ 1 \end{pmatrix} = \begin{pmatrix} 1 \\ 2 \end{pmatrix} + \begin{pmatrix} -3 \\ -1 \end{pmatrix} = \begin{pmatrix} -2 \\ 1 \end{pmatrix} \tag{8.7}$$

The resulting vector corresponds to the diagonal of the parallelogram spanned by the first vector and the opposite vector of the second vector.

Finally, Figure 8.4 visualizes the scalar multiplication

$$3 \begin{pmatrix} 1 \\ 1 \end{pmatrix} = \begin{pmatrix} 3 \\ 3 \end{pmatrix} \tag{8.8}$$

Multiplication of a vector by a positive scalar changes only its length, but not its direction.

8.2 Euclidean vector space

By defining the *inner product*, the real vector space can be endowed with spatial-geometric intuition in the sense of a *Euclidean vector space*. In particular, this makes it possible to define and compute concepts such as the *length of a vector*, the *distance between two vectors*, and, not least, the *angle between two vectors*. We first introduce the *inner product*.

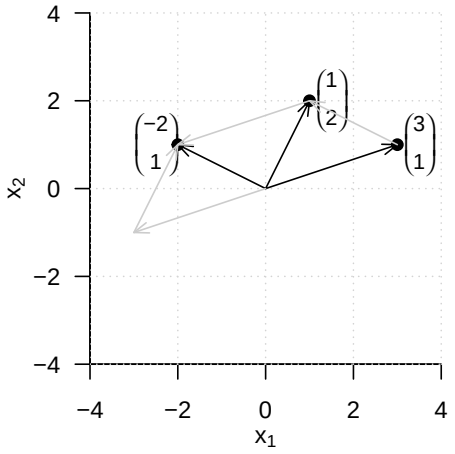


Figure 8.3 Vector subtraction in $\mathbb{R}^2$

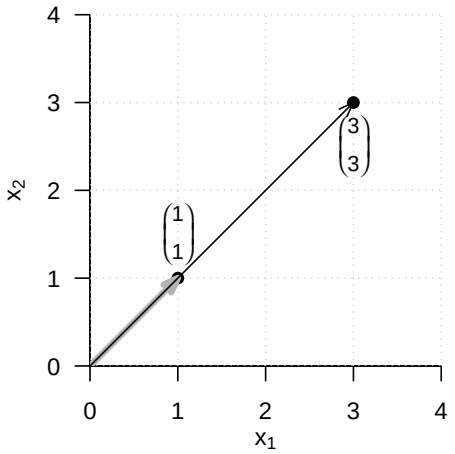


Figure 8.4 Scalar multiplication in $\mathbb{R}^2$

Definition 8.4 (Inner product on $\mathbb{R}^m$)**.** The *inner product on* $\mathbb{R}^m$ is defined as the mapping

$$\langle\rangle : \mathbb{R}^m \times \mathbb{R}^m \to \mathbb{R}, (x, y) \mapsto \langle (x, y) \rangle := \langle x, y \rangle := \sum_{i=1}^{m} x_i y_i. \tag{8.9}$$

•

The inner product is called an inner product because it yields a scalar, not because it involves multiplication by scalars. The inner product is closely related to the matrix product, as we will see later. We first consider an example and its implementation in **R**.

Example

Let

$$x := \begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix} \text{ and } y := \begin{pmatrix} 2 \\ 0 \\ 1 \end{pmatrix} \tag{8.10}$$

Then

$$\langle x, y \rangle = x_1 y_1 + x_2 y_2 + x_3 y_3 = 1 \cdot 2 + 2 \cdot 0 + 3 \cdot 1 = 2 + 0 + 3 = 5. \tag{8.11}$$

In **R**, there are several ways to evaluate an inner product. We list two of them for the given example below.

```
# vector definitions
x = matrix(c(1,2,3), nrow = 3)
y = matrix(c(2,0,1), nrow = 3)

# inner product using R's componentwise multiplication and sum() function
sum(x*y)
```

```
[1] 5
```

```
# inner product using R's matrix transposition and multiplication
t(x) %*% y
```

```
     [,1]
[1,]    5
```

With the help of the inner product, the concept of the real vector space can be extended to the concept of the *real canonical Euclidean vector space*.

Definition 8.5 (Euclidean vector space)**.** The tuple $((\mathbb{R}^m, +, \cdot), \langle\rangle)$ consisting of the real vector space $(\mathbb{R}^m, +, \cdot)$ and the inner product $\langle\rangle$ on $\mathbb{R}^m$ is called the *real canonical Euclidean vector space*.

•

In general, every tuple consisting of a vector space and an inner product is called a "Euclidean vector space". Informally, however, we often simply speak of $\mathbb{R}^m$ as a "Euclidean vector space" and, in particular, of $((\mathbb{R}^m, +, \cdot), \langle\rangle)$ as the "Euclidean vector space". A Euclidean vector space is a vector space with geometric structure induced by the inner product. In particular, the geometric concepts of *length*, *distance*, and *angle* now acquire meaning in the Euclidean vector space. We define them as follows.

Definition 8.6. $((\mathbb{R}^m, +, \cdot), \langle\rangle)$ is the Euclidean vector space.

(1) The *length* of a vector $x \in \mathbb{R}^m$ is defined as

$$\|x\| := \sqrt{\langle x, x \rangle}. \tag{8.12}$$

(2) The *distance* between two vectors $x, y \in \mathbb{R}^m$ is defined as

$$d(x, y) := \|x - y\|. \tag{8.13}$$

(3) The *angle* α between two vectors $x, y \in \mathbb{R}^m$ with $x, y \neq 0$ is defined by

$$0 \leq \alpha \leq \pi \text{ and } \cos\alpha := \frac{\langle x, y \rangle}{\|x\|\|y\|} \tag{8.14}$$

•

The length $\|x\|$ of a vector $x \in \mathbb{R}^m$ is also called the *Euclidean norm of* x, the ℓ_2*-norm of* x, or simply the *norm of* x. It is often denoted by $\|x\|_2$. We consider three examples for determining the length of a vector and their corresponding **R** implementation. We illustrate these examples in Figure 8.5.

Example (1)

$$\left\| \begin{pmatrix} 2 \\ 0 \end{pmatrix} \right\| = \sqrt{\left\langle \begin{pmatrix} 2 \\ 0 \end{pmatrix}, \begin{pmatrix} 2 \\ 0 \end{pmatrix} \right\rangle} = \sqrt{2^2 + 0^2} = \sqrt{4} = 2.00 \tag{8.15}$$

```
norm(matrix(c(2,0),nrow = 2), type = "2")            # vector length = l_2 norm
```

```
[1] 2
```

Example (2)

$$\left\| \begin{pmatrix} 2 \\ 2 \end{pmatrix} \right\| = \sqrt{\left\langle \begin{pmatrix} 2 \\ 2 \end{pmatrix}, \begin{pmatrix} 2 \\ 2 \end{pmatrix} \right\rangle} = \sqrt{2^2 + 2^2} = \sqrt{8} \approx 2.83 \tag{8.16}$$

```
norm(matrix(c(2,2),nrow = 2), type = "2")            # vector length = l_2 norm
```

```
[1] 2.828427
```

Example (3)

$$\left\| \begin{pmatrix} 2 \\ 4 \end{pmatrix} \right\| = \sqrt{\left\langle \begin{pmatrix} 2 \\ 4 \end{pmatrix}, \begin{pmatrix} 2 \\ 4 \end{pmatrix} \right\rangle} = \sqrt{2^2 + 4^2} = \sqrt{20} \approx 4.47 \tag{8.17}$$

```
norm(matrix(c(2,4),nrow = 2), type = "2")            # vector length = l_2 norm
```

```
[1] 4.472136
```

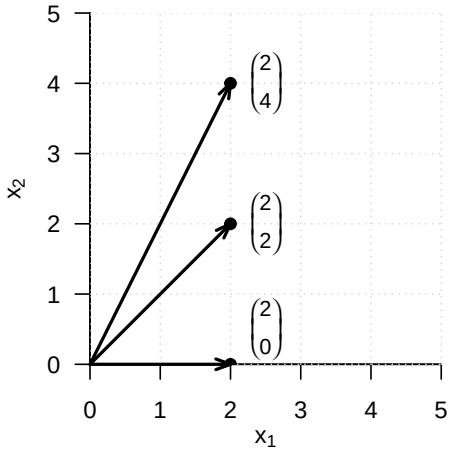


Figure 8.5 Vector length in $\mathbb{R}^2$

For the distance $d(x, y)$ between two vectors $x, y \in \mathbb{R}^m$, we note without proof that it is nonnegative and symmetric, that is,

$$d(x, y) \geq 0, d(x, x) = 0 \text{ and } d(x, y) = d(y, x) \tag{8.18}$$

hold. Moreover, $d(x, y)$ satisfies the so-called *triangle inequality*, which states that the direct path between two points in space is always shorter than an indirect path through a third point,

$$d(x, y) \leq d(x, z) + d(z, y). \tag{8.19}$$

Thus, $d(x, y)$ satisfies important aspects of spatial intuition. We give two examples for determining distances between vectors in $\mathbb{R}^2$, which we visualize in Figure 8.6.

Example (1)

$$d\left(\begin{pmatrix}1\\1\end{pmatrix}, \begin{pmatrix}2\\2\end{pmatrix}\right) = \left\|\begin{pmatrix}1\\1\end{pmatrix} - \begin{pmatrix}2\\2\end{pmatrix}\right\| = \left\|\begin{pmatrix}-1\\-1\end{pmatrix}\right\| = \sqrt{(-1)^2 + (-1)^2} = \sqrt{2} \approx 1.41 \tag{8.20}$$

```
norm(matrix(c(1,1),nrow = 2) - matrix(c(2,2),nrow = 2), type = "2")
```

```
[1] 1.414214
```

Example (2)

$$d\left(\begin{pmatrix}1\\1\end{pmatrix}, \begin{pmatrix}4\\1\end{pmatrix}\right) = \left\|\begin{pmatrix}1\\1\end{pmatrix} - \begin{pmatrix}4\\1\end{pmatrix}\right\| = \left\|\begin{pmatrix}-3\\0\end{pmatrix}\right\| = \sqrt{(-3)^2 + 0^2} = \sqrt{9} = 3 \tag{8.21}$$

```
norm(matrix(c(1,1),nrow = 2) - matrix(c(1,4),nrow = 2), type = "2")
```

```
[1] 3
```

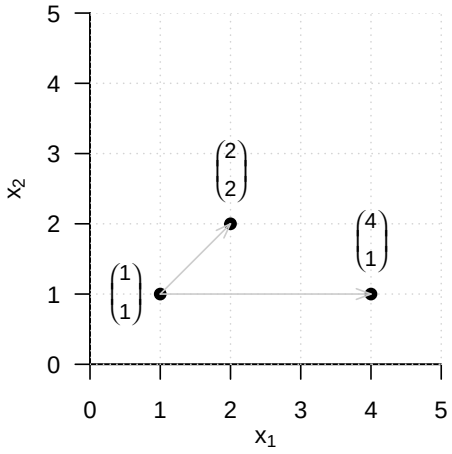


Figure 8.6 Vector distances in $\mathbb{R}^2$

Finally, we note that for the calculation of the angle between two vectors based on the above definition, the cosine function cos is bijective on $[0, \pi]$ and hence invertible with inverse function *acos*, the arccosine function. We also consider two examples for the concept of angle. Note in particular that Definition 8.6 gives the angle in radians. For a specification in degrees, a corresponding conversion is required.

Example (1)

$$\text{acos}\left(\frac{\left\langle \begin{pmatrix}3\\0\end{pmatrix}, \begin{pmatrix}3\\3\end{pmatrix} \right\rangle}{\left\| \begin{pmatrix}3\\0\end{pmatrix} \right\| \left\| \begin{pmatrix}3\\3\end{pmatrix} \right\|}\right) = \text{acos}\left(\frac{3 \cdot 3 + 3 \cdot 0}{\sqrt{3^2 + 0^2} \cdot \sqrt{3^2 + 3^2}}\right) = \text{acos}\left(\frac{9}{3 \cdot \sqrt{18}}\right) = \frac{\pi}{4} \approx 0.785 \tag{8.22}$$

The conversion to degrees then gives

$$0.785 \cdot \frac{180^\circ}{\pi} = 45^\circ \tag{8.23}$$

In **R**, one implements this as follows.

```
x = matrix(c(3,0), nrow = 2)                                # vector 1
y = matrix(c(3,3), nrow = 2)                                # vector 2
w = acos(sum(x*y)/(sqrt(sum(x*x))*sqrt(sum(y*y)))) * 180/pi  # angle in degrees
print(w)
```

```
[1] 45
```

Example (2)

$$\alpha = \text{acos}\left(\frac{\left\langle \begin{pmatrix}3\\0\end{pmatrix}, \begin{pmatrix}0\\3\end{pmatrix} \right\rangle}{\left\| \begin{pmatrix}3\\0\end{pmatrix} \right\| \left\| \begin{pmatrix}0\\3\end{pmatrix} \right\|}\right) = \text{acos}\left(\frac{3 \cdot 0 + 0 \cdot 3}{\sqrt{3^2 + 0^2} \cdot \sqrt{0^2 + 3^2}}\right) = \text{acos}\left(\frac{0}{3 \cdot 3}\right) = \frac{\pi}{2} \approx 1.57 \tag{8.24}$$

The conversion to degrees then gives

$$\frac{\pi}{2} \cdot \frac{180^\circ}{\pi} = 90^\circ \tag{8.25}$$

The corresponding **R** implementation is as follows.

```
x = matrix(c(3,0), nrow = 2)                                  # vector 1
y = matrix(c(0,3), nrow = 2)                                  # vector 2
w = acos(sum(x*y)/(sqrt(sum(x*x))*sqrt(sum(y*y)))) * 180/pi   # angle in degrees
print(w)
```

```
[1] 90
```

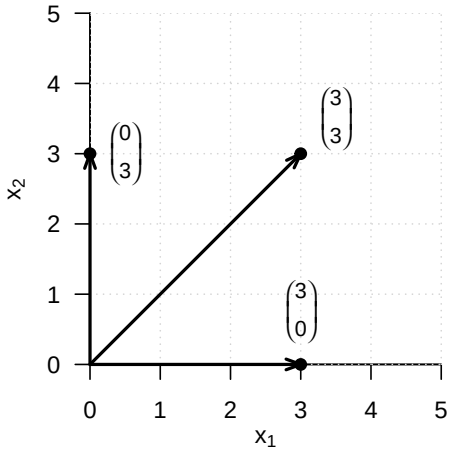


Figure 8.7 Angles in $\mathbb{R}^2$

The fact that two vectors can form a right angle, that is, can in a sense be maximally non-parallel, is an important geometric principle and is therefore singled out by the following definition.

Definition 8.7 (Orthogonality and orthonormality of vectors)**.** Let $((\mathbb{R}^m, +, \cdot), \langle\rangle)$ be the Euclidean vector space.

(1) Two vectors $x, y \in \mathbb{R}^m$ are called *orthogonal* if

$$\langle x, y\rangle = 0 \tag{8.26}$$

(2) Two vectors $x, y \in \mathbb{R}^m$ are called *orthonormal* if

$$\langle x, y\rangle = 0 \text{ and } \|x\| = \|y\| = 1. \tag{8.27}$$

•

For orthogonal and orthonormal vectors, in particular, we therefore also have

$$\cos\alpha = \frac{\langle x, y\rangle}{\|x\|\|y\|} = \frac{0}{\|x\|\|y\|} = 0, \tag{8.28}$$

and hence

$$\alpha = \frac{\pi}{2} = 90^\circ. \tag{8.29}$$

8.3 Linear independence

In this section, we introduce the concept of *linear independence* of vectors. To this end, we first define the concept of a *linear combination* of vectors.

Definition 8.8 (Linear combination)**.** Let $\{v_1, v_2, ..., v_k\}$ be a set of k vectors of a vector space V, and let $a_1, a_2, ..., a_k$ be scalars. Then the *linear combination* of the vectors in $\{v_1, v_2, ..., v_k\}$ with the *coefficients* $a_1, a_2, ..., a_k$ is defined as the vector

$$w := \sum_{i=1}^{k} a_i v_i \in V. \tag{8.30}$$

•

Example

Let

$$v_1 := \begin{pmatrix} 2 \\ 1 \end{pmatrix}, v_2 := \begin{pmatrix} 1 \\ 1 \end{pmatrix}, v_3 := \begin{pmatrix} 0 \\ 1 \end{pmatrix} \text{ and } a_1 := 2, a_2 := 3, a_3 := 0. \tag{8.31}$$

Then the linear combination of v_1, v_2, v_3 with coefficients a_1, a_2, a_3 is

$$\begin{aligned} w &= a_1 v_1 + a_2 v_2 + a_3 v_3 \\ &= 2 \cdot \begin{pmatrix} 2 \\ 1 \end{pmatrix} + 3 \cdot \begin{pmatrix} 1 \\ 1 \end{pmatrix} + 0 \cdot \begin{pmatrix} 0 \\ 1 \end{pmatrix} \\ &= \begin{pmatrix} 4 \\ 2 \end{pmatrix} + \begin{pmatrix} 3 \\ 3 \end{pmatrix} + \begin{pmatrix} 0 \\ 0 \end{pmatrix} \\ &= \begin{pmatrix} 7 \\ 5 \end{pmatrix}. \end{aligned} \tag{8.32}$$

Based on the concept of a linear combination, one can now define the concept of *linear independence* of vectors.

Definition 8.9 (Linear independence)**.** Let V be a vector space. A set $W := \{w_1, w_2, ..., w_k\}$ of vectors in V is called *linearly independent* if the only representation of the zero element $0 \in V$ by a linear combination of the $w \in W$ is the so-called *trivial representation*

$$0 = a_1 w_1 + a_2 w_2 + \cdots + a_k w_k \text{ with } a_1 = a_2 = \cdots = a_k = 0 \tag{8.33}$$

If the set W is not linearly independent, it is called *linearly dependent*.

•

To check whether a given set of vectors is linearly dependent or independent, in principle one has to check whether a possible linear combination of the given vectors is zero. Theorem 8.2 and Theorem 8.3 show how this can be achieved with less effort for two and for finitely many vectors, respectively.

Theorem 8.2 (Linear dependence of two vectors)**.** *Let V be a vector space. Two vectors $v_1, v_2 \in V$ are linearly dependent if one of the vectors is a scalar multiple of the other vector.*

∘

Proof. Let v_1 be a scalar multiple of v_2, that is,

$$v_1 = \lambda v_2 \text{ with } \lambda \neq 0. \tag{8.34}$$

Then

$$v_1 - \lambda v_2 = 0. \tag{8.35}$$

But this corresponds to the linear combination

$$a_1 v_1 + a_2 v_2 = 0 \tag{8.36}$$

with $a_1 = 1 \neq 0$ and $a_2 = -\lambda \neq 0$. Thus, there exists a linear combination of the zero element that is not the trivial representation, and hence v_1 and v_2 are not linearly independent. □

Theorem 8.3 (Linear dependence of a set of vectors)**.** *Let V be a vector space, and let $w_1, ..., w_k \in V$ be a set of vectors in V. If one of the vectors w_i with $i = 1, ..., k$ is a linear combination of the other vectors, then the set of vectors is linearly dependent.*

∘

Proof. The vectors $w_1, ..., w_k$ are linearly dependent if and only if $\sum_{i=1}^{k} a_i w_i = 0$ with at least one $a_i \neq 0$. Thus, for example, let $a_j \neq 0$. Then

$$0 = \sum_{i=1}^{k} a_i w_i = \sum_{i=1, i\neq j}^{k} a_i w_i + a_j w_j \tag{8.37}$$

Therefore,

$$a_j w_j = -\sum_{i=1, i\neq j}^{k} a_i w_i \tag{8.38}$$

and thus

$$w_j = -a_j^{-1} \sum_{i=1, i\neq j}^{k} a_i w_i = -\sum_{i=1, i\neq j}^{k} (a_j^{-1} a_i) w_i \tag{8.39}$$

Thus, w_j is a linear combination of the w_i for $i = 1, ..., k$ with $i \neq j$. □

8.4 Vector space bases

In this section, we introduce the concept of a *vector space basis*. A basis of a vector space is a subset of vectors of the vector space that can be used to represent all vectors of the vector space. In the sense of linear combinations of vectors, a vector space basis therefore contains all information needed to construct the corresponding vector space. However, a vector space basis is usually not unique, and many vector spaces indeed have infinitely many bases. The following definition first states how infinitely many vectors can be formed from a limited number of vectors by means of linear combinations.

Definition 8.10 (Span and spanning)**.** Let V be a vector space, and let $W := \{w_1, ..., w_k\} \subset V$. Then the *span* of W is defined as the set of all linear combinations of the elements of W,

$$\text{Span}(W) := \left\{ \sum_{i=1}^{k} a_i w_i | a_1, ..., a_k \text{ are scalar coefficients} \right\} \tag{8.40}$$

A set of vectors $W \subseteq V$ is said to *span a vector space* V if every $v \in V$ can be written as a linear combination of vectors in W.

•

We now define the concept of a *basis* of a vector space.

Definition 8.11 (Basis)**.** Let V be a vector space, and let $B \subseteq V$. B is called a *basis of* V if

(1) the vectors in B are linearly independent, and
(2) the vectors in B span the vector space V.

•

Bases of vector spaces have the following important properties.

Theorem 8.4 (Properties of bases)**.**

(1) All bases of a vector space contain the same number of vectors.
(2) Every set of m linearly independent vectors is a basis of an m-dimensional vector space.

◦

For a proof of this very deep theorem, we refer to the advanced literature. The unique number of vectors in a basis of a vector space named by the above theorem is called the *dimension of the vector space*. Since there are usually infinitely many sets of m linearly independent vectors in a vector space, vector spaces usually have infinitely many bases.

If one considers a single vector in a vector space, one may ask how this vector can be represented with the help of a vector space basis. This leads to the following concepts.

Definition 8.12 (Basis representation and coordinates)**.** Let $B := \{b_1, ..., b_m\}$ be a basis of an m-dimensional vector space V, and let $v \in V$. Then the linear combination

$$v = \sum_{i=1}^{m} c_i b_i \tag{8.41}$$

is called the *representation of v with respect to the basis B*, and the coefficients $c_1, ..., c_m$ are called the *coordinates of v with respect to the basis B*.

•

For a fixed basis, the coordinates of a vector with respect to this basis are also fixed and unique. This is the statement of the following theorem.

Theorem 8.5 (Uniqueness of basis representation)**.** *The basis representation of a $v \in V$ with respect to a basis B is unique.*

∘

Proof. Without loss of generality, assume that the vector space has dimension m. Assume that two representations of v with respect to the basis B exist, that is,

$$\begin{aligned} v &= a_1 b_1 + \cdots + a_m b_m \\ v &= c_1 b_1 + \cdots + c_m b_m \end{aligned} \tag{8.42}$$

Subtracting the lower equation from the upper equation gives

$$0 = (a_1 - c_1)b_1 + \cdots + (a_m - c_m)b_m \tag{8.43}$$

Because $b_1, ..., b_m$ are linearly independent, however, $(a_i - c_i) = 0$ for all $i = 1, ..., m$, and thus the two representations of v with respect to the basis B are identical. □

At the end of this section, we consider a special basis of the real vector space.

Definition 8.13 (Orthonormal basis of $\mathbb{R}^m$)**.** A set of m vectors $v_1, ..., v_m \in \mathbb{R}^m$ is called an *orthonormal basis* of $\mathbb{R}^m$ if $v_1, ..., v_m$ each have length 1 and are mutually orthogonal, that is, if

$$\langle v_i, v_j \rangle = \begin{cases} 1 & \text{for } i = j \\ 0 & \text{for } i \neq j \end{cases}. \tag{8.44}$$

•

We first consider an example of an orthonormal basis.

Example (1)

$$B_1 := \left\{ \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 0 \\ 1 \end{pmatrix} \right\} \tag{8.45}$$

is an orthonormal basis of $\mathbb{R}^2$, because B_1 consists of two vectors and

$$\left\langle \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 \\ 0 \end{pmatrix} \right\rangle = 1 \cdot 1 + 0 \cdot 0 = 1 + 0 = 1 \tag{8.46}$$

as well as

$$\left\langle \begin{pmatrix} 0 \\ 1 \end{pmatrix}, \begin{pmatrix} 0 \\ 1 \end{pmatrix} \right\rangle = 0 \cdot 0 + 1 \cdot 1 = 0 + 1 = 1 \tag{8.47}$$

and

$$\left\langle \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 0 \\ 1 \end{pmatrix} \right\rangle = 1 \cdot 0 + 0 \cdot 1 = 0 + 0 = 0 \tag{8.48}$$

hold.

For general real vector spaces, bases of the form of B_1 are singled out by the concept of the *canonical basis*.

Definition 8.14 (Canonical basis and standard unit vectors)**.** The orthonormal basis

$$B := \left\{e_1, ..., e_m | (e_i)_j = 1 \text{ for } i = j \text{ and } (e_i)_j = 0 \text{ for } i \neq j,\ i, j = 1, ..., m\right\} \subset \mathbb{R}^m \quad (8.49)$$

is called the *canonical basis* of $\mathbb{R}^m$, and the e_i are called *standard unit vectors.*

•

B_1 from Example (1) is therefore the canonical basis of $\mathbb{R}^2$.

The canonical basis of $\mathbb{R}^3$ is

$$B := \left\{ \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix} \right\}. \quad (8.50)$$

However, there are also non-canonical orthonormal bases. To see this, we consider another example.

Example (2)

$$B_2 := \left\{ \begin{pmatrix} \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} \end{pmatrix}, \begin{pmatrix} -\frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} \end{pmatrix} \right\} \quad (8.51)$$

is also an orthonormal basis of $\mathbb{R}^2$, because B_2 consists of two vectors and

$$\left\langle \begin{pmatrix} \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} \end{pmatrix}, \begin{pmatrix} \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} \end{pmatrix} \right\rangle = \frac{1}{\sqrt{2}} \cdot \frac{1}{\sqrt{2}} + \frac{1}{\sqrt{2}} \cdot \frac{1}{\sqrt{2}} = \frac{1}{2} + \frac{1}{2} = 1, \quad (8.52)$$

as well as

$$\left\langle \begin{pmatrix} -\frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} \end{pmatrix}, \begin{pmatrix} -\frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} \end{pmatrix} \right\rangle = \left(-\frac{1}{\sqrt{2}}\right) \cdot \left(-\frac{1}{\sqrt{2}}\right) + \frac{1}{\sqrt{2}} \cdot \frac{1}{\sqrt{2}} = \frac{1}{2} + \frac{1}{2} = 1 \quad (8.53)$$

and

$$\left\langle \begin{pmatrix} -\frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} \end{pmatrix}, \begin{pmatrix} \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} \end{pmatrix} \right\rangle = -\frac{1}{\sqrt{2}} \cdot \frac{1}{\sqrt{2}} + \frac{1}{\sqrt{2}} \cdot \frac{1}{\sqrt{2}} = -\frac{1}{2} + \frac{1}{2} = 0 \quad (8.54)$$

hold.

We visualize the two orthonormal bases B_1 and B_2 of $\mathbb{R}^2$ in Figure 8.8.

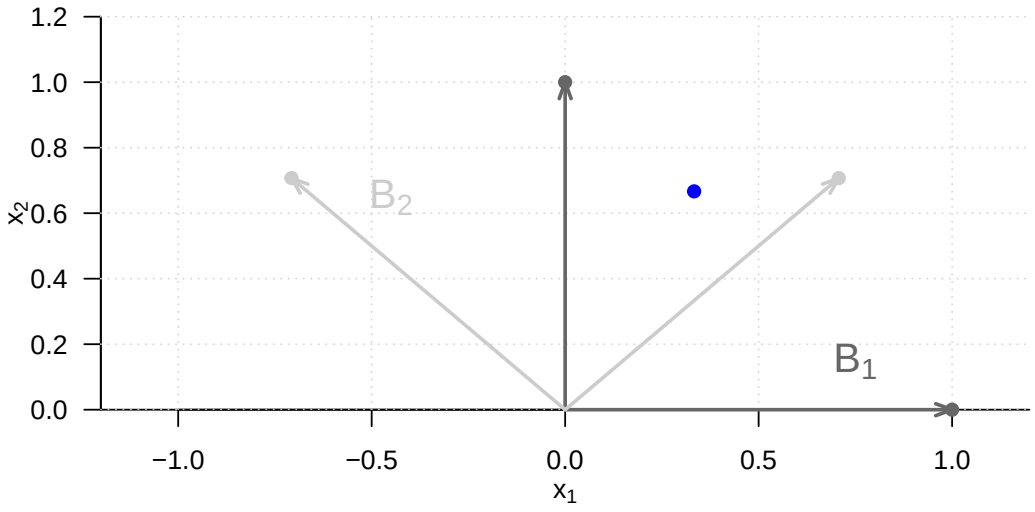


Figure 8.8 Two bases of $\mathbb{R}^2$

9 Matrices

Matrices are the words of the language of modern data analysis. An understanding of modern data-analytic methods and their implementation is not possible without a basic understanding of the concept of a matrix and knowledge of the fundamental matrix operations. Matrices can play very different roles. For example, matrices can represent data, experimental designs, and model parameters. In the context of linear algebra, matrices are used to represent linear mappings and vector spaces; here vectors are then understood as special matrices.

In this chapter, we introduce working with matrices while largely avoiding abstract terminology from linear algebra. We first introduce the concept of a matrix and then discuss the first fundamental matrix operations, namely matrix addition, matrix subtraction, scalar multiplication, and matrix transposition (Section 9.1 and Section 9.2). We then introduce the central concepts of matrix multiplication and matrix inversion (Section 9.3 and Section 9.4). With the matrix determinant, we discuss a first measure for describing matrices in Section 9.5. We close in Section 9.6 with an overview of particularly common matrices.

9.1 Definition

We begin with the definition of a matrix.

Definition 9.1. A matrix is a rectangular arrangement of numbers denoted as follows:

$$A := \begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1m} \\ a_{21} & a_{22} & \cdots & a_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nm} \end{pmatrix} := (a_{ij})_{1 \leq i \leq n,\, 1 \leq j \leq m}. \tag{9.1}$$

•

Matrices consist of *rows* and *columns*. The matrix entries a_{ij} are indexed with a *row index* i and a *column index* j. For example, for

$$A := \begin{pmatrix} 2 & 7 & 5 & 2 \\ 8 & 2 & 5 & 6 \\ 6 & 4 & 0 & 9 \\ 9 & 2 & 1 & 2 \end{pmatrix}, \tag{9.2}$$

we have $a_{32} = 4$. The *size* or *dimension* of a matrix is determined by the number of its rows $n \in \mathbb{N}$ and columns $m \in \mathbb{N}$. Matrices with $n = m$ are called *square matrices*.

In what follows, we only need matrices with real entries, that is, $a_{ij} \in \mathbb{R}$ for all $i = 1, ..., n$ and $j = 1, ..., m$. We call matrices with real entries *real matrices* and denote the set of real matrices with n rows and m columns by $\mathbb{R}^{n \times m}$. From the expression

$$A \in \mathbb{R}^{n \times m} \tag{9.3}$$

we can therefore see that A is a real matrix with n rows and m columns. We identify the set $\mathbb{R}^{1 \times 1}$ with the set $\mathbb{R}$ and the set $\mathbb{R}^{n \times 1}$ with the set $\mathbb{R}^n$. Thus, real matrices with one column and n rows correspond to n-dimensional real vectors, and real matrices with one column and one row correspond to real numbers.

Defining matrices in R

In **R**, matrices are defined by transforming **R** vectors into the representation of a mathematical matrix with the `matrix()` function. The entries of an **R** vector are distributed over the matrix according to their total number and the specified number of rows `nrow`. For example, if we want to define the matrix

$$A := \begin{pmatrix} 2 & 3 & 0 \\ 1 & 6 & 5 \end{pmatrix} \tag{9.4}$$

in **R**, we obtain

```
# columnwise definition of A (R default)
A = matrix(c(2,1,3,6,0,5), nrow = 2)
print(A)
```

```
     [,1] [,2] [,3]
[1,]    2    3    0
[2,]    1    6    5
```

By default, **R** follows a so-called *column-major order*, that is, the elements of the **R** vector `c(2,1,3,6,0,5)` are transferred one after another from top to bottom into the columns of the matrix from left to right. A somewhat clearer connection between the visual layout of the **R** code and the resulting matrix is obtained by formatting the **R** vector with line breaks according to the intended matrix layout and then changing the column-major order to *row-major order* with the argument `byrow = TRUE`. Then the first row of the matrix is filled from left to right with the elements of the **R** vector, then the second row, and so on until all elements of the vector are distributed over the matrix.

```
# rowwise definition of A (byrow = TRUE)
A = matrix(c(2,3,0,
             1,6,5),
           nrow = 2,
           byrow = TRUE)
print(A)
```

```
     [,1] [,2] [,3]
[1,]    2    3    0
[2,]    1    6    5
```

```
# rowwise definition of B
B = matrix(c(4,1,0,
            -4,2,0),
           nrow = 2,
           byrow = TRUE)
print(B)
```

```
     [,1] [,2] [,3]
[1,]    4    1    0
[2,]   -4    2    0
```

9.2 Basic matrix operations

One can compute with matrices. The following matrix operations are fundamental:

- the addition of matrices of the same size, called *matrix addition*,
- the subtraction of matrices of the same size, called *matrix subtraction*,
- the multiplication of a matrix by a scalar, called *scalar multiplication*,
- the interchange of the rows and columns of a matrix, called *matrix transposition*.

In what follows, we introduce these operations in operator form, that is, as functions. This serves in particular to emphasize, for each operation, by means of its domain what kind of objects the respective operation takes, and by means of its codomain what kind of object the result of the respective operation is.

9.2.1 Matrix addition

Definition 9.2. Let $A, B \in \mathbb{R}^{n\times m}$. Then the *addition* of A and B is defined as the mapping

$$+ : \mathbb{R}^{n\times m} \times \mathbb{R}^{n\times m} \to \mathbb{R}^{n\times m},\ (A, B) \mapsto +(A, B) := A + B \tag{9.5}$$

with

$$\begin{aligned} A + B &= \begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1m} \\ a_{21} & a_{22} & \cdots & a_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nm} \end{pmatrix} + \begin{pmatrix} b_{11} & b_{12} & \cdots & b_{1m} \\ b_{21} & b_{22} & \cdots & b_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ b_{n1} & b_{n2} & \cdots & b_{nm} \end{pmatrix} \\ &:= \begin{pmatrix} a_{11} + b_{11} & a_{12} + b_{12} & \cdots & a_{1m} + b_{1m} \\ a_{21} + b_{21} & a_{22} + b_{22} & \cdots & a_{2m} + b_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} + b_{n1} & a_{n2} + b_{n2} & \cdots & a_{nm} + b_{nm} \end{pmatrix}. \end{aligned} \tag{9.6}$$

•

In particular, the definition of matrix addition specifies that only matrices of the same size can be added and that the operation of matrix addition is defined elementwise.

Example

Let $A, B \in \mathbb{R}^{2\times 3}$ be defined as

$$A := \begin{pmatrix} 2 & -3 & 0 \\ 1 & 6 & 5 \end{pmatrix} \text{ and } B := \begin{pmatrix} 4 & 1 & 0 \\ -4 & 2 & 0 \end{pmatrix}. \tag{9.7}$$

Because A and B have the same size, we can add them:

$$\begin{aligned} C = A + B &= \begin{pmatrix} 2 & -3 & 0 \\ 1 & 6 & 5 \end{pmatrix} + \begin{pmatrix} 4 & 1 & 0 \\ -4 & 2 & 0 \end{pmatrix} \\ &= \begin{pmatrix} 2+4 & -3+1 & 0+0 \\ 1-4 & 6+2 & 5+0 \end{pmatrix} \\ &= \begin{pmatrix} 6 & -2 & 0 \\ -3 & 8 & 5 \end{pmatrix}. \end{aligned} \tag{9.8}$$

In **R**, one carries out the above calculation as follows.

```r
# definition
A = matrix(c(2, -3, 0,
             1,  6, 5),
           nrow  = 2,
           byrow = TRUE)
B = matrix(c( 4, 1, 0,
             -4, 2, 0),
            nrow = 2,
           byrow = TRUE)

# addition
C = A + B
print(C)
```

```
     [,1] [,2] [,3]
[1,]    6   -2    0
[2,]   -3    8    5
```

9.2.2 Matrix subtraction

The subtraction of matrices of the same size is defined analogously to addition.

Definition 9.3 (Matrix subtraction)**.** Let $A, B \in \mathbb{R}^{n \times m}$. Then the *subtraction* of A and B is defined as the mapping

$$- : \mathbb{R}^{n \times m} \times \mathbb{R}^{n \times m} \to \mathbb{R}^{n \times m},\ (A, B) \mapsto -(A, B) := A - B \tag{9.9}$$

with

$$\begin{aligned} A - B &= \begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1m} \\ a_{21} & a_{22} & \cdots & a_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nm} \end{pmatrix} - \begin{pmatrix} b_{11} & b_{12} & \cdots & b_{1m} \\ b_{21} & b_{22} & \cdots & b_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ b_{n1} & b_{n2} & \cdots & b_{nm} \end{pmatrix} \\ &:= \begin{pmatrix} a_{11} - b_{11} & a_{12} - b_{12} & \cdots & a_{1m} - b_{1m} \\ a_{21} - b_{21} & a_{22} - b_{22} & \cdots & a_{2m} - b_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} - b_{n1} & a_{n2} - b_{n2} & \cdots & a_{nm} - b_{nm} \end{pmatrix}. \end{aligned} \tag{9.10}$$

•

As with matrix addition, the definition of matrix subtraction specifies that only matrices of the same size can be subtracted from one another and that the subtraction of two equally sized matrices is defined elementwise.

Example

We can also subtract the matrices A and B defined in the example on matrix addition from one another:

$$\begin{aligned} D = A - B &= \begin{pmatrix} 2 & -3 & 0 \\ 1 & 6 & 5 \end{pmatrix} - \begin{pmatrix} 4 & 1 & 0 \\ -4 & 2 & 0 \end{pmatrix} \\ &= \begin{pmatrix} 2-4 & -3-1 & 0-0 \\ 1+4 & 6-2 & 5-0 \end{pmatrix} \\ &= \begin{pmatrix} -2 & -4 & 0 \\ 5 & 4 & 5 \end{pmatrix}. \end{aligned} \tag{9.11}$$

In **R**, one carries out this calculation as follows.

```
# subtraction
D = A - B
print(D)
```

```
     [,1] [,2] [,3]
[1,]   -2   -4    0
[2,]    5    4    5
```

9.2.3 Scalar multiplication

The *scalar multiplication* of a matrix denotes the multiplication of a scalar by a matrix.

Definition 9.4 (Scalar multiplication)**.** Let $c \in \mathbb{R}$ be a scalar and let $A \in \mathbb{R}^{n \times m}$. Then the *scalar multiplication* of c and A is defined as the mapping

$$\cdot : \mathbb{R} \times \mathbb{R}^{n \times m} \to \mathbb{R}^{n \times m},\ (c, A) \mapsto \cdot(c, A) := cA \tag{9.12}$$

with

$$cA = c \begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1m} \\ a_{21} & a_{22} & \cdots & a_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nm} \end{pmatrix} := \begin{pmatrix} ca_{11} & ca_{12} & \cdots & ca_{1m} \\ ca_{21} & ca_{22} & \cdots & ca_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ ca_{n1} & ca_{n2} & \cdots & ca_{nm} \end{pmatrix}. \tag{9.13}$$

•

With this definition, scalar multiplication is therefore defined elementwise.

Example

Let $c := -3$ and let $A \in \mathbb{R}^{4 \times 3}$ be defined as

$$A := \begin{pmatrix} 3 & 1 & 1 \\ 5 & 2 & 5 \\ 2 & 7 & 1 \\ 3 & 4 & 2 \end{pmatrix}. \tag{9.14}$$

Then

$$B := cA = -3 \begin{pmatrix} 3 & 1 & 1 \\ 5 & 2 & 5 \\ 2 & 7 & 1 \\ 3 & 4 & 2 \end{pmatrix} = \begin{pmatrix} -3 \cdot 3 & -3 \cdot 1 & -3 \cdot 1 \\ -3 \cdot 5 & -3 \cdot 2 & -3 \cdot 5 \\ -3 \cdot 2 & -3 \cdot 7 & -3 \cdot 1 \\ -3 \cdot 3 & -3 \cdot 4 & -3 \cdot 2 \end{pmatrix} = \begin{pmatrix} -9 & -3 & -3 \\ -15 & -6 & -15 \\ -6 & -21 & -3 \\ -9 & -12 & -6 \end{pmatrix}. \tag{9.15}$$

In **R**, one carries out this scalar multiplication as follows.

```
# definitions
A = matrix(c(3,1,1,
             5,2,5,
             2,7,1,
             3,4,2),
           nrow = 4,
           byrow = TRUE)
c = -3

# scalar multiplication
B = c*A
print(B)
```

```
     [,1] [,2] [,3]
[1,]   -9   -3   -3
[2,]  -15   -6  -15
[3,]   -6  -21   -3
[4,]   -9  -12   -6
```

With the help of the definition of matrix addition and scalar multiplication, it is possible to define a vector space whose elements are the real matrices. In particular, this definition also specifies the arithmetic rules for working with matrix addition and scalar multiplication.

Theorem 9.1 (Vector space of real-valued matrices)**.** *The triple* $(\mathbb{R}^{n\times m}, +, \cdot)$ *with the matrix addition and scalar multiplication defined above is a vector space. In particular, for* $A, B, C \in \mathbb{R}^{n\times m}$ *and* $r, s, t \in \mathbb{R}$*, the following arithmetic rules therefore hold:*

(1) Commutativity of addition: $A + B = B + A$.
(2) Associativity of addition: $(A + B) + C = A + (B + C)$.
(3) Existence of an identity element for addition: $\exists 0 \in \mathbb{R}^{n\times m}$ *with* $A + 0 = 0 + A = A$.
(4) Existence of inverse elements for addition: $\forall A \exists -A \in \mathbb{R}^{n\times m}$ *with* $A + (-A) = 0$.
(5) Existence of an identity element for scalar multiplication: $\exists 1 \in \mathbb{R}$ *with* $1 \cdot A = A$.
(6) Associativity of scalar multiplication: $r \cdot (s \cdot A) = (r \cdot s) \cdot A$.
(7) Distributivity with respect to matrix addition: $r \cdot (A + B) = r \cdot A + r \cdot B$.
(8) Distributivity with respect to scalar addition: $(r + s) \cdot A = r \cdot A + s \cdot A$.

∘

We omit a proof, which follows directly, with some notational effort, from the elementwise character of matrix addition and scalar multiplication as well as the arithmetic rules known from working with real numbers. The identity element of addition mentioned in the theorem is called the *zero matrix*; we will introduce a general notation for it later. The inverse elements of addition are given by

$$-A := (-a_{ij})_{1\leq i\leq n, 1\leq j\leq m} \tag{9.16}$$

and make it possible to view matrix subtraction as a special case of matrix addition.

9.2.4 Matrix transposition

Another frequently occurring basic matrix operation is the interchange of the row and column arrangement of a matrix, called *matrix transposition*.

Definition 9.5 (Matrix transposition)**.** Let $A \in \mathbb{R}^{n\times m}$. Then the *transposition* of A is defined as the mapping

$$.^T : \mathbb{R}^{n\times m} \to \mathbb{R}^{m\times n},\ A \mapsto .^T(A) := A^T \tag{9.17}$$

with

$$A^T = \begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1m} \\ a_{21} & a_{22} & \cdots & a_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nm} \end{pmatrix}^T := \begin{pmatrix} a_{11} & a_{21} & \cdots & a_{n1} \\ a_{12} & a_{22} & \cdots & a_{n2} \\ \vdots & \vdots & \ddots & \vdots \\ a_{1m} & a_{2m} & \cdots & a_{nm} \end{pmatrix}. \tag{9.18}$$

•

Thus, for $A \in \mathbb{R}^{n \times m}$, we always have $A^T \in \mathbb{R}^{m \times n}$. Furthermore, the following arithmetic rules of matrix transposition hold, as one can see from examples:

(1) For $A \in \mathbb{R}^{1 \times 1}$,

$$A^T = A. \tag{9.19}$$

(2) We have

$$\left(A^T\right)^T = A. \tag{9.20}$$

(3) We have

$$(a_{ii})_{1 \leq i \leq \min(n,m)} = (a_{ii})^T_{1 \leq i \leq \min(n,m)}. \tag{9.21}$$

The latter property of transposition states that the elements on the main diagonal of a matrix remain unchanged under transposition.

Example

Let $A \in \mathbb{R}^{2 \times 3}$ be defined by

$$A := \begin{pmatrix} 2 & 3 & 0 \\ 1 & 6 & 5 \end{pmatrix}, \tag{9.22}$$

Then $A^T \in \mathbb{R}^{3 \times 2}$ and, specifically,

$$A^T := \begin{pmatrix} 2 & 1 \\ 3 & 6 \\ 0 & 5 \end{pmatrix}. \tag{9.23}$$

Furthermore, apparently $\min(m, n) = 2$, and consequently

$$(a_{11}) = (a_{11})^T \text{ and } (a_{22}) = (a_{22})^T. \tag{9.24}$$

In **R**, one transposes a matrix as follows.

```
# definition
A = matrix(c(2,3,0,
             1,6,5),
           nrow = 2,
           byrow = TRUE)
print(A)
```

```
     [,1] [,2] [,3]
[1,]    2    3    0
[2,]    1    6    5
```

```
# transposition
AT = t(A)
print(AT)
```

```
     [,1] [,2]
[1,]    2    1
[2,]    3    6
[3,]    0    5
```

Finally, in connection with matrix addition, matrix subtraction, and scalar multiplication, the following arithmetic rules hold, as one can see from examples:

(1) For $A, B \in \mathbb{R}^{n \times m}$,

$$(A + B)^T = A^T + B^T. \tag{9.25}$$

(2) For $A, B \in \mathbb{R}^{n \times m}$,

$$(A - B)^T = A^T - B^T. \tag{9.26}$$

(3) For $c \in \mathbb{R}$ and $A \in \mathbb{R}^{n \times m}$,

$$(cA)^T = cA^T. \tag{9.27}$$

9.3 Matrix multiplication

Matrix multiplication is the central operation when computing with matrices. It is defined as follows.

Definition 9.6 (Matrix multiplication)**.** Let $A \in \mathbb{R}^{n\times m}$ and $B \in \mathbb{R}^{m\times k}$. Then the *matrix multiplication* of A and B is defined as the mapping

$$\cdot : \mathbb{R}^{n\times m} \times \mathbb{R}^{m\times k} \to \mathbb{R}^{n\times k},\ (A, B) \mapsto \cdot(A, B) := AB \tag{9.28}$$

with

$$\begin{aligned} AB &= \begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1m} \\ a_{21} & a_{22} & \cdots & a_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nm} \end{pmatrix} \begin{pmatrix} b_{11} & b_{12} & \cdots & b_{1k} \\ b_{21} & b_{22} & \cdots & b_{2k} \\ \vdots & \vdots & \ddots & \vdots \\ b_{m1} & b_{m2} & \cdots & b_{mk} \end{pmatrix} \\ &:= \begin{pmatrix} \sum_{i=1}^{m} a_{1i}b_{i1} & \sum_{i=1}^{m} a_{1i}b_{i2} & \cdots & \sum_{i=1}^{m} a_{1i}b_{ik} \\ \sum_{i=1}^{m} a_{2i}b_{i1} & \sum_{i=1}^{m} a_{2i}b_{i2} & \cdots & \sum_{i=1}^{m} a_{2i}b_{ik} \\ \vdots & \vdots & \ddots & \vdots \\ \sum_{i=1}^{m} a_{ni}b_{i1} & \sum_{i=1}^{m} a_{ni}b_{i2} & \cdots & \sum_{i=1}^{m} a_{ni}b_{ik} \end{pmatrix} \\ &= \left(\sum_{i=1}^{m} a_{ji}b_{il} \right)_{1\le j\le n, 1\le l\le k} \end{aligned} \tag{9.29}$$

•

The matrix product AB is therefore defined only if A has exactly as many columns as B has rows. Informally, the following mnemonic applies to the involved matrix sizes:

$$(n \times m)(m \times k) = (n \times k). \tag{9.30}$$

The entry $(AB)_{ij}$ in AB corresponds to the sum of the multiplied ith row of A and jth column of B. To compute $(AB)_{ij}$ for $i = 1, ..., n$ and $j = 1, ..., k$, one can therefore proceed mentally as follows:

(1) Place the transpose of the ith row of A over the jth column of B.
(2) Because A has exactly m columns and B has exactly m rows, each element of the row from A has a corresponding element in the column of B.
(3) Multiply the corresponding elements with one another.
(4) The sum of these products is then the entry with index ij in AB.

Example

Let $A \in \mathbb{R}^{2\times 3}$ and $B \in \mathbb{R}^{3\times 2}$ be defined as

$$A := \begin{pmatrix} 2 & -3 & 0 \\ 1 & 6 & 5 \end{pmatrix} \text{ and } B := \begin{pmatrix} 4 & 2 \\ -1 & 0 \\ 1 & 3 \end{pmatrix}. \tag{9.31}$$

We want to compute $C := AB$ and $D := BA$. With $n = 2, m = 3$, and $k = 2$, we already know that $C \in \mathbb{R}^{2\times 2}$ and $D \in \mathbb{R}^{3\times 3}$, because

$$(2 \times 3)(3 \times 2) = (2 \times 2) \tag{9.32}$$

and

$$(3 \times 2)(2 \times 3) = (3 \times 3). \tag{9.33}$$

Thus here surely $AB \neq BA$. For C, we obtain

$$\begin{aligned}
C &= AB \\
&= \begin{pmatrix} 2 & -3 & 0 \\ 1 & 6 & 5 \end{pmatrix} \begin{pmatrix} 4 & 2 \\ -1 & 0 \\ 1 & 3 \end{pmatrix} \\
&= \begin{pmatrix} 2 \cdot 4 + (-3) \cdot (-1) + 0 \cdot 1 & 2 \cdot 2 + (-3) \cdot 0 + 0 \cdot 3 \\ 1 \cdot 4 + 6 \cdot (-1) + 5 \cdot 1 & 1 \cdot 2 + 6 \cdot 0 + 5 \cdot 3 \end{pmatrix} \\
&= \begin{pmatrix} 8 + 3 + 0 & 4 + 0 + 0 \\ 4 - 6 + 5 & 2 + 0 + 15 \end{pmatrix} \\
&= \begin{pmatrix} 11 & 4 \\ 3 & 17 \end{pmatrix}.
\end{aligned} \tag{9.34}$$

In **R**, the `%*%` operator is used for matrix multiplication.

```
# definitions
A = matrix(c(2,-3,0,
             1, 6,5),
           nrow  = 2,
           byrow = TRUE)
B = matrix(c( 4,2,
             -1,0,
              1,3),
           nrow  = 3,
           byrow = TRUE)

# matrix multiplication
C = A %*% B
print(C)
```

```
     [,1] [,2]
[1,]   11    4
[2,]    3   17
```

For D, we further obtain

$$\begin{aligned}
D &= BA \\
&= \begin{pmatrix} 4 & 2 \\ -1 & 0 \\ 1 & 3 \end{pmatrix} \begin{pmatrix} 2 & -3 & 0 \\ 1 & 6 & 5 \end{pmatrix} \\
&= \begin{pmatrix} 4 \cdot 2 + 2 \cdot 1 & 4 \cdot (-3) + 2 \cdot 6 & 4 \cdot 0 + 2 \cdot 5 \\ (-1) \cdot 2 + 0 \cdot 1 & (-1) \cdot (-3) + 0 \cdot 6 & (-1) \cdot 0 + 0 \cdot 5 \\ 1 \cdot 2 + 3 \cdot 1 & 1 \cdot (-3) + 3 \cdot 6 & 1 \cdot 0 + 3 \cdot 5 \end{pmatrix} \\
&= \begin{pmatrix} 8 + 2 & -12 + 12 & 0 + 5 \\ -2 + 0 & 3 + 0 & 0 + 0 \\ 2 + 3 & -3 + 18 & 0 + 15 \end{pmatrix} \\
&= \begin{pmatrix} 10 & 0 & 10 \\ -2 & 3 & 0 \\ 5 & 15 & 15 \end{pmatrix}
\end{aligned} \tag{9.35}$$

In **R**, one checks this calculation as follows.

```
# definitions
A = matrix(c(2,-3,0,
             1, 6,5),
           nrow  = 2,
           byrow = TRUE)
B = matrix(c( 4,2,
             -1,0,
              1,3),
           nrow  = 3,
           byrow = TRUE)

# matrix multiplication
D = B %*% A
print(D)
```

```
     [,1] [,2] [,3]
[1,]   10    0   10
[2,]   -2    3    0
[3,]    5   15   15
```

However, if a matrix multiplication is not defined because of incompatible matrix sizes, then it also cannot be evaluated numerically.

```
# example of an undefined matrix multiplication
E = t(A) %*% B      # (3 x 2)(3 x 2)
```

```
Error in `t(A) %*% B`:
! nicht passende Argumente
```

The following theorem, which we do not prove, establishes the relation between the inner product of two vectors and the multiplication of two matrices. This essentially follows from the identification of $\mathbb{R}^n$ and $\mathbb{R}^{n\times 1}$ and from the fact that, by definition, the entry $(AB)_{ij}$ in the product of $A \in \mathbb{R}^{n\times m}$ and $B \in \mathbb{R}^{m\times k}$ corresponds to the vector inner product of the ith column of A^T and the jth column of B.

Theorem 9.2 (Matrix multiplication and vector inner product)**.** *Let $x, y \in \mathbb{R}^n$. Then*

$$\langle x, y\rangle = x^T y. \tag{9.36}$$

Furthermore, for $A \in \mathbb{R}^{n\times m}$ and $i = 1, ..., n$, let

$$\bar{a}_i := (a_{ij})_{1\leq j\leq m} \in \mathbb{R}^m \tag{9.37}$$

be the columns of A^T, and for $B \in \mathbb{R}^{m\times k}$ and $j = 1, ..., k$, let

$$\bar{b}_j := (b_{ij})_{1\leq i\leq m} \in \mathbb{R}^m \tag{9.38}$$

be the columns of B, that is,

$$A^T = \begin{pmatrix} \bar{a}_1 & \bar{a}_2 & \cdots & \bar{a}_n \end{pmatrix} \in \mathbb{R}^{m\times n} \text{ and } B = \begin{pmatrix} \bar{b}_1 & \bar{b}_2 & \cdots & \bar{b}_k \end{pmatrix} \in \mathbb{R}^{m\times k}. \tag{9.39}$$

Then

$$AB = \left(\langle \bar{a}_i, \bar{b}_j\rangle\right)_{1\leq i\leq n, 1\leq j\leq k}. \tag{9.40}$$

∘

9.3.1 Arithmetic rules for matrix multiplication

In the following, we collect a few basic arithmetic rules for matrix multiplication, especially also in combination with other matrix operations.

For proofs of the following two theorems on associativity and distributivity, which essentially follow from the corresponding arithmetic rules for sums and products of real numbers, we refer to the advanced literature.

Theorem 9.3 (Associativity)**.** *Let $A \in \mathbb{R}^{n\times m}$, $B \in \mathbb{R}^{m\times k}$, $C \in \mathbb{R}^{k\times p}$, and $c \in \mathbb{R}$. Then:*

(1) Matrix multiplication is associative, that is,

$$A(BC) = (AB)C. \tag{9.41}$$

(2) The combination of matrix multiplication and scalar multiplication is associative,

$$c(AB) = (cA)B = A(cB). \tag{9.42}$$

∘

The associativity of matrix multiplication and scalar multiplication can be seen easily by considering the j, lth element of $c(AB)$, $(cA)B$, and $A(cB)$:

$$c\left(\sum_{i=1}^{m} a_{ji}b_{il}\right) = \sum_{i=1}^{m} \left(ca_{ji}\right) b_{il} = \sum_{i=1}^{m} a_{ji}\left(cb_{il}\right). \tag{9.43}$$

Theorem 9.4 (Distributivity)**.** *Let $A \in \mathbb{R}^{n\times m}$, $B \in \mathbb{R}^{n\times m}$, and $C \in \mathbb{R}^{m\times p}$. Then*

$$(A + B)C = AC + BC \tag{9.44}$$

and

$$C^T(A + B) = C^TA + C^TB \tag{9.45}$$

∘

In contrast to the commutativity of multiplication of real numbers, matrix multiplication is in general not commutative.

Theorem 9.5 (Noncommutativity)**.** *Let $A \in \mathbb{R}^{n\times m}$ and $B \in \mathbb{R}^{m\times p}$. In general,*

$$AB \neq BA. \tag{9.46}$$

∘

Proof. If $p \neq n$, then BA is not defined, so we consider only the case $p = n$. We show by giving a counterexample with $A, B \in \mathbb{R}^{2\times 2}$ that in general $AB = BA$ does *not* hold. Let

$$A \coloneqq \begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix} \text{ and } B \coloneqq \begin{pmatrix} 0 & 0 \\ 1 & 0 \end{pmatrix}. \tag{9.47}$$

Then

$$AB = \begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix}\begin{pmatrix} 0 & 0 \\ 1 & 0 \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix} \neq \begin{pmatrix} 0 & 0 \\ 0 & 1 \end{pmatrix} = \begin{pmatrix} 0 & 0 \\ 1 & 0 \end{pmatrix}\begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix} = BA. \tag{9.48}$$

□

Theorem 9.6 (Combination of matrix multiplication and transposition)**.** *Let $A \in \mathbb{R}^{m \times n}$ and $B \in \mathbb{R}^{n \times k}$. Then*

$$(AB)^T = B^T A^T. \tag{9.49}$$

∘

Proof. A proof is obtained as follows:

$$\begin{aligned}
(AB)^T &= \left(\left(\sum_{i=1}^{n} a_{ji} b_{il} \right)_{1 \leq j \leq m, 1 \leq l \leq k} \right)^T \\
&= \left(\sum_{i=1}^{n} a_{li} b_{ij} \right)_{1 \leq j \leq k, 1 \leq l \leq m} \\
&= \left(\sum_{i=1}^{n} b_{ij} a_{li} \right)_{1 \leq j \leq k, 1 \leq l \leq m} \\
&= B^T A^T.
\end{aligned} \tag{9.50}$$

□

9.4 Matrix inversion

To motivate the concept of the inverse matrix, we first consider the problem of *solving a system of linear equations*. To this end, let $A \in \mathbb{R}^{n \times n}$, $x \in \mathbb{R}^n$, and $b \in \mathbb{R}^n$, and suppose that

$$Ax = b. \tag{9.51}$$

Let A and b be known, and let x be unknown. Specifically, for example, let

$$A := \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} \text{ and } b := \begin{pmatrix} 5 \\ 11 \end{pmatrix}. \tag{9.52}$$

Then the following system of linear equations with two equations and two unknowns is given:

$$Ax = b \Leftrightarrow \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} 5 \\ 11 \end{pmatrix} \Leftrightarrow \begin{matrix} 1x_1 + 2x_2 & = 5 \\ 3x_1 + 4x_2 & = 11 \end{matrix}. \tag{9.53}$$

The goal of solving systems of linear equations is, as is well known, to find those x for which the system is satisfied. To introduce the concept of the inverse matrix of A in this context, we simplify the situation further. We assume that $A = a$ is a 1×1 matrix, that is, a scalar, and likewise that x and b are scalars, so that for $a, x, b \in \mathbb{R}$ we have the equation

$$ax = b \tag{9.54}$$

To solve this equation for x, one would naturally multiply both sides of the equation by the *multiplicative inverse* of a, where the *multiplicative inverse* of a denotes the value which, when multiplied by a, yields 1. As is well known, this is given by

$$a^{-1} = \frac{1}{a} \tag{9.55}$$

Then

$$ax = b \Leftrightarrow a^{-1}ax = a^{-1}b \Leftrightarrow 1 \cdot x = a^{-1}b \Leftrightarrow x = \frac{b}{a}. \tag{9.56}$$

For example, quite concretely,

$$2x = 6 \Leftrightarrow 2^{-1}2x = 2^{-1}6 \Leftrightarrow \frac{1}{2}2x = \frac{1}{2}6 \Leftrightarrow x = 3. \tag{9.57}$$

Analogously to the case in which all matrices in $Ax = b$ are scalars, in the case of a system of linear equations one would like to be able to multiply both sides of the equation by the *multiplicative inverse* A^{-1} of A, so that an equation of the form

$$A^{-1}A = \text{"1"}. \tag{9.58}$$

results. Then one would have

$$Ax = b \Leftrightarrow A^{-1}Ax = A^{-1}b \Leftrightarrow x = A^{-1}b. \tag{9.59}$$

This intuitive idea of the multiplicative inverse of a matrix A is formalized below under the concept of the *inverse matrix*. To this end, we first need the concept of the *identity matrix*.

Definition 9.7 (Identity matrix)**.** The matrix

$$I_n := (a_{ij})_{1 \leq i \leq n, 1 \leq j \leq n} \in \mathbb{R}^{n \times n} := \begin{pmatrix} 1 & 0 & \cdots & 0 \\ 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & 1 \end{pmatrix} \tag{9.60}$$

with $a_{ij} = 1$ for $i = j$ and $a_{ij} = 0$ for $i \neq j$ is called the *n-dimensional identity matrix*.

•

In **R**, I_n is generated with the command `diag(n)`. For matrix multiplication, the identity matrix is the analogue of the 1 in the multiplication of real numbers. This is the statement of the following theorem.

Theorem 9.7 (Identity element of matrix multiplication)**.** *I_n is the identity element of matrix multiplication, that is, for $A \in \mathbb{R}^{n \times m}$ we have*

$$I_n A = A \text{ and } AI_m = A. \tag{9.61}$$

∘

Proof. Let $B = (b_{ij}) = I_n A \in \mathbb{R}^{n \times m}$. Then, for all $1 \leq i \leq n$ and all $1 \leq j \leq m$,

$$b_{ij} = 0 \cdot a_{1j} + 0 \cdot a_{2j} + \cdots + 0 \cdot a_{i-1,j} + 1 \cdot a_{ij} + \cdots + 0 \cdot a_{i+1,j} + 0 \cdot a_{nj} = a_{ij}. \tag{9.62}$$

The proof for AI_m is analogous. □

With the concept of the identity matrix, we can now define the concepts of the inverse matrix and the invertible matrix:

Definition 9.8 (Invertible matrix and inverse matrix)**.** A square matrix $A \in \mathbb{R}^{n \times n}$ is called *invertible* if there exists a square matrix $A^{-1} \in \mathbb{R}^{n \times n}$ such that

$$A^{-1}A = AA^{-1} = I_n \tag{9.63}$$

The matrix A^{-1} is called the *inverse matrix of* A.

•

Note that the concepts of the inverse matrix and invertibility refer *only* to square matrices. In particular, square matrices can be invertible, but need not be (systems of linear equations can therefore have solutions, but need not). Non-invertible matrices are also called *singular* matrices, and invertible matrices are sometimes called *nonsingular* matrices. Finally, note that Definition 9.8 merely states what an inverse matrix is, but not how to compute it.

Example of an invertible matrix

The matrix

$$A := \begin{pmatrix} 2.0 & 1.0 \\ 3.0 & 4.0 \end{pmatrix} \tag{9.64}$$

is invertible, and its inverse matrix is given by

$$A^{-1} = \begin{pmatrix} 0.8 & -0.2 \\ -0.6 & 0.4 \end{pmatrix}, \tag{9.65}$$

because

$$\begin{pmatrix} 2.0 & 1.0 \\ 3.0 & 4.0 \end{pmatrix} \begin{pmatrix} 0.8 & -0.2 \\ -0.6 & 0.4 \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} = \begin{pmatrix} 0.8 & -0.2 \\ -0.6 & 0.4 \end{pmatrix} \begin{pmatrix} 2.0 & 1.0 \\ 3.0 & 4.0 \end{pmatrix}, \tag{9.66}$$

as can be verified by direct calculation.

Example of a non-invertible matrix

The matrix

$$B := \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix} \tag{9.67}$$

is not invertible, because if B were invertible, then there would exist

$$\begin{pmatrix} a & b \\ c & d \end{pmatrix} \tag{9.68}$$

with

$$\begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix} \begin{pmatrix} a & b \\ c & d \end{pmatrix} = \begin{pmatrix} a & b \\ 0 & 0 \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}. \tag{9.69}$$

But this would mean that $0 = 1$ in $\mathbb{R}$, which is a contradiction. Therefore, B cannot be invertible.

On computing inverse matrices

2×2 up to about 5×5 matrices can in principle be inverted by hand; linear algebra provides various methods for this. We omit an introduction to matrix inversion by hand here, since matrices are inverted numerically by default in applications. Numerical matrix inversion is also a large field of research in numerical mathematics, which provides a variety

of algorithms for this purpose. In **R**, matrices are inverted by default with the function `solve()`, in analogy to solving systems of linear equations. For the above example of an invertible matrix, this yields the following **R** code.

```
# definition
A = matrix(c(2,1,
             3,4),
           nrow  = 2,
           byrow = TRUE)

# computing A^{-1}
print(solve(A))
```

```
     [,1] [,2]
[1,]  0.8 -0.2
[2,] -0.6  0.4
```

```
# checking the properties of an inverse matrix
print(solve(A) %*% A)
```

```
     [,1] [,2]
[1,]    1    0
[2,]    0    1
```

```
# the reverse calculation yields small rounding errors
print(A %*% solve(A))
```

```
     [,1]          [,2]
[1,]    1 -5.551115e-17
[2,]    0  1.000000e+00
```

Non-invertible matrices are, of course, also numerically non-invertible, as the following error message in **R** demonstrates for the above example of a non-invertible matrix.

```
B = matrix(c(1,0,
             0,0),
           nrow  = 2,
           byrow = TRUE)
solve(B)
```

```
Error in `solve.default()`:
! Lapackroutine dgesv: System ist genau singulär: U[2,2] = 0
```

9.5 Determinants

The determinant is a versatile measure of a square matrix. For understanding *eigenanalysis* and *matrix decomposition*, the concept of the determinant is fundamental in the context of the *characteristic polynomial*.

In general, a determinant is a nonlinear mapping of the form

$$|\cdot| : \mathbb{R}^{n \times n} \to \mathbb{R}, A \mapsto |A|, \tag{9.70}$$

that is, a determinant assigns the real number $|A|$ to a square matrix A. The number $|A|$ is determined recursively by the following definition.

Definition 9.9 (Determinant)**.** For $A = (a_{ij})_{1 \leq i,j \leq n} \in \mathbb{R}^{n \times n}$ with $n > 1$, let $A_{ij} \in \mathbb{R}^{n-1 \times n-1}$ be the matrix obtained from A by removing the ith row and the jth column. Then the number

$$|A| \coloneqq a_{11} \qquad \text{for } n = 1 \tag{9.71}$$

$$|A| \coloneqq \sum_{j=1}^{n} a_{1j}(-1)^{1+j} \det\left(A_{1j}\right) \text{ for } n > 1 \tag{9.72}$$

is called the *determinant of A*.

•

The definition thus successively reduces the determination of the determinant of a square matrix, by deleting rows and columns, to the determinant of a 1×1 matrix, which is given by its only element. For

$$A \coloneqq \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix} \in \mathbb{R}^{3 \times 3} \tag{9.73}$$

we obtain, for example, the following matrices of the form $A_{ij} \in \mathbb{R}^{3-1 \times 3-1}$:

$$A_{11} = \begin{pmatrix} 5 & 6 \\ 8 & 9 \end{pmatrix}, A_{12} = \begin{pmatrix} 4 & 6 \\ 7 & 9 \end{pmatrix}, A_{21} = \begin{pmatrix} 2 & 3 \\ 8 & 9 \end{pmatrix}, A_{22} = \begin{pmatrix} 1 & 3 \\ 7 & 9 \end{pmatrix}. \tag{9.74}$$

For computing determinants of two- and three-dimensional square matrices, there are direct, non-recursive arithmetic rules, which are recorded in the following theorem.

Theorem 9.8 (Determinants of two- and three-dimensional matrices)**.**

Let $A = (a_{ij})_{1 \leq i,j \leq 2} \in \mathbb{R}^{2 \times 2}$*. Then*

$$|A| = a_{11}a_{22} - a_{12}a_{21}. \tag{9.75}$$

Let $A = (a_{ij})_{1 \leq i,j \leq 3} \in \mathbb{R}^{3 \times 3}$*. Then*

$$|A| = a_{11}a_{22}a_{33} + a_{12}a_{23}a_{31} + a_{13}a_{21}a_{32} - a_{12}a_{21}a_{33} - a_{11}a_{23}a_{32} - a_{13}a_{22}a_{31}. \tag{9.76}$$

∘

Proof. For $A \in \mathbb{R}^{2 \times 2}$, by definition

$$\begin{aligned} |A| &= \sum_{j=1}^{n} a_{1j}(-1)^{1+j}|A_{1j}| \\ &= a_{11}(-1)^{1+1}|A_{11}| + a_{12}(-1)^{1+2}|A_{12}| \\ &= a_{11}|(a_{22})| - a_{12}|(a_{21})| \\ &= a_{11}a_{22} - a_{12}a_{21}. \end{aligned} \tag{9.77}$$

For $A \in \mathbb{R}^{3\times 3}$, by definition and with the formula for determinants of 2×2 matrices,

$$\begin{aligned}
|A| &= \sum_{j=1}^{n} a_{1j}(-1)^{1+j}|A_{1j}| \\
&= a_{11}(-1)^{1+1}|A_{11}| + a_{12}(-1)^{1+2}|A_{12}| + a_{13}(-1)^{1+3}|A_{13}| \\
&= a_{11}|A_{11}| - a_{12}|A_{12}| + a_{13}|A_{13}| \\
&= a_{11}\left|\begin{pmatrix} a_{22} & a_{23} \\ a_{32} & a_{33} \end{pmatrix}\right| - a_{12}\left|\begin{pmatrix} a_{21} & a_{23} \\ a_{31} & a_{33} \end{pmatrix}\right| + a_{13}\left|\begin{pmatrix} a_{21} & a_{22} \\ a_{31} & a_{32} \end{pmatrix}\right| \\
&= a_{11}(a_{22}a_{33} - a_{23}a_{32}) - a_{12}(a_{21}a_{33} - a_{23}a_{31}) + a_{13}(a_{21}a_{32} - a_{22}a_{31}) \\
&= a_{11}a_{22}a_{33} - a_{11}a_{23}a_{32} - a_{12}a_{21}a_{33} + a_{12}a_{23}a_{31} + a_{13}a_{21}a_{32} - a_{13}a_{22}a_{31} \\
&= a_{11}a_{22}a_{33} + a_{12}a_{23}a_{31} + a_{13}a_{21}a_{32} - a_{12}a_{21}a_{33} - a_{11}a_{23}a_{32} - a_{13}a_{22}a_{31}.
\end{aligned} \tag{9.78}$$

□

For determining the determinants of 2×2 and 3×3 matrices, the so-called *rule of Sarrus* therefore applies:

"sum of the products on the diagonals minus sum of the products on the counterdiagonals."

For 3×3 matrices, the mnemonic refers to the scheme

$$\left(\begin{array}{ccc|cc} a_{11} & a_{12} & a_{13} & a_{11} & a_{12} \\ a_{21} & a_{22} & a_{23} & a_{21} & a_{22} \\ a_{31} & a_{32} & a_{33} & a_{31} & a_{32} \end{array}\right). \tag{9.79}$$

Examples for determinants of 2×2 and 3×3 matrices

Let

$$A := \begin{pmatrix} 2 & 1 \\ 3 & 4 \end{pmatrix}, B := \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix} \text{ and } C := \begin{pmatrix} 2 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 3 \end{pmatrix} \tag{9.80}$$

Then

$$|A| = 2 \cdot 4 - 1 \cdot 3 = 8 - 3 = 5 \tag{9.81}$$

and

$$|B| = 1 \cdot 0 - 0 \cdot 0 = 0 - 0 = 0 \tag{9.82}$$

and

$$|C| = 2 \cdot 1 \cdot 3 + 0 \cdot 0 \cdot 0 + 0 \cdot 0 \cdot 0 - 0 \cdot 0 \cdot 3 - 0 \cdot 0 \cdot 0 - 0 \cdot 1 \cdot 0 = 2 \cdot 1 \cdot 3 = 6. \tag{9.83}$$

In **R**, one checks this with the `det()` function as follows.

```
# matrix definition and determinant calculation
A = matrix(c(2,1,
             3,4),
           nrow = 2,
           byrow = TRUE)
det(A)
```

```
[1] 5
```

```
# matrix definition and determinant calculation
B = matrix(c(1,0,
             0,0),
           nrow = 2,
           byrow = TRUE)
det(B)
```

```
[1] 0
```

```r
# matrix definition and determinant calculation
C = matrix(c(2,0,0,
             0,1,0,
             0,0,3),
           nrow = 3,
           byrow = TRUE)
det(C)
```

```
[1] 6
```

There are numerous arithmetic rules for determinants in conjunction with matrix multiplication and matrix inversion. Without proof, we collect these in the following theorem.

Theorem 9.9 (Arithmetic rules for determinants)**.**

(Multiplication theorem for determinants.) For $A, B \in \mathbb{R}^{n\times n}$*,*

$$|AB| = |A||B|. \tag{9.84}$$

(Transposition.) For $A \in \mathbb{R}^{n\times n}$*,*

$$|A| = \left|A^T\right|. \tag{9.85}$$

(Inversion.) For an invertible matrix $A \in \mathbb{R}^{n\times n}$*,*

$$\left|A^{-1}\right| = \frac{1}{|A|}. \tag{9.86}$$

(Triangular matrices.) For matrices $A = (a_{ij})_{1\leq i,j\leq n} \in \mathbb{R}^{n\times n}$ *with* $a_{ij} = 0$ *for* $i > j$ *or* $a_{ij} = 0$ *for* $j > i$*,*

$$|A| = \prod_{i=1}^{n} a_{ii}. \tag{9.87}$$

∘

The following very deep theorem, which we do not want to prove completely, provides a way to determine from the determinant of a square matrix whether it is invertible.

Theorem 9.10. $A \in \mathbb{R}^{n\times n}$ *is invertible if and only if* $|A| \neq 0$*. Thus,*

$$A \text{ is invertible} \Leftrightarrow |A| \neq 0 \text{ and } A \text{ is not invertible} \Leftrightarrow |A| = 0. \tag{9.88}$$

∘

Proof. We only indicate a proof and show that invertibility of A implies that $|A|$ cannot be zero. Suppose, then, that A is invertible. Then there exists a matrix B with $AB = I_n$, and by the multiplication theorem for determinants it follows that

$$|AB| = |A||B| = |I_n| = 1. \tag{9.89}$$

Thus, $|A| = 0$ cannot hold, because otherwise $0 = 1$. □

Visual Intuition

The abstract concept of the determinant of a square matrix can be illustrated somewhat with the concept of a vector space. To this end, let $a_1, ..., a_n \in \mathbb{R}^n$ be the columns of $A \in \mathbb{R}^{n \times n}$. Then (as we do not prove) $|A|$ corresponds to the signed volume of the parallelotope spanned by $a_1, ..., a_n \in \mathbb{R}^n$. To illustrate this visually, we consider the matrices

$$A_1 = \begin{pmatrix} 3 & 1 \\ 1 & 2 \end{pmatrix}, A_2 = \begin{pmatrix} 2 & 0 \\ 0 & 2 \end{pmatrix}, A_3 = \begin{pmatrix} 2 & 2 \\ 2 & 2 \end{pmatrix} \tag{9.90}$$

with the respective determinants

$$|A_1| = 3 \cdot 2 - 1 \cdot 1 = 5, \quad |A_2| = 2 \cdot 2 - 0 \cdot 0 = 4, \quad |A_3| = 2 \cdot 2 - 2 \cdot 2 = 0. \tag{9.91}$$

Figure 9.1 visualizes the corresponding intuition.

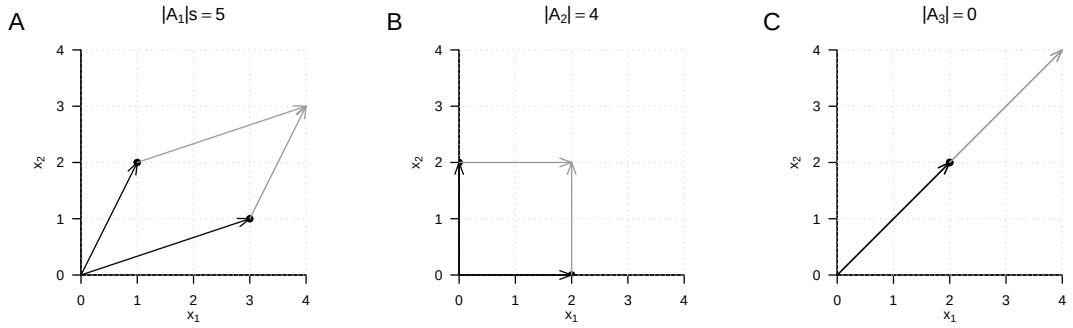


Figure 9.1 Determinants as parallelotope volumes.

9.6 Special matrices

In this section, we collect several frequently occurring types of matrices and their properties. For proofs of the vast majority of properties, we refer to the advanced literature.

9.6.1 Identity matrices

We have already encountered the identity matrix and the unit vectors. We collect them here once more in a joint definition.

Definition 9.10 (Identity matrix and unit vectors)**.** We denote the *identity matrix* by

$$I_n := (i_{jk})_{1 \leq j \leq n, 1 \leq k \leq n} \in \mathbb{R}^{n \times n} \text{ with } i_{jk} = 1 \text{ for } j = k \text{ and } i_{jk} = 0 \text{ for } j \neq k. \tag{9.92}$$

We denote the *unit vectors* $e_i, i = 1, ..., n$ by

$$e_i := (e_{i_j})_{1 \leq j \leq n} \in \mathbb{R}^n \text{ with } e_{i_j} = 1 \text{ for } i = j \text{ and } e_{i_j} = 0 \text{ for } i \neq j. \tag{9.93}$$

•

The identity matrix I_n consists only of zeros and diagonal elements equal to one; the unit vectors consist only of zeros and one single 1 in the respectively indexed component. We have

$$I_n = \begin{pmatrix} e_1 & \cdots & e_n \end{pmatrix} \in \mathbb{R}^{n \times n} \tag{9.94}$$

For $n = 3$, for example,

$$I_3 = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} \text{ and } e_1 = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}, e_2 = \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix}, e_3 = \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix}. \tag{9.95}$$

Furthermore, as is well known, for the unit vectors and $1 \leq i, j \leq n$,

$$e_i^T e_j = 0 \text{ for } i \neq j, e_i^T e_i = 1 \text{ and } e_i^T v = v^T e_i = v_i \text{ for } v \in \mathbb{R}^n. \tag{9.96}$$

9.6.2 One matrices and zero matrices

Definition 9.11 (Zero matrices, zero vectors, one matrices, one vectors)**.** We denote *zero matrices* and *zero vectors* by

$$0_{nm} := (0)_{1 \leq i \leq n, 1 \leq j \leq m} \in \mathbb{R}^{n \times m} \text{ and } 0_n := (0)_{1 \leq i \leq n} \in \mathbb{R}^n. \tag{9.97}$$

We denote *one matrices* and *one vectors* by

$$1_{nm} := (1)_{1 \leq i \leq n, 1 \leq j \leq m} \in \mathbb{R}^{n \times m} \text{ and } 1_n := (1)_{1 \leq i \leq n} \in \mathbb{R}^n. \tag{9.98}$$

•

0_{nm} and 0_n therefore consist only of zeros, and 1_{nm} and 1_n consist only of ones. For example,

$$0_{32} = \begin{pmatrix} 0 & 0 \\ 0 & 0 \\ 0 & 0 \end{pmatrix}, 0_3 = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix}, 1_{32} = \begin{pmatrix} 1 & 1 \\ 1 & 1 \\ 1 & 1 \end{pmatrix} \text{ and } 1_3 = \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix}. \tag{9.99}$$

Furthermore, for example,

$$0_n 0_n^T = 0_{nn} \text{ and } 1_n 1_n^T = 1_{nn}, \tag{9.100}$$

as can be verified by direct calculation.

9.6.3 Diagonal matrices

Definition 9.12 (Diagonal matrix)**.** A matrix $D \in \mathbb{R}^{n \times m}$ is called a *diagonal matrix* if $d_{ij} = 0$ for $1 \leq i \leq n, 1 \leq j \leq m$ with $i \neq j$.

•

A square diagonal matrix $D \in \mathbb{R}^{n \times n}$ with diagonal elements $d_1, ..., d_n \in \mathbb{R}$ is also written as

$$D = \text{diag}(d_1, ..., d_n). \tag{9.101}$$

For example,

$$D := \text{diag}(1, 2, 3) = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 3 \end{pmatrix} \tag{9.102}$$

and for $\sigma^2 \in \mathbb{R}$,

$$\Sigma = \text{diag}(\sigma^2, \sigma^2, \sigma^2) = \begin{pmatrix} \sigma^2 & 0 & 0 \\ 0 & \sigma^2 & 0 \\ 0 & 0 & \sigma^2 \end{pmatrix} = \sigma^2 I_3. \tag{9.103}$$

In the following theorem, we collect a few important properties of square diagonal matrices.

Theorem 9.11 (Properties of square diagonal matrices)**.**

(Determinant.) Let $D := diag(d_1, ..., d_n) \in \mathbb{R}^{n \times n}$ be a square diagonal matrix. Then

$$|D| = \prod_{i=1}^{n} d_i. \tag{9.104}$$

∘

9.6.4 Symmetric matrices

Symmetric matrices are square matrices that remain unchanged under transposition:

Definition 9.13. A matrix $S \in \mathbb{R}^{n \times n}$ is called *symmetric* if $S^T = S$.

•

An example of a symmetric matrix is

$$S := \begin{pmatrix} 1 & 2 & 3 \\ 2 & 1 & 2 \\ 3 & 2 & 1 \end{pmatrix}. \tag{9.105}$$

In the following theorem, we collect a few important properties of symmetric matrices.

Theorem 9.12 (Properties of symmetric matrices)**.**

(Summation.) Let $S_1 \in \mathbb{R}^{n \times n}$ and $S_2 \in \mathbb{R}^{n \times n}$ be symmetric matrices. Then

$$S_1 + S_2 = (S_1 + S_2)^T. \tag{9.106}$$

(Inverse.) Let S be an invertible symmetric matrix and let S^{-1} be its inverse. Then S^{-1} is also a symmetric matrix, that is,

$$\left(S^{-1}\right)^T = S^{-1}. \tag{9.107}$$

∘

9.6.5 Orthogonal matrices

Definition 9.14. A matrix $Q \in \mathbb{R}^{n \times n}$ is called *orthogonal* if $Q^T Q = I_n$.

•

The columns of an orthogonal matrix are therefore pairwise orthogonal. For

$$Q = \begin{pmatrix} q_1 & \cdots & q_n \end{pmatrix} \text{ with } q_i \in \mathbb{R}^n \text{ for } 1 \leq i \leq n, \tag{9.108}$$

we have

$$q_i^T q_j = 0 \text{ for } i \neq j \text{ and } q_i^T q_j = 1 \text{ for } i = j \text{ with } 1 \leq i, j \leq n. \tag{9.109}$$

Theorem 9.13 (Properties of orthogonal matrices)**.** *Let $Q \in \mathbb{R}^{n \times n}$ be an orthogonal matrix. Then the following properties of Q hold.*

(Inverse.) The inverse of Q is Q^T, that is,

$$Q^{-1} = Q^T. \tag{9.110}$$

(Transposition.) The rows of Q are orthonormal, that is,

$$QQ^T = I_n \tag{9.111}$$

◦

Proof. (Inverse.) Under the assumption that Q^{-1} exists, we have

$$Q^T Q = I_n \Leftrightarrow Q^T Q Q^{-1} = I_n Q^{-1} \Leftrightarrow Q^{-1} = Q^T. \tag{9.112}$$

(Transposition.) We have

$$Q^T Q = I_n \Leftrightarrow QQ^T Q = QI_n \Leftrightarrow QQ^T QQ^T = QQ^T \Leftrightarrow QQ^T = I_n. \tag{9.113}$$

□

9.6.6 Positive-definite matrices

Positive-definite matrices are central to probabilistic modeling using multivariate normal distributions.

Definition 9.15. A square matrix $C \in \mathbb{R}^{n \times n}$ is called positive-definite (p.d.) if

- C is a symmetric matrix and
- for all $x \in \mathbb{R}^n, x \neq 0_n$, $x^T C x > 0$.

•

In the following theorem, we collect a few important properties of positive-definite matrices.

Theorem 9.14 (Properties of positive-definite matrices)**.**

(Inverse.) Let $C \in \mathbb{R}^{n \times n}$ be a positive-definite matrix. Then C^{-1} exists and is also positive-definite.

◦

9.7 Eigenanalysis

With the *eigenanalysis of a square matrix*, the *orthonormal decomposition of a symmetric matrix*, and the *singular value decomposition of an arbitrary matrix*, we discuss in this section three closely related concepts of matrix theory that play central roles in many areas of data-analytic applications. However, the meaning of these concepts becomes apparent above all in the respective application context, so this section may necessarily seem somewhat abstract.

Eigenvectors and eigenvalues

The *eigenanalysis* of a square matrix is understood as determining its *eigenvectors* and *eigenvalues*. These are defined for a square matrix as follows.

Definition 9.16 (Eigenvector and eigenvalue)**.** Let $A \in \mathbb{R}^{m \times m}$ be a square matrix. Then every vector $v \in \mathbb{R}^m$ different from the zero vector 0_m for which, with a scalar $\lambda \in \mathbb{R}$,

$$Av = \lambda v \tag{9.114}$$

holds is called an *eigenvector of* A, and λ is then called an *eigenvalue of* A.

•

By definition, every eigenvector therefore has an associated eigenvalue, although the eigenvalues of different eigenvectors can certainly be identical. Intuitively, the definition of eigenvector and eigenvalue means that an eigenvector of a matrix is changed in its length, but not in its direction, by multiplication with this very matrix. The associated eigenvalue of the eigenvector corresponds to the factor of the change in length. However, the assignment of eigenvectors and eigenvalues is not unique, as the following theorem shows.

Theorem 9.15 (Multiplicativity of eigenvectors)**.** *Let $A \in \mathbb{R}^{m \times m}$ be a square matrix. If $v \in \mathbb{R}^m$ is an eigenvector of A with eigenvalue $\lambda \in \mathbb{R}$, then for $c \in \mathbb{R} \setminus \{0\}$, $cv \in \mathbb{R}^m$ is also an eigenvector of A, again with eigenvalue $\lambda \in \mathbb{R}$.*

∘

Proof. This follows from

$$A(cv) = cAv = c\lambda v = \lambda(cv). \tag{9.115}$$

Thus, cv is an eigenvector of A with eigenvalue λ. □

To resolve the non-uniqueness in the definition of the eigenvector associated with an eigenvalue, we use the convention of considering only those vectors as eigenvectors for an eigenvalue λ that have length 1 and hence are elements of the unit sphere in $\mathbb{R}^m$:

$$\mathbb{S}^{m-1} := \{x \in \mathbb{R}^m : \|x\|_2 = 1\}. \tag{9.116}$$

Thus, if we should have found an eigenvector v for an eigenvalue λ of a matrix A with length other than 1, then

$$\frac{v}{\|v\|_2} \tag{9.117}$$

is of length 1 and, by Theorem 9.15, is likewise an eigenvector of A with eigenvalue λ. Before turning to the determination of eigenvalues and eigenvectors, we illustrate the concepts of eigenvalue and eigenvector for the case of a 2×2 matrix using an example.

Example

Consider the matrix

$$A := \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix} \tag{9.118}$$

and the vector

$$v := \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix}. \tag{9.119}$$

Then v is an eigenvector of A with eigenvalue $\lambda = 3$, because

$$\begin{aligned} Av &= \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix} \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix} \\ &= \frac{1}{\sqrt{2}} \begin{pmatrix} 3 \\ 3 \end{pmatrix} = 3 \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix} = \lambda v. \end{aligned} \tag{9.120}$$

Inspection of Figure 9.2 accordingly shows that for the matrix A defined here, the vectors v and Av point in the same direction, but that Av is three times as long as v. Now consider the vector

$$w := \begin{pmatrix} 1 \\ 0 \end{pmatrix}. \tag{9.121}$$

This vector also has length 1, but in contrast to v it is not an eigenvector of A. We have

$$\begin{aligned} Aw &= \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \end{pmatrix} \\ &= \begin{pmatrix} 2 \\ 1 \end{pmatrix}. \end{aligned} \tag{9.122}$$

Because the second entry of w is 0, there can be no scalar λ that would transform the second entry of w into the second entry of Aw by multiplication. Figure 9.2 therefore shows accordingly that the vector resulting from multiplication of w by A no longer points in the same direction as w.

Determining eigenvalues and eigenvectors

The following theorem states how the eigenvalues and eigenvectors of a square matrix can be computed.

Theorem 9.16 (Determining eigenvalues and eigenvectors)**.** *Let $A \in \mathbb{R}^{m \times m}$ be a square matrix. Then the eigenvalues of A are obtained as the roots of the characteristic polynomial*

$$\chi_A(\lambda) := |A - \lambda I_m| \tag{9.123}$$

of A. Furthermore, let $\lambda_i^*, i = 1, 2, \ldots$ be the eigenvalues of A determined in this way. The corresponding eigenvectors $v_i, i = 1, 2, \ldots$ of A can then be determined by solving the systems of linear equations

$$(A - \lambda_i^* I_m) v_i = 0_m \text{ for } i = 1, 2, \ldots \tag{9.124}$$

∘

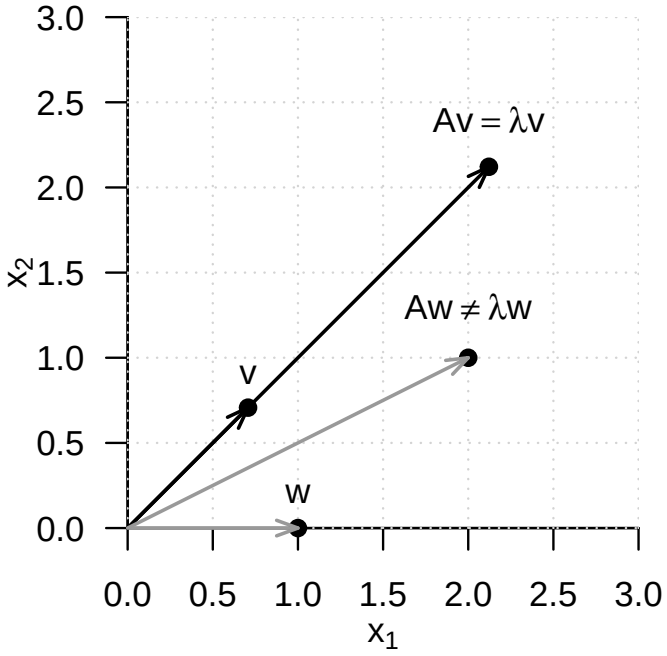


Figure 9.2 Eigenvector of a 2×2 matrix. For the matrix A defined in the text, v is an eigenvector, but w is not.

Proof. (1) Determining eigenvalues

We first note that, by the definition of eigenvectors and eigenvalues,

$$Av = \lambda v \Leftrightarrow Av - \lambda v = 0_m \Leftrightarrow (A - \lambda I_m)v = 0_m. \tag{9.125}$$

For the eigenvalue λ, the eigenvector v is therefore mapped to the zero vector 0_m by multiplication with $(A - \lambda I_m)$. But because by definition $v \neq 0_m$, the matrix $(A - \lambda I_m)$ is not invertible: both the zero vector and v are mapped to 0_m by $(A - \lambda I_m)$, so the mapping

$$f : \mathbb{R}^m \to \mathbb{R}^m, x \mapsto (A - \lambda I_m)x \tag{9.126}$$

is not bijective, and $(A - \lambda I_m)^{-1}$ cannot exist. The fact that $(A - \lambda I_m)$ is not invertible, however, is equivalent to the determinant of $(A - \lambda I_m)$ being equal to zero. Thus,

$$\chi_A(\lambda) = |A - \lambda I_m| = 0 \tag{9.127}$$

is a necessary and sufficient condition for λ to be an eigenvalue of A.

(2) Determining eigenvectors

Let λ_i^* be an eigenvalue of A. Then the above considerations imply that solving

$$(A - \lambda_i^* I_m)v_i^* = 0_m \tag{9.128}$$

for v_i^* yields an eigenvector for the eigenvalue λ_i^*. □

In general, determining eigenvalues and eigenvectors therefore requires determining polynomial roots and solving systems of linear equations. For small matrices with $m \leq 4$, this can indeed be done manually. The matrices that occur in applications, however, are usually much larger, so numerical methods for root finding and for solving systems of linear equations are used for eigenanalysis, for example in functions such as **R**'s `eigen()`, **SciPy's** `linalg.eig()`, or **Julia**'s `eigvals()` and `eigvecs()`. For details on these methods, we refer to the advanced literature, for example Burden et al. (2016) and Richter & Wick (2017). Here we merely illustrate Theorem 9.16 by means of an example.

Example

To this end, let again

$$A := \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix} \tag{9.129}$$

We first want to compute the eigenvalues of A. By Theorem 9.16, these are the roots of the characteristic polynomial of A. We therefore first compute the characteristic polynomial of A by

$$\chi_A(\lambda) = \left| \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix} - \begin{pmatrix} \lambda & 0 \\ 0 & \lambda \end{pmatrix} \right| = \left| \begin{pmatrix} 2-\lambda & 1 \\ 1 & 2-\lambda \end{pmatrix} \right| = (2-\lambda)^2 - 1. \tag{9.130}$$

Using the quadratic formula to solve quadratic equations, one then finds

$$(2 - \lambda_{1/2}^*)^2 - 1 = 0 \Leftrightarrow \lambda_1^* = 3 \text{ or } \lambda_2^* = 1. \tag{9.131}$$

The eigenvalues of A are therefore $\lambda_1 = 3$ and $\lambda_2 = 1$. The associated eigenvectors are then obtained for $i = 1, 2$ by solving the system of linear equations

$$(A - \lambda_i I_2)v_i = 0_2. \tag{9.132}$$

Specifically, for $\lambda_1 = 3$,

$$(A - 3I_2)v_1 = 0_2 \Leftrightarrow \begin{pmatrix} -1 & 1 \\ 1 & -1 \end{pmatrix} \begin{pmatrix} v_{1_1} \\ v_{1_2} \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix} \tag{9.133}$$

implies that

$$v_1 = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix} \tag{9.134}$$

is an eigenvector for the eigenvalue λ_1, and for $\lambda_2 = 1$,

$$(A - I_2)v_2 = 0_2 \Leftrightarrow \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} v_{2_1} \\ v_{2_2} \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix} \tag{9.135}$$

implies that

$$v_2 = \frac{1}{\sqrt{2}} \begin{pmatrix} -1 \\ 1 \end{pmatrix} \tag{9.136}$$

is an eigenvector for the eigenvalue $\lambda_2 = 1$. Furthermore, here obviously

$$v_1^T v_2 = 0 \text{ and } \|v_1\| = \|v_2\| = 1. \tag{9.137}$$

The following **R** code demonstrates the determination of the eigenvalues and eigenvectors of the matrix considered here with the help of the `eigen()` function.

```
# matrix definition
A = matrix(c(2,1,
             1,2),
           nrow  = 2,
           byrow = TRUE)

# eigenanalysis
eigen(A)
```

```
eigen() decomposition
$values
[1] 3 1

$vectors
          [,1]       [,2]
[1,] 0.7071068 -0.7071068
[2,] 0.7071068  0.7071068
```

At the end of this section, we consider two technical theorems that make statements about the relation between special matrix products and their eigenvalues and eigenvectors. We need these theorems in the context of canonical correlation analysis.

Theorem 9.17 (Eigenvalues and eigenvectors of matrix products)**.** *For $A \in \mathbb{R}^{n \times m}$ and $B \in \mathbb{R}^{m \times n}$, the eigenvalues of $AB \in \mathbb{R}^{n \times n}$ and $BA \in \mathbb{R}^{m \times m}$ are equal. Furthermore, if v is an eigenvector for a nonzero eigenvalue λ of AB, then $w := Bv$ is an eigenvector of BA for the eigenvalue λ.*

∘

For a proof, we refer to Mardia et al. (1979), p. 468. We demonstrate the statement of this theorem using the **R** code below.

```
A   = matrix(1:6, nrow = 2,  byrow = T)          # matrix A \in \mathbb{R}^{2 x 3}
B   = matrix(1:6, ncol = 2,  byrow = T)          # matrix B \in \mathbb{R}^{3 x 2}
EAB = eigen(A %*% B)                             # eigenanalysis of AB \in \mathbb{R}^{2 \times 2}
EBA = eigen(B %*% A)                             # eigenanalysis of BA \in \mathbb{R}^{3 \times 3}
w   = B %*% EAB$vectors[,1]                      # eigenvector of BA
cat("Eigenvalues of AB :"  , EAB$values[1:2],
    "\nEigenvalues of BA :", EBA$values[1:2],
    "\nBAw with w = Bv   :", B %*% A %*% w,
    "\nlw with w = Bv    :", EBA$values[1] * w)
```

```
Eigenvalues of AB : 85.57934 0.4206623
Eigenvalues of BA : 85.57934 0.4206623
BAw with w = Bv   : -191.1333 -416.7586 -642.3839
lw with w = Bv    : -191.1333 -416.7586 -642.3839
```

Theorem 9.18. *For $A \in \mathbb{R}^{n \times m}$, $B \in \mathbb{R}^{p \times n}$, $a \in \mathbb{R}^m$ and $b \in \mathbb{R}^p$, the only nonzero eigenvalue of $Aab^TB \in \mathbb{R}^{n \times n}$ is equal to b^TBAa, with associated eigenvector Aa.*

∘

For a proof, we refer to Mardia et al. (1979), p. 468. We demonstrate the statement of this theorem using the **R** code below.

```
A      = matrix(1:6, nrow = 2,  byrow = T)     # matrix A \in \mathbb{R}^{2 x 3}
B      = matrix(1:8, ncol = 2,  byrow = T)     # matrix B \in \mathbb{R}^{4 x 2}
a      = matrix(1:3, nrow = 3,  byrow = T)     # vector a \in \mathbb{R}^{3 x 1}
b      = matrix(1:4, nrow = 4,  byrow = T)     # vector b \in \mathbb{R}^{4 x 1}
EAabTB = eigen(A %*% a %*% t(b) %*% B)         # eigenanalysis of Aab^TB \in \mathbb{R}^{2 x 2}
cat("Eigenvalues of AabTB :", EAabTB$values,
    "\nbTBAa                :", t(b) %*% B %*% A %*% a,
    "\nAa                   :", A %*% a,
    "\n(AabTB)Aa            :",(A %*% a %*% t(b) %*% B) %*% A %*% a,            # mv
    "\n(bTBAa)Aa            :",as.vector((t(b) %*% B %*% A %*% a)) * (A %*% a)) # = \lambda v
```

```
Eigenvalues of AabTB : 2620 0
bTBAa                : 2620
Aa                   : 14 32
(AabTB)Aa            : 36680 83840
(bTBAa)Aa            : 36680 83840
```

9.8 Orthonormal decomposition

The term *decomposition* of a matrix denotes the splitting of a given matrix into the matrix product of several matrices. A wide variety of matrix decompositions play an important role in many mathematical applications; for an overview, see for example Golub & Van Loan (2013). In this section, we introduce a special matrix decomposition, the *orthonormal decomposition of a symmetric matrix*, which builds directly on eigenanalysis. We first record the following fundamental theorem on the eigenvalues and eigenvectors of symmetric matrices.

Theorem 9.19 (Eigenvalues and eigenvectors of symmetric matrices)**.**

Let $S \in \mathbb{R}^{m \times m}$ be a symmetric matrix. Then:

(1) The eigenvalues of S are real.

(2) The eigenvectors corresponding to any two distinct eigenvalues of S are orthogonal.

∘

Proof. We take the fact that a symmetric matrix has m real eigenvalues as given and show only that the eigenvectors corresponding to any two distinct eigenvalues of a symmetric matrix are orthogonal. Without loss of generality, let $\lambda_i, \lambda_j \in \mathbb{R}$ with $1 \leq i, j \leq m$ and $\lambda_i \neq \lambda_j$ be two distinct eigenvalues of S with associated eigenvectors q_i and q_j, respectively. Then, as shown below,

$$\lambda_i q_i^T q_j = \lambda_j q_i^T q_j. \tag{9.138}$$

With $q_i \neq 0_m, q_j \neq 0_m$ and $\lambda_i \neq \lambda_j$, it follows that $q_i^T q_j = 0$, because there is no number c other than zero for which, with $a, b \in \mathbb{R}$ and $a \neq b$,

$$ac = bc \tag{9.139}$$

holds. To finally show

$$\lambda_i q_i^T q_j = \lambda_j q_i^T q_j, \tag{9.140}$$

we first note that

$$Sq_i = \lambda_i q_i \Leftrightarrow (Sq_i)^T = (\lambda_i q_i)^T \Leftrightarrow q_i^T S^T = \lambda_i q_i^T \Leftrightarrow q_i^T S = \lambda_i q_i^T \Leftrightarrow q_i^T S q_j = \lambda_i q_i^T q_j \tag{9.141}$$

and

$$Sq_j = \lambda_j q_j \Leftrightarrow q_j^T S = \lambda_j q_j^T \Leftrightarrow q_j^T S q_i = \lambda_j q_j^T q_i \Leftrightarrow (q_j^T S q_i)^T = (\lambda_j q_j^T q_i)^T \Leftrightarrow q_i^T S q_j = \lambda_j q_i^T q_j \tag{9.142}$$

hold. Thus, both $\lambda_i q_i^T q_j$ and $\lambda_j q_i^T q_j$ are identical with $q_i^T S q_j$ and therefore also with each other. □

Obviously, we have proven only statement (2) of Theorem 9.19. A complete proof of the theorem can be found, for example, in Strang (2009). We also note that, because by convention we consider eigenvectors of length 1, the orthogonal eigenvectors addressed in Theorem 9.19 are in particular also orthonormal. With the help of Theorem 9.19, we can now formulate the orthonormal decomposition of a symmetric matrix and prove its existence.

Theorem 9.20 (Orthonormal decomposition of a symmetric matrix)**.** *Let $S \in \mathbb{R}^{m \times m}$ be a symmetric matrix with m distinct eigenvalues. Then S can be written as*

$$S = Q \Lambda Q^T, \tag{9.143}$$

where $Q \in \mathbb{R}^{m \times m}$ is an orthogonal matrix and $\Lambda \in \mathbb{R}^{m \times m}$ is a diagonal matrix.

∘

Proof. Let $\lambda_1 > \lambda_2 > \ldots > \lambda_m$ be the eigenvalues of S ordered by size, and let $q_1, \ldots, q_m$ be the associated orthonormal eigenvectors. With

$$Q \coloneqq \begin{pmatrix} q_1 & q_2 & \cdots & q_m \end{pmatrix} \in \mathbb{R}^{m \times m} \text{ and } \Lambda \coloneqq \operatorname{diag}(\lambda_1, \lambda_2, \ldots, \lambda_m) \in \mathbb{R}^{m \times m}, \tag{9.144}$$

the definitions of eigenvalues and eigenvectors first imply that

$$Sq_i = \lambda_i q_i \text{ for } i = 1, \ldots, m \Leftrightarrow SQ = Q\Lambda. \tag{9.145}$$

Right multiplication by Q^T then gives, with $QQ^T = I_m$,

$$SQQ^T = Q\Lambda Q^T \Leftrightarrow SI_m = Q\Lambda Q^T \Leftrightarrow S = Q\Lambda Q^T. \tag{9.146}$$

□

Because of the diagonality of Λ, the splitting of S into the matrix product $Q\Lambda Q^T$ is also called a *diagonalization of S*. As shown in the proof, to represent S in diagonal form, the diagonal elements of Λ are chosen as the eigenvalues of S ordered by size, and the columns of Q are chosen as the corresponding eigenvectors of S. We illustrate this with an example.

Example

For the symmetric matrix

$$A = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix} \tag{9.147}$$

with the eigenvalues $\lambda_1 = 3$ and $\lambda_2 = 1$ determined above and the associated orthonormal eigenvectors

$$v_1 = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix}, v_2 = \frac{1}{\sqrt{2}} \begin{pmatrix} -1 \\ 1 \end{pmatrix} \tag{9.148}$$

let

$$Q := \begin{pmatrix} v_1 & v_2 \end{pmatrix} \text{ and } \Lambda = \text{diag}(\lambda_1, \lambda_2). \tag{9.149}$$

Then obviously

$$\begin{aligned} Q\Lambda Q^T &= \begin{pmatrix} v_1 & v_2 \end{pmatrix} \text{diag}(\lambda_1, \lambda_2) \begin{pmatrix} v_1 & v_2 \end{pmatrix}^T \\ &= \left(\frac{1}{\sqrt{2}} \begin{pmatrix} 1 & -1 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} 3 & 0 \\ 0 & 1 \end{pmatrix} \right) \left(\frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ -1 & 1 \end{pmatrix} \right) \\ &= \left(\frac{1}{\sqrt{2}} \begin{pmatrix} 3 & -1 \\ 3 & 1 \end{pmatrix} \right) \left(\frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ -1 & 1 \end{pmatrix} \right) \\ &= \frac{1}{2} \begin{pmatrix} 4 & 2 \\ 2 & 4 \end{pmatrix} \\ &= \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix} \\ &= A \end{aligned}$$

and we have verified Theorem 9.20 for this example.

Symmetric square root of a matrix

The definition of the orthonormal decomposition of a symmetric matrix allows us to introduce the concept of the *symmetric square root of a matrix*.

Definition 9.17 (Symmetric square root of a matrix)**.** Let $S \in \mathbb{R}^{m \times m}$ be an invertible symmetric matrix with positive eigenvalues. Then for $r \in \mathbb{N}^0$ and $s \in \mathbb{N}$, the rational powers of S are defined, with the orthonormal matrix $Q \in \mathbb{R}^{m \times m}$ of the eigenvectors of S and the diagonal matrix $\Lambda = \text{diag}(\lambda_i) \in \mathbb{R}^{m \times m}$ of the associated eigenvalues $\lambda_1, ..., \lambda_m$ of S, as

$$S^{r/s} = Q\Lambda^{r/s}Q^T \text{ with } \Lambda^{r/s} = \text{diag}\left(\lambda_i^{r/s}\right). \tag{9.150}$$

The special case $r := 1, s := 2$ is called the *symmetric square root of* S and has the form

$$S^{1/2} = Q\Lambda^{1/2}Q^T \text{ with } \Lambda^{1/2} = \text{diag}\left(\lambda_i^{1/2}\right). \tag{9.151}$$

•

We note that Definition 9.17 immediately implies

$$\left(S^{1/2}\right)^2 = Q\Lambda^{1/2}Q^TQ\Lambda^{1/2}Q^T = Q\Lambda^{1/2}\Lambda^{1/2}Q^T = Q\Lambda Q^T = S. \tag{9.152}$$

Furthermore,

$$\left(S^{-1/2}\right)^2 = Q\Lambda^{-1/2}Q^TQ\Lambda^{-1/2}Q^T = Q\Lambda^{-1/2}\Lambda^{-1/2}Q^T = Q\Lambda^{-1}Q^T = S^{-1}. \tag{9.153}$$

Finally,

$$\begin{aligned} S^{-1/2}SS^{-1/2} &= Q\Lambda^{-1/2}Q^TQ\Lambda Q^TQ\Lambda^{-1/2}Q^T \\ &= Q\Lambda^{-1/2}\Lambda\Lambda^{-1/2}Q^T \\ &= Q\Lambda\Lambda^{-1}Q^T \\ &= I_m \end{aligned} \tag{9.154}$$

9.9 Singular value decomposition

A versatile matrix decomposition of an arbitrary matrix is the *singular value decomposition*. At this point, we are interested only in the relation between singular value decomposition and eigenanalysis and refer to the advanced literature, for example Strang (2009), for a detailed discussion of the singular value decomposition. The concept of singular value decomposition is defined as follows.

Definition 9.18 (Singular value decomposition)**.** Let $Y \in \mathbb{R}^{m \times n}$ be a matrix. Then the decomposition

$$Y = USV^T, \tag{9.155}$$

where $U \in \mathbb{R}^{m \times m}$ is an orthogonal matrix, $S \in \mathbb{R}^{m \times n}$ is a diagonal matrix, and $V \in \mathbb{R}^{n \times n}$ is an orthogonal matrix, is called the *singular value decomposition* of Y. The diagonal elements of S are called the *singular values* of Y.

•

Singular value decompositions are abbreviated as *SVD*. We omit a discussion of the computation of a singular value decomposition and merely note that singular value decompositions can be computed, for example, in **R** with the function `svd()`, in **SciPy** with `scipy.linalg.svd()`, and in **Julia** with `svd()`. The following theorem describes the relation between singular value decomposition and eigenanalysis and is used in many places.

Theorem 9.21 (Singular value decomposition and eigenanalysis)**.**

Let $Y \in \mathbb{R}^{m \times n}$ be a matrix and

$$Y = USV^T \tag{9.156}$$

be its singular value decomposition. Then:

- *the columns of U are the eigenvectors of YY^T,*
- *the columns of V are the eigenvectors of Y^TY, and*
- *the singular values are the square roots of the associated eigenvalues.*

∘

Proof. We first note that, with

$$\left(YY^T\right)^T = YY^T \text{ and } \left(Y^TY\right)^T = Y^TY, \tag{9.157}$$

YY^T and Y^TY are symmetric matrices and thus have orthonormal decompositions. We further note that, with $V^TV = I_n$ and $U^TU = I_m$,

$$YY^T = USV^T\left(USV^T\right)^T = USV^TVS^TU^T = USS^TU^T =: U\Lambda_U U^T \tag{9.158}$$

and

$$Y^TY = \left(USV^T\right)^T USV^T = VS^TU^TUSV^T = VS^TSV^T =: V\Lambda_V V^T \tag{9.159}$$

where we have defined $\Lambda_U := SS^T$ and $\Lambda_V := S^TS$. Because the product of diagonal matrices is again a diagonal matrix, Λ_U and Λ_V are diagonal matrices, and by definition U and V are orthogonal matrices. Thus, we have written YY^T and Y^TY in the form of the orthonormal decompositions

$$YY^T = U\Lambda_U U^T \text{ and } Y^TY = V\Lambda_V V^T \tag{9.160}$$

where, for the nonzero diagonal elements of Λ_U and Λ_V, these elements are the squared singular values, that is, the squared diagonal elements of S. □

10 Descriptive statistics

The aim of evaluating descriptive statistics is to make data sets accessible to human observation as efficiently as possible. As soon as the number of data points in a data set exceeds a certain number, pure numerical representations are usually difficult for human cognition to grasp. This is where descriptive statistics comes in and, in addition to graphical representations of larger data sets, also offers measures that describe certain characteristic aspects of a data set, such as its "average" value or its variability, in the sense of data reduction. In this section we want to use univariate descriptive statistics to get to know the first procedures and measures that make it possible to summarize data sets in which there is one number per study unit. We look in particular at *frequency distributions* and *histograms*, *distribution functions* and *quantiles*, as well as *measures of central tendency*, which allow statements about "average" data values, and *measures of data variability*, which allow statements about the spread of data values.

In contrast to probabilistic modeling in the context of frequentist and Bayesian inference, descriptive statistics is not based on probability theory and can therefore be viewed independently of it. However, many aspects of descriptive statistics ultimately only make sense against the background of probability theory considerations and, conversely, many concepts in probability theory are motivated by descriptive statistical concepts. If the present section does not have a background in probability theory, its full meaning will certainly only become apparent in the context of the concepts of probability theory.

In this section, we focus on descriptive statistics for describing a data set of the form

$$y := (y_1, ..., y_n) \text{ with } y_i \in \mathbb{R} \text{ for } i = 1, ..., n. \tag{10.1}$$

It is initially irrelevant whether such a data set is in the form of a column vector $y \in \mathbb{R}^{n \times 1}$ or a row vector $y \in \mathbb{R}^{1 \times n}$. Furthermore, we focus on the practical evaluation of the descriptive statistics considered, so we provide many examples for their calculation with **R**.

Application example

As an application example, we consider a data set for the evidence-based evaluation of psychotherapy for depression. To do this, we assume that $n = 100$ patients were randomly divided into a treatment and a control study condition and were asked about their depression severity using the BDI-II questionnaire at the beginning of the respective intervention. We do not want to consider the BDI-II scores collected after the interventions in this chapter. So we assume that we have a data set as shown by the following **R** code in Table 10.1.

```
D = read.csv("./_data/110-descriptive-statistics.csv")  # loading the data set
knitr::kable(D[,1:2], digits = 2, align = "c")          # r markdown table output
```

Table 10.1 BDI-II scores (PRE) of n = 100 patients in a treatment (T) and a control study group (C) before the start of a psychological intervention

GROUP	PRE
T	17
T	20
T	16
T	18
T	21
T	17
T	17
T	17
T	18
T	18
T	20
T	17
T	16
T	18
T	16
T	18
T	17
T	14
T	18
T	18
T	20
T	20
T	21
T	19
T	19
T	17
T	20
T	21
T	17
T	17
T	19
T	20
T	22
T	20
T	21
T	20
T	16
T	15
T	15
T	18
T	21
T	17
T	18
T	21
T	17
T	19
T	16
T	17
T	17
T	18
C	22
C	19
C	21
C	18
C	19
C	17
C	20
C	19
C	19
C	19
C	17
C	20
C	18
C	20
C	18
C	18
C	15
C	18
C	17
C	19
C	19
C	19
C	20
C	19
C	20
C	19
C	21
C	19
C	19
C	18
C	20
C	16
C	21
C	19
C	20
C	18
C	23
C	18
C	19
C	18

Table 10.1 BDI-II scores (PRE) of n = 100 patients in a treatment (T) and a control study group (C) before the start of a psychological intervention

GROUP	PRE
C	21
C	17
C	20
C	21
C	21
C	20
C	18
C	18
C	19
C	19

10.1 Frequency distributions

We start with the concepts of *absolute* and *relative frequency distribution*. For data sets with values in which the same value occurs multiple times, frequency distributions can provide a first impression of the nature of the data set.

Definition 10.1 (Absolute and relative frequency distributions)**.** $y := (y_1, ..., y_n)$ is a data set and $W := \{w_1, ..., w_k\}$ with $k \leq n$ are the values occurring in the data set. Then the function is called

$$h : W \to \mathbb{N}, w \mapsto h(w) := \text{number of } y_i \text{ in } y \text{ with } y_i = w \tag{10.2}$$

the *absolute frequency distribution* of the values of y and the function

$$r : W \to [0, 1], w \mapsto r(w) := \frac{h(w)}{n} \tag{10.3}$$

is called the *relative frequency distribution* of the values of y.

•

Frequency distributions can be evaluated and displayed in **R** by combining the `table()` and `barplot()` functions. The following **R** code first generates the absolute and relative frequency distributions of the pre-intervention BDI-II data of the example data set.

```
D        = read.csv("./_data/110-descriptive-statistics.csv")  # loading the data set
y        = D$PRE                                               # double vector of the PRE values
n        = length(y)                                           # number of data values (100)
H        = as.data.frame(table(y))                             # absolute frequency distribution (data frame)
names(H) = c("w", "ah")                                        # column naming
H$rh     = H$ah/n                                              # relative frequency distribution
print(H, digits = 1)                                           # output
```

```
    w ah   rh
1  14  1 0.01
2  15  3 0.03
3  16  6 0.06
4  17 17 0.17
5  18 21 0.21
6  19 20 0.20
7  20 17 0.17
8  21 12 0.12
9  22  2 0.02
10 23  1 0.01
```

From the output of the data frame `H` we can see that in the data set 10 under consideration, different values `w` occur between 14 and 23, whereby, for example, the values 18 and 22 occur 21 times and twice respectively. Both the absolute and the relative frequency distribution show that values around 18 occur more frequently in the present data set, whereas extreme values such as 13 or 23 occur rather rarely. Note that the relative frequency distribution is simply a scaled form of the absolute frequency distribution. Qualitatively, i.e. with regard to the frequent or rare occurrence of certain values, the two frequency distributions do not differ. This becomes particularly clear when both frequency distributions are visualized, as in Figure 10.1. The following **R** code uses the `barplot()` function for this purpose. The height of the bar plotted above a value w corresponds to the function values $h(w)$ or $r(w)$. The visual insight naturally corresponds to the inspection of the `H` data frame. Both frequency distributions show that values around 18 occur more frequently in the present data set, whereas lower or higher values such as 13 or 23 are rather rare.

```
pdf(                                                    # pdf copy function
file     = "./_figures/110-frequency-distributions.pdf",# filename
width    = 7,                                           # width (inches)
height   = 4)                                           # height (inches)
par(                                                    # figure parameters
mfcol    = c(1,2),                                      # rows and columns layout
las      = 1,                                           # x-tick orientation
cex      = 0.7,                                         # annotation enlargement
cex.main = 1.3)                                         # title enlargement
h        = H$ah                                         # h(w) values
r        = H$rh                                         # r(w) values
names(h) = H$w                                          # barplot needs w values as names
names(r) = H$w                                          # barplot needs w values as names
barplot(                                                # bar chart
h,                                                      # absolute frequencies
col      = "gray90",                                    # bar color
xlab     = "w",                                         # x axis label
ylab     = "h(w)",                                      # y axis label
ylim     = c(0,25),                                     # y axis limits
main     = "Absolute frequency distribution")           # title
barplot(                                                # bar chart
r,                                                      # absolute frequencies
col      = "gray90",                                    # bar color
xlab     = "w",                                         # x axis label
ylab     = "r(w)",                                      # y axis label
ylim     = c(0,.25),                                    # y axis limits
main     = "Relative frequency distribution")           # title
dev.off()                                               # end image editing
```

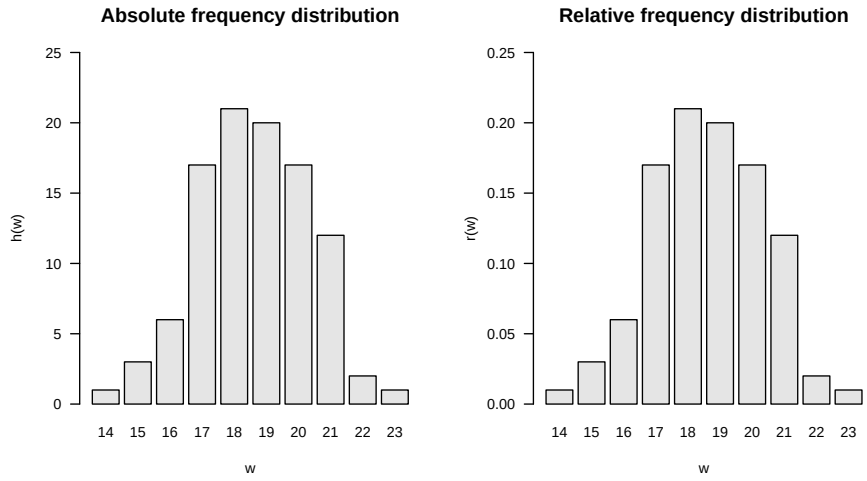


Figure 10.1 Absolute and relative frequency distributions of pre-intervention BDI-II scores.

10.2 Histograms

If no values appear multiple times in a data set, the absolute and relative frequency distributions for each of these values take the value 1 and $1/n$, respectively, and then provide no value beyond visual inspection of the data set. However, the histograms presented in this section offer a way to generate visual representations of frequency distributions in this case too. Different values that are "close to each other" in a well-defined sense are combined into classes and the absolute or relative frequencies of these classes are determined. Since in data sets in which the same values occur multiple times, these values are naturally in the same class, histograms can also be usefully used in the case considered in Section 10.1. Histograms generalize the frequency distributions considered in the previous section for data sets with multiple values.

We first give a definition of the concept of histogram. Note that in this definition the process of class formation, the determination of their absolute and relative frequencies, and the visualization of these frequencies are closely interlinked. This is due to the convention that the term histogram describes both a specific form of data analysis and its visualization.

Definition 10.2 (Histogram)**.** A *histogram* is a diagram in which, for a data set $y = (y_1, ..., y_n)$ with values $W := \{w_1, ..., w_m\}$ with $m \leq n$ for $j = 1, ..., k$ over adjacent intervals $[b_{j-1}, b_j[$ with $b_0 := \min W$ and $b_k := \max W$, rectangles with the *width* $d_j := b_j - b_{j-1}$ and the *height* $h(w)$ or $r(w)$ for $w \in [b_{j-1}, b_j[$ are shown. The intervals $[b_{j-1}, b_j[$ are called *classes* (*bins*).

•

Of course, the visual appearance of a histogram depends heavily on how precisely the values of a data set are grouped into classes. According to Definition 10.2, the decisive parameter for this is the number of classes k that are selected between the minimum value b_0 and the maximum value b_k of the data set. Below we present some conventional values for the number of classes k and calculate and visualize them using the example data set as an example. We always assume that the neighboring intervals $[b_{j-1}, b_j[$ with $j = 1, ..., k$ are chosen for the number of classes k with a constant width $d = b_j - b_{j-1}$.

To calculate and visualize histograms we use the **R** function `hist()`. In this case, the classes $[b_{j-1}, b_j[$ are specified for $j = 1, ..., k$ as the argument `breaks`. Specifically, `breaks` is the **R** vector `c(b_0,b_1,...,b_k)` with length $k+1$. To illustrate the effect of different choices of the number of classes, we first read the data set and determine its minimum and maximum values b_0 and b_k.

```
# dataset
D   = read.csv("./_data/110-descriptive-statistics.csv")  # loading the data set
y   = D$PRE                                               # pre-intervention BDI-II data
b_0 = min(y)                                              # b_0
b_k = max(y)                                              # b_k
```

```
Minimum b_0               :  14
Maximum b_k               :  23
```

A first possibility for choosing the classes k is to achieve a desired constant width d as precisely as possible. This is what you rely on

$$k := \left\lceil \frac{b_k - b_0}{d} \right\rceil . \tag{10.4}$$

Note that the application of the rounding function implies that the resulting class width is at most the desired value, but can also be smaller. The following **R** code illustrates this effect.

```
# histogram with desired class width d = 2
d   = 2                                 # desired class width
b_0 = min(y)                            # b_0
b_k = max(y)                            # b_k
k   = ceiling((b_k - b_0)/d)            # number of classes
b   = seq(b_0, b_k, length.out = k + 1)  # classes [b_{j-1}, b_j[ (breaks)
```

```
Desired class width d      :  2
Number of classes k        :  5
Intervals [b_j-1, b_j]     :  14 15.8 17.6 19.4 21.2 23
Achieved class width d     :  1.8
```

Choosing a desired class width of $d = 1.5$, on the other hand, leads to a resulting class width for the present data set that corresponds to the desired one.

```
# histogram with desired class width d = 1.5
d = 1.5                                 # desired class width
k = ceiling((b_k - b_0)/d)              # number of classes
b = seq(b_0, b_k, length.out = k + 1)  # classes [b_{j-1}, b_j[ (breaks)
```

```
Desired class width d      :  1.5
Number of classes k        :  6
Intervals [b_j-1, b_j]     :  14 15.5 17 18.5 20 21.5 23
Achieved class width d     :  1.5
```

Another option is to make the number of classes dependent on the number of data points in the data set. The Excel standard choice is included

$$k := \lceil \sqrt{n} \rceil . \tag{10.5}$$

The following **R** code implements this choice for the present data set.

```
# histogram with Excel standard
n = length(y)                           # number of data points
k = ceiling(sqrt(n))                    # number of classes
b = seq(b_0, b_k, length.out = k + 1)  # classes [b_{j-1}, b_j[ (breaks)
```

```
Number of data points n    :  100
Number of classes k        :  10
Intervals [b_j-1, b_j]     :  14 14.9 15.8 16.7 17.6 18.5 19.4 20.3 21.2 22.1 23
Achieved class width d     :  0.9
```

The number of classes suggested by Sturges (1926) is similar to the Excel standard, which optimizes the representation under an implicit normal distribution assumption

$$k := \lceil \log_2 n + 1 \rceil \tag{10.6}$$

is given. In this case the result is:

```
# class number according to Sturges (1926)
n = length(y)                          # number of data points
k = ceiling(log2(n)+1)                 # number of classes
b = seq(b_0, b_k, length.out = k + 1)  # classes [b_{j-1}, b_j[ (breaks)
```

```
Number of data points n    :  100
Number of classes k        :  8
Intervals [b_j-1, b_j]     :  14 15.125 16.25 17.375 18.5 19.625 20.75 21.875 23
Achieved class width d     :  1.125
```

Another possibility is to incorporate descriptive statistics of the data set into the choice of the number of classes. As a measure of the dispersion of the data in the data set, the number of classes according to Scott (1979) uses the empirical standard deviation (cf. Section 10.6) in order to use the histogram to achieve a density estimate for a normal distribution. We do not want to delve into the non-parametric background of this approach here, but simply note that the choice of the number of classes according to Scott (1979) by determining the class width

$$d := 3.49S/\sqrt[3]{n} \tag{10.7}$$

and the subsequent choice of the number of classes

$$k := \left\lceil \frac{b_k - b_0}{d} \right\rceil \tag{10.8}$$

is given, where S denotes the standard deviation of the data set. In this case the result is:

```
# number of classes according to Scott (1979)
n = length(y)                          # number of data values
S = sd(y)                              # empirical standard deviation
d = ceiling(3.49*S/(n^(1/3)))          # class width
k = ceiling((b_k - b_0)/d)             # number of classes
b = seq(b_0, b_k, length.out = k + 1)  # classes [b_{j-1}, b_j[ (breaks)
```

```
Number of data points n    :  100
Standard deviation         :  1.740167
Class width by Scott d     :  2
Number of classes k        :  5
Intervals [b_j-1, b_j]     :  14 15.8 17.6 19.4 21.2 23
Achieved class width d     :  1.8
```

By default, the `hist()` function implemented in **R** uses a modification of the number of classes suggested by Sturges (1926) and offers a number of additional options for selecting classes. `?hist` provides an overview of this. The following **R** code visualizes the absolute frequencies of the values of the example data set using the classes specified above.

```
# figure parameters
pdf(
file     = "./_figures/110-histograms.pdf",                    # pdf copy function
width    = 6,                                                  # filename
height   = 9)                                                  # width (inches)
par(                                                           # height (inches)
mfcol    = c(3,2),                                             # figure parameters
las      = 1,                                                  # rows and columns layout
cex      = .7,                                                 # x-tick orientation
cex.main = .9)                                                 # annotation enlargement
x_min    = 12                                                  # title enlargement
x_max    = 25                                                  # x-axis limit
y_min    = 0                                                   # x-axis limit
y_max    = 45                                                  # y-axis limit
label    = c("", "", "Excel", "Sturges", "Scott")              # y-axis limit
                                                               # title
# histograms with predefined number of classes and class width
for(i in 1:5){
    hist(                                                      # histogram
    y,                                                         # dataset
    breaks   = bs[[i]],                                        # breaks argument
    xlim     = c(x_min, x_max),                                # x-axis limits
```

```
    ylim     = c(y_min, y_max),                                          # y-axis limits
    ylab     = "frequency",                                              # y-axis label
    xlab     = "BDI",                                                    # x-axis label
    main     = sprintf(paste(label[i], "k = %.0f, d = %.2f"), ks[i],ds[i]))  # title
}

# r default histogram
hist(                                                                    # histogram
y,                                                                       # dataset
xlim     = c(x_min, x_max),                                              # x-axis limits
ylim     = c(y_min, y_max),                                              # y-axis limits
ylab     = "frequency",                                                  # y-axis label
xlab     = "",                                                           # x-axis label
main     = "R Default")                                                  # title
mtext(LETTERS[6], adj=0, line=2, cex = 1.2, at = 8)                      # subplot letter
dev.off()                                                                # end image editing
```

10.3 Empirical distribution functions

In addition to the question of how frequently certain values or classes of values occur in a data set, it can be interesting to examine how frequently values that are smaller than a certain value occur. Although the answer to this question is of course implicit in the absolute and relative frequencies or the corresponding histograms, it is sometimes advantageous to have this information directly available. The concepts of *cumulative absolute and relative frequency distribution* and *empirical distribution function* introduced in this section denote the corresponding analogues to the concepts discussed in Section 10.1 and Section 10.2. We begin by defining the *cumulative absolute* and *cumulative relative frequency distributions*.

Definition 10.3 (Cumulative absolute and relative frequency distributions)**.** Let $y = (y_1, ..., y_n)$ be a data set, $W := \{w_1, ..., w_k\}$ with $k \leq n$ the numerical values occurring in the data set and h and r the absolute and relative frequency distributions of the values of y, respectively. Then the function is called

$$H : W \to \mathbb{N}, w \mapsto H(w) := \sum_{w' \leq w} h(w') \tag{10.9}$$

the *cumulative absolute frequency distribution* of y and the function

$$R : W \to [0, 1], w \mapsto R(w) := \sum_{w' \leq w} r(w') \tag{10.10}$$

the *cumulative relative frequency distribution* of the numerical values of y.

•

In the sense of the definition Definition 10.1 applies to the cumulative frequency distributions

$$H(w) = \text{number of } y_i \text{ in } y \text{ with } y_i \leq w \tag{10.11}$$

and

$$R(w) = \text{number of } y_i \text{ in } y \text{ with } y_i \leq w \text{ divided by } n. \tag{10.12}$$

In **R**, the function `cumsum()` allows the calculation of cumulative sums and thus enables the evaluation of cumulative frequency distributions, as the following **R** code demonstrates using the example data set.

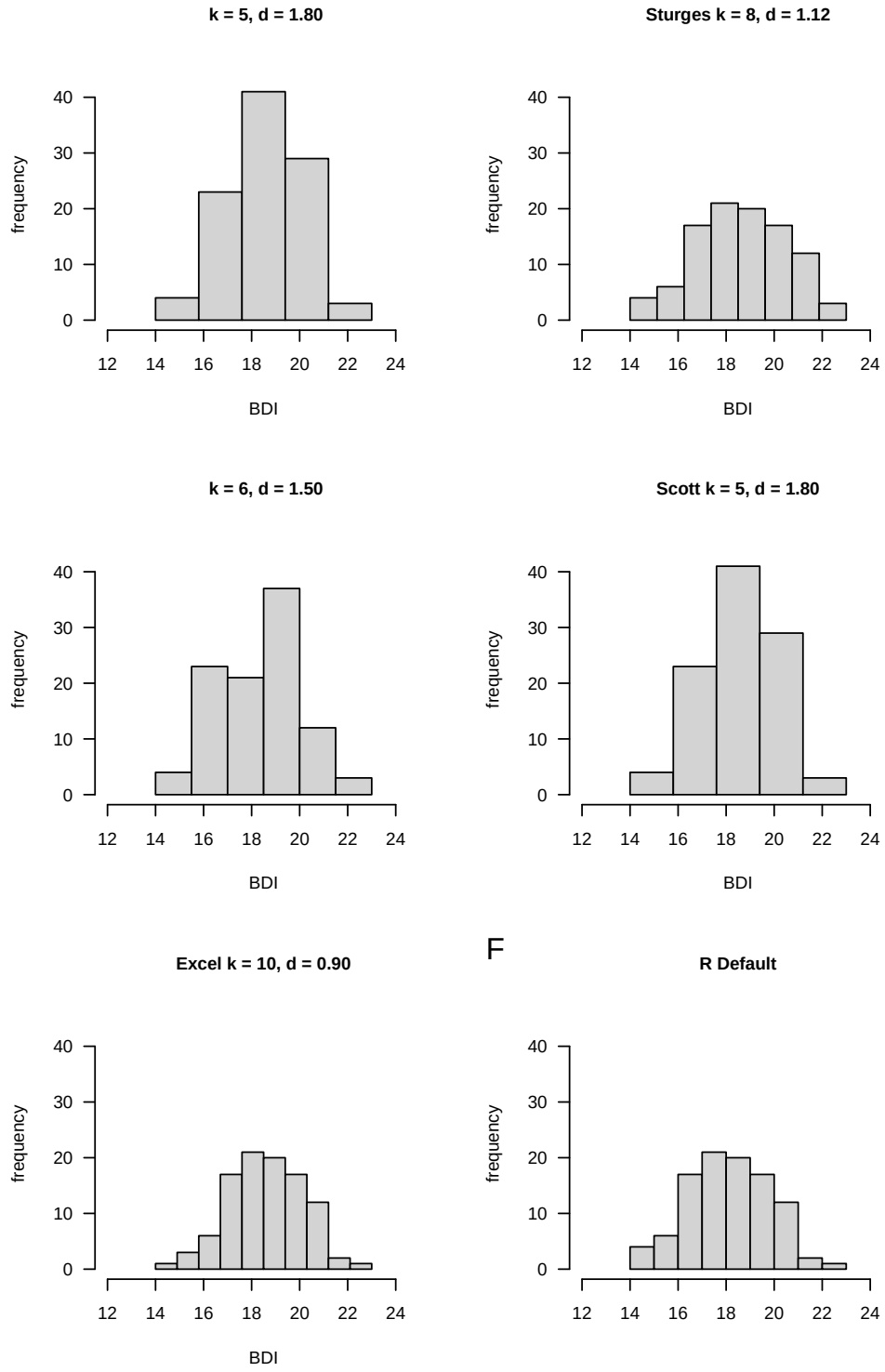


Figure 10.2 Histograms of the absolute frequencies of the pre-intervention BDI-II scores. Note that the qualitative impression that values around 18 occur frequently in the data set and that lower and higher values are rarer is constant across all choices of the number of classes. The exact quantitative appearance, however, depends on the exact choice of the number of classes and the class width.

```
D        = read.csv("./_data/110-descriptive-statistics.csv")  # loading the data set
y        = D$PRE                                               # pre-intervention BDI-II data
n        = length(y)                                           # number of data values
H        = as.data.frame(table(y))                             # absolute frequency distribution as a data frame
names(H) = c("w", "h")                                         # column naming
H$H      = cumsum(H$h)                                         # cumulative absolute frequency distribution
H$r      = H$h/n                                               # relative frequency distribution
H$R      = cumsum(H$r)                                         # cumulative relative frequency distribution
print(H, digits = 1)
```

```
    w  h   H    r    R
1  14  1   1 0.01 0.01
2  15  3   4 0.03 0.04
3  16  6  10 0.06 0.10
4  17 17  27 0.17 0.27
5  18 21  48 0.21 0.48
6  19 20  68 0.20 0.68
7  20 17  85 0.17 0.85
8  21 12  97 0.12 0.97
9  22  2  99 0.02 0.99
10 23  1 100 0.01 1.00
```

Pay particular attention to the comparison of `h` and `H` as well as `r` and `R`, which illustrate the cumulative totals in Definition 10.3. The following **R** code enables the representation of the cumulative frequency distributions determined as shown in Figure 10.3. Obviously, the absolute and relative frequency of values less than the smallest value in the data set is zero. On the other hand, the absolute and relative frequency of values smaller than the largest value in the data set are n and $n/n = 1$, respectively.

```
pdf(                                                           # pdf copy function
file     = "./_figures/110-cumulative-frequency-distributions.pdf",  # filename
width    = 10,                                                 # width (inches)
height   = 5)                                                  # height (inches)
Hw       = H$H                                                 # h(w) values
Rw       = H$R                                                 # r(w) values
names(Hw) = H$w                                                # barplot needs w values as names
names(Rw) = H$w                                                # barplot needs w values as names
par(                                                           # figure parameters
mfcol    = c(1,2),                                             # rows and columns layout
las      = 1,                                                  # x-tick orientation
cex      = 1)                                                  # annotation enlargement
barplot(                                                       # bar chart
Hw,                                                            # h(w) values
col      = "gray90",                                           # bar color
xlab     = "w",                                                # x-axis label
ylab     = "H(w)",                                             # y-axis label
ylim     = c(0,110),                                           # y-axis limits
las      = 1,                                                  # axis tick label orientation
main     = "Absolute cumulative frequency PRE")                # title
barplot(                                                       # bar chart
Rw,                                                            # r(w) values
col      = "gray90",                                           # bar color
xlab     = "w",                                                # x-axis label
ylab     = "R(w)",                                             # y-axis label
ylim     = c(0,1),                                             # y-axis limits
las      = 1,                                                  # axis tick label orientation
main     = "Relative cumulative frequency PRE")                # title
dev.off()                                                      # end image editing
```

The cumulative analogue of a histogram is the *empirical distribution function*. In contrast to a histogram, no definition of classes is required to determine an empirical distribution function. Instead of the interval division of the real numbers between the minimum and the maximum of a data set, the empirical distribution function simply uses the real numbers, as the following definition makes clear.

Definition 10.4 (Empirical distribution function)**.** Let $y = (y_1, ..., y_n)$ be a data set. Then the function is called

$$F : \mathbb{R} \to [0, 1], x \mapsto F(x) := \frac{\text{number of } y_i \text{ in } y \text{ with } y_i \leq x}{n} \tag{10.13}$$

the empirical distribution function (EVF) of y.

•

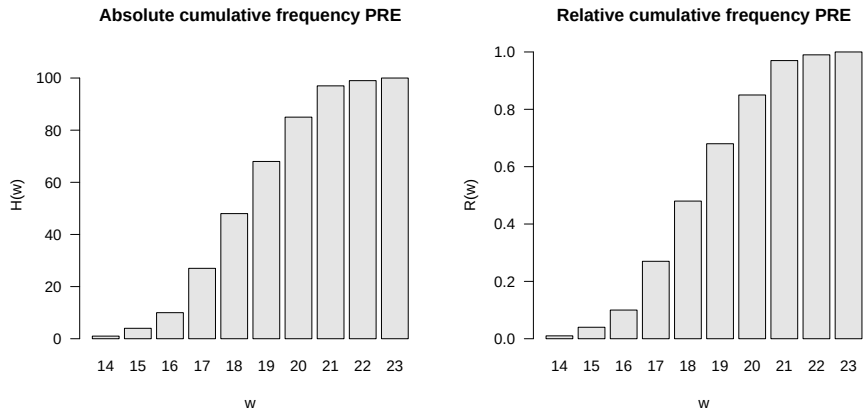


Figure 10.3 Absolute and relative cumulative frequency distributions of pre-intervention BDI-II scores.

The empirical distribution function is sometimes called the *empirical cumulative distribution function*. In general, the empirical distribution function as in Definition 10.4 is limited to specifying the relative frequencies. Of course, not all real numbers appear in data sets. This implies that the relative frequency assigned to real numbers that lie between two adjacent values in the data set is equal to that of the smaller of the two values. Typically, empirical distribution functions are step functions. In **R**, the function `ecdf()` can be used to evaluate and visualize the empirical distribution function, as the following **R** code demonstrates using the example data set and is visualized in Figure 10.4. For example, you can read from Figure 10.4 that the relative frequency of values smaller than 17.9 is approximately 0.3. Values smaller than 17.8 have the same relative frequency.

```
D      = read.csv("./_data/110-descriptive-statistics.csv")  # loading the data set
y      = D$PRE                                               # pre-intervention BDI-II data
evf    = ecdf(y)                                             # evaluation of the EVF
pdf(                                                         # pdf copy function
file   = "./_figures/110-ecdf.pdf",                          # filename
width  = 8,                                                  # width (inches)
height = 5)                                                  # height (inches)
plot(                                                        # plot() knows how to handle ecdf object
evf,                                                         # ecdf object
xlab   = "BDI",                                              # x-axis label
ylab   = "Relative frequency",                               # y-axis label
main   = "Empirical distribution function")                  # title
dev.off()                                                    # end image editing
```

10.4 Quantiles and boxplots

The empirical distribution function provides an answer, at least graphically represented, to the question of how large the relative proportion of values in a data set is that are smaller than a given value. The quantiles introduced in this section can be understood as an inversion of this data set interrogation. Specifically, with quantiles you specify a relative proportion $0 \leq p \leq 1$ of the values of a data set and ask which value of the data set is such that $p \cdot 100\%$ of the values of the data set are less than or equal to this value. We use the following definition of the term p *quantile*.

Definition 10.5 (p quantile)**.** Let $y = (y_1, ..., y_n)$ be a data set and

$$y_s = \left(y_{(1)}, y_{(2)}, ..., y_{(n)}\right) \text{ with } \min_{1 \leq i \leq n} y_i = y_{(1)} \leq y_{(2)} \leq \cdots \leq y_{(n)} = \max_{1 \leq i \leq n} y_i \tag{10.14}$$

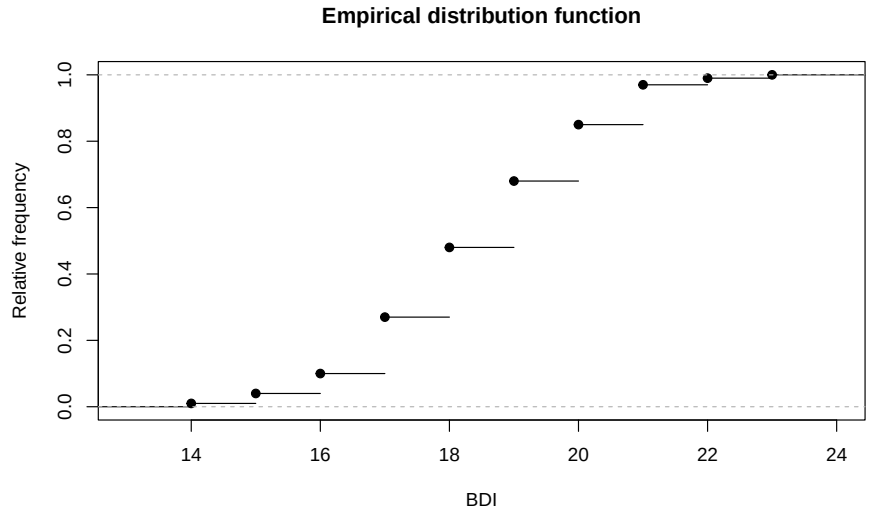


Figure 10.4 Empirical distribution function of the PRE values.

be the corresponding data set sorted in ascending order. Furthermore, let $\lfloor \cdot \rfloor$ denote the rounding function. Then for a $p \in [0, 1]$ is the number

$$y_p := \begin{cases} y_{(\lfloor np+1 \rfloor)} & \text{if } np \notin \mathbb{N} \\ \frac{1}{2}\left(y_{(np)} + y_{(np+1)}\right) & \text{if } np \in \mathbb{N} \end{cases} \tag{10.15}$$

the p *quantile* of the data set y.

•

Note that the p quantile y_p to Definition 10.5 is either a value of the data set or a value between two values of the data set. After constructing y_p, it applies in particular that at least $p \cdot 100\%$ of all values in the data set are less than or equal to y_p and at least $(1-p) \cdot 100\%$ of all values in the data set are greater than or equal to y_p. The p quantile therefore divides the ordered data set y_s in the ratio p to $(1-p)$. Depending on the choice of p, the p quantiles receive special names:

- $y_{0.25}, y_{0.50}, y_{0.75}$ are called *lower quartile*, *median* and *upper quartile*, respectively,
- $y_{j \cdot 0.10}$ for $j = 1, ..., 9$ are called *deciles* and
- $y_{j \cdot 0.01}$ for $j = 1, ..., 99$ are called *percentiles*.

To clarify the concept of the p quantile, let us consider an example based on Henze (2018), chapter 5. First of all, the data set shown in the table below and its version sorted in ascending order are given.

i	1	2	3	4	5	6	7	8	9	10
y_i	8.5	1.5	75	4.5	6.0	3.0	3.0	2.5	6.0	9.0
$y_{(i)}$	1.5	2.5	3.0	3.0	4.5	6.0	6.0	8.5	9.0	75

We first want to determine the lower quartile, i.e. the 0.25 quantile, of this data set. It's apparently $n = 10$ and we chose $p := 0.25$. Based on Definition 10.5 we find that

$$np = 10 \cdot 0.25 = 2.5 \notin \mathbb{N}. \tag{10.16}$$

So according to Definition 10.5 it applies that

$$y_{0.25} = y_{(\lfloor 2.5+1 \rfloor)} = y_{(3)} = 3.0. \tag{10.17}$$

The lower quartile of the data set under consideration is therefore the value 3.0 of the data set.

Next we want to determine the 0.80 quantile of the data set. Apparently it is still $n = 10$ and we have now chosen $p := 0.80$. In this case we find that

$$np = 10 \cdot 0.80 = 8 \in \mathbb{N}. \tag{10.18}$$

So follows after Definition 10.5

$$y_{0.80} = \frac{1}{2}\left(y_{(8)} + y_{(8+1)}\right) = \frac{1}{2}\left(y_{(8)} + y_{(9)}\right) = \frac{8.5 + 9.0}{2} = 8.75. \tag{10.19}$$

The $y_{0.80}$ quantile of the data set under consideration is with 8.75 the value between the values 8.5 and 9 of the data set. In **R** the quantiles determined in this way are evaluated as follows.

```
y   = c(8.5,1.5,75,4.5,6.0,3.0,3.0,2.5,6.0,9.0)  # sample data
n   = length(y)                                  # number of data values
y_s = sort(y)                                    # sorted data set
p   = 0.25                                       # np \notin \mathbb{N}
y_p = y_s[floor(n*p + 1)]                        # 0.25 quantile
cat("0.25 quantile: ", round(y_p,2))             # output
```

```
0.25 quantile:  3
```

```
p   = 0.80                                       # np \in \mathbb{N}
y_p = (1/2)*(y_s[n*p] + y_s[n*p + 1])            # 0.80 quantile
cat("0.80 quantile: ", round(y_p,2))             # output
```

```
0.80 quantile:  8.75
```

With `quantile()`, **R** also provides a function that automates the quantile determination of a data set. However, it should be noted that the definition of the quantile term given in Definition 10.5 is only one of at least nine different quantile definitions and the exact values can differ from definition to definition. The specific quantile definition used by `quantile()` is selected via the function argument `type`. Hyndman & Fan (1996) provide an overview on the theoretical level, and `?quantile` on the implementation level. As an example, we look at determining the 0.25 quantile of the pre-intervention BDI-II sample data set using `type = 8`.

```
D   = read.csv("./_data/110-descriptive-statistics.csv")  # loading the data set
y   = D$PRE                                              # pre-intervention BDI-II data
y_p = quantile(y, 0.25, type = 8)                        # 0.25 quantile, definition 1
```

```
0.25 quantile (type = 8):  17
```

Finally, a *boxplot* visualizes a quantile-based summary of a data set, typically visualizing the minimum and maximum of a data set as well as the lower quartile, median and upper quartile. In **R** you create a boxplot using the `boxplot()` function, as the following **R** code demonstrates using the pre-intervention BDI-II data set.

```
D          = read.csv("./_data/110-descriptive-statistics.csv")  # loading the data set
pdf(                                                            # pdf copy function
file       = "./_figures/110-boxplot.pdf",                      # filename
width      = 8,                                                 # width (inches)
height     = 5)                                                 # height (inches)
boxplot(                                                        # box plot
D$PRE,                                                          # dataset
horizontal = T,                                                 # horizontal representation
range      = 0,                                                 # whiskers up to min y and max y
xlab       = "BDI",                                             # x axis label
main       = "Box plot")                                        # title
dev.off()                                                       # end image editing
```

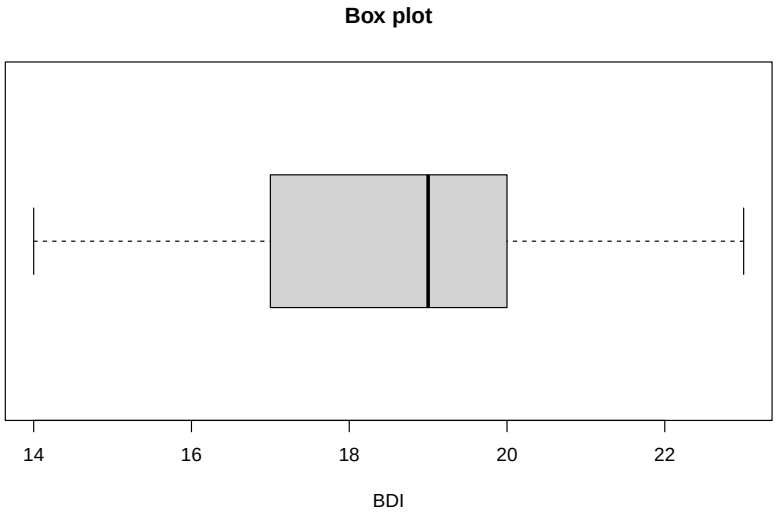


Figure 10.5 Boxplot of the pre-intervention BDI-II scores of the example data set.

In Figure 10.5, $\min y$ and $\max y$ are represented as so-called "whisker endpoints", $y_{0.25}$ and $y_{0.75}$ form the lower and upper boundaries of the central gray box and the median $y_{0.50}$ is represented as a line in the central gray box. Here too, there are a lot of box plot variations, see McGill et al. (1978). A precise explanation of the data set properties displayed graphically is always necessary. Finally, it should be noted that the representation of data sets using boxplots has become somewhat out of fashion in the last twenty years and is only very rarely found in current scientific publications.

10.5 Measures of central tendency

In this section we look at some basic data set metrics that are central to descriptive statistics and provide information about the "average" value of a data set. We start by defining the mean.

10.5.1 Mean

Definition 10.6 (Mean)**.** Let $y = (y_1, ..., y_n)$ be a data set. Then that means

$$\bar{y} := \frac{1}{n} \sum_{i=1}^{n} y_i \tag{10.20}$$

the *mean* of y.

•

The mean of a data set is the sum of the data values divided by the number of data values. The mean defined here is also referred to as the *arithmetic mean* or *average*. In the context of frequentist inference, we will refer to the mean as the *sample mean* in the context of the random sampling model. In particular, the models of frequentist inference also allow the functional form of the mean to be more deeply substantiated, as we will see at a given point. Using the sum function, you can calculate the mean of a data set in **R** as follows.

```
D     = read.csv("./_data/110-descriptive-statistics.csv")  # loading the data set
y     = D$PRE                                               # pre-intervention BDI-II data
n     = length(y)                                           # number of values
y_bar = (1/n)*sum(y)                                        # average calculation
```

```
Mean of PRE-BDI-II data 18.61
```

With the `mean()` function, **R** of course also provides a function for automated calculation of the mean value.

```
cat("Mean of PRE-BDI-II data", mean(y))  # output
```

```
Mean of PRE-BDI-II data 18.61
```

In the following theorem we first note two properties of the mean value of a data set that make its classification as a measure of the central tendency of a data set plausible.

Theorem 10.1 (Centrality properties of the mean)**.** *Let $y = (y_1, ..., y_n)$ be a data set and $\bar{y}$ be the mean of y. Then apply*

(1) The sum of the deviations from the mean is zero,

$$\sum_{i=1}^{n}(y_i - \bar{y}) = 0. \tag{10.21}$$

(2) The absolute sums of negative and positive deviations from the mean are equal, i.e. if $j = 1, ..., n_j$ denotes the data point indices with $(y_j - \bar{y}) < 0$ and $k = 1, ..., n_k$ denotes the data point indices with $(y_k - \bar{y}) \geq 0$, then with $n = n_j + n_k$ it applies that

$$|\sum_{j=1}^{n_j}(y_j - \bar{y})| = |\sum_{k=1}^{n_k}(y_k - \bar{y})|. \tag{10.22}$$

∘

Proof. (1) It applies

$$\begin{aligned}
\sum_{i=1}^{n}(y_i - \bar{y}) &= \sum_{i=1}^{n} y_i - \sum_{i=1}^{n} \bar{y} \\
&= \sum_{i=1}^{n} y_i - n\bar{y} \\
&= \sum_{i=1}^{n} y_i - \frac{n}{n}\sum_{i=1}^{n} y_i \\
&= \sum_{i=1}^{n} y_i - \sum_{i=1}^{n} y_i \\
&= 0.
\end{aligned} \tag{10.23}$$

(2) Let $j = 1, ..., n_j$ be the indices with $(y_j - \bar{y}) < 0$ and $k = 1, ..., n_k$ the indices with $(y_k - \bar{y}) \geq 0$, such that $n = n_j + n_k$. Then applies

$$\begin{aligned} & \sum_{i=1}^{n}(y_i - \bar{y}) = 0 \\ \Leftrightarrow & \sum_{j=1}^{n_j}(y_j - \bar{y}) + \sum_{k=1}^{n_k}(y_k - \bar{y}) = 0 \\ \Leftrightarrow & \sum_{k=1}^{n_k}(y_k - \bar{y}) = -\sum_{j=1}^{n_j}(y_j - \bar{y}) \\ \Leftrightarrow & |\sum_{j=1}^{n_j}(y_j - \bar{y})| = |\sum_{k=1}^{n_k}(y_k - \bar{y})|. \end{aligned} \tag{10.24}$$

□

According to Theorem 10.1, the mean value of a data set is just such that the total deviations of the data points from this value equalize to zero and that positive and negative deviations from the mean balance each other out. In **R** you can verify Theorem 10.1 using the example data set as follows.

```
# dataset
D   = read.csv("./_data/110-descriptive-statistics.csv")  # loading the data set
y   = D$PRE                                               # pre-intervention BDI-II data
s   = sum(y - mean(y))                                    # sum of deviations from the mean
s_1     = sum(y[y <= mean(y)] - mean(y))                  # sum of all negative deviations
s_2 = sum(y[y > mean(y)]  - mean(y))                      # sum of all positive deviations
```

```
Sum of deviations from the mean: 0
Sums of positive and negative deviations: -71.28 71.28
```

Theorem 10.2 (Summation properties of the mean)**.** *Given two data sets $y = (y_1, ..., y_n)$ and $z = (z_1, ..., z_n)$ of the same size and let it be by*

$$y + z := (y_1 + z_1, ..., y_n + z_n) \tag{10.25}$$

the sum of these data sets is defined. Furthermore, let $\bar{y}$ and $\bar{z}$ be the mean values of the data sets y and z, respectively. Then

$$\overline{y + z} = \bar{y} + \bar{z} \tag{10.26}$$

∘

Proof. It applies

$$\begin{aligned} \overline{y + z} &:= \frac{1}{n}\sum_{i=1}^{n}(y_i + z_i) \\ &= \frac{1}{n}\sum_{i=1}^{n} y_i + \frac{1}{n}\sum_{i=1}^{n} z_i \\ &=: \bar{y} + \bar{z}. \end{aligned} \tag{10.27}$$

□

If you look at the sum of two data sets, it is irrelevant to determining their mean value whether you first add up the data sets and then determine the mean value of the summed data set or whether you determine the mean value for each of the two data sets and add them up. We verify this result using the pre- and post-intervention BDI-II data from the example data set in **R** as follows.

```
D        = read.csv("./_data/110-descriptive-statistics.csv")  # loading the data set
y        = D$PRE                                               # pre-intervention BDI-II data
y_bar    = mean(y)                                             # mean of pre-intervention BDI-II data
z        = D$POS                                               # post-intervention BDI-II data
z_bar    = mean(z)                                             # mean of post-intervention BDI-II data
yz_bar_1 = mean(y + z)                                         # mean of summed pre and post data
yz_bar_2 = y_bar + z_bar                                       # sum of the means of the pre- and post-data
```

```
Mean of z: 31.68
Sum of the means of y and z :  31.68
```

Theorem 10.3 (Mean under linear-affine transformation)**.** *Let $y = (y_1, ..., y_n)$ be a data set and $\bar{y}$ be the mean of y. Furthermore, be for $a, b \in \mathbb{R}$*

$$ay + b := (ay_1 + b, ..., ay_n + b) \tag{10.28}$$

a linear-affine transformed data set of y is defined. Then that applies

$$\overline{ay + b} = a\bar{y} + b \tag{10.29}$$

∘

Proof. It applies

$$\begin{aligned}
\overline{ay + b} &:= \frac{1}{n}\sum_{i=1}^{n}(ay_i + b) \\
&= \sum_{i=1}^{n}\left(\frac{1}{n}ay_i + \frac{1}{n}b\right) \\
&= \left(\sum_{i=1}^{n}\frac{1}{n}ay_i\right) + \left(\sum_{i=1}^{n}\frac{1}{n}b\right) \\
&= a\left(\frac{1}{n}\sum_{i=1}^{n}y_i\right) + \frac{1}{n}\sum_{i=1}^{n}b \\
&= a\bar{y} + \frac{1}{n}nb \\
&= a\bar{y} + b.
\end{aligned} \tag{10.30}$$

□

A linear-affine transformation of a data set transforms the mean value of the data set in a linear-affine manner. You can take advantage of this if you think about what the mean value should be when rescaling a data set, for example converting from grams to kilograms. Of course, to be on the safe side, you can also simply determine the average of the rescaled data set. We demonstrate this property as an example by rescaling the pre-intervention BDI-II data set with the value $a = 2$ while simultaneously adding the constant $b := 5$.

```
D        = read.csv("./_data/110-descriptive-statistics.csv")  # loading the data set
y        = D$PRE                                               # pre-intervention BDI-II data
y_bar    = mean(y)                                             # mean of pre-intervention BDI-II data
a        = 2                                                   # multiplication constant
b        = 5                                                   # addition constant
z        = a*y + b                                             # linear-affine transformation of the PRE data
z_bar    = mean(z)                                             # mean value of transformed PRE data
ay_bar_b = a*y_bar + b                                         # pre data mean transformation
```

```
Mean value of the transformed data set: 42.22
Transformation of the mean       : 42.22
```

10.5.2 Median

In the mean value defined above, each value of a data set is included summatively with the same weight $1/n$. This particularly applies to values that are very atypical compared to the other values in the data set. One way to determine the "center" of a data set independent of such atypical values is to determine its median, which we define below. We already learned about the median in section Section 10.4 as the 0.5 quantile, with which it is identical.

Definition 10.7 (Median)**.** Let $y = (y_1, ..., y_n)$ be a data set and $y_s = (y_{(1)}, ..., y_{(n)})$ the corresponding data set sorted in ascending order. Then the median of y is defined as

$$\tilde{y} := \begin{cases} y_{((n+1)/2)} & \text{if } n \text{ ungerade,} \\ \frac{1}{2}\left(y_{(n/2)} + y_{(n/2+1)}\right) & \text{if } n \text{ gerade.} \end{cases} \tag{10.31}$$

•

Definition 10.7 implies that at least 50% of all data values y_i of the data set are less than or equal to $\tilde{y}$ and at the same time at least 50% of all data values y_i of the data set are greater than or equal to $\tilde{y}$. Instead of a formal proof of these statements, we refer to Figure 10.6. As in the definition of the median, we differentiate between the cases where the number of data points is odd (here $n = 5$, Figure 10.6 A) or even ($n = 6$, Figure 10.6 B). In the case of an odd number of data points, the median is the data value with the index number that lies in the middle of the index numbers of the sorted data set, in the example Figure 10.6 A, i.e. the value $y_{(3)}$. In this case, 3/5, i.e. 60% of the data values (specifically $y_{(1)}$, $y_{(2)}$ and $y_{(3)}$) are less than or equal to the median, but at the same time 3/5, i.e. 60% of the data values (specifically $y_{(3)}$, $y_{(4)}$ and $y_{(5)}$) greater than or equal to the median. In the case of an even number of data points, the location of the median is somewhat more intuitive: the median lies in the middle of the two middle values of the sorted data set (in Figure 10.6 B specifically in the middle of the values $y_{(3)}$ and $y_{(4)}$). This means that 3/6, i.e. 50% of the data values (specifically $y_{(1)}$, $y_{(2)}$ and $y_{(3)}$) are smaller than the median and 3/6, i.e. 50% of the data values (specifically $y_{(4)}$, $y_{(5)}$ and $y_{(6)}$) are larger than the median.

The following **R** code determines the median of the example data set using Definition 10.7.

```
D       = read.csv("./_data/110-descriptive-statistics.csv")  # loading the data set
y       = D$PRE                                               # pre-intervention BDI-II data
n       = length(y)                                           # number of values
y_s     = sort(y)                                             # ascending sorted vector
if(n %% 2 == 1){                                              # n odd, n mod 2 == 1
   y_tilde = y_s[(n+1)/2]                                     # median formula, case n odd
} else {                                                      # n even, n mod 2 == 0
   y_tilde = (y_s[n/2] + y_s[n/2 + 1])/2}                     # median formula, case n even
```

```
Median of PRE-BDI-II data: 19
```

Of course, **R** also provides a function with `median()` for directly determining the median, as the following **R** code demonstrates.

```
cat("Median of PRE-BDI-II data:", median(y))  # output
```

```
Median of PRE-BDI-II data: 19
```

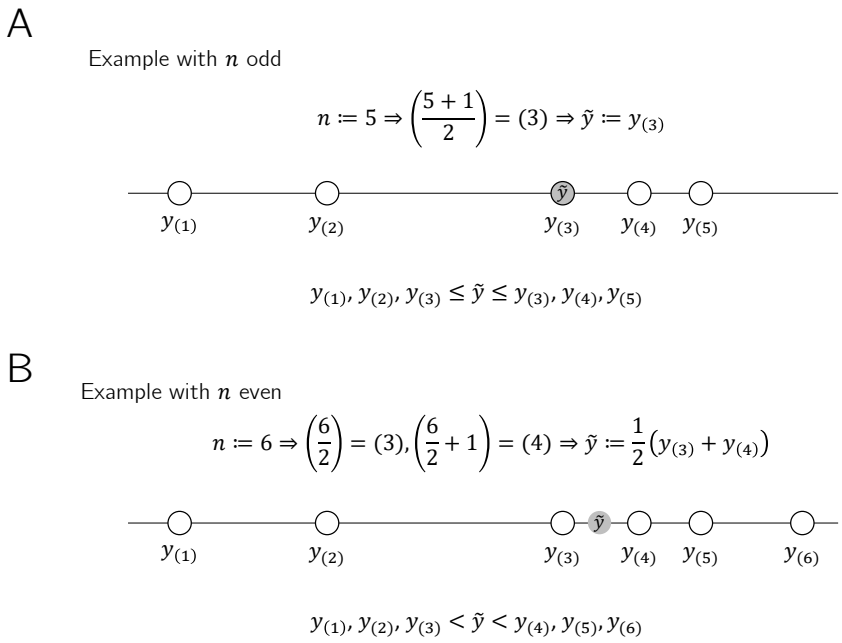


Figure 10.6 Determination and properties of the median.

The following simulation shows as an example that the median, as a measure of the middle of a data set, is less susceptible to outliers than the mean.

```
D = read.csv("./_data/110-descriptive-statistics.csv")  # loading the data set
y = D$PRE                                               # pre-intervention BDI-II data
```

```
Mean and median for data set without outliers: 18.61 19
```

```
z    = y      # new simulated data set
z[1] = 10000  # ... with an extreme value
```

```
Mean and median for data set with outliers: 118.44 19
```

So, although the median appears to be a more robust measure of the central tendency of a data set than the mean, determining averages is certainly the more common approach. On the one hand, this is due to the fact that the mean value is easier to use in an analytical context for designing probabilistic models due to its mathematically less complex definition. On the other hand, the occurrence of extreme values is usually a sign of data collection errors (e.g. a BDI-II value of 10.000 simply should not occur) or large differences between the experimental units under consideration, which are usually noticed by visual inspection of the data set before determining measures of central tendency. However, there is a whole subfield of statistics that deals with - in the broadest sense - measures that are independent of outliers, see e.g. Huber (1981).

10.5.3 Mode

If some values occur repeatedly in a data set, a third measure of central tendency is given with the *mode* as the value that occurs most frequently in a data set. We use the following definition.

Definition 10.8 (Mode)**.** $y := (y_1, ..., y_n)$ with $y_i \in \mathbb{R}$ is a data set, $W := \{w_1, ..., w_k\}$ with $k \leq n$ are the different numerical values occurring in the data set and $h : W \to \mathbb{N}$ is the absolute frequency distribution of the numerical values of y. Then the *mode (or mode)* of y is defined as

$$\text{argmax}_{w \in W} h(w), \tag{10.32}$$

i.e. the value that occurs most frequently in the data set.

•

In **R** you can determine the mode of a corresponding data set as follows.

```
D        = read.csv("./_data/110-descriptive-statistics.csv")  # loading the data set
y        = D$PRE                                               # pre-intervention BDI-II data
n        = length(y)                                           # number of data values (100)
H        = as.data.frame(table(y))                             # absolute frequency distribution (data frame)
names(H) = c("a", "w")                                         # consistent naming
mode     = H$w[which.max(H$w)]                                 # mode
```

```
cat("Mode of PRE-BDI-II data:", as.numeric(as.vector(mode)))  # output as numeric vector, not factor
```

```
Mode of PRE-BDI-II data: 21
```

10.5.4 Data distribution shapes and measures of central tendency

How useful it is to use the mean, median or mode to provide information about the central tendency of a data set depends largely on the nature of the data set. For example, if you have a data set of high-resolution data in which no data value appears multiple times, the mode is certainly not suitable for making a quantitative statement about the central tendency of the data set. The specification of a measure of central tendency should always be viewed in the context of the overall nature of the data set and, if applicable, the inferential statistical model that is used to interrogate the data set. In Figure 10.7 we give four examples of how the measures of central tendency considered above can behave in different data value distribution scenarios. Of course, these four scenarios are not exhaustive and each data set should be viewed critically for a meaningful measure of central tendency.

Figure 10.7 A visualizes the position of the mean, median and mode in a data set that is symmetrically distributed around a value of approximately $y = 50$ and for which values close to this central value occur frequently and values far from this central value occur rarely. As we see later, this corresponds to the typical characteristics of normally distributed data values. In this case, the mean, median and mode are quite close to each other and each of these measures in fact describes the central tendency of the data set.

Figure 10.7 B visualizes the position of the mean, median and mode in a data set in which the data values occur clustered around two values of approximately $y = 25$ and $y = 75$ and rarely occur in the positive and negative directions from these values. For example, there are very few data points in this data set around $y = 50$. The mean and median now take on values around $y = 50$. As such, they do not provide any information about which values occur particularly frequently in this data set, but only about where the average "physical center of gravity" of the data set is. The mode, on the other hand, is at the larger of the two data accumulation points due to the slightly more frequent occurrence of values around $y = 75$. Overall, none of the three measures really satisfactorily describes the central tendency of the data values for such a data set, which is also called *bimodal*.

Figure 10.7 C visualizes the position of the mean, median and mode in a data set in which the data values in the width under consideration occur with approximately the same frequency throughout. Here the mode is relatively arbitrary at around $y = 15$, as there is a minimal accumulation of data values there. In the sense of the "physical center of gravity" of the data set, the mean and median are around $y = 50$, even if the values of this data set have no tendency to occur more frequently there than elsewhere. Here too, the description of the central tendency of the data set by the measures considered is not really satisfactory. Taken together, using Figure 10.7 B and Figure 10.7 C, one might be able to conclude that if a data set does not have a clear central tendency, the measures of central tendency cannot reflect it either.

Finally, Figure 10.7 D shows the location of the mean, median and mode for a data set that is skew-symmetrically distributed around values of approximately $y = 0.5$. The data in this data set only takes positive values, many data values are in fact around $y = 0.5$, but there are also higher data values. We will see later that this appearance is typical for squared normally distributed data values. In a psychological context, reaction times in particular often have a similar distribution, as they cannot be negative, in most cases being in the range of 0.3 to 0.6 seconds, but in some cases can be much longer due to inattention or stimulus variability. In this case, the median describes the actual central tendency of the data values somewhat better than the mean, as this is slightly shifted towards higher values due to the rarely occurring outlier values.

Overall, the specification of one or more measures of central tendency must always be viewed critically against the background of the distribution of the values of a data set and classified qualitatively.

10.6 Measures of variability

In addition to the question of what "average value" a data set assumes, it is often of interest to quantify how much the values in the data set are scattered or, in other words, how "variable" they are. Here we consider the *range*, the *empirical variance* and the *empirical standard deviation* as quantitative measures of the variability of a data set. As with the mean, the full meaning of the latter measures becomes apparent primarily in the context of inferential statistical methods.

10.6.1 Range

The range of a data set is the difference between its largest and smallest values.

Definition 10.9 (Range)**.** Let $y = (y_1, ..., y_n)$ be a data set. Then the *range* of $y_1, ..., y_n$ is defined as

$$r := \max(y_1, ..., y_n) - \min(y_1, ..., y_n). \tag{10.33}$$

•

In **R** you determine the range either by evaluating the maximum and minimum of the data set using the `max()` and `min()` functions or directly using the `range()` function. The following **R** code demonstrates the use of the `max()` and `min()` functions.

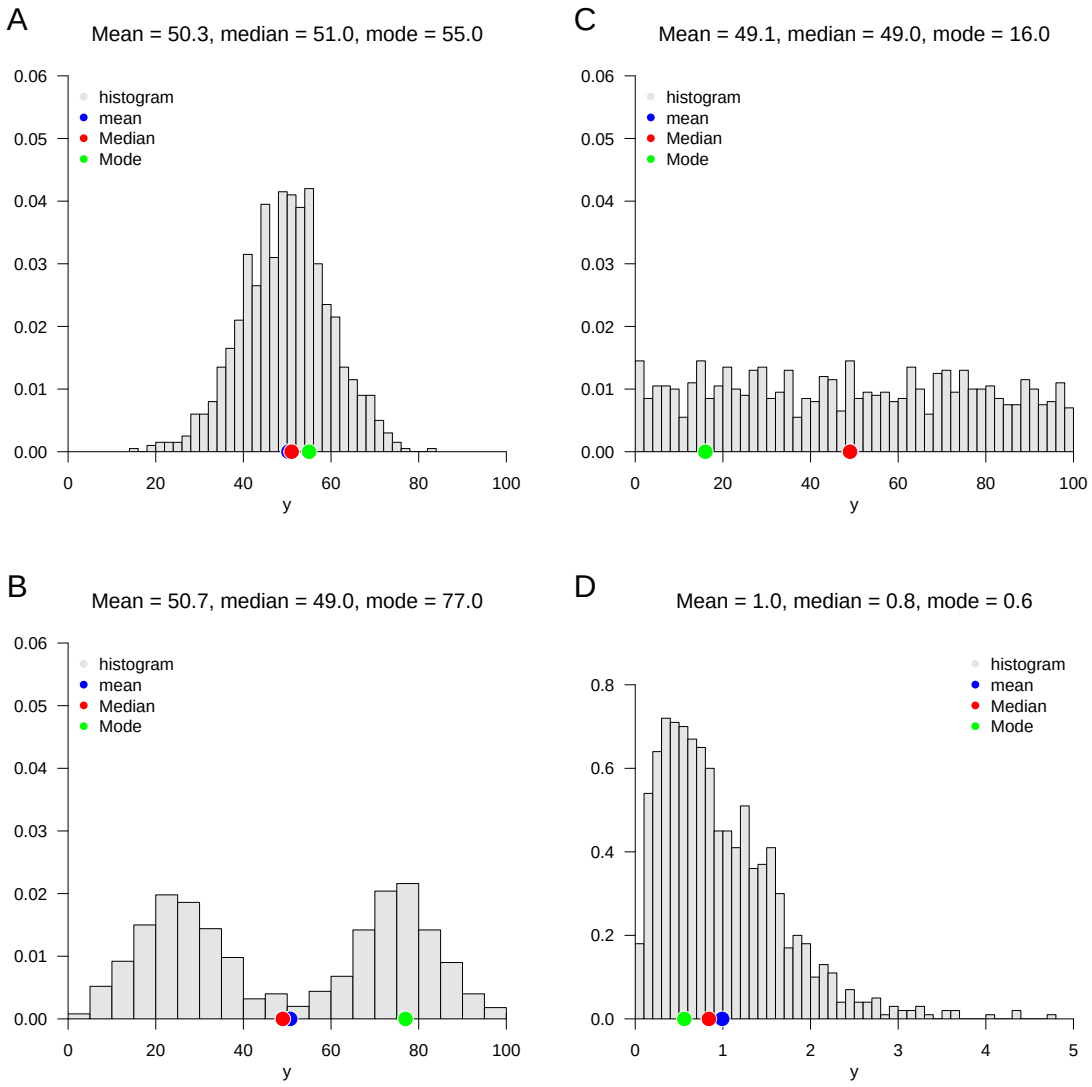


Figure 10.7 Visual intuition on measures of central tendency in different data distribution forms

```
D     = read.csv("./_data/110-descriptive-statistics.csv")  # loading the data set
y     = D$PRE                                               # pre-intervention BDI-II data
y_max = max(y)                                              # maximum of PRE values
y_min = min(y)                                              # minimum of PRE values
r     = y_max - y_min                                       # range
```

```
Range of PRE-BDI-II data: 9
```

The following **R** code demonstrates the use of the `range()` function.

```
MinMax = range(y)              # "automatic" calculation of min(x), max(x)
r      = MinMax[2] - MinMax[1]  # range
```

```
Range of PRE-BDI-II data: 9
```

10.6.2 Empirical variance

A typical statistical measure of the variability of data sets is the standardized sum of the squared differences between the individual data values and their mean. The squared differences between the individual data values and their mean are also referred to as *difference squares*. Depending on the form of standardization, i.e. the formation of an average of the squared deviations, a distinction is made between *uncorrected empirical variance* and *empirical variance*, as the following definition states.

Definition 10.10 (Uncorrected empirical variance and empirical variance)**.** Let $y = (y_1, ..., y_n)$ be a data set with $n > 1$ and $\bar{y}$ be its mean. Then the *uncorrected empirical variance* of y is defined as

$$\tilde{s}^2 := \frac{1}{n}\sum_{i=1}^{n}(y_i - \bar{y})^2 \tag{10.34}$$

and the *empirical variance* of y is defined as

$$s^2 := \frac{1}{n-1}\sum_{i=1}^{n}(y_i - \bar{y})^2. \tag{10.35}$$

•

We saw in Theorem 10.1 that the sum of the differences between the data values and their mean is always zero. Squaring the differences

$$y_i - \bar{y} \text{ for } i = 1, ..., n \tag{10.36}$$

in Definition 10.10 now leads to the fact that negative and positive differences between data points do not balance each other out and, depending on the level of the positive and negative differences, with

$$\sum_{i=1}^{n}(y_i - \bar{y})^2 \tag{10.37}$$

a higher or lower measure of the overall deviation of the data values from their mean can be determined. The sum of the squares of the deviations is zero if all data values are identical and therefore identical to their mean. Alternatively, for example, the determination of the absolute amounts $|y_i - \bar{y}|$ would also be conceivable, but the resulting measure of variability

has different properties than the measure of the squared deviations in Definition 10.10, which is why this is usually preferred, also in view of its proximity to probabilistic normal distribution models. The uncorrected empirical variance $\tilde{s}^2$ and the (corrected) empirical variance s^2 then differ in terms of the form of the averaging of the squared deviations. For $\tilde{s}^2$ the sum of the squares of the deviations is divided by n, for s^2 by $n-1$. So $\tilde{s}^2$ is always slightly smaller than s^2. However, this difference will be rather small for large n, think for example of the numerical difference between $\frac{1}{3}$ and $\frac{1}{2}$ for small n and between $\frac{1}{1001}$ and $\frac{1}{1000}$ for large n. Note that the empirical variance for $n = 1$ is not defined, but the uncorrected empirical variance can in principle be determined even in this special case. The term *uncorrected* (and implicitly *corrected*) cannot be understood at this point, but only makes sense against the background of a frequentist inference model and will be explained in the later section on parameter estimation.

We summarize the above discussion in the following theorem.

Theorem 10.4 (Ratio of uncorrected empirical variance and empirical variance)**.** *Let $y = (y_1, ..., y_n)$ be a data set with $n > 1$ and let $\tilde{s}^2$ and s^2 be its uncorrected empirical variance and empirical variance, respectively. Then apply*

(1) (conversion)

$$\tilde{s}^2 = \frac{n-1}{n} s^2 \text{ and } s^2 = \frac{n}{n-1} \tilde{s}^2. \tag{10.38}$$

(2) (inequality)

$$0 \leq \tilde{s}^2 \leq s^2. \tag{10.39}$$

∘

Proof. (1) After Definition 10.10 applies on the one hand

$$\begin{aligned} \tilde{s}^2 &= \frac{1}{n} \sum_{i=1}^{n} (y_i - \bar{y})^2 \\ \Leftrightarrow \frac{n-1}{n-1} \tilde{s}^2 &= \frac{n-1}{n-1} \frac{1}{n} \sum_{i=1}^{n} (y_i - \bar{y})^2 \\ \Leftrightarrow \tilde{s}^2 &= \frac{n-1}{n} \frac{1}{n-1} \sum_{i=1}^{n} (y_i - \bar{y})^2 \\ \Leftrightarrow \tilde{s}^2 &= \frac{n-1}{n} s^2 \end{aligned} \tag{10.40}$$

and on the other

$$\begin{aligned} s^2 &= \frac{1}{n-1} \sum_{i=1}^{n} (y_i - \bar{y})^2 \\ \Leftrightarrow \frac{n}{n} s^2 &= \frac{n}{n} \frac{1}{n-1} \sum_{i=1}^{n} (y_i - \bar{y})^2 \\ \Leftrightarrow s^2 &= \frac{n}{n-1} \frac{1}{n} \sum_{i=1}^{n} (y_i - \bar{y})^2 \\ \Leftrightarrow s^2 &= \frac{n}{n-1} \tilde{s}^2. \end{aligned} \tag{10.41}$$

(2) With the non-negativity of the square function and $\frac{1}{n} > 0$ and $\frac{1}{n-1} > 0$ for $n > 1$, $\tilde{s}^2 \geq 0$ and $s^2 \geq 0$ result directly. Finally applies to $\sum_{i=1}^{n} (y_i - \bar{y})^2 \neq 0$

$$\frac{1}{n} < \frac{1}{n-1} \Leftrightarrow \frac{1}{n} \sum_{i=1}^{n} (y_i - \bar{y})^2 < \frac{1}{n-1} \sum_{i=1}^{n} (y_i - \bar{y})^2 \Leftrightarrow \tilde{s}^2 < s^2. \tag{10.42}$$

and for $\sum_{i=1}^{n}(y_i - \bar{y})^2 = 0$ apparently $\tilde{s}^2 = s^2$. So overall:

$$0 \leq \tilde{s}^2 \leq s^2. \tag{10.43}$$

□

The following **R** code demonstrates the determination of the uncorrected empirical variance and the empirical variance using the formulas specified in Definition 10.10.

```
D        = read.csv("./_data/110-descriptive-statistics.csv")  # loading the data set
y        = D$PRE                                               # pre-intervention BDI-II data
n        = length(y)                                           # number of values
s2_tilde = (1/n)*sum((y - mean(y))^2)                          # uncorrected empirical variance
s2       = (1/(n-1))*sum((y - mean(y))^2)                      # empirical variance
```

```
Uncorrected empirical variance of PRE-BDI-II data: 2.998
Empirical variance of PRE-BDI-II data: 3.028
```

The **R** function `var()` determines the empirical variance by default. Using statement (1) in Theorem 10.4, the uncorrected empirical variance can also be determined based on this function.

```
s2       = var(y)            # empirical variance
s2_tilde = ((n-1)/n)*var(y)  # uncorrected empirical variance
```

```
Uncorrected empirical variance of PRE-BDI-II data: 2.998
Empirical variance of PRE-BDI-II data: 3.028
```

The following theorem states how the empirical variance behaves when a data set is transformed with linear affinity.

Theorem 10.5 (Variance in linear-affine transformations)**.** *Let $y = (y_1, ..., y_n)$ be a data set with empirical variance S_y^2 and $z = (ay_1 + b, ..., ay_n + b)$ be the data set with empirical variance S_z^2 transformed linearly with $a, b \in \mathbb{R}$. Then applies*

$$s_z^2 = a^2 s_y^2. \tag{10.44}$$

∘

Proof.

$$\begin{aligned}
s_z^2 &= \frac{1}{n-1}\sum_{i=1}^{n}(z_i - \bar{z})^2 \\
&= \frac{1}{n-1}\sum_{i=1}^{n}(ay_i + b - (a\bar{y} + b))^2 \\
&= \frac{1}{n-1}\sum_{i=1}^{n}(ay_i + b - a\bar{y} - b)^2 \\
&= \frac{1}{n-1}\sum_{i=1}^{n}(a(y_i - \bar{y}))^2 \\
&= \frac{1}{n-1}\sum_{i=1}^{n}a^2(y_i - \bar{y})^2 \\
&= a^2\frac{1}{n-1}\sum_{i=1}^{n}(y_i - \bar{y})^2 \\
&= a^2 S_y^2.
\end{aligned} \tag{10.45}$$

□

For example, converting a data set from meters to centimeters changes the variance of the original data set by the factor 100^2. We reproduce Theorem 10.5 using the following **R** codes as an example.

```
y   = D$PRE                                                    # pre-intervention BDI-II data
s2y = var(y)                                                   # empirical variance of y_1,...,y_n
a   = 2                                                        # multiplication constant
b   = 5                                                        # addition constant
z   = a*y + b                                                  # z_i = ay_i + b
s2z = var(z)                                                   # empirical variance of z_1,...,z_n
cat("Empirical variance of z_1,...,z_n by direct calculation:", round(s2z,3))  # output
```

```
Empirical variance of z_1,...,z_n by direct calculation: 12.113
```

```
s2z = a^2*s2y                                                  # empirical variance of z_1,...,z_n
cat("Empirical variance of z_1,...,z_n according to theorem:", round(s2z,3))   # output
```

```
Empirical variance of z_1,...,z_n according to theorem: 12.113
```

The following theorem shows a way to determine the uncorrected empirical variance of a data set solely by determining mean values. The theorem finds its analytical equivalent in the variance shift theorem.

Theorem 10.6 (Shift theorem for uncorrected empirical variance)**.** *Let $y = (y_1, ..., y_n)$ be a data set, $y^2 := (y_1^2, ..., y_n^2)$ be its element-wise square, and $\bar{y}$ and $\overline{y^2}$ be the mean values, respectively. Then applies*

$$\tilde{s}^2 = \overline{y^2} - \bar{y}^2 \tag{10.46}$$

∘

Proof.

$$\begin{aligned}
\tilde{s}^2 &:= \frac{1}{n}\sum_{i=1}^{n}(y_i - \bar{y})^2 \\
&= \frac{1}{n}\sum_{i=1}^{n}(y_i^2 - 2y_i\bar{y} + \bar{y}^2) \\
&= \frac{1}{n}\sum_{i=1}^{n}y_i^2 - 2\bar{y}\frac{1}{n}\sum_{i=1}^{n}y_i + \frac{1}{n}\sum_{i=1}^{n}\bar{y}^2 \\
&= \overline{y^2} - 2\bar{y}\bar{y} + \frac{1}{n}n\bar{y}^2 \\
&= \overline{y^2} - 2\bar{y}^2 + \bar{y}^2 \\
&= \overline{y^2} - \bar{y}^2.
\end{aligned} \tag{10.47}$$

□

We reproduce Theorem 10.6 using the following **R** codes as an example.

```
y        = D$PRE                    # pre-intervention BDI-II data
s2_tilde = mean(y^2) - (mean(y))^2  # \bar{y^2} - \bar{y}^2
cat("Uncorrected empirical variance according to the shift theorem:", round(s2_tilde,3))  # output
```

```
Uncorrected empirical variance according to the shift theorem: 2.998
```

Note that the shift theorem of the uncorrected empirical variance does not apply to the empirical variance, since in this case the multiplication factor $\frac{1}{n-1}$ does not lead to the corresponding mean values. In particular, we have already seen above that for the example data set in question, $s^2 = 3.028$.

10.6.3 Empirical standard deviation

As seen above, the empirical variance is the average squared deviation of a data set value from its mean. If, for example, the unit of the data set in a reaction time task is the millisecond, then subtracting and then squaring results in the unit millisecond^2, a not very intuitive unit. In order to obtain a measure of dispersion whose unit corresponds to the unit of the original data set, the square root function is often applied to the empirical variance. This leads to the concept of *empirical standard deviation*.

Definition 10.11 (Empirical standard deviation)**.** Let $y = (y_1, ..., y_n)$ be a data set. The *empirical standard deviation* of y is defined as

$$s := \sqrt{s^2} \tag{10.48}$$

and the *uncorrected empirical standard deviation* of y is defined as

$$\tilde{s} := \sqrt{\tilde{s}^2}. \tag{10.49}$$

•

In analogy to the properties of the empirical variance and its uncorrected version, the uncorrected empirical standard deviation can also easily be converted into the empirical standard deviation and vice versa, apparently the following apply here

$$\tilde{s} = \sqrt{\frac{n-1}{n}} s \text{ and } s = \sqrt{\frac{n}{n-1}} \tilde{s}. \tag{10.50}$$

The following **R** code demonstrates the determination of the uncorrected empirical standard deviation and the empirical standard deviation using the formulas specified in Definition 10.11.

```
y = D$PRE                                # pre-intervention BDI-II data
n = length(y)                            # number of values
s = sqrt((1/(n-1))*sum((y - mean(y))^2))  # standard deviation
```

```
Empirical standard deviation: 1.74
```

The **R** function `sd()` determines the empirical standard deviation by default.

```
s = sd(y)
```

```
Empirical standard deviation: 1.74
```

If you transform a data set with linear affinity, the multiplication constant of the transformation is only included in the transformation of the empirical standard deviation in terms of its amount. This is the statement of the following theorem.

Theorem 10.7 (Empirical standard deviation for linear-affine transformations)**.** *Let $y = (y_1, ..., y_n)$ be a data set with empirical standard deviation s_y and $z = (ay_1 + b, ..., ay_n + b)$ be the data set linearly-affinically transformed with $a, b \in \mathbb{R}$ with empirical standard deviation s_z. Then applies*

$$s_z = |a| s_y. \tag{10.51}$$

∘

Proof.

$$
\begin{aligned}
s_z &:= \left(\frac{1}{n-1} \sum_{i=1}^{n} (z_i - \bar{z})^2 \right)^{1/2} \\
&= \left(\frac{1}{n-1} \sum_{i=1}^{n} (ay_i + b - (a\bar{y} + b))^2 \right)^{1/2} \\
&= \left(\frac{1}{n-1} \sum_{i=1}^{n} (a(y_i - \bar{y}))^2 \right)^{1/2} \\
&= \left(\frac{1}{n-1} \sum_{i=1}^{n} a^2 (y_i - \bar{y})^2 \right)^{1/2} \\
&= (a^2)^{1/2} \left(\frac{1}{n-1} \sum_{i=1}^{n} (y_i - \bar{y})^2 \right)^{1/2}.
\end{aligned}
\tag{10.52}
$$

So $s_z = as_y$ applies when $a \geq 0$ and $s_z = -as_y$ applies when $a < 0$. But this corresponds to $s_z = |a|s_y$. □

We reproduce Theorem 10.7 as an example using the following **R** codes once for the $a \geq 0$ case and once for the $a < 0$ case

```
# a >= 0
D  = read.csv("./_data/110-descriptive-statistics.csv")  # loading the data set
y  = D$PRE                                               # pre-intervention BDI-II data
a  = 2                                                   # multiplication constant
b  = 5                                                   # addition constant
z  = a*y + b                                             # z_i = ay_i + b
sz = sd(z)                                               # empirical standard deviation of z
```

```
Empirical standard deviation after transformation: 3.48
```

```
sz = a*sd(y)  # empirical standard deviation according to theorem
```

```
Empirical standard deviation after transformation: 3.48
```

```
# a < 0
D  = read.csv("./_data/110-descriptive-statistics.csv")  # loading the data set
y  = D$PRE                                               # pre-intervention BDI-II data
a  = -3                                                  # multiplication constant
b  = 10                                                  # addition constant
z  = a*y + b                                             # z_i = ay_i + b
sz = sd(z)                                               # empirical standard deviation of z
```

```
Empirical standard deviation after transformation: 5.221
```

```
sz = -a*sd(y)  # empirical standard deviation according to theorem
```

```
Empirical standard deviation after transformation: 5.221
```

Part II

Imperative programming

On the importance of imperative programming in psychology

Today, scientific data are usually available in digital form. To evaluate these digital data, they are analyzed with the help of a computer. For this purpose, one writes special computer programs that are tailored precisely to the research question at hand. Such programs are called data analysis scripts.

Computer-assisted data analysis follows a typical structure. First, a digital data set is imported and cleaned in order to correct erroneous or incomplete data and prepare them for analysis. Descriptive statistics are then usually computed and visualized in order to obtain an initial overview of the data. This is often followed by a step of probabilistic modeling and inference, in which models based on probability theory are used to test hypotheses or make predictions. Finally, the results are documented and presented clearly, again with the help of the computer.

The central element of computer-assisted data analysis is the *data analysis script*, a text file that documents all steps from the raw data to the final data visualization and thus makes data analysis processes transparent and reproducible. A data analysis script enables third parties to reproduce scientific results, which is a cornerstone of good scientific practice. Data analysis scripts are therefore indispensable tools of everyday scientific work and essential components of scientific publications. In the following sections, starting from a discussion of some basic concepts from computer science and structure-forming elements of imperative programming, we will become familiar with the development of data analysis scripts in the programming environment R.

Various digital tools are used to analyze psychological data. Particularly common are freely available programming languages and platforms such as R, which is widely used in data science, statistics, and psychology, and Python, with its broad range of applications in data science, machine learning, and artificial intelligence. The free programming language Julia, which is in fact optimized for data analysis, still occupies more of a niche and currently has not yet found wide adoption, especially in industrial applications. Free programming languages have the advantage that they are often further developed by a broad community of users and can in this way develop a versatile infrastructure.

In addition, commercial programming environments whose source code cannot be inspected continue to be used in psychology. In neuroimaging and engineering applications, for example, the commercial programming environment MATLAB remains popular even today. Some programs that now seem rather old-fashioned also continue to be used, including SPSS, which was very widespread in the social sciences and psychology around the turn of the millennium, JMP, with areas of application in biology and psychology, and STATA, which was popular in medicine and economics, among other fields.

The popularity of digital tools is subject to constant change. One way to measure the current popularity of different programming languages and environments is to quantify Google searches for programming language tutorials. An overview of current trends is provided by the PYPL Index, which is based on the analysis of such search queries.

In summary, digitalization has a lasting impact on science in particular. Effective research data management poses an acute challenge that researchers must face. Programming is increasingly becoming a central craft of scientific work, and basic knowledge of computer science has become indispensable in the modern working world. This applies not least to psychotherapists, for example in the context of online interventions, digitally supported treatment services, or psychotherapy research.

11 Basic concepts of computer science

In ordinary usage (cf. Wikipedia), computer science is the science of the systematic representation, storage, processing, and transmission of information, with particular emphasis on the automatic processing of information by computers. Computer science is both a basic and formal science and an engineering discipline.

Central components of computer science are, on the one hand, *computers* and, on the other hand, *algorithms and programs*. *Computers* are machines that can store data and carry out simple data operations. They perform these simple operations at extremely high speed. The universality of computers rests on the fact that they can store both data and programs.

Programs are *algorithms* formulated in a programming language. *Algorithms*, in turn, are sequences of *instructions* that prescribe certain *operations*. In algorithms, one distinguishes different levels: the *description level*, as found, for example, in a recipe, assembly instructions, or a data analysis script; the *instructions* of an algorithm, such as "mix flour and water", the pictorial depiction of screwing two parts together, or the instruction `x = c(1, 2, 3)` in **R**; and finally the *execution level*, that is, the actual cooking process, the assembly of a piece of furniture, or the execution of the data analysis script.

Computer science is divided into several subfields that may also be relevant to psychology. *Theoretical computer science* deals with the foundations of computation, such as automata theory, computability theory, and complexity theory. *Applied computer science* concerns the development and use of application software, questions of human-computer interaction, and the social effects of computer science. *Computer engineering* examines the hardware side of computer science, including microprocessor technology, computer architecture, and network technology. Finally, *practical computer science* deals with concrete implementation in the form of programming, the development and analysis of algorithms, and the construction and use of databases. In psychological basic research, practical computer science is the subfield that is applied most often. However, questions from complexity theory, and thus from theoretical computer science, are also of interest in the development of quantitative psychological theories (cf. Van Rooij & Wareham (2012)), and human-computer interaction is one of the topic areas of modern work psychology (cf. Dahm (2005)).

In addition, there are many special areas of computer science that are closely connected with psychology. These include, in particular, the areas of *machine learning* and *artificial intelligence*, which have their origins in quantitative cognitive psychology and today make it possible to analyze large amounts of data automatically and to make predictions. Another important field is *computer graphics and visual computing*, which deals with image recognition and image synthesis as well as the design of virtual reality (VR) and augmented reality (AR). These technologies use findings from perceptual psychology on the one hand and, on the other hand, open up new possibilities for experiments, therapies, and training in psychology. *Computational linguistics* contributes methods of speech recognition and speech synthesis that help analyze, model, and make human communication usable

in applications such as chatbots or assistance systems. Finally, *bioinformatics* also plays a major role for psychology, for example by developing methods for evaluating medical imaging procedures that are also used in neuropsychological research and diagnostics.

11.1 Computer architecture

Even though we do not want to go into the technical aspects of computer science in depth here, for imperative programming it is helpful to have at least a rudimentary understanding of the typical physical structure, that is, the hardware, of a computer. A computer consists of several essential hardware components that work together to handle all computation and storage tasks. At the center is the main circuit board, also called the motherboard or mainboard. It is the central printed circuit board on which important components such as the processor, memory, and connectors are linked to one another.

The heart of a computer is the CPU (central processing unit, also called the microprocessor), which combines the arithmetic unit, the control unit, and the registers of the system. It performs all basic computations, controls the flow of data, and coordinates the processes. In addition, it has a small but very fast *cache*, a volatile memory that temporarily stores frequently needed data in order to speed up processing. Typical examples of current CPUs are Intel Core i processors, AMD Ryzen processors, or Apple Silicon M processors.

The CPU is complemented by RAM (random access memory), the temporary, volatile working memory of the system. During the computer's runtime, the programs and data currently needed are stored here to enable fast access. RAM is limited in its capacity, for example to 32 to 64 GB in many modern systems.

Mass storage, the computer's stationary storage, is used for the permanent storage of data and programs. Today, SSDs (solid state drives) are usually used for this purpose; they are much faster and more robust than conventional magnetic hard drives. Cloud storage is also increasingly used as a complement to, or replacement for, local mass storage.

Another important component is the GPU (graphics processing unit), a powerful processor optimized specifically for visualization tasks. The GPU relieves the CPU in graphics-intensive applications such as 3D rendering or video editing and also plays a growing role in computationally intensive tasks such as training neural networks, where it can effectively support the CPU.

The basic functional architecture of modern computers goes back to Von Neumann (1945) and is therefore referred to as the *Von Neumann architecture*. The decisive characteristic of the Von Neumann architecture is that both instructions (program instructions) and data are stored in the same memory, namely in the form of binary numbers. The central processing unit (CPU) reads both the instructions and the data from this shared memory. As a result, programs can be changed dynamically during execution or even generated anew, since in memory they are present merely as data.

Specifically, Von Neumann (1945) describes the following computer structure: the computer is composed of the basic functional units control unit, arithmetic unit, memory, input unit, and output unit. Programs and data are entered into memory together, so that data, programs, and intermediate and final results are stored in the same memory area. The memory itself is divided into equally sized, numbered (addressed) cells whose

contents can be retrieved or changed selectively via the respective address. The instructions of a program typically lie in consecutive memory cells, so that the control unit can call the next instruction simply by increasing the instruction address by one. This principle of sequentially processing instructions is the basic principle of imperative programming and is therefore very closely connected with the Von Neumann architecture. Special jump instructions also make it possible to deviate from this fixed order and jump to another point in the program. The basic instructions include arithmetic operations such as addition and multiplication, logical comparisons such as logical AND or OR, and transport instructions by which data are transferred, for example, from the input unit to memory or from memory to the arithmetic unit. All data, that is, both instructions and addresses, are encoded in binary. A suitable circuit technology in the computer handles the binary encoding and decoding of the information.

11.2 Algorithms and programs

A *real-world problem* denotes the problem that is to be solved with the help of a computer. A typical example is the evaluation of questionnaire data from a psychological study. To work on such a problem, one first creates a *problem specification*, that is, the precise verbal formulation of the real-world problem, as one might find it in the methods section of a scientific publication. On this basis, an *algorithm* is designed, that is, a sequence of instructions for solving the problem, for example reading in the data, computing descriptive statistics, and carrying out a statistical test. Finally, the algorithm is implemented in the form of a *program*, that is, as a text file written in a programming language that can be executed by a computer.

Definition 11.1 (Algorithm)**.** An algorithm is a sequence of instructions for deriving certain output data from certain input data, where the following conditions must be satisfied:

- Finiteness. The sequence of instructions is completely described in a finite text.
- Effectiveness. Each instruction must actually be executable.
- Termination. The algorithm ends after finitely many instructions.
- Determinacy. The course of the algorithm is prescribed at every point in time.

If E denotes the set of admissible input data and A denotes the set of admissible output data, then an algorithm is therefore a function:

$$f : E \to A, \quad e \mapsto f(e). \tag{11.1}$$

Conversely, functions that can be described by an algorithm are called *computable functions*.

•

11.3 Programming languages

Programming languages specify the rules that a program must obey. They define a *syntax*, that is, the vocabulary and structure of a program, as well as the *semantics*, that is,

the meaning of the permitted instructions. In the following, we give some terminological clarifications for categorizing different forms of programming languages. Here one distinguishes *machine language* and so-called *higher-level programming languages*, different *generations of programming languages*, the *imperative programming* that is the focus here from *declarative programming*, and *compiled* from *interpreted* programming languages.

Machine language and higher-level programming languages

Machine language consists of elementary operation instructions such as storing, comparing, or adding, which are encoded as binary numbers. Table 11.1 shows some examples of such instructions:

Table 11.1 Examples of machine language

Instruction	Binary code
Add contents of R1 to contents of R2	1001 0010
Increase contents of R by 1	1001 0110
Transfer contents of R1 to R3	0010 0011

Programs written in machine language are called machine programs. De facto, a computer executes only machine programs, because these are understood directly by the hardware. For humans, however, programming in machine language is very laborious, which is why higher-level programming languages are predominantly used in practice.

Higher-level programming languages use words and sentences modeled on human language to make programming more understandable and efficient. Programs in higher-level programming languages are then translated into machine language by a *compiler* or an *interpreter* so that they can be executed by the computer. Well-known examples of higher-level programming languages are FORTRAN, COBOL, C++, Python, and **R**.

Generations of programming languages

The *first-generation programming languages (1GL)* are the machine languages. Programs are written directly in the form of binary numbers, for example `10110000 01100001`, which corresponds to `B0 61` in hexadecimal notation. Such machine languages were used mainly on the first vacuum-tube and relay computers such as ENIAC or early IBM mainframes of the 1940s and early 1950s.

With the *second generation (2GL)*, assembly languages emerged from the 1950s on as the first form of symbolic programming. Here, human-readable abbreviations are used, although they are still processor-specific. An example of such a processor-specific Intel instruction is `MOV AL, 61H`. This generation of programming languages was used mainly on mainframe computers such as the IBM 704 or early mainframes of the 1950s and 1960s, which already used transistors instead of vacuum tubes.

From the 1970s onward, the *third-generation programming languages (3GL)* became established. These include higher-level programming languages such as FORTRAN, C, C++, and Java. They are much more programming-friendly and, above all, processor-independent, which considerably improves the portability of programs. Such languages were first used widely on minicomputers such as the DEC PDP-11 and later on workstations, PCs, and servers.

Finally, from the 1980s onward, the *fourth-generation programming languages (4GL)* developed; they include languages such as Python, MATLAB, and **R**. These are characterized by minimizing code overhead, high flexibility, support for multiparadigmatic approaches, and a high degree of automation. They are used mainly on modern PCs, high-performance computing clusters (HPC clusters), and cloud-based systems, which became increasingly available from the 1990s onward.

Imperative and declarative programming

In *imperative programming* (from *imperare*, Latin for "to command"), the path to a solution is specified as a sequence of instructions or commands. These commands process data that are addressed via variables. Within imperative programming, two different approaches are distinguished: *procedural* imperative programming and *object-oriented* imperative programming.

In *procedural imperative programming*, data and the commands that manipulate them are treated separately. The central structural concept here is procedures or functions, which encapsulate reusable units of instructions. By contrast, in *object-oriented imperative programming*, data and the commands that act on them are combined as objects. These objects are at the same time data carriers and providers of functions and form the central structural concept of this approach.

In practice, mixed forms of the two paradigms are often used, because the approaches can be combined well depending on the problem. An example of such a mixed form is the newer programming language Julia, which does not offer a classical object-oriented approach but instead a *multiple-dispatch* approach: functions can have different implementations depending on the types of their arguments and can thus combine aspects of object-oriented and procedural programming.

In *declarative programming*, the focus is not on the path to the solution but on the description of the problem itself. The programmer formulates what is to be achieved, and not in detail how the computer is to reach this goal step by step. The path to the solution is determined automatically by the system on the basis of given rules and mechanisms.

A typical feature of declarative approaches is that the focus is on defining relations, conditions, or goals rather than on an explicit sequence of commands. The programmer thus describes the desired result in a formal language, and the computer independently searches for a solution that satisfies these conditions.

Well-known approaches to declarative programming are, for example, logic programming, as implemented in the language Prolog, or functional programming, as in Haskell. In logic programming, facts and rules are formulated from which the system draws conclusions by inference. In functional programming, computations are described by the composition of functions, while the state of the data is not changed explicitly. Declarative programming is particularly suitable for problems in which the path to the solution can be complex and varied, but the desired properties of the result can be clearly specified. Typical application areas include database queries (for example SQL), knowledge bases, constraint solving, and formal proofs.

A modern example of the use of declarative programming is the system Lean, a so-called *theorem prover*. Lean is used to formally prove mathematical theorems. Here the user formulates the axioms, definitions, and statements to be proved in a mixture of declarative and interactive commands. The system automatically checks whether the specified steps

are valid and completes the missing parts of the proof according to fixed rules. In this way, Lean combines declarative programming with automated proof and makes it possible to develop large mathematical theories in a formally correct way. For an introduction to Lean in the context of mathematical proof, see, for example, The Mechanics of Proof.

Compiled and interpreted programming languages

Compiled programming languages are characterized by the fact that the entire source code is translated into machine language before execution. A special program, the so-called *compiler*, is used for this translation. The machine code generated by the compiler is executed directly by the processor, which enables the program to run very quickly. Since the executable program is already available in machine language, it no longer has to be translated at runtime. However, whenever the source code is changed, it is necessary to compile the program again. Typical examples of compiled programming languages are C, C++, and Rust.

By contrast, in *interpreted programming languages*, the source code is translated into a machine-like language during execution. This is done by a special program, the so-called *interpreter*. Since translation and execution take place at the same time, execution is usually slower than in compiled programs. One advantage, however, is that after changes no separate compilation step is required; the program can be executed immediately. Typical examples of interpreted programming languages are Python and **R**.

In the sense of the concepts introduced here, **R** is an interpreted multiparadigmatic fourth-generation programming language that supports object-oriented concepts but is often used procedurally and functionally and is tailored to statistical data analysis.

11.4 R

R is a programming language and a software package originally developed by Ross Ihaka (Ihaka & Gentleman (1996)). It is a free dialect of the proprietary software S, introduced by Becker et al. (Becker et al. (1988)). Today, **R** is developed and maintained by the R Core Team and the R Foundation. **R** has a large and active community that has so far contributed more than 20,000 **R** packages, which extend the system with numerous additions. The core of **R** deliberately develops in an evolutionary and conservative way, while the large number of R packages enables consistent and progressive further development. **R** can be easily downloaded and installed from CRAN.

With **R**, one can load, manipulate, and save data sets in different formats. The language makes it possible to carry out a wide variety of computations on different data structures. In addition, **R** offers a broad spectrum of statistical analysis methods that can be applied directly to the data. It is also typical to write reproducible data analysis scripts with which analyses can be automated and results can be documented transparently. Finally, **R** is very well suited for creating figures and visualizations in order to present data clearly.

However, some things that are important in psychological data science can be implemented only to a limited extent with **R**. For example, **R** per se lacks a modern and appealing development environment of its own; embedding **R** in VS Code remedies this. In the area of scientific computing, **R** reaches its limits. Here, languages such as Python, MATLAB, or Julia often offer more powerful and flexible tools. Finally, **R** is hardly suitable for

programming psychological experiments, for which Python or MATLAB are also more commonly used.

There are many ways to get help with **R**. Quite classically, a quick search on Google, a request to ChatGPT, or an interaction with Copilot in VS Code often already helps. During programming, and when one already knows the name of a function, one can call help directly in **R**:

```
?mean
help(mean)
```

For more comprehensive, longer tutorials and background information, it is worth taking a look at the so-called *vignettes* of installed packages:

```
browseVignettes()
```

In addition, there are numerous helpful online resources, for example the specialized search engine rseek or the many articles and guides on R-Bloggers, although most information on these platforms is certainly also part of the databases accessible through chatbots such as ChatGPT, Gemini, Claude, or DeepSeek.

11.5 Visual studio code

Visual Studio Code (VS Code) is a free source code editor from Microsoft that has been available for Windows, macOS, and Linux since 2015. Roughly speaking, VS Code is something like the perhaps better-known RStudio environment, except that it is for all programming languages. VS Code can be flexibly extended through extensions and used as an integrated development environment (IDE) for almost any language, for example **R**, Python, Julia, Shell, Quarto, and many more. Since 2018, VS Code has been regarded as the most popular editor overall, as annual surveys show. Remarkably, this means that a Microsoft product in particular is also the most-used editor in the Linux world. VS Code is highly configurable, strongly shaped by the community, and enables the synchronization of settings and extensions across multiple devices via Microsoft's GitHub. Detailed documentation with instructions for use can be found at www.code.visualstudio.com/docs. The corresponding documentation R in Visual Studio Code provides an introduction to working with **R** in VS Code.

12 Arithmetic and variables

12.1 Arithmetic

If one opens an **R** terminal, one can first use it as a calculator, as the following example shows.

```
1+1                                    # addition
```

```
[1] 2
```

In the terminal, this shows, on the one hand, the result 2 and, on the other hand, with `[1]`, an indication that the result is the first entry in the result vector generated by **R**. We will discuss vectors in detail later. Further examples of using an **R** terminal as a calculator are the following.

```
2*3                                    # multiplication
```

```
[1] 6
```

```
sqrt(2)                                # square root
```

```
[1] 1.414214
```

```
exp(0)                                 # exponential function
```

```
[1] 1
```

```
log(1)                                 # logarithm function
```

```
[1] 0
```

Table 12.1 gives an initial overview of the arithmetic operators available in Base **R**. In addition to the familiar operations of addition, subtraction, multiplication, division, and exponentiation, Base **R** also offers special operators for more advanced arithmetic computations. These include, for example, *matrix multiplication*, *integer division* (`5 %/% 2 = 2`), and the *modulo operation*, which returns the remainder of an integer division (`5 %% 2 = 1`).

Table 12.1 Examples of arithmetic operators in Base **R**.

Operation	Operator
Addition	`+`
Subtraction	`-`
Multiplication	`*`
Division	`/`
Power	`^`
Matrix multiplication	`%*%`
Integer division	`%/%`
Modulo operation	`%%`

In addition to the arithmetic operators for combining two numbers, Base **R** naturally also offers the possibility of evaluating standard mathematical functions. Table 12.2 gives an initial overview.

Table 12.2 Examples of mathematical functions in Base **R**.

Function	Call
Exponential function	`exp(x)`
Logarithm function	`log(x)`
Absolute value	`abs(x)`
Square root	`sqrt(x)`
Round up	`ceiling(x)`
Round down	`floor(x)`
Mathematical rounding	`round(x)`

Although the operators listed in Table 12.1 and the functions listed in Table 12.2 seem to have a somewhat different character at first glance, Base **R** does not formally distinguish between operators and functions. In particular, operators can also be used and understood as functions of several arguments by means of so-called *infix notation*. The **R** code below shows how the arithmetic combination $2 + 3$ can be understood as the execution of a function of the form $+ : \mathbb{R}^2 \to \mathbb{R}$ with the name "+".

```
`+`(2,3)                                    # infix notation for 2 + 3
```

```
[1] 5
```

Finally, like every programming language, Base **R** also offers the possibility of evaluating expressions with respect to their logical content. Here, logic is to be understood in the sense of the propositional logic from Chapter 1, in which statements can take one of two values, true (TRUE) or false (FALSE). The relational operators `<`, `<=`, `>`, and `>=` listed in Table 12.3 are usually applied to numerical values. The equality operators `==` and `!=` can be applied to arbitrary data structures. Finally, the operators for combining logical values, `&` and `|`, correspond to the concepts of logical conjunction ("and") and disjunction ("non-exclusive or") known from Chapter 1.

Table 12.3 Examples of logical operators in Base **R**.

Logical connection	Operator
Equal	`==`
Unequal	`!=`
Conjunction	`&`
Disjunction	`|`
Less than	`<`
Less than or equal	`<=`

The operators and functions listed in Table 12.1, Table 12.2, and Table 12.3 represent only a small excerpt of the operators and functions provided by Base **R**. A complete overview is provided by the call `names(methods:::.BasicFunsList)`, which lists the names of all functions provided by Base **R**. We will get to know many of these functions later.

```
names(methods:::.BasicFunsList)
```

```
  [1] "$"                 "$<-"               "["
  [4] "[<-"               "[["                "[[<-"
  [7] "crossprod"         "tcrossprod"        "xtfrm"
 [10] "c"                 "all"               "any"
 [13] "sum"               "prod"              "max"
 [16] "min"               "range"             "cummax"
 [19] "rep"               "^"                 "cos"
 [22] "levels<-"          "cumsum"            "asin"
 [25] "anyNA"             "<="                "Conj"
 [28] "exp"               "is.matrix"         "log10"
 [31] "as.environment"    "ceiling"           "asinh"
 [34] "abs"               "as.raw"            "is.infinite"
 [37] "is.array"          "floor"             "=="
 [40] "sign"              "cummin"            "|"
 [43] "round"             "Re"                "!"
 [46] "is.numeric"        "acosh"             "!="
 [49] "names<-"           "&"                 "atanh"
 [52] "sinh"              ">="                "sin"
 [55] "*"                 "atan"              "+"
 [58] "length"            "length<-"          "sinpi"
 [61] "-"                 "is.nan"            "/"
 [64] "sqrt"              "%/%"               "as.numeric"
 [67] "seq.int"           "trunc"             "digamma"
 [70] "acos"              "<"                 "as.logical"
 [73] "cosh"              "Mod"               "tanpi"
 [76] "%*%"               "dimnames<-"        "log2"
 [79] ">"                 "tanh"              "is.na"
 [82] "dim"               "signif"            "gamma"
 [85] "is.finite"         "as.integer"        "dim<-"
 [88] "as.double"         "lgamma"            "Arg"
 [91] "log1p"             "trigamma"          "%%"
 [94] "tan"               "cumprod"           "Im"
 [97] "expm1"             "as.call"           "log"
[100] "as.complex"        "cospi"             "as.character"
[103] "dimnames"          "names"             "("
[106] ":"                 "="                 "@"
[109] "{"                 "~"                 "&&"
[112] ".C"                "baseenv"           "quote"
[115] "::"                "<-"                "is.name"
[118] "if"                "||"                "attr<-"
[121] "untracemem"        ".cache_class"      "substitute"
[124] "interactive"       "is.call"           "switch"
[127] "function"          "is.single"         "is.null"
[130] "is.language"       "is.pairlist"       ".External.graphics"
[133] "declare"           "globalenv"         "class<-"
[136] ".Primitive"        "is.logical"        "enc2utf8"
[139] "UseMethod"         ".subset"           "proc.time"
[142] "enc2native"        "repeat"            ":::"
[145] "<<-"               "@<-"               "missing"
[148] "nargs"             "isS4"              ".isMethodsDispatchOn"
[151] "forceAndCall"      "Exec"              ".primTrace"
[154] "storage.mode<-"    ".Call"             "unclass"
[157] "gc.time"           ".subset2"          "environment<-"
[160] "emptyenv"          "seq_len"           ".External2"
[163] "is.symbol"         "class"             "on.exit"
[166] "is.raw"            "for"               "is.complex"
[169] "list"              "invisible"         "is.character"
[172] "oldClass<-"        "is.environment"    "attributes"
[175] "break"             "return"            "attr"
[178] "tracemem"          "next"              ".Call.graphics"
[181] "standardGeneric"   "is.atomic"         "retracemem"
```

```
[184] "expression"          "is.expression"       "call"
[187] "is.object"           "pos.to.env"          "attributes<-"
[190] ".primUntrace"        "...length"           "Tailcall"
[193] ".External"           "oldClass"            ".Internal"
[196] ".Fortran"            "browser"             "is.double"
[199] ".class2"             "while"               "nzchar"
[202] "is.list"             "lazyLoadDBfetch"     "unCfillPOSIXlt"
[205] "...elt"              "...names"            "is.integer"
[208] "is.function"         "is.recursive"        "seq_along"
[211] "unlist"              "as.vector"           "lengths"
```

12.2 Precedence

An important property when using operators is their *precedence*, as defined by the respective programming language. These are rules specified by the developers of programming languages that determine the order of operations. From mathematics, one is especially familiar with the precedence rule "multiplication and division before addition and subtraction". This rule says that in expressions containing both products or divisions and sums or differences, the products or divisions are carried out first, and their results then enter the formation of sums or differences second. Thus, for example,

$$2 \cdot 5 - 3 = 10 - 3 = 7 \tag{12.1}$$

and not, for instance,

$$2 \cdot 5 - 3 \neq 2 \cdot 2 = 4. \tag{12.2}$$

We assume as known that precedence rules can be emphasized or overridden by parentheses. For example, the expression in Equation 12.1 is equivalent to

$$(2 \cdot 5) - 3 = 10 - 3 = 7 \tag{12.3}$$

and the calculation intended in Equation 12.2 can be made correct by placing parentheses as

$$2 \cdot (5 - 3) = 2 \cdot 2 = 4 \tag{12.4}$$

These familiar calculation rules and their overriding by parentheses are implemented correspondingly in **R**:

```
2*5 - 3
```

```
[1] 7
```

```
2*(5 - 3)
```

```
[1] 4
```

Another precedence rule assumed to be known is that powers have higher precedence than products or divisions and sums or differences. Thus, for example,

$$2^3 + 3 = (2 \cdot 2 \cdot 2) + 3 = 8 + 3 = 11 \tag{12.5}$$

and not, for instance,

$$2^3 + 3 \neq 2^{3+3} = 2^6 = 64. \tag{12.6}$$

Correspondingly, in **R** we have

```
2^3 + 3
```

```
[1] 11
```

```
2^(3 + 3)
```

```
[1] 64
```

An important special feature of the precedence rules in **R** is that *unary signs*, that is, signs that identify a number as a negative number, also fall within the domain of operator precedence. Although one might be inclined to interpret an expression of the form -1^2 as $(-1)^2 = 1$, **R** nevertheless follows the rule that in -1^2 the power is evaluated first and the sign afterward, so that $-1^2 = -(1^2) = -1$ holds:

```
-1^2                    # -(1^2)   = -1
```

```
[1] -1
```

```
(-1)^2                  # (-1)^2   =  1
```

```
[1] 1
```

Furthermore, in **R**, powers are processed from right to left, whereas products, divisions, sums, and differences are processed from left to right. For example,

```
2^2^3                   # 2^(2^3)  = 2^8 = 256
```

```
[1] 256
```

```
(2^2)^3                 # (2^2)^3  = 4^3 = 64
```

```
[1] 64
```

but

```
2+3/4*5                 # 2+(3/4)*5 = 2+(0.75*5) = 2+3.75 = 5.75
```

```
[1] 5.75
```

```
2+3/(4*5)               # 2+3/(4*5) = 2+3/20 = 2+0.15 = 2.15
```

```
[1] 2.15
```

In general, in programming one should not guess precedence rules or try to derive them logically. Nor should one trust that the developers of a programming language will probably have implemented the precedence rules according to one's own state of knowledge. Instead, one should always check computations critically and, to be safe, emphasize the intended calculation with parentheses once too often rather than once too little.

In addition to the precedence rules considered here for the basic arithmetic operations, **R** has a whole series of further rules for dealing with many other operators, of which we have so far become familiar with only very few. When becoming familiar with a new operator while learning a programming language, one should therefore also make sure to clarify the precedence of this operator. The text from the **R** documentation shown in Figure 12.1, which can be called with the command `?Syntax`, discusses the precedence rules that apply in **R** in full and should be consulted frequently when working with **R**.

Operator Syntax and Precedence

Description

Outlines `R` syntax and gives the precedence of operators.

Details

The following unary and binary operators are defined. They are listed in precedence groups, from highest to lowest.

`:: :::`	access variables in a namespace
`$ @`	component / slot extraction
`[ [[`	indexing
`^`	exponentiation (right to left)
`- +`	unary minus and plus
`:`	sequence operator
`%any% |>`	special operators (including `%%` and `%/%`)
`* /`	multiply, divide
`+ -`	(binary) add, subtract
`< > <= >= == !=`	ordering and comparison
`!`	negation
`& &&`	and
`| ||`	or
`~`	as in formulae
`-> ->>`	rightwards assignment
`<- <<-`	assignment (right to left)
`=`	assignment (right to left)
`?`	help (unary and binary)

Within an expression operators of equal precedence are evaluated from left to right except where indicated. (Note that `=` is not necessarily an operator.)

Figure 12.1 Precedence rules in **R**.

12.3 Variables

In programming, *variables* are abstract containers for data whose contents can be changed during the execution of a program. Variables are denoted by names in program code

and, during program execution, have an address in working memory. Variables can be of different *types*. This means that variables represent different kinds of data, and only those kinds. In traditional programming languages, the type of a variable is explicitly *declared*. At the beginning of program execution, it is specified which kind of data a particular variable can represent. The declaration of a variable `A`, which, for example, can represent integers and only integers, has roughly the form

```
VAR A : INTEGER     # a is a variable of type integer
```

Subsequently, the variable `A` can be assigned a value of type integer, for example in the form

```
A := 1              # variable A is assigned the numerical value 1
```

In a 4GL language such as **R**, the variable type is usually specified directly by an initialization statement. Thus, the following **R** code simultaneously declares the variable `a` as being of type `double` (decimal number) with the value `1`,

```
a = 1               # a is a variable of type double; its value is 1
```

In the vast majority of programming languages, the assignment command is `=`. **R** is special in this respect because it allows the additional assignment commands `<-` and `->`: `<-` assigns from right to left, `->` from left to right. The usual assignment from right to left is also achieved by `=`. The officially recommended assignment command for **R** is `<-`. This is, however, highly idiosyncratic, and we mostly use `=`, as in every other programming language. A first example should illustrate how to work with variables.

Example

We consider the following task:

"Lina goes to the stationery store and buys four notebooks and two pens. One notebook costs 1 euro and one pen costs 3 euros. How many items does Lina buy in total, and how many euros does Lina have to pay in total?"

To solve the task, we first define the variables `number_notebooks` and `number_pens` and assign to them their respective values from the task.

```
number_notebooks = 4
number_pens      = 2
```

After assignment, the variables with their assigned values now exist in working memory, the so-called *workspace*. The command `ls()` displays the existing usable variables in working memory.

```
ls()
```

```
[1] "number_notebooks" "number_pens"      "pandoc_dir"       "quarto_bin_path"
```

The crucial point is that the variable names can now be used like numbers in computations. `print()` outputs variable values in an **R** terminal:

```
total_items = number_notebooks + number_pens
print(total_items)
```

```
[1] 6
```

To calculate the total price, we next define the variables `price_notebook` and `price_pen` according to the task specification.

```
price_notebook = 1
price_pen      = 2
```

The total price is then calculated as follows.

```
total_price = number_notebooks*price_notebook + number_pens*price_pen
print(total_price)
```

```
[1] 8
```

Variables in the workspace can also be deleted again. `rm()` (remove) allows individual variables to be deleted.

```
rm(total_price)
ls()
```

```
[1] "number_notebooks" "number_pens"      "pandoc_dir"       "price_notebook"
[5] "price_pen"        "quarto_bin_path"  "total_items"
```

`rm(list=ls())` deletes all variables.

```
rm(list = ls())
ls()
```

```
character(0)
```

Variable names

In principle, one is free to choose names for variables, with minor restrictions. Admissible variable names in **R**

- consist of letters, numbers, periods (.), and underscores (_),
- begin with a letter or a period, but then not followed by a number,
- must not be expressions already reserved in **R**, such as `for`, `if`, `NaN`, …

The help page `?reserved` provides assistance with the restrictions on variable names. A function for generating a syntactically admissible variable name from arbitrary suggestions is `make.names()`. Its help page also describes the rules for variable naming sketched here in further detail. In general, useful variables are short, to save space in the code, and meaningful, to increase human understanding of the code. Variables usually consist only of lowercase letters and underscores.

Variable representation

We now want to examine the relationship between variable names (identifiers) and variable contents (objects in working memory) in a little more detail. The following concepts and relationships are especially important when computations reach the limits of the available working memory and must be optimized.

We begin with the concept of *binding*. To do so, we first consider the following variable initialization:

```
x = 1
```

Intuitively, this statement creates a variable named `x` with the value `1`. At the technical level, two things happen (cf. Figure 12.2):

1. **R** creates an object (a vector with value 1) with working-memory address `lobstr::obj_addr(x)`.
2. **R** connects this object with the name `x` (*binding*), which references the object in working memory.

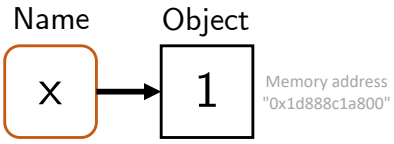


Figure 12.2 Binding of a name to an object in working memory.

Now consider the following statement:

```
y = x
```

Intuitively, this creates a variable named `y` with a value equal to the value of `x`. At the technical level, a new name `y` is created that references the same object in working memory as `x` (cf. Figure 12.3).

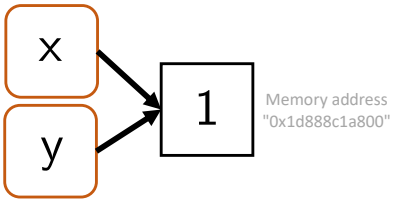


Figure 12.3 Binding of two names to one object in working memory.

One can use `lobstr::obj_addr(x)` and `lobstr::obj_addr(y)` to verify that the addresses of the object in working memory are identical:

```
print(lobstr::obj_addr(x))
```

```
[1] "0x21b2a51d508"
```

```
print(lobstr::obj_addr(y))
```

```
[1] "0x21b2a51d508"
```

Thus, the relevant object is *not* copied when assigned to `y`. This saves working memory.

Furthermore, **R** uses the so-called *copy-on-modify* principle. This principle states that an object in working memory is copied and receives a new address as soon as it is modified. The original object in working memory remains unchanged. Figure 12.4 and the following **R** code first illustrate the *copy-on-modify* principle:

```
x = 1       # object (e.g., 0x74b) is created; x references the object's memory address
y = x       # y references the same memory address as x (0x74b)
y = 3       # y is modified; a modified copy (e.g., 0xcd2) is stored
y           # y now references (0xcd2)
```

```
[1] 3
```

```
x           # x continues to reference (0x74b)
```

```
[1] 1
```

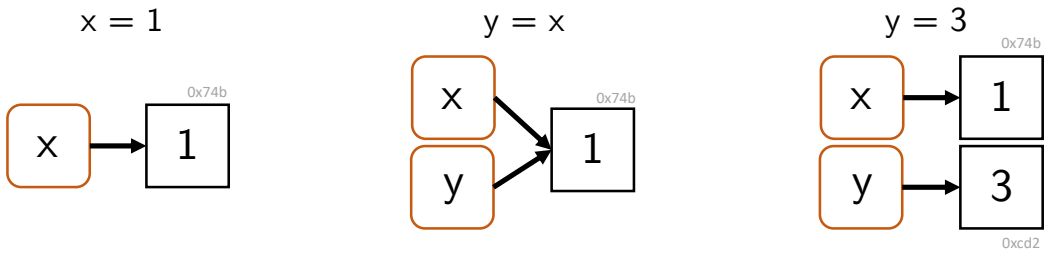


Figure 12.4 Copy-on-modify. Only after changing the value assigned to the identifier `y` is the value copied in working memory.

The fact that objects are copied in working memory when they are changed, while the original objects are in principle preserved unchanged, is also referred to as the *immutability* of objects in **R**. However, the immutability of objects in **R** does not go very far, as the following example shows. First, an object of a `1` is created in working memory and can be referenced with the variable name `x`. By *indexing* (cf. Chapter 13), the object is then readdressed in working memory when an intended modification takes place.

```
x   = 1                    # object is created; x references the object's memory address
print(lobstr::obj_addr(x))  # object's memory address
```

```
[1] "0x21b2c31ec48"
```

```
x[1] = 2                    # object is changed
print(lobstr::obj_addr(x))  # new memory address
```

```
[1] "0x21b2c445cb8"
```

The copy-on-modify principle also applies in the reverse order. That is, if an object to which another variable name points is changed, the originally referenced object does not change. The following **R** code may illustrate this.

```
x = 1       # object (e.g., 0x74b) is created; x references the object's memory address
y = x       # y references the same memory address as x (0x74b)
x = 3       # a new object (e.g., 0x2a08) is created; x now references 0x2a08
y           # y continues to reference the original object (0x74b)
```

```
[1] 1
```

Finally, *unbinding* of objects in working memory is also possible, for example as a consequence of the following **R** commands (cf. Figure 12.5):

```
x = 1       # x references object (0x74b)
x = "a"     # x references object (0x2a08); object (0x74b) is now unreferenced
```

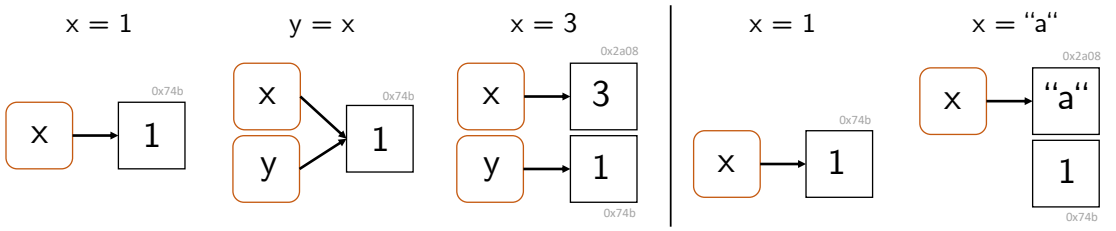


Figure 12.5 Unbinding. Allocated working memory remains that is not referenced by any identifier.

The deletion of unreferenced objects in working memory is called *garbage collection*. **R** automatically deletes unreferenced objects in working memory, although usually only when the memory is needed. In general, however, this works well, and it is usually not necessary to actively use the function `gc()` provided by **R** for garbage collection.

Representation of missing values

In data-analytic programming, one often has to deal with missing values, for example when an experimental measurement for a participant had to be aborted. To represent missing values, **R** uses `NA` (*not available*). If one attempts to calculate with `NA`, the result is usually again `NA`:

```
3 * NA      # multiplication of an NA value yields NA
```

```
[1] NA
```

```
NA < 2       # relational comparison of an NA value yields NA
```

```
[1] NA
```

```
NA^0         # missing value to the power of 0 yields 1 because every value to the power of 0 is one (?)
```

```
[1] 1
```

```
NA & FALSE   # logical conjunction with FALSE yields FALSE
```

```
[1] FALSE
```

One tests for `NA` with `is.na()`:

```
x = c(NA, 5, NA, 10)  # vector with NAs
x == NA               # no testing for NAs: 5 == NA is NA, not FALSE
```

```
[1] NA NA NA NA
```

```
is.na(x)              # logical test for NA
```

```
[1]  TRUE FALSE  TRUE FALSE
```

In addition to `NA`, **R** also has `NaN` as a way to represent mathematically undefined values. In contrast to `NA`, `NaN` is always of numerical type. The results when calculating with `NaN` are similar to those with `NA`.

```
3 * NaN      # multiplication of a NaN value yields NaN
```

```
[1] NaN
```

```
NaN < 2      # relational comparison of a NaN value yields NA
```

```
[1] NA
```

```
NaN^0        # undefined value to the power of 0 yields 1 because every value to the power of 0 is one (?)
```

```
[1] 1
```

```
NaN & FALSE  # logical conjunction with FALSE yields FALSE
```

```
[1] FALSE
```

One tests for `NaN` with `is.nan()`.

```
x = c(NaN, 5, NaN, 10)  # vector with NaNs
is.nan(x)               # logical test for NaN
```

13 Data structures

In every programming language, there are abstract containers for representing different kinds of data and working with them. In classical 3GL programming languages, one distinguishes *fundamental*, *composite*, and *user-defined* data structures.

Fundamental data structures are predefined in the programming language. Typical examples are containers for logical values (`TRUE`, `FALSE`), integers (`int8`), which, for example, represent the 256 values from -128 to 127, floating-point numbers (`single`, `double`) with different precision, or characters (`a`, `b`), which are often placed in quotation marks. These basic data types are typically associated with suitable operations, such as `AND/OR` for logical values, `+` and `-` for integers, `+`, `-`, `*`, and `/` for floating-point numbers, or concatenation for characters.

Composite data structures are also predefined and serve to combine several variables of the same type. In **R**, these include, for example, vectors, lists, arrays, matrices, and data frames. Specific operations also exist for composite structures, such as vector indexing or matrix multiplication.

Finally, in 3GL programming languages, *user-defined data structures* can also be created. They allow the programming environment to be adapted to specific application cases and are therefore especially relevant in practice. As a rule, they are assembled from predefined structures and, in the context of object-oriented programming, can also be associated with their own operations.

When one learns a programming language, an important first step is to become at least rudimentarily familiar with its data structures. One then gets to know the subtleties of data-structure representation in daily practical work with the programming language. To become familiar with the data structures of a programming language, the following guiding questions are therefore useful:

Fundamental data structures

- Which fundamental data structures does the language provide?
- Which operations on them are already defined?
- What is the syntax for defining a variable of a fundamental data type?
- What is the syntax for calling predefined operations?

Composite data structures

- Which containers and associated operations does the programming language provide?
- What is the syntax for working with a container?

User-defined data structures

- How does one create user-defined data structures and associated operations?
- What is the syntax for working with a user-defined data structure?

13.1 Data structures in R

As a 4GL language, **R** goes beyond the classical data structures of 3GL languages in its data structures. In **R**, first of all, everything is an *object*. Regardless of whether it is a single value, a vector, a function, or a complex data structure, everything is represented by **R** as an object. In general, the concept of an *object* denotes the possibility of representing both data and functionality in a single unit of meaning. Here, however, we are primarily interested only in data representation.

In general, one distinguishes in **R** between *atomic* objects and *recursive* objects. *Atomic objects* consist of components of the same data type. Typical examples are numerical vectors, logical vectors, or character strings. *Recursive objects*, by contrast, can contain components of different types. These include especially lists and data frames, which make it possible to combine heterogeneous data.

Every **R** object can be described uniquely by three central properties: its *mode*, its *length*, and, optionally, further *attributes*. The *mode* of an object specifies the basic data type of an object and is closely related to the *type* and *class* of an object, as explained below. In addition to the mode, every object has a *length*, which indicates how many elements it contains. For a vector, for example, the length is the number of values it contains; for a list, it is the number of its components. In addition, objects in **R** can be provided with additional *attributes*. Attributes are metadata that provide additional information about the object, such as the names of elements (`names`), dimensions (`dim`) for matrices and arrays, or class information (`class`), which is relevant for object-oriented programming in **R**.

Mode, type, and class

The classification of data structures in **R** is somewhat complex, because the concepts of the *mode*, the *type*, and the *class* of an object exist side by side. In practical application, the subtleties of this classification are usually not important, and the mode, type, and class of an object are also often identical.

Roughly speaking, the *mode* corresponds to the basic nature of an object and is chosen especially so that there is high compatibility with the data-structure classification system of **S** (Becker et al. (1988)). Examples of modes are `numeric`, `character`, `logical`, or `list`. The mode of an object can be retrieved with the function `mode()`, as the following example shows.

```
mode(42)                        # numeric
```

```
[1] "numeric"
```

```
mode(1L)                        # numeric
```

```
[1] "numeric"
```

```
mode("hello")                   # character
```

```
[1] "character"
```

```
mode(TRUE)                          # logical
```

```
[1] "logical"
```

```
mode(list(1, 2, 3))                 # list
```

```
[1] "list"
```

```
mode(factor(c("a", "b")))           # r factor => numeric
```

```
[1] "numeric"
```

```
mode(Sys.Date())                    # r date => numeric
```

```
[1] "numeric"
```

```
mode(lm(mpg ~ wt, data=mtcars))     # r linear model => list
```

```
[1] "list"
```

Somewhat closer to the internal representation of an object in memory, and therefore somewhat more technical in nature, is the *type* of an **R** object. This becomes particularly clear with respect to objects of mode `numeric`. Here, the classical 3GL type distinctions `integer` or `double` exist. Another example is the mode *factor*. As we will see later, factors in **R** look to the user like categorical data, often of character type. Their mode, however, is `numeric` and their type is `integer`. This is related to the fact that factors in **R** are often used at the user level to specify experimental conditions that usually have meaningful names such as "Treatment" or "Control", but in data analysis are represented by mathematical matrices with integer entries. The type of an object can be retrieved with the function `typeof()`, as the following example shows. Note the similarities and differences with the modes of the same objects called above.

```
typeof(42)                          # double
```

```
[1] "double"
```

```
typeof(1L)                          # integer
```

```
[1] "integer"
```

```
typeof("hello")                     # character
```

```
[1] "character"
```

```
typeof(TRUE)                    # logical
```

```
[1] "logical"
```

```
typeof(list(1,2,3))             # list
```

```
[1] "list"
```

```
typeof(factor(c("a", "b")))     # r factor => integer
```

```
[1] "integer"
```

```
typeof(Sys.Date())              # r date => double
```

```
[1] "double"
```

```
typeof(lm(mpg ~ wt, data=mtcars))   # r linear model => list
```

```
[1] "list"
```

Finally, the class of an object specifies the kind of object from the perspective of object-oriented programming in **R**. For an object, it is especially relevant how generic functions such as `print()` or `summary()` deal with it. In contrast to modes and types, programmers can specify the class of objects themselves and thus develop generic functions that are applicable to many kinds of objects and, despite having the same name, carry out different operations with different objects. The class of an object can be retrieved with the function `class()`, as the following example shows. Note the similarities and differences with the modes and types of the same objects called above.

```
class(42)                       # numeric
```

```
[1] "numeric"
```

```
class(1L)                       # integer
```

```
[1] "integer"
```

```
class("hello")                  # character
```

```
[1] "character"
```

```
class(TRUE)                     # logical
```

```
[1] "logical"
```

```
class(list(1,2,3))                  # list
```

```
[1] "list"
```

```
class(factor(c("a", "b")))          # r factor => "factor"
```

```
[1] "factor"
```

```
class(Sys.Date())                   # r date => "Date"
```

```
[1] "Date"
```

```
class(lm(mpg ~ wt, data=mtcars))    # r linear model => "lm"
```

```
[1] "lm"
```

In summary, the mode describes the general kind of an object, the type describes its internal, technical storage, and the class defines **R**'s perspective for the functional processing of the object. For many objects, these three concepts agree to a large extent, but for special objects such as factors, date values, or user-defined objects, they differ.

14 Vectors

In this section, we now want to get to know **R** vectors in more detail as a first fundamental data structure. Many principles that can be explained using vectors carry over analogously to more complex data structures in **R**, so that competent handling of the basic data structure considered here is indispensable in application contexts as well. It should be noted that the term *vector* in this section refers strictly to the corresponding **R** data structure, not to the mathematical object of the same name. Although **R** vectors resemble their mathematical counterpart in some properties and certain operations between **R** vectors can represent operations between the corresponding mathematical objects, vectors in imperative programming often have many more and different properties than the corresponding mathematical objects.

In general, when discussing a data structure, we first consider how it is *generated* using **R** code, by which properties it is *characterized*, how individual elements of the data structure can be *indexed* and edited, and which *arithmetic operations* are admissible for the data structure. Finally, we consider possible further special features of the data structure. In the context of **R** vectors, these include, for example, the concepts of *data type coercion*, *recycling*, or the representation of missing values in data sets.

R vectors are ordered sequences of data values. The individual data values of a vector are called the *elements* of the vector. Vectors whose elements are all of the same data type are called *atomic*. Here, the term *vector* always means an atomic vector. The central data types for vectors are the previously introduced *numeric*, (*double*, *integer*), *logical*, and *character* types. Figure 14.1 shows graphical representations of corresponding vectors.

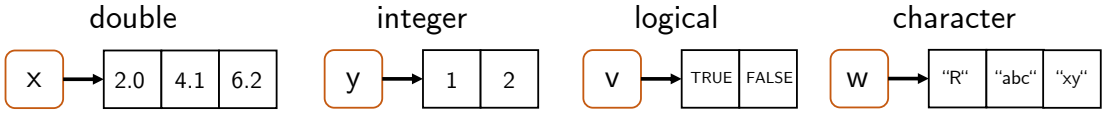


Figure 14.1 Vectors in **R**.

14.1 Generation

We first consider the generation of one-element vectors, which are also called *elementary values*. Like all data structures, elementary values are generated by assignments of the form

```
elementary_value = value
```

Here, the specification of `value` determines what kind of elementary value is generated.

Elementary values of type `numeric` can be generated by assigning a number. By default, this creates an elementary value of type `double`. The numbers can be specified without

decimal places, in decimal notation, or in scientific notation. Further possible values for generating an elementary value of type `double` are `Inf` and `-Inf` for positive and negative infinity and `NaN` (Not-a-Number) as a placeholder for numerical values.

```
x = 1      # one-element vector
typeof(x)  # of type double
```

```
[1] "double"
```

```
y = 2.1e2  # one-element vector
typeof(y)  # of type double
```

```
[1] "double"
```

```
z = Inf    # one-element vector
typeof(z)  # of type double
```

```
[1] "double"
```

Numeric elementary values of type `integer` are generated like `double` values without decimal places, followed by an `L` for `long integer`.

```
x = 1L     # one-element vector
typeof(x)  # of type integer
```

```
[1] "integer"
```

```
y = 200L   # one-element vector
typeof(y)  # of type integer
```

```
[1] "integer"
```

Elementary values of type `logical` can be generated by assigning the logical values `TRUE` or `FALSE`, or abbreviated as `T` or `F`.

```
x = TRUE   # one-element vector
typeof(x)  # of type logical
```

```
[1] "logical"
```

```
y = F      # one-element vector
typeof(y)  # of type logical
```

```
[1] "logical"
```

Finally, elementary values of type `character` are generated by assigning values in single quotes (`'a'`) or double quotes (`"a"`).

```
x = "a"     # one-element vector
typeof(x)   # of type character
```

```
[1] "character"
```

```
y = 'test'  # one-element vector
typeof(y)   # of type character
```

```
[1] "character"
```

To combine several elementary values into one vector, one concatenates them with the function `c()`. The term concatenation is derived from the Latin *catena* ("the chain").

```
x = c(1,2,3)          # numeric vector [1,2,3]
s = c("a", "b", "c")  # character vector ["a", "b", "c"]
l = c(TRUE, FALSE)    # logical vector [TRUE, FALSE]
```

```
x
```

```
[1] 1 2 3
```

```
s
```

```
[1] "a" "b" "c"
```

```
l
```

```
[1]  TRUE FALSE
```

Not only elementary values, but also vectors with elementary values or with other vectors can be concatenated. With the vector `x` above, for example, one generates the following vectors `y` and `z`:

```
y = c(0,x,4)  # numeric vector [0,1,2,3,4]
z = c(x,y)    # numeric vector [1,2,3,0,1,2,3,4]
```

```
y
```

```
[1] 0 1 2 3 4
```

```
z
```

```
[1] 1 2 3 0 1 2 3 4
```

Vectors do not have to be generated from already existing values; "empty" vectors of a desired data type can also be generated. **R** provides the functions `vector()` as well as `double()`, `integer()`, `logical()`, and `character()` for this purpose. This is potentially helpful for already occupying computer memory at the beginning of a longer computation, so that no memory has to be searched for during program execution, which can increase program runtime under certain circumstances.

```
# generate "empty" vectors with vector()
v = vector("double",3)     # double vector [0,0,0]
w = vector("integer",3)    # integer vector [0,0,0]
l = vector("logical",2)    # logical vector [FALSE, FALSE]
s = vector("character",4)  # character vector ["", "", "", ""]

# alternative generation of "empty" vectors
v = double(3)              # double vector [0,0,0]
w = integer(3)             # integer vector [0,0,0]
l = logical(2)             # logical vector [FALSE, FALSE]
s = character(4)           # character vector ["", "", "", ""]
```

However, the "empty" vectors generated in this way are not actually empty: `v` and `w` consist of zeros, `l` consists of the logical values `FALSE`, and `s` consists of the characters `""`. For initializing vectors as containers and then filling them in the context of a program, `vector()`, `double()`, `integer()`, `logical()`, and `character()` are therefore suitable only to a limited extent: if the code overlooks the filling of individual entries, computations are performed with the values generated by these functions, which is not necessarily intended. In the chapter on control structures, we will learn a method for initializing data structures while generating reasonably safe code.

Sequences

A frequent tool in programming is vectors that represent sequences of numerical values, for example as representations of the domains of univariate functions. **R** provides the so-called *colon operator* `:` and the function `seq()` and its variants for generating them.

The colon operator in the form `a:b` generates sequences of integer steps between a and b according to

$$(s_i)_{i=0,1,2...} \text{ with } s_i = a + i \text{ for } i = 0, 1, 2, ... \text{ and } s_i \leq b. \tag{14.1}$$

Here, "between `a` and `b`" specifically means that `a` is the first number of the generated sequence and `b` is the largest possible value. The following examples illustrate this.

```
x = 0:5      # 0 is the start of the sequence, 5 can be included
y = 1.5:6.1  # 1.5 is the start, the sequence ends at 5.5 < 6.1
```

```
x
```

```
[1] 0 1 2 3 4 5
```

```
y
```

```
[1] 1.5 2.5 3.5 4.5 5.5
```

The function `seq()` with the syntax `seq(from, to, by)`, where `from` denotes the start value, `to` the maximal end value, and `by` the step size, offers further possibilities for generating sequences with non-integer step sizes as well. The following examples illustrate this.

```
x_1 = seq(0,5)           # like 0:5
x_2 = seq(0,1,len = 5)   # 5 numbers between 0 (incl.) and 1 (incl.)
x_3 = seq(0,2,by = .15)  # 0.15 steps between 0 (incl.) and 2 (max.)
x_4 = seq(1,0,by = -.1)  # -0.1 steps between 1 (incl.) and 0 (min.)
```

```
x_1
```

```
[1] 0 1 2 3 4 5
```

```
x_2
```

```
[1] 0.00 0.25 0.50 0.75 1.00
```

```
x_3
```

```
 [1] 0.00 0.15 0.30 0.45 0.60 0.75 0.90 1.05 1.20 1.35 1.50 1.65 1.80 1.95
```

```
x_4
```

```
 [1] 1.0 0.9 0.8 0.7 0.6 0.5 0.4 0.3 0.2 0.1 0.0
```

The `seq()` variant `seq.int()` can be used to generate integer sequences and may be faster than `seq()` under certain circumstances. `seq_len()` generates integer sequences of a given length, and `seq_along()` generates integer sequences based on the entries of a vector.

```
x_1 = seq.int(0,5)           # like 0:5, [0,1,2,3,4,5]
x_2 = seq_len(5)             # natural numbers up to 5, [1,2,3,4,5]
x_3 = seq_along(c("a","b"))  # like seq_len(length(c("a", "b")))
```

```
x_1
```

```
[1] 0 1 2 3 4 5
```

```
x_2
```

```
[1] 1 2 3 4 5
```

```
x_3
```

```
[1] 1 2
```

14.2 Characterization

For characterizing vectors, the functions `length()` and `typeof()` are available; they output the number of entries of a vector and the data type of its entries, as the following examples illustrate.

```
x = 0:10   # vector
length(x)  # number of elements of the vector
```

```
[1] 11
```

```
x = 1:3L   # vector
typeof(x)  # data type of the atomic vector
```

```
[1] "integer"
```

```
y = c(T,F,T)  # vector
typeof(y)     # data type of the atomic vector
```

```
[1] "logical"
```

`is.logical()`, `is.double()`, `is.integer()`, and `is.character()` test the data type of a vector and return a logical binary value.

```
is.double(x)  # test the vector above
```

```
[1] FALSE
```

```
is.logical(y)  # test the vector above
```

```
[1] TRUE
```

Data type coercion

If one tries to concatenate different data types in **R** in one vector, for example elementary values of type `numeric`, `character`, and `logical`, then **R** enforces a uniform data type for the resulting vector. The following ranking applies:

$$\text{character} > \text{double} > \text{integer} > \text{logical}$$

Thus, for example, if one of the elementary values is of type `character`, then the resulting vector is of type `character` and numerical or logical values are converted to this type.

```
x = c(1, "a", TRUE)  # concatenation of different data types
typeof(x)            # coerced data type
```

```
[1] "character"
```

```
x                    # character vector
```

```
[1] "1"    "a"    "TRUE"
```

If, by contrast, one of the elementary values is of type `integer` and another is of type `logical`, then the resulting vector is of type `integer` and the logical values are converted to integers.

```
x = c(1L, TRUE, FALSE)  # combination of mixed data types (integer beats logical)
typeof(x)               # generated vector is of type integer
```

```
[1] "integer"
```

```
x
```

```
[1] 1 1 0
```

Since **R** performs this data type coercion implicitly, one should be careful to concatenate only values of the same data type with the function `c()`. To be as flexible as possible, **R** also performs more subtle data coercions. Thus, for example, it is possible to compute the sum of logical values, where the logical values `TRUE` and `FALSE` are implicitly treated as the numerical values `0` and `1`.

```
x = c(TRUE, FALSE, TRUE, TRUE)  # logical vector
s = sum(x)                      # sum of logical elements converted to integer
s
```

```
[1] 3
```

Explicit data type coercion is allowed by the functions `as.logical()`, `as.integer()`, `as.double()`, and `as.character()`. If one applies these to a vector, the resulting vector is of the desired type.

```
x = c(0,1,1,0)     # double vector
typeof(x)
```

```
[1] "double"
```

```
x
```

```
[1] 0 1 1 0
```

```
y = as.logical(x)  # converted to logical
typeof(y)
```

```
[1] "logical"
```

```
y
```

```
[1] FALSE  TRUE  TRUE FALSE
```

```
z = as.integer(x)
typeof(z)
```

```
[1] "integer"
```

```
z
```

```
[1] 0 1 1 0
```

14.3 Indexing

In **R** and many other programming languages, individual or several vector entries are addressed by *indexing*. Indexing is also called *subsetting* or *slicing*. In **R**, square brackets of the form [ ] are used for indexing. **R** uses *one-based indexing*, which means that the index of the first element is `1` (and not, for example, `0` as in Python).

Indexing of vector entries can be used to copy a specific entry of a vector into another data structure:

```
x = c("a", "b", "c")  # character vector ["a", "b", "c"]
y = x[2]              # copy of "b" in y
y                     # output
```

```
[1] "b"
```

Likewise, indexing of vector entries can be used to manipulate a specific entry of the indexed vector:

```
x    = c("a", "b", "c")  # character vector ["a", "b", "c"]
x[3] = "d"               # change of x to x = ["a", "b", "d"]
x                        # output
```

```
[1] "a" "b" "d"
```

In principle, in **R**, indexing

(1) with a vector of positive numbers addresses the corresponding entries,
(2) with a vector of negative numbers addresses the corresponding complementary entries,
(3) with a logical vector addresses the entries with logical value `TRUE`, and
(4) with a `character` vector addresses the correspondingly named entries.

We want to illustrate these principles using the following examples.

(1) Indexing with a vector of positive numbers

```
x = c(1,4,9,16,25)  # [1,4,9,16,25] = [1^2, 2^2, 3^2, 4^2, 5^2]
y = x[1:3]          # 1:3 generates vector [1,2,3], x[1:3] = [1,4,9]
z = x[c(1,3,5)]     # c(1,3,5) generates vector [1,3,5], x[c(1,3,5)] = [1,9,25]
x
```

```
[1]  1  4  9 16 25
```

```
y
```

```
[1] 1 4 9
```

```
z
```

```
[1]  1  9 25
```

(2) Indexing with a vector of negative numbers

```
x = c(1,4,9,16,25)  # [1,4,9,16,25] = [1^2, 2^2, 3^2, 4^2, 5^2]
y = x[c(-2,-4)]     # all components except 2 and 4, x[c(-2,-4)] = [1,9,25]
x
```

```
[1]  1  4  9 16 25
```

```
y
```

```
[1]  1  9 25
```

```
z = x[c(-1,2)]      # mixed indexing is not allowed (error message)
```

```
Error in `x[c(-1, 2)]`:
! nur Nullen dürfen mit negativen Indizes gemischt werden
```

(3) Indexing with a logical vector

```
x = c(1,4,9,16,25)   # [1,4,9,16,25] = [1^2, 2^2, 3^2, 4^2, 5^2]
y = x[c(T,T,F,F,T)]  # components with TRUE, x[c(T,T,F,F,T)] = [1,4,25]
z = x[x > 5]         # x > 5 = [F,F,T,T,T], x[x > 5] = [9,16,25]
x
```

```
[1]  1  4  9 16 25
```

```
y
```

```
[1]  1  4 25
```

```
z
```

```
[1]  9 16 25
```

(4) Indexing with a `character` vector

```
x = c(1,4,9,16,25)     # [1,4,9,16,25] = [1^2, 2^2, 3^2, 4^2, 5^2]
names(x) = c("a","b")  # naming of components as [a  b <NA> <NA> <NA>]
y = x["a"]             # x["a"] = 1
```

It remains to be noted that **R** exhibits high flexibility in indexing, that is, it often does not issue errors even though an indexing operation makes no sense per se. For example, out-of-range indices, that is, indices that exceed the number of entries of a vector, do not cause errors, but instead return `NA` (not available).

```
x = c(1,4,9,16,25)  # [1,4,9,16,25] = [1^2, 2^2, 3^2, 4^2, 5^2]
y = x[10]           # x[10] = NA (not available)
y
```

```
[1] NA
```

Furthermore, non-integer indices do not cause errors, but are rounded down to the next integer.

```
y = x[4.9]   # x[4.9]  = x[4]  = 16
y
```

```
[1] 16
```

```
z = x[-4.9]  # x[-4.9] = x[-4] = [1,4,9,25]
z
```

```
[1]  1  4  9 25
```

Finally, empty indices index the entire vector.

```
y = x[]  # y = x
```

The flexibility of **R** in nonsensical indexing has the advantage that programs are less often interrupted with an error. On the other hand, it has the disadvantage that unintended misindexing remains unnoticed and data analysis programs can thus produce unintended results. When working with indexing in **R**, a high degree of attention is therefore required, and ideally the validation of created data analysis programs using simulated data with known results. A similar degree of caution is advisable because of **R**'s *recycling* functionality, which we will get to know in the next section.

14.4 Arithmetic of numeric vectors

With **R** vectors, one can carry out arithmetic operations, that is, one can compute with them. We distinguish between *unary arithmetic operations*, that is, arithmetic operations applied to one vector, and *binary arithmetic operations*, in which two vectors are linked with each other. In general, arithmetic vector operators in **R** are evaluated elementwise, but some special features must be observed for binary operators.

Let us first consider some unary arithmetic operators and the application of functions to a vector. In the following example, the unary operators `-` and `^2`, as well as the function `log()`, are applied to a vector `a`. The evaluation of the operators or the function occurs *elementwise*, which means that the operator or function is applied to each element of the vector independently of its other elements.

```
a = seq(0,1,len=11)  # a = [ 0.0   ,  0.1   , ...,  0.9    , 1.0    ]
a
```

```
 [1] 0.0 0.1 0.2 0.3 0.4 0.5 0.6 0.7 0.8 0.9 1.0
```

```
b = -a               # b = [-0.0   , -0.1   , ..., -0.9    ,-1.0    ]
b
```

```
 [1]  0.0 -0.1 -0.2 -0.3 -0.4 -0.5 -0.6 -0.7 -0.8 -0.9 -1.0
```

```
v = a^2              # v = [ 0.0^2 ,  0.1^2 , ...,  0.9^2  , 1.0^2  ]
v
```

```
 [1] 0.00 0.01 0.04 0.09 0.16 0.25 0.36 0.49 0.64 0.81 1.00
```

```
w = log(a)           # w = [ln(0.0), ln(0.1), ...,  ln(0.9), ln(1.0)]
w
```

```
 [1]       -Inf -2.3025851 -1.6094379 -1.2039728 -0.9162907 -0.6931472
 [7] -0.5108256 -0.3566749 -0.2231436 -0.1053605  0.0000000
```

Binary arithmetic operators, that is, the linking of two vectors, are likewise evaluated elementwise. When linking two vectors of the same length, the vector entries with the same indices are linked using the binary arithmetic operator. The result is a vector with the same length as the two initial vectors.

```
a = c(1,2,3)  # a = [1,2,3]
b = c(2,1,4)  # b = [2,1,4]
v = a + b     # v = [1,2,3] + [2,1,4] = [1+2,2+1,3+4] = [   3,  3,    7]
v
```

```
[1] 3 3 7
```

```
w = a - b     # w = [1,2,3] - [2,1,4] = [1-2,2-1,3-4] = [  -1,  1,   -1]
w
```

```
[1] -1  1 -1
```

```
x = a * b     # x = [1,2,3] * [2,1,4] = [1*2,2*1,3*4] = [   2,  2,   12]
x
```

```
[1]  2  2 12
```

```
y = a / b     # y = [1,2,3] / [2,1,4] = [1/2,2/1,3/4] = [0.50,  2, 0.75]
y
```

```
[1] 0.50 2.00 0.75
```

If one links a vector and a *scalar*, that is, a vector of length 1, in **R**, then **R** expands the scalar to a vector of the same length and performs the elementwise linking of these vectors. The following examples illustrate this.

```
a = c(1,2,3)  # a = [1,2,3]
b = 2         # b = [2]
v = a + b     # v = [1,2,3] + [2,2,2] = [1+2,2+2,3+2] = [  3,  4,   5]
v
```

```
[1] 3 4 5
```

```
w = a - b     # w = [1,2,3] - [2,2,2] = [1-2,2-2,3-2] = [ -1,  2,   1]
w
```

```
[1] -1  0  1
```

```
x = a * b     # x = [1,2,3] * [2,2,2] = [1*2,2*2,3*2] = [  2,  4,   6]
x
```

```
[1] 2 4 6
```

```
y = a / b     # y = [1,2,3] / [2,2,2] = [1/2,2/2,3/2] = [0.5,  1, 1.5]
y
```

```
[1] 0.5 1.0 1.5
```

Under the term *recycling*, **R** allows, in addition to elementwise arithmetic for vectors of the same length or for scalars and vectors, arithmetic with vectors of different lengths. For this purpose, elements of the shorter of two vectors are used to extend it to the length of the longer vector, that is, they are *recycled* in this sense. This property of vector arithmetic in **R** again means that programs abort with an error less often and that recycling can, under certain circumstances, offer elegant solutions for computations in informed code. However, it has the clear disadvantage that unintended misallocations of vectors, for example due to the absence of data points, can remain undetected. Here, too, a high degree of attention and, in particular, extensive testing are therefore indicated when creating programs. Specifically, one can distinguish two cases in the recycling of vector arithmetic in **R**: If the length of the longer of the two vectors is an integer multiple of the length of the shorter vector, then the shorter vector is expanded by repeating its elements in their original order the corresponding number of times. The following example illustrates this.

```
x = 1:2    # x = [1,2], length(x) = 2
x
```

```
[1] 1 2
```

```
y = 3:6    # y = [3,4,5,6], length(y) = 4
y
```

```
[1] 3 4 5 6
```

```
v = x + y  # v = [1,2,1,2] + [3,4,5,6] = [4,6,6,8]
v
```

```
[1] 4 6 6 8
```

In this sense, the arithmetic of vectors and scalars, that is, vectors of length 1, is a special case of this principle. If the length of the longer of the two vectors is not an integer multiple of the length of the shorter vector, then elements of the shorter vector are used in their original order to fill up the shorter vector until both vectors have the same length. The following example illustrates this.

```
x = c(1,3,5)       # x = [1,3,5], length(x) = 3
x
y = c(2,4,6,8,10)  # y = [2,4,6,8,10], length(y) = 5
y
v = x + y          # v = [1,3,5,1,3] + [2,4,6,8,10] = [3,7,11,9,13]
v
```

14.5 Attributes

Attributes in **R** are metadata of objects in the form of key-value pairs. In general, the function `attributes()` calls all attributes of an object. Vectors do not have attributes per se.

```
a = 1:3        # a numeric vector
attributes(a)  # call all attributes of a
```

```
NULL
```

The function `attr()` can be used to call and define attributes. The following code defines an attribute with the key (name) `"S"` and the value `"W"` for the vector `a`.

```
attr(a, "S") = "W"  # a receives attribute with key S and value W
attr(a, "S")        # the attribute with key S has value W
```

```
[1] "W"
```

However, attributes are often removed during operations.

```
b = a[1]       # copy of the first element of a in vector b
attributes(b)  # call all attributes of b
```

```
NULL
```

The specification of the attribute `names` gives names to the elements of a vector that can be used for indexing and are not removed during operations.

```
v = c(x=1,y=2,z=3)  # element-name generation during vector creation
v                   # vector output
```

```
x y z
1 2 3
```

Indexing using names has the following form:

```
v["y"]  # indexing by name
```

```
y
2
```

After the definition of a vector, the function `names()` can be used to define names for the vector entries and to look them up.

```
y = 4:6                    # generation of a vector
names(y) = c("a","b","c")  # definition of names
names(y)                   # call element names
```

```
[1] "a" "b" "c"
```

Named vector entries can be helpful if the vector forms a unit of meaning and, for example, represents properties of a participant such as age, height, and weight.

```
p = c(age    = 31,   # age (years)
      height = 198,  # height (cm)
      weight = 75)   # weight (kg)
p                    # vector output
```

```
   age height weight
    31    198     75
```

However, working with named vector entries is rather rare. Nevertheless, the principle of providing entries of a data structure with a meaningful label instead of a numerical index is common practice in **R**, as we will see when working with the central data structure of data frames in the chapter on lists, data frames, and tibbles.

15 Matrices and arrays

R matrices and arrays can be understood as higher-dimensional generalizations of **R** vectors. As with vectors, it is important in programming to distinguish matrices from the mathematical objects of the same name. Thus, in programming, matrices can simply serve for the tabular storage of numerical data, without the primary goal necessarily being the application of mathematical matrix calculus. Conversely, elementwise arithmetic operations with matrices in programming represent rather special mathematical operations such as the Hadamard product in mathematics. In this section, we primarily want to examine matrices and arrays as data containers in imperative programming with **R**. An introduction to matrix calculus with the same data structures is given in Chapter 9.

15.1 Matrices

Abstractly speaking, **R** matrices are two-dimensional, rectangular data structures of the form

$$M = \begin{pmatrix} m_{11} & m_{12} & \cdots & m_{1n_c} \\ m_{21} & m_{22} & \cdots & m_{2n_c} \\ \vdots & \vdots & \ddots & \vdots \\ m_{n_r1} & m_{n_r2} & \cdots & m_{n_rn_c} \end{pmatrix} \tag{15.1}$$

The elements $m_{ij}, i = 1, ..., n_r, j = 1, ..., n_c$ of the matrix are necessarily of the same data type. In the above scheme, n_r denotes the number of rows and n_c denotes the number of columns of the matrix M. Each element of a matrix therefore has a row index i and a column index j. Intuitively, **R** matrices are thus numerically indexed tables. Formally, matrices in **R** are atomic vectors interpreted two-dimensionally.

15.1.1 Generation

The **R** function `matrix()` is used to generate a matrix; it fills matrices with the elements of a vector. The general syntax of this function is `M = matrix(v, nrow, ncol, byrow)`, where

- `v` denotes a vector of length `nrow`· `ncol`,
- `nrow` denotes the desired number of rows of `M`,
- `ncol` denotes the desired number of columns of `M`,
- the logical variable `byrow` denotes the filling order of `M`.

Since, for a given length of `v` and a given number of rows or columns, the corresponding number of columns or rows of the matrix follows, it is only necessary to specify either `nrow` or `ncol`, as the following examples show.

```
matrix(c(1:12), nrow = 3)               # 3 x 4 matrix of the numbers 1,...,12, byrow = FALSE
```

```
     [,1] [,2] [,3] [,4]
[1,]    1    4    7   10
[2,]    2    5    8   11
[3,]    3    6    9   12
```

```
matrix(c(1:12), ncol = 4)               # 3 x 4 matrix of the numbers 1,...,12, byrow = FALSE
```

```
     [,1] [,2] [,3] [,4]
[1,]    1    4    7   10
[2,]    2    5    8   11
[3,]    3    6    9   12
```

```
matrix(c(1:12), nrow = 3, byrow = TRUE)  # 3 x 4 matrix of the numbers 1,...,12, byrow = TRUE
```

```
     [,1] [,2] [,3] [,4]
[1,]    1    2    3    4
[2,]    5    6    7    8
[3,]    9   10   11   12
```

Existing matrices with the same number of rows can be concatenated columnwise with the function `cbind()`.

```
A = matrix(c(1:4) , nrow = 2)  # 2 x 2 matrix of the numbers 1,...,4
A
```

```
     [,1] [,2]
[1,]    1    3
[2,]    2    4
```

```
B = matrix(c(5:10), nrow = 2)  # 2 x 3 matrix of the numbers 5,...,10
B
```

```
     [,1] [,2] [,3]
[1,]    5    7    9
[2,]    6    8   10
```

```
C = cbind(A,B)                 # columnwise concatenation of A and B
C
```

```
     [,1] [,2] [,3] [,4] [,5]
[1,]    1    3    5    7    9
[2,]    2    4    6    8   10
```

Equivalently, existing matrices with the same number of columns can be concatenated rowwise with the function `rbind()`.

```
A = matrix(c(1:6) , nrow = 2, byrow = TRUE)  # 2 x 3 matrix of the numbers 1,...,6
print(A)
```

```
     [,1] [,2] [,3]
[1,]    1    2    3
[2,]    4    5    6
```

```
B = matrix(c(7:9), nrow = 1)                # 1 x 3 matrix of the numbers 7,...,9
print(B)
```

```
     [,1] [,2] [,3]
[1,]    7    8    9
```

```
C = rbind(A,B)                              # rowwise concatenation of A and B
print(C)
```

```
     [,1] [,2] [,3]
[1,]    1    2    3
[2,]    4    5    6
[3,]    7    8    9
```

15.1.2 Characterization

The familiar function `typeof()` can be used to output the elementary data type of a matrix:

```
typeof(matrix(c(1,0,0,1)    , nrow = 2))  # 2 x 2 double matrix
```

```
[1] "double"
```

```
typeof(matrix(c(1L,0L,0L,1L), nrow = 2))  # 2 x 2 integer matrix
```

```
[1] "integer"
```

```
typeof(matrix(c(T,T,F,F)    , nrow = 2))  # 2 x 2 logical matrix
```

```
[1] "logical"
```

```
typeof(matrix(c("a","b","c"), nrow = 1))  # 1 x 3 character matrix
```

```
[1] "character"
```

The number of rows or columns of a matrix can be output with the functions `nrow()` (number of rows) and `ncol()` (number of columns).

```
C = matrix(1:12, nrow = 3)  # 3 x 4 matrix
nrow(C)                     # number of rows
```

```
[1] 3
```

```
ncol(C)                     # number of columns
```

```
[1] 4
```

15.1.3 Indexing

In general, matrix elements are indexed with a row index and a column index. The first of two matrix indices always indexes the row, and the second always indexes the column. The remaining principles of matrix indexing correspond essentially to the principles of vector indexing. One special feature is that a missing row or column index indexes all elements of the corresponding dimension.

```
A = matrix(c(2:7)^2, nrow = 2)  # 2 x 3 matrix of the numbers 2^2,...,7^2
A
```

```
     [,1] [,2] [,3]
[1,]    4   16   36
[2,]    9   25   49
```

```
A[1,3]                          # element in row 1, column 3 of A [36]
```

```
[1] 36
```

```
A[2,2]                          # element in row 2, column 2 of A [25]
```

```
[1] 25
```

```
A[2,]                           # all elements of row 2 [9,25,49]
```

```
[1]  9 25 49
```

```
A[,3]                           # all elements of column 3 [36,49]
```

```
[1] 36 49
```

```
A[1:2,1:2]                      # submatrix of the first two rows and columns
```

```
     [,1] [,2]
[1,]    4   16
[2,]    9   25
```

```
A[A>10]                         # elements of A greater than 10 [16,25,36,49]
```

```
[1] 16 25 36 49
```

```
A[1,c(F,F,T)]                   # element in row 1, column 3 of A [36]
```

```
[1] 36
```

15.1.4 Arithmetic

If one applies unary arithmetic operators and functions to matrices, these are evaluated elementwise, as with vectors.

```
A = matrix(c(1:4), nrow = 2)  # 2 x 2 matrix of the numbers 1,2,3,4
A
```

```
     [,1] [,2]
[1,]    1    3
[2,]    2    4
```

```
B = A^2                       # result B[i,j] = A[i,j]^2, 1 <= i,j <= 2
B
```

```
     [,1] [,2]
[1,]    1    9
[2,]    4   16
```

```
C = sqrt(B)                   # result C[i,j] = sqrt(A[i,j]^2), 1 <= i,j <= 2
C
```

```
     [,1] [,2]
[1,]    1    3
[2,]    2    4
```

```
D = exp(A)                    # result D[i,j] = exp(A[i,j]), 1 <= i,j <= 2
D
```

```
         [,1]     [,2]
[1,] 2.718282 20.08554
[2,] 7.389056 54.59815
```

Like vectors, matrices with identical numbers of rows and columns can be linked element-wise with the binary arithmetic operators `+`, `-`, `*`, and `/`. The following examples illustrate this:

```
A = matrix(c(1:4), nrow = 2)  # 2 x 2 matrix of the numbers 1,2,3,4
A
```

```
     [,1] [,2]
[1,]    1    3
[2,]    2    4
```

```
B = matrix(c(5:8), nrow = 2)  # 2 x 2 matrix of the numbers 5,6,7,8
B
```

```
     [,1] [,2]
[1,]    5    7
[2,]    6    8
```

```
C = A + B                     # result C[i,j] = A[i,j] + B[i,j], 1 <= i,j <= 2
C
```

```
     [,1] [,2]
[1,]    6   10
[2,]    8   12
```

```
D = A * B                    # result D[i,j] = A[i,j] * B[i,j], 1 <= i,j <= 2
D
```

```
     [,1] [,2]
[1,]    5   21
[2,]   12   32
```

In addition, **R** matrices can be used for computations in the sense of matrix calculus from linear algebra. We show only a few examples here; a detailed presentation of the mathematical principles and their **R** implementation can be found in Chapter 9.

```
C = A %*% B       # 2 x 2 matrix product
C
```

```
     [,1] [,2]
[1,]   23   31
[2,]   34   46
```

```
A_T = t(A)        # transposition of A
A_T
```

```
     [,1] [,2]
[1,]    1    2
[2,]    3    4
```

```
A_inv = solve(A)  # inverse of A
A_inv
```

```
     [,1] [,2]
[1,]   -2  1.5
[2,]    1 -0.5
```

```
A_det = det(A)    # determinant of A
A_det
```

```
[1] -2
```

15.1.5 Attributes

Formally, matrices are atomic vectors with a `dim` attribute.

```
A = matrix(1:12, nrow = 4 )  # 4 x 3 matrix
attributes(A)                # call the attributes of A
```

```
$dim
[1] 4 3
```

As with vectors, one can also give names to matrix elements, although this is rather unusual for matrices as well. Specifically, the functions `rownames()` and `colnames()` define the matrix attribute `dimnames`.

```
rownames(A) = c("P1","P2","P3","P4")  # naming the rows of A
colnames(A) = c("Age", "Hgt", "Wgt")  # naming the columns of A
A                                     # matrix with attribute dimnames
```

```
   Age Hgt Wgt
P1   1   5   9
P2   2   6  10
P3   3   7  11
P4   4   8  12
```

```
attr(A, "dimnames")                   # call the attribute dimnames
```

```
[[1]]
[1] "P1" "P2" "P3" "P4"

[[2]]
[1] "Age" "Hgt" "Wgt"
```

15.2 Arrays

R arrays are d-dimensional, hyperrectangular data structures. Intuitively, arrays can be understood as the generalization of matrices to more than two dimensions. For $d = 1$, an array corresponds to a vector; for $d = 2$, an array corresponds to a matrix. As with vectors and matrices, the elements of an array must be of the same data type. Arrays are characterized by how many elements $d_i, i = 1, ..., d$ the i-th dimension can represent. Each element of an array is indexed by a d-dimensional index $i_1, i_2, ..., i_d$.

15.2.1 Generation

Analogously to the function `matrix()`, the function `array()` is used to fill an array with vector elements. The general syntax of this function is `A = array(v, dim)`, where `v` is a vector of array data and `dim` is a vector of integer values that represents the maximal indices in each of the `length(dim)` dimensions of the array. For example, the following command generates a $2 \times 2 \times 3$ array.

```
A = array(1:12, dim = c(2,2,3))  # 2 x 2 x 3 array of the numbers 1,...,12
A
```

```
, , 1

     [,1] [,2]
[1,]    1    3
[2,]    2    4

, , 2

     [,1] [,2]
[1,]    5    7
[2,]    6    8

, , 3
```

```
     [,1] [,2]
[1,]    9   11
[2,]   10   12
```

Three-dimensional arrays can be imagined intuitively as matrices placed one behind another in space. The third dimension then represents the "pages" of the array. However, when working with higher-dimensional arrays, it is useful to try to detach oneself from spatial conceptions of arrays and instead understand arrays as data containers whose elements can be addressed uniquely by their indices.

15.2.2 Characterization

The `length()` function outputs the number of elements of an array.

```
A = array(1:12, dim = c(2,2,3))  # 2 x 2 x 3 array of the numbers 1,...,12
length(A)                        # number of elements of the array
```

```
[1] 12
```

The function `typeof()` outputs the elementary data type of an array:

```
typeof(A)  # type of the array
```

```
[1] "integer"
```

The function `dim()` outputs the dimensions of an array:

```
dim(A)  # dimension of the array
```

```
[1] 2 2 3
```

15.2.3 Indexing

Indexing of arrays is analogous to vector and matrix indexing, with the difference that three or more indices are needed to index an array element.

```
A     = array(1:12, dim = c(2,2,3))  # 2 x 2 x 3 array of the numbers 1,...,12
a_223 = A[2,2,3]                     # array element with index address [2,2,3] (12)
```

15.2.4 Arithmetic

As with vectors and matrices, unary arithmetic operators are evaluated elementwise for arrays. Likewise, binary arithmetic operators are evaluated elementwise for arrays of identical dimensionality.

```
A = array(1:4, dim = c(1,2,2))  # 1 x 2 x 2 array of the numbers 1,...,4
A
```

```
, , 1

     [,1] [,2]
[1,]    1    2

, , 2

     [,1] [,2]
[1,]    3    4
```

```
B = array(5:8, dim = c(1,2,2))  # 1 x 2 x 2 array of the numbers 5,...,8
B
```

```
, , 1

     [,1] [,2]
[1,]    5    6

, , 2

     [,1] [,2]
[1,]    7    8
```

```
C = A^2                         # elementwise exponentiation
C
```

```
, , 1

     [,1] [,2]
[1,]    1    4

, , 2

     [,1] [,2]
[1,]    9   16
```

```
D = A + B                       # elementwise addition
D
```

```
, , 1

     [,1] [,2]
[1,]    6    8

, , 2

     [,1] [,2]
[1,]   10   12
```

15.2.5 Attributes

Formally, arrays are atomic vectors with a `dim` attribute.

```
A = array(1:12, dim = c(2,2,3))  # 2 x 2 x 3 array of the numbers 1,...,12
attributes(A)                    # call the attributes of A
```

```
$dim
[1] 2 2 3
```

As with matrices, the function `dimnames()` can be used to name the array dimensions, but here too the naming of array dimensions is rather unusual.

16 Lists, dataframes, and tibbles

16.1 Lists

R lists are ordered sequences of **R** objects. Lists are called a recursive data type because they can bundle objects of different data types, including lists themselves. Although the intuitive view that lists "contain" objects is natural, lists de facto do not "contain" objects, but only the references to the corresponding objects in memory (Figure 16.1). Lists are an essential building block of **R** dataframes.

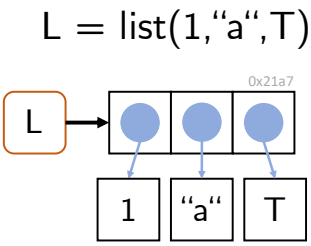


Figure 16.1 List representation. A list in **R** is an indexable sequence of references to **R** objects in memory.

Generation

Lists can be generated by directly concatenating the list elements with the function `list()`.

```
# generation of a list with a vector, matrix, and function element
L = list(c(1,4,5), matrix(1:8, nrow = 2), exp)
print(L)
```

```
[[1]]
[1] 1 4 5

[[2]]
     [,1] [,2] [,3] [,4]
[1,]    1    3    5    7
[2,]    2    4    6    8

[[3]]
function (x)  .Primitive("exp")
```

In particular, lists themselves can also be elements of lists, which motivates their designation as a *recursive* data type.

```
L = list(list(1))  # list with element 1 in a list
print(L)           # output
```

The function `c()`, familiar from **R** vectors, can also be used to concatenate several lists.

```
L = c(list(pi), list("a"))  # concatenation of two lists
print(L)                    # output
```

Characterization

The data type of lists is `list`:

```
L = list(1:2, "a", log)  # generation of a list
typeof(L)                # type determination
```

```
[1] "list"
```

`length()` returns the number of list elements at the top list level; the element contents are therefore ignored:

```
L = list(1:2, list("a", pi), exp)  # list with three top-level elements
length(L)                          # length() ignores element contents
```

```
[1] 3
```

The dimension, number of rows, and number of columns of lists are `NULL`:

```
L = list(1:2, "a", sin)  # a list
dim(L)                   # the dimension of lists is NULL
```

```
NULL
```

```
nrow(L)                  # the number of rows of lists is NULL
```

```
NULL
```

```
ncol(L)                  # the number of columns of lists is NULL
```

```
NULL
```

Indexing

When indexing lists, one must note that lists can be indexed in two different ways. Single square brackets `[ ]` index list elements as lists:

```
L  = list(1:3, "a", exp)  # a list
l1 = L[1]                 # indexing a list element
print(l1)                 # output
```

```
[[1]]
[1] 1 2 3
```

```
typeof(l1)                # type determination of l1
```

```
[1] "list"
```

Double square brackets `[[ ]]` index the contents of list elements:

```
L  = list(1:3, "a", exp)  # a list
i2 = L[[2]]               # indexing the list-element content
print(i2)                 # output of the content of list element L[2]
typeof(i2)                # type determination of i2
```

These conventions must also be observed when replacing list contents. For example, it is not possible to change the first list element with the expression `L[1] = c(1,2,3)`, because the vector `c(1,2,3)` is not a list:

```
L      = list(1:3, "a", exp)  # a list
L[1]   = 4:6                  # no type conversion, error message
```

```
Warning in L[1] = 4:6: Anzahl der zu ersetzenden Elemente ist kein Vielfaches
der Ersetzungslänge
```

```
L[1]   = list(4:6)            # replacement of the 1st list element
L[[3]] = "c"                  # replacement of the 3rd list-element content
```

Indexing

The principles of list indexing are analogous to those of vector indexing. Vectors of positive numbers address the corresponding elements:

```
L = list(1:3, "a", pi)  # a list
l = L[c(1,3)]           # 1st and 3rd list element
l
```

```
[[1]]
[1] 1 2 3

[[2]]
[1] 3.141593
```

Vectors of negative numbers address the elements complementary to their values:

```
L = list(1:3, "a", pi)  # a list
l = L[-c(1,3)]          # 2nd list element
l
```

```
[[1]]
[1] "a"
```

Logical vectors address elements with index `TRUE`:

```
L = list(1:3, "a", pi)  # a list
l = L[c(T,T,F)]         # 1st and 2nd list element
l
```

```
[[1]]
[1] 1 2 3

[[2]]
[1] "a"
```

For non-numerical addressing, list elements can also be given names. This is possible after the generation of a list with the `names()` function:

```
K        = list(1:2, TRUE)    # an unnamed list
names(K) = c("Frodo", "Sam")  # naming with names()
K                             # output
```

```
$Frodo
[1] 1 2

$Sam
[1] TRUE
```

It is likewise possible to assign the names of the list elements directly during the generation of the list:

```
L = list(frodo = 1:3, sam = "a", eowyn = "b")  # list generation with names
```

Both list elements and list-element contents can then be addressed with their names:

```
L = list(frodo = 1:3, sam = "a", eowyn = "b")  # list generation with names
L["frodo"]                                     # list-element output
```

```
$frodo
[1] 1 2 3
```

```
typeof(L["frodo"])                             # list-element type
```

```
[1] "list"
```

```
L[["frodo"]]                                    # list-element content output
```

```
[1] 1 2 3
```

```
typeof(L[["sam"]])                              # list-element content type
```

```
[1] "character"
```

In particular, list-element contents can also be indexed with the $ operator and their name. In the context of dataframes, this is the most common kind of list indexing:

```
L = list(frodo = 1:3, sam = "a", eowyn = "b")  # list generation with names
L$frodo                                         # list-element content output
```

```
[1] 1 2 3
```

```
typeof(L$frodo)                                 # list-element content type
```

```
[1] "integer"
```

```
L$sam                                           # list-element content output
```

```
[1] "a"
```

```
typeof(L$sam)                                   # list-element content type
```

```
[1] "character"
```

```
L$eowyn                                         # list-element content output
```

```
[1] "b"
```

```
typeof(L$eowyn)                                 # list-element content type
```

```
[1] "character"
```

Arithmetic

General list arithmetic is not defined, because list-element contents can be of different data types for which unary or binary operations are not defined:

```
L1 = list(1:3, "a" )  # a list
L2 = list(T, exp )    # a list
L1+L2                 # attempt at list addition
```

```
Error in `L1 + L2`:
! nicht-numerisches Argument für binären Operator
```

If, however, the contents of list elements are of a common data type for which certain binary operations are defined, then they can also be linked, for example arithmetically:

```
L1 = list(1:3, pi )   # a list
L2 = list(4:6, exp )  # a list
L1[[1]] + L2[[1]]     # addition of the 1st list-element contents, [1+4, 2+5,3+6]
```

```
[1] 5 7 9
```

```
L2[[2]](1)            # application of the 2nd list-element content, exp(1)
```

```
[1] 2.718282
```

Copy-on-modify

As with vectors, the *copy-on-modify* principle applies to lists. Specifically, for lists, so-called *shallow copies* are generated when a list is modified. In this process, only the list object itself is copied and receives a new address in memory, but not the objects bound by the list, which remain at the same addresses in memory. `lobstr::ref()` makes it possible to trace this behavior.

```
L1 = list(1,2,3)  # generation of a list as an object
lobstr::ref(L1)   # output of the references
```

```
 [1:0x221ab69b598] <list>
[2:0x221a9487818] <dbl>
[3:0x221a94871f8] <dbl>
[4:0x221a9487038] <dbl>
```

```
L2 = L1           # same object referenced by L1 and L2
lobstr::ref(L2)   # output of the references
```

```
 [1:0x221ab69b598] <list>
[2:0x221a9487818] <dbl>
[3:0x221a94871f8] <dbl>
[4:0x221a9487038] <dbl>
```

```
L2[[3]] = 4       # copy-on-modify with shallow copy
lobstr::ref(L2)   # output of the references
```

```
 [1:0x221a963d2c8] <list>
[2:0x221a9487818] <dbl>
[3:0x221a94871f8] <dbl>
[4:0x221a9515e00] <dbl>
```

Apparently, the assignment `L2 = L1` initially changes nothing about the memory addresses to which `L1` and `L2` as well as their elements refer. The modification of the content of the third list element of `L2` by `L2[[3]] = 4`, however, then changes the address of the list object in memory because a copy of the list object is generated. At the same time, according to the shallow-copy principle, the addresses of the first two contents of `L2` do not change. Figure 16.2 visualizes these principles with an example.

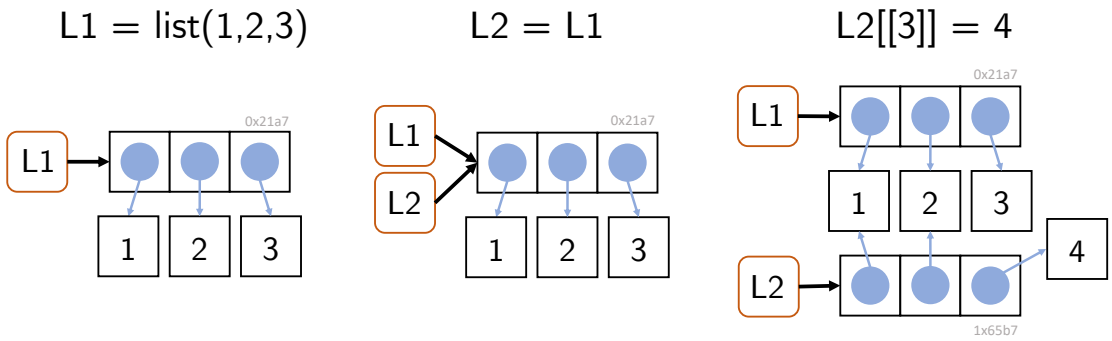


Figure 16.2 Effects of the assignment `L2 = L1` for a list `L1` and of the modification of an element of `L2`.

16.2 Dataframes

R dataframes are the central data structure in **R**. Dataframes are best imagined as tables whose columns have names. Technically, a dataframe is a list whose elements are vectors of the same length. The list elements correspond to the columns of a table, and the vector elements at the same position correspond to the rows of a table.

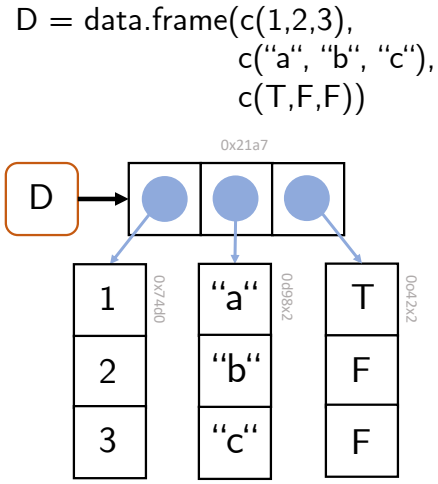


Figure 16.3 Dataframe representation. An **R** dataframe is an **R** list whose elements are **R** vectors of the same length.

Generation

Dataframes are generated with the function `data.frame()`. The column names and their vector-valued contents are specified as arguments of the function:

```
D = data.frame(x = letters[1:4],  # 1st column with name x
               y = 1:4,           # 2nd column with name y
               z = c(T,T,F,T))    # 3rd column with name z
print(D)                          # output of the dataframe as a table
```

```
  x y     z
1 a 1  TRUE
2 b 2  TRUE
3 c 3 FALSE
4 d 4  TRUE
```

The columns of the dataframe must have the same length; otherwise a dataframe is not defined:

```
D = data.frame(x = letters[1:4],  # 1st column with name x
               y = 1:4,           # 2nd column with name y
               z = c(T,T,F))      # 3rd column with name z
```

```
Error in `data.frame()`:
! Argumente implizieren unterschiedliche Anzahl Zeilen: 4, 3
```

The columns of a dataframe can evidently be of different data types; this is, of course, congruent with the character of **R** lists.

Characterization

The functions `names()` and `colnames()` can be used to output the names of the columns of a dataframe, and the function `rownames()` can be used to output its row names. By default, the row names of a dataframe are the integers 1,2,...,n:

```
D = data.frame(age    = c(30,35,40,45),      # 1st column
               height = c(178,189,165,171),  # 2nd column
               weight = c(67, 76, 81, 92))   # 3rd column
names(D)                                     # names outputs the column names
```

```
[1] "age"    "height" "weight"
```

```
colnames(D)                                  # colnames corresponds to names
```

```
[1] "age"    "height" "weight"
```

```
rownames(D)                                  # default rownames are 1,2,...
```

```
[1] "1" "2" "3" "4"
```

Furthermore, a dataframe is characterized by its number of rows and columns. These can be output with the functions `nrow()` and `ncol()`, respectively. For dataframes, as for lists, the function `length()` returns the number of list entries; for dataframes this correspondingly means the number of columns.

```
nrow(D)    # number of rows
```

```
[1] 4
```

```
ncol(D)    # number of columns
```

```
[1] 3
```

```
length(D)  # length is the number of columns
```

```
[1] 3
```

A helpful function for characterizing a dataframe is the function `str()`, which displays essential aspects of a dataframe in compact form. `str()` stands for *structure* and denotes the columns of the dataframe as *variables* and row entries as *observations*:

```
str(D)
```

```
'data.frame':   4 obs. of  3 variables:
 $ age   : num  30 35 40 45
 $ height: num  178 189 165 171
 $ weight: num  67 76 81 92
```

Attributes

Technically, dataframes are lists with attributes for (column) `names` and `row.names`. Dataframes are of `class` "data.frame":

```
typeof(D)
```

```
[1] "list"
```

```
attributes(D)
```

```
$names
[1] "age"    "height" "weight"

$class
[1] "data.frame"

$row.names
[1] 1 2 3 4
```

Indexing

Dataframes can be indexed both like matrices and like lists. The indexing principles correspond to those of vectors. The following example first shows how a dataframe is indexed with single brackets in the sense of a list. The corresponding list element is then itself a dataframe for dataframes:

```
D = data.frame(x = letters[1:4],  # 1st column with name x
               y = 1:4,           # 2nd column with name y
               z = c(T,T,F,T))    # 3rd column with name z
class(D)                          # dataframe D
```

```
[1] "data.frame"
```

```
v = D[1]                          # 1st list element as dataframe
v
```

```
  x
1 a
2 b
3 c
4 d
```

```
class(v)                          # v is a dataframe
```

```
[1] "data.frame"
```

Furthermore, dataframes can be indexed like lists with double brackets. This addresses the contents of a list element:

```
w = D[[1]]  # content of the 1st list element
w
```

```
[1] "a" "b" "c" "d"
```

```
class(w)    # w is a character vector
```

```
[1] "character"
```

The typical indexing of a dataframe uses the $ indexing familiar from lists, which addresses the content of the corresponding dataframe column:

```
y = D$y   # $ for indexing the y column
y
```

```
[1] 1 2 3 4
```

```
class(y)  # y is a vector of type "integer" (!)
```

```
[1] "integer"
```

If one is interested in specific contents of the dataframe in specific rows and columns, the principles of matrix indexing are useful, as the following example shows:

```
D[2:3,-2]        # 1st index for rows, 2nd index for columns
```

```
  x     z
2 b  TRUE
3 c FALSE
```

```
D[c(T,F,T,F),]   # 1st index for rows, 2nd index for columns
```

```
  x y     z
1 a 1  TRUE
3 c 3 FALSE
```

```
D[,c("x", "z")]  # 1st index for rows, 2nd index for columns
```

```
  x     z
1 a  TRUE
2 b  TRUE
3 c FALSE
4 d  TRUE
```

Copy-on-modify

For optimal use of memory, the copy-on-modify principles for lists also apply to dataframes. Furthermore, the modification of a column leads to the copy of the corresponding column, whereas the modification of a row results in the copy of the entire dataframe.

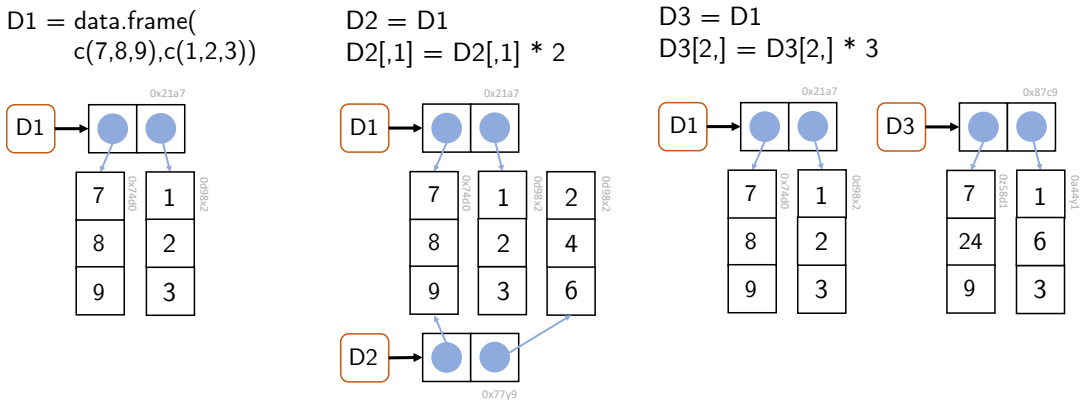


Figure 16.4 Effects of column and row modifications in dataframes. The modification of a column leads to the copy of the corresponding column, and the modification of a row to the copy of the entire dataframe.

16.3 Tibbles

In addition to **R** dataframes, *tibbles* (diminutive for *tables*) have become widespread in the last ten years as part of the `tidyverse` package. Tibbles are a more modern, technically improved form of dataframes and show how **R** has been extended for data science tasks beyond the application of classical linear models. To use tibbles, the `tidyverse` package must be installed and the tibble code must be loaded:

```
install.packages("tidyverse")  # one-time package download and installation
library(tibble)                # session-specific loading of the tibble code
```

By and large, tibbles correspond to dataframes. As with dataframes, tibbles are lists of vector references, where the vectors again have the same length and represent the named columns of a table. Tibbles can be generated with the function `tibble()`. The display in the **R** terminal is optimized compared with dataframes, as the following example shows:

```
library(tibble)                                                    # package
```

```
Warning: Paket 'tibble' wurde unter R Version 4.5.3 erstellt
```

```
T = tibble(x = c("a", "c", "d"), y = c(1,2,3), z = c(TRUE,FALSE,FALSE))  # generation
print(T)                                                           # output
```

```
# A tibble: 3 x 3
  x         y z
  <chr> <dbl> <lgl>
1 a         1 TRUE
2 c         2 FALSE
3 d         3 FALSE
```

```
attributes(T)                                                      # attributes
```

```
$class
[1] "tbl_df"     "tbl"        "data.frame"

$row.names
[1] 1 2 3

$names
[1] "x" "y" "z"
```

16.3.1 Tibbles vs. dataframes

Tibbles were introduced in 2016 as part of the `tidyverse` package to compensate for perceived weaknesses of **R** dataframes. These included, among other things, that the function `data.frame()` automatically converted character vectors into the **R** data type `factor`. Since an **R** revision from 2019, however, this is no longer the case, as the following example shows:

```
D = data.frame(x = 1:3, y = c("a", "b", "c"))  # character vector y
str(D)                                         # remains character vector
```

```
'data.frame':   3 obs. of  2 variables:
 $ x: int  1 2 3
 $ y: chr  "a" "b" "c"
```

By default, `data.frame()` still converts inadmissible column names such as a number (`1`) into legal column names (`X1`). If one does not want this, however, one can turn off this behavior by setting the argument `check.names`.

```
names(data.frame(`1` = 1))                          # conversion of nonlegal variable names
```

```
[1] "X1"
```

```
names(data.frame(`1` = 1, check.names = FALSE))  # turning off the default behavior
```

```
[1] "1"
```

As seen in Chapter 14, **R** tends to generate data itself through recycling when data types have unsuitable lengths, which can easily lead to errors. Thus, for example, the function `data.frame()` recycles vectors when generating a dataframe if one of them is an integer multiple of the other:

```
D = data.frame(x = 1:2, y = 3:6)  # length(y) = 2 * length(x)
print(D)                          # output
```

```
  x y
1 1 3
2 2 4
3 1 5
4 2 6
```

By contrast, `tibble()` is more precise here and issues an error in the case above:

```
T = tibble(x = 1:2, y = 3:6)  # tibble issues an error here
```

```
Error in `tibble()`:
! Tibble columns must have compatible sizes.
* Size 2: Existing data.
* Size 4: Column `y`.
i Only values of size one are recycled.
```

Vectors of length 1, however, are also recycled by `tibble()`:

```
T = tibble(x = 1:3, y = 3)  # vectors with length 1
print(T)                    # output
```

```
# A tibble: 3 x 2
      x     y
  <int> <dbl>
1     1     3
2     2     3
3     3     3
```

Dataframes are sometimes inconsistent with respect to the data types they return. As the example below shows, indexing a dataframe column by its name yields a dataframe, whereas indexing the same column by its name in matrix indexing yields a vector:

```
D = data.frame(x = 1:3, y = 4:6)  # dataframe
str(D["x"])                       # indexing with names returns a dataframe
```

```
'data.frame':   3 obs. of  1 variable:
 $ x: int  1 2 3
```

```
str(D[,"x"])                      # matrix indexing with names returns a vector
```

```
 int [1:3] 1 2 3
```

Tibbles are more consistent in this respect:

```
T = tibble(x = 1:3, y = 4:6)  # tibble
str(T["x"])                   # indexing with names yields tibble
```

```
tibble [3 x 1] (S3: tbl_df/tbl/data.frame)
 $ x: int [1:3] 1 2 3
```

```
str(T[,"x"])                  # matrix indexing with names yields tibble
```

```
tibble [3 x 1] (S3: tbl_df/tbl/data.frame)
 $ x: int [1:3] 1 2 3
```

An additional feature of tibbles compared with dataframes is that the function `tibble()` allows column names to be referenced already during the generation of a tibble, which is not possible with dataframes.

```
T = tibble(x = 1, y = 2*x)      # column x is defined and used
D = data.frame(x = 1, y = 2*x)  # data.frame does not allow this
```

```
Error:
! Objekt 'x' nicht gefunden
```

In summary, tibbles can be said to function less flexibly and therefore more precisely than dataframes in some respects. In general, tibbles behave more consistently than dataframes in `tidyverse` contexts. However, as seen above, the differences between dataframes and tibbles are rather subtle, so that efficient handling of dataframes also enables efficient handling of tibbles. Since not all **R** projects use the packages of the `tidyverse`, dataframes will probably remain the standard data structure for tabular data in **R** in the future.

17 Data management

In this section, we want to introduce some general terminology for dealing with research data, discuss common formats for the longer-term storage of data, and finally consider the practical reading and saving of data in the context of **R** directory management.

17.1 The FAIR data ideal

The FAIR data ideal is a principle for the management of research data that takes into account the broad dissemination, usefulness, and transparency of research data in the digital age. *Research data* are generally understood as "electronically represented analog or digital data that arise or are used in the course of scientific projects, for example through observations, experiments, simulations, surveys, questionnaires, source research, recordings of audio and video sequences, digitization of objects, and analyses" (Rat für Informationsinfrastrukturen (2017)). Specifically, these include numerical arrays or character arrays, but also computer software such as **R** scripts, digital tools, workflows, and analysis pipelines.

In addition to these data forms, sometimes also referred to as *primary data*, so-called *metadata* are important for the FAIR data ideal. These are data that represent information about other data. One distinguishes, for example, *descriptive*, *structural*, and *administrative metadata* (Riley (2017), Ulrich et al. (2022)).

- *Descriptive metadata* serve to find and identify a data source. Examples of descriptive metadata include the title of a scientific publication or its Digital Object Identifier (DOI, Rosenblatt (1997)). DOIs are alphanumeric identifiers that uniquely identify digital objects and function as persistent links to objects on the internet.
- *Structural metadata* are metadata about data containers and represent the structural composition of a data source. Examples are the order of the pages of a book or the for-loop encoding of three-dimensional data objects.
- *Administrative metadata*, finally, are data that facilitate the management of a data source. Examples of administrative metadata are provenance, file format, access rights, or other technical information about a data source.

The FAIR ideal for research data management has its origin in discussions within the FORCE11 conference series and was communicated academically in 2016 by Wilkinson et al. (2016). In essence, it states that research data should be prepared and maintained as *Findable*, *Accessible*, *Interoperable*, and *Reusable* for humans and machines. Specifically, the FAIR data ideal formulates the following requirements for the management of research (meta)data:

Findability

- F1. (Meta)data have a persistent globally unique identifier.

- F2. Data are enriched with metadata.
- F3. Metadata are clearly assigned to a dataset.
- F4. (Meta)data are indexed in a searchable resource.

Accessibility

- A1. (Meta)data are retrievable with standardized protocols.
- A1.1. The protocol used is open, free, and usable.
- A1.2. The protocol enables authentication and assignment of rights.
- A2. Metadata remain accessible even if data are no longer available.

Interoperability

- I1. (Meta)data use a formal, accessible, shared, and broadly applicable language for knowledge representation.
- I2. (Meta)data use vocabularies that follow the FAIR principles.
- I3. (Meta)data contain qualified references to other (meta)data.

Reusability

- R1. (Meta)data have a variety of accurate and relevant attributes.
- R1.1. (Meta)data contain a clear usage license.
- R1.2. (Meta)data contain detailed provenance information.
- R1.3. (Meta)data meet the standards of the respective disciplinary community.

The demand for fair research data management is now quite widespread, although the FAIR principles can be implemented with varying degrees of bureaucratic effort. It remains to be noted, however, that at present the FAIR principles are essentially an ideal of data management to be aspired to, and by no means all research data are available in FAIR form. In fact, dealing with digital research data remains very unstructured, and the German university system in particular only very slowly understands digital data management as a core task. On a larger, although project-based and thus not sustainable, scale, the initiative for a National Research Data Infrastructure (NFDI) is trying to improve German digital research data management. Unfortunately, it must still be noted that many publicly funded researchers neither want to systematically prepare the research data they collect nor make them available to the public. Here, a cultural change toward greater scientific transparency therefore remains to be hoped for, and open, publicly funded research (Open Research, cf., e.g., Toelch & Ostwald (2018), Miedema (2022), Bertram et al. (2023)) remains an ideal to be aspired to.

17.2 File formats

In general, file formats define the syntax and semantics of data within a file. File formats are bijective mappings of information onto binary long-term storage. In general, one distinguishes *data file formats* from *software file formats*, *textual* from *binary file formats*, and *open* from *proprietary*, that is, copyright-protected, file formats. For binary data file formats, reading data, inspecting them, and manipulating them is generally possible only with special software. Examples of binary file formats are *.docx*, *.pdf*, *.xlsx*, *.jpg*, and *.mp4* files. Binary file formats are often proprietary. In the past, binary file formats were also preferred for research data because of their lower storage requirements. Today, by

contrast, textual data file formats such as *.txt*, *.csv*, *.tsv*, and *.json* are favored. In these formats, reading the stored data, inspecting them, and manipulating them with simple general editors such as VS Code is straightforward, and the file formats are generally not proprietary. Figure 17.1 shows the representation of a binary data format and a textual data format in Windows Notepad. In general, open textual data file formats are therefore appropriate in the research context and in the spirit of the FAIR principles. As examples, we want to consider the .csv, .tsv, and .json file formats as textual file formats in more detail here.

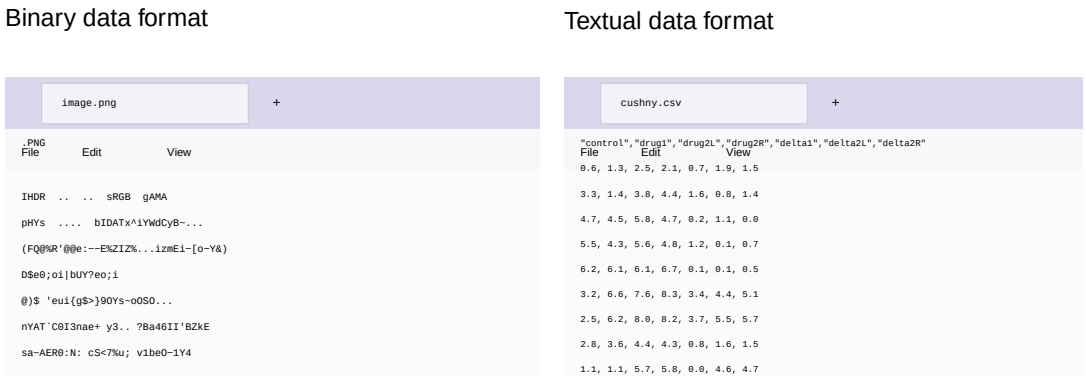


Figure 17.1 Binary and textual file formats as represented in an editor.

17.2.1 .csv and .tsv files

.csv (comma-separated values) and .tsv files (tab-separated values) are important file formats for storing simple, tabularly structured data. In general, .csv and .tsv represent datasets linked row by row. Data fields within a row are separated by commas (.csv) or tab characters (.tsv), and individual datasets are separated from one another by line breaks. The dataset in the first row of a .csv or .tsv file typically contains the definition of the column names and is called the header record. A typical example of a .csv file is shown in Figure 17.1 (right). Here, the rows represent experimental units (here participants), and the columns represent different variables for which a value was determined for the participant.

In the context of data analysis, one distinguishes a *wide-format* and a *long-format* data representation for .csv and .tsv files. In wide-format data representation, all variables of an experimental unit are represented in one row, as in the example above; cf. Table 17.1. For some data analyses, however, it is more helpful to distribute the data of an experimental unit across rows. This leads to the so-called long format, as shown in Table 17.2. In general, the wide format is often more helpful for a first intuitive understanding of a dataset, whereas the long format usually represents data more consistently with the model architecture in model-based analyses.

Table 17.1 Wide format (EU: experimental unit, V1,V2,V3: variables)

EU	V1	V2	V3
1	10.1	67.5	4
2	12.9	51.2	2

EU	V1	V2	V3
3	20.4	70.8	5
4	17.5	60.3	7

Table 17.2 Long format (EU: experimental unit, V1,V2,V3: variables)

Unit	Variable	Measured value
1	V1	10.1
1	V2	67.5
1	V3	4
2	V1	12.9
2	V2	51.2
2	V3	2
3	V1	20.4
3	V2	70.8
3	V3	5
4	V1	17.5
4	V2	60.3
4	V3	7

17.2.2 .json files

.json (JavaScript Object Notation) is a textual data format used to store structured data in key-value form. It has similarities to R lists, especially to named list elements, and is therefore well suited as a format for storing metadata.

A .json file consists of objects that are written in curly braces { } and contain a comma-separated list of properties. Each property is a key-value pair, where the key is always written as a string in quotation marks (" "). The value can in turn be an object, an array, a string, a Boolean, or a number. Figure Figure 17.2 shows the content of a .json file for storing student information.

```
{
  "first_name" : "Maxi",
  "last_name" : "Samplewoman",
  "student_id" : 12345,
  "semester" : 2,
  "program" : "BSc Psychology",
  "modules" :
  {
    "Descriptive statistics" : { "completed" : true,  "grade" : 1.0 },
    "Inferential statistics" : { "completed" : false, "grade" : null }
  }
}
```

Figure 17.2 .json file format.

17.3 Directory management

Because computer directory systems are usually represented in words (e.g., `C:\Users`) rather than numerically, a basis for the automated reading of long-term stored data into memory and, conversely, for saving data from memory in long-term storage is working with so-called *strings* (character strings). Strings are generated in **R** with quotation marks or apostrophes:

```
# quotation marks are the string standard
c("This is a character vector")
```

```
[1] "This is a character vector"
```

```
# apostrophes can help with quotation marks in the string
c('This is a "string"')
```

```
[1] "This is a \"string\""
```

The function `paste()` converts vectors to character and concatenates them element by element.

```
# conversion and concatenation of one-element double vectors
paste(1,2)
```

```
[1] "1 2"
```

```
# concatenation of one-element character strings
paste("This is", "a string")
```

```
[1] "This is a string"
```

The function `paste()` also has a number of other functionalities that can sometimes be helpful:

```
# vector recycling, elementwise links
paste(c("Red", "Yellow"), "flower")
```

```
[1] "Red flower"    "Yellow flower"
```

```
# separator specification
paste(c("Red", "Yellow"), "flower", sep = "-")
```

```
[1] "Red-flower"    "Yellow-flower"
```

```
# concatenation with specified separator
paste(c("Red", "Yellow"), "flower", collapse = ", ")
```

```
[1] "Red flower, Yellow flower"
```

The function `toString()` is a `paste()` variant for numerical vectors:

```
# conversion of a double vector to a formatted string
toString(1:10)
```

```
[1] "1, 2, 3, 4, 5, 6, 7, 8, 9, 10"
```

```
# with the option to restrict output to width characters
toString(1:10, width = 10)
```

```
[1] "1, 2, ...."
```

To load the data of a file in long-term storage into memory, one needs its address in the computer's directory structure. Figure 17.3 shows an excerpt from the directory structure of a typical Windows computer.

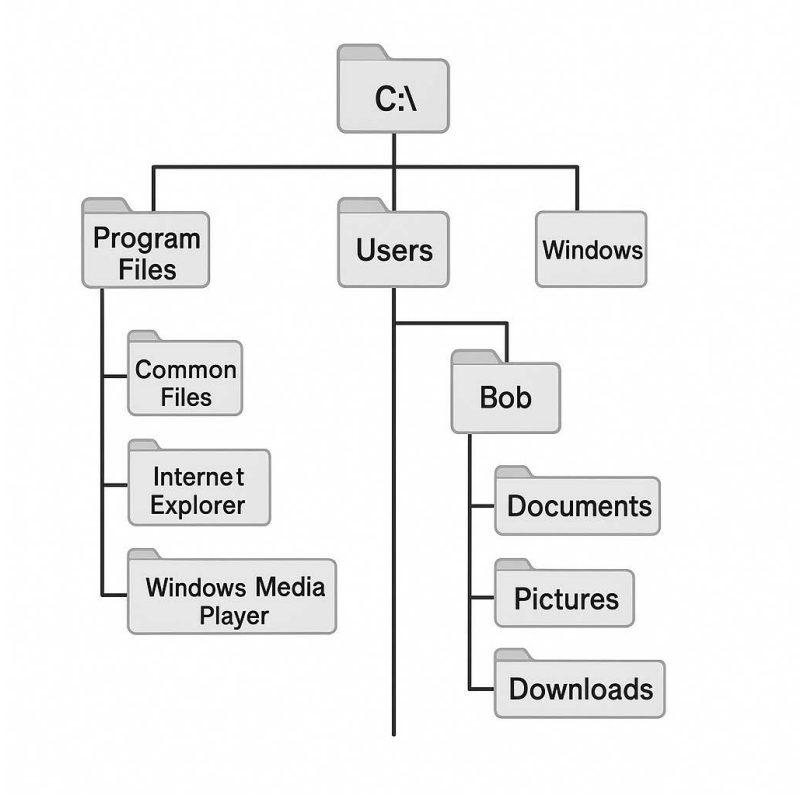


Figure 17.3 Windows directory structure

The addresses of files in the directory structure are called *file paths* (*paths*). A path consists of a list of directory names separated by slashes. Under Windows, left-leaning slashes (\, *backslash*) are traditionally used, whereas under Mac and Linux right-leaning slashes (/, *slash*) are used. Under Windows, the disk name (e.g., `C:\` or `D:\`) is usually found at the top level of the directory structure, under Mac the user name (`/Users/`), and under Linux the home directory (`/home/`). The following examples are Windows-based.

```
# path that ends in a directory name
C:\Users\dirko\Nextcloud\Home\pdwp-online\dirk-ostwald.github.io\_data

# path that ends in a file name
C:\Users\dirko\Nextcloud\Home\pdwp-online\dirk-ostwald.github.io\_data\cushny.csv
```

Absolute file paths give the address of a directory or file in the overall directory structure of the hard drive and are not very susceptible to file confusions. *Relative file paths*, by contrast, refer to a storage location in relation to the current directory, where `.` and `..` denote the current and the superordinate directory. Relative file paths can certainly lead to file confusions. In general, therefore, the use of adaptively generated absolute paths is recommended.

```
# file name in absolute path form
fname =  "D:\Teaching\Data\cushny.csv"
```

R has a so-called *working directory* from which files are read by default. When an **R** terminal is open, `getwd()` indicates the working directory.

```
getwd()
```

```
[1] "C:/Users/dirko/Nextcloud/Home/pdwp-online/dirk-ostwald.github.io-en/_part_2"
```

`setwd()` changes the working directory. Note that **R** works only with forward slashes `/`. The manual specification of Windows paths therefore requires double backward slashes `\\`

```
setwd("C:\\Users\\dirko\\Nextcloud\\Home\\pdwp-online\\dirk-ostwald.github.io\\_data")
getwd()
```

With the function `file.path()`, operating-system-compliant directory and file paths can be generated:

```
file.path("D:", "Nextcloud", "Home", "Teaching")
```

The function `dirname()` gives the directory that contains a directory or file:

```
getwd()
```

```
[1] "C:/Users/dirko/Nextcloud/Home/pdwp-online/dirk-ostwald.github.io-en/_part_2"
```

```
dirname(getwd())
```

```
[1] "C:/Users/dirko/Nextcloud/Home/pdwp-online/dirk-ostwald.github.io-en"
```

The function `basename()` gives the lowest level of a file or directory path:

```
getwd()
```

```
[1] "C:/Users/dirko/Nextcloud/Home/pdwp-online/dirk-ostwald.github.io-en/_part_2"
```

```
basename(getwd())
```

```
[1] "_part_2"
```

17.4 Data import and data export

For reading simply structured .csv files, the **R** function `read.csv()` is a good choice. This function reads a file and stores its contents in a dataframe.

```
D = read.csv("C:\\Users\\dirko\\Nextcloud\\Home\\pdwp-online\\dirk-ostwald.github.io\\_part_2\\_data\\cushny
print(D)
```

```
   Control drug1 drug2L drug2R delta1 delta2L delta2R
1      0.6   1.3    2.5    2.1    0.7     1.9     1.5
2      3.3   1.4    3.8    4.4    1.6     0.8     1.4
3      4.7   4.5    5.8    4.7    0.2     1.1     0.0
4      5.5   4.3    5.6    4.8    1.2     0.1     0.7
5      6.2   6.1    6.1    6.7    0.1     0.1     0.5
6      3.2   6.6    7.6    8.3    3.4     4.4     5.1
7      2.5   6.2    8.0    8.2    3.7     5.5     5.7
8      2.8   3.6    4.4    4.3    0.8     1.6     1.5
9      1.1   1.1    5.7    5.8    0.0     4.6     4.7
10     2.9   4.9    6.3    6.4    2.0     3.4     3.5
```

The following **R** code defines the absolute path of the file adaptively in the directory structure of the computer with the help of the **R** working directory.

```
wdir  = getwd()                            # working directory
print(wdir)                                # output
ddir  = file.path(wdir, "_part_2/_data")   # data directory path
print(ddir)                                # output
fname = "cushny.csv"                       # (base) filename
print(fname)                               # output
fpath = file.path(ddir, fname)             # filepath
print(fpath)                               # output
D     = read.csv(fpath)                    # reading the file
print(D)                                   # output of the dataframe
```

In addition, **R** offers a variety of possibilities for data import using specialized functions:

For .csv and other text files:

- `read.csv()`, `read.csv2()`, `read.delim()`, `read.delim2()` as `read.table()` variants.
- `readLines()` for low-level text file import.
- `fromJSON()` from the `rjson` package for .json files.

For binary files:

- `read.xlsx()` and `read.xlsx2()` from the `xlsx` package for Excel .xlsx files.
- `read.spss()` from the `foreign` package for SPSS .sav files.
- `readMat()` from the `R.matlab` package for MATLAB .mat files.

For databases:

- SQL databases can be queried with the help of the `DBI` and `RSQLite` packages.

18 Control structures

Per se, imperative program code is executed strictly sequentially, command by command. Sometimes, however, one wants to deviate from this purely sequential order of commands and control the execution order of commands more flexibly. Principal tools for this purpose are *conditional control structures* and *iterative control structures*. *Conditional control structures* control program execution by specifying that certain commands are executed only when defined conditions are met. For this purpose, **R** provides `if()` and `switch()` statements. Often, one also wants to repeat (*iterate*) certain sections of code frequently in loops. For this purpose, **R** provides `for()`, `while()`, and `repeat()` loops. In this section, we introduce the basic conditional and iterative control structures in **R**, following the presentation in Cotton (2013), Chapter 8 and Chapter 9, as well as Wickham (2019), Chapter 5.

18.1 Conditional control structures

Conditional control structures allow the condition-dependent execution of commands.

if-statements

The basic conditional control structure is the if-statement. The general syntax for an if-statement in **R** has the following form:

```
if (condition){
  true_actions
}
```

Here, if the variable `condition` has the value `TRUE`, that is, if it is true, the commands denoted by `true_actions` are executed. If the value of the variable `condition` is `FALSE`, by contrast, the commands `true_actions` are not executed, and program execution continues on the next line after the if-statement. For example, in the following code the command `print("x is greater than 0")` is executed, since `x` has the value `1` after the preceding assignment.

```
x = 1
if(x > 0){
  print("x is greater than 0")
}
```

```
[1] "x is greater than 0"
```

The if-statement can be extended by an `else` condition to specify which commands are to be executed in the case that the variable `condition` has the value `FALSE`.

```
if (condition){
  true_actions
} else {
  false_actions
}
```

Here, the commands `true_actions` are executed if the variable `condition` has the value `TRUE`, and the commands `false_actions` are executed if the variable `condition` has the value `FALSE`. For example, in the following code the command `print("y is not greater than 0")` is executed because the condition `y > 0` is not met.

```
y = -1
if(y > 0){
  print("y is greater than 0")
} else{
  print("y is not greater than 0")
}
```

```
[1] "y is not greater than 0"
```

R includes the relation operators listed in Table 18.1 for testing binary logical conditions.

Table 18.1 Relation operators in **R**

Relation operator	Meaning
`==`	Equal
`!=`	Unequal
`<`, `>`	Smaller, greater
`<=`, `>=`	Smaller or equal, greater or equal
`&`	AND
`|`	OR

The relation operators `<`, `<=`, `>`, `>=` are mostly applied only to numerical values. The equality operators `==`,`!=` can be applied to arbitrary data structures to check their equality. The logical links `&` and `|` are mostly applied to logical values. Finally, the function `xor()` implements the exclusive OR. For example, the following tests of equality and relation can be performed:

```
# variable definition
x = 1
y = 2

# test of equality
if(x == y){
  print("x is equal to y")
} else {
  print("x is unequal to y")
}
```

```
[1] "x is unequal to y"
```

```
# test of inequality
if(x < y){
  print("x is smaller than y")
} else {
  print("x is greater than or equal to y")
}
```

```
[1] "x is smaller than y"
```

The if-statement tests can be extended by logical conditions, as the following example is intended to demonstrate:

```
# variable definition
x = 2
y = 2

# logical AND/OR
if(x == y | x < y){
  print("x is smaller than or equal to y")
} else {
  print("x is greater than y")
}
```

```
[1] "x is smaller than or equal to y"
```

```
# logical AND
if(x > y & x != y){
  print("x is greater than y")
} else {
  print("x is smaller than or equal to y")
}
```

```
[1] "x is smaller than or equal to y"
```

For if-statements, it is crucial that the condition corresponds to a single logical value. The following conditions, for example, are erroneous:

```
if("x"){
  print(1)
}
```

```
Error in `if ("x") ...`:
! argument is not interpretable as logical
```

```
if(NA){
  print(1)
}
```

```
Error in `if (NA) ...`:
! missing value where TRUE/FALSE needed
```

```
if(c(T,F)){
  print(1)
}
```

```
Error in `if (c(T, F)) ...`:
! the condition has length > 1
```

switch-statements

Another conditional control structure is the switch-statement. Switch-statements are motivated by the fact that combined if-else-statements can easily become unclear, as the following example shows:

```
x = 2
if (x == 1){
  print("action 1")
} else if(x == 2){
  print("action 2")
} else if(x == 3){
  print("action 3")
} else if(x == 4){
  print("action 4")
}
```

```
[1] "action 2"
```

Thus, if one has a case in which one of many actions is to be executed depending on the value of a variable, a switch-statement can be useful for improving code readability. **R** implements switch-statements in the `switch()` function, which works slightly differently depending on the data type of its variable. If the switch variable is of type integer, the ith action is executed, as the following example demonstrates:

```
x = 2               # switch variable
switch(x,           # x = 2
print("action 1"),  # 1st action
print("action 2"),  # 2nd action
print("action 3"),  # 3rd action
print("action 4"))  # 4th action
```

```
[1] "action 2"
```

If the switch variable is of type character, by contrast, the action with the corresponding name is executed:

```
x = "a"                # switch variable
switch(x,              # x = "a"
a = print("action a"),  # a action
b = print("action b"),  # b action
c = print("action c"),  # c action
d = print("action d"))  # d action
```

```
[1] "action a"
```

In general, it can be said of if- and switch-statements that every if-statement can be formulated equivalently as a switch-statement and vice versa. In practice, if-statements are certainly encountered more frequently than switch-statements. In general, for human code comprehension it makes sense not to formulate if-else-statements too complexly. If this is necessary, however, complex if-else-statements can be simplified by switch-statements.

18.2 Iterative control structures

Iterative control structures serve to execute certain commands repeatedly. The basic iterative control structure in **R** is the `for` loop. It has the general form

```
for (item in vector){
  perform_action
}
```

The command `perform_action` is executed once for each `item` in `vector`, and the value of `item` is updated each time. The following example is intended to illustrate this:

```
for (i in 1:3){
  print(i)
}
```

```
[1] 1
[1] 2
[1] 3
```

Here the vector consists of `1:3 = [1,2,3]`, the command executed is `print(i)`, and the `items` are the entries of the vector. However, the command within the `for` loop does not necessarily have to refer to the `items`, as the following example shows:

```
for (i in 1:3){
  print('a')
}
```

```
[1] "a"
[1] "a"
[1] "a"
```

Here, the constant value `a` is output three times. Typically, something is generated and stored within a `for` loop. In doing so, the corresponding storage structure should generally be preallocated, because the corresponding space in memory is then already provided and does not have to be provided during the execution of the `for` loop. The efficiency gain of this strategy may be negligible in most cases, but this procedure also makes it easier in particular to detect errors, for example if there are discrepancies in dimensional expectations or missing entries that are then already allocated as `NaN`. The following example demonstrates the iterative filling of a vector with means.

```
ns    = 3               # number of simulations
X_bar = rep(NaN, ns)    # storage structure

# simulation iterations
for(i in 1:ns){         # iteration indices are typically i,j,k
  X        = rnorm(12)  # realization of 12 Z variables
  X_bar[i] = mean(X)    # mean of the ith realization
  print(X_bar)          # display for demonstration
}
```

```
[1] 0.2003372       NaN       NaN
[1] 0.2003372 0.3012615       NaN
[1]  0.2003372  0.3012615 -0.1172935
```

To iterate linearly along a vector with variable entries, the function `seq_along()` is useful. It generates the linear indices 1,2, 3, … up to the length of the vector in question, as the following example demonstrates:

```
mu    = c(0,5,50)                # three expected-value parameters
X_bar = rep(NaN, ns)             # storage structure

# simulation iterations
for(i in seq_along(mu)){         # iteration indices are typically i,j,k
  X        = rnorm(12, mu[i], 1) # realization of 12 N(mu,1) variables
  X_bar[i] = mean(X)             # mean of the ith realization
  print(X_bar)                   # display for demonstration
}
```

```
[1] -0.2736334        NaN        NaN
[1] -0.2736334  5.1632245        NaN
[1] -0.2736334  5.1632245 49.8329047
```

`for` loops can also be nested, that is, further `for` loops can be executed within `for` loops. This may take some getting used to. The crucial point here is to realize that for *each* iteration of the outer loop, *all* iterations of the inner loop are run through. It is often helpful first to clarify the general logic of a nested loop structure with the help of `print()` statements, as the following example demonstrates:

```
n_i = 2              # number of iterations of the outer loop
n_j = 3              # number of iterations of the inner loop
for(i in 1:n_i){     # outer loop
  for (j in 1:n_j){  # inner loop
    print(c(i,j))    # display of the iteration indices [i,j]
    }
}
```

```
[1] 1 1
[1] 1 2
[1] 1 3
[1] 2 1
[1] 2 2
[1] 2 3
```

The following example, for instance, generates a 4 x 5 matrix whose elements correspond to the respective column indices. The inner `for` loop with iterator variable `j` serves the column indices, and the outer `for` loop with iterator variable `i` serves the row indices:

```
     [,1] [,2] [,3] [,4] [,5]
[1,]    1    2    3    4    5
[2,]    1    2    3    4    5
[3,]    1    2    3    4    5
[4,]    1    2    3    4    5
```

Analogously, the following example generates a 4 x 5 matrix whose elements correspond to the respective row indices. The inner `for` loop with iterator variable `j` again serves the column indices, and the outer `for` loop with iterator variable `i` serves the row indices:

```
     [,1] [,2] [,3] [,4] [,5]
[1,]    1    1    1    1    1
[2,]    2    2    2    2    2
[3,]    3    3    3    3    3
[4,]    4    4    4    4    4
```

Finally, in the following example, two nested `for` loops and an if-statement are used to generate a 4 x 5 matrix whose elements are equal to the column indices if the respective column index is greater than or equal to the row index, and whose elements are otherwise zero:

```
     [,1] [,2] [,3] [,4] [,5]
[1,]    1    2    3    4    5
[2,]    0    2    3    4    5
[3,]    0    0    3    4    5
[4,]    0    0    0    4    5
```

Part III

Probability theory

Preliminary remarks

Probability theory is a mathematical model for describing and quantitatively reasoning about *random processes* of reality (Figure 18.1). By random processes we mean all phenomena that cannot be predicted by us with absolute certainty, and whose outcome is therefore subject to uncertainty. Obvious and familiar examples of random processes are throwing a die or tossing a coin. However, the concept of a random process, and thus the domain of application of probability theory, should be understood much more broadly. For example, the outcome of an election, tomorrow's weather, the measurement value of an electroencephalography electrode at a particular point in time, or the effect of a psychotherapy intervention on the health status of a patient cannot be predicted with complete certainty and are therefore subject to uncertainty. Once one begins to think about which phenomena of reality are subject to uncertainty, it becomes difficult to name nontrivial phenomena about whose outcome one has complete certainty.

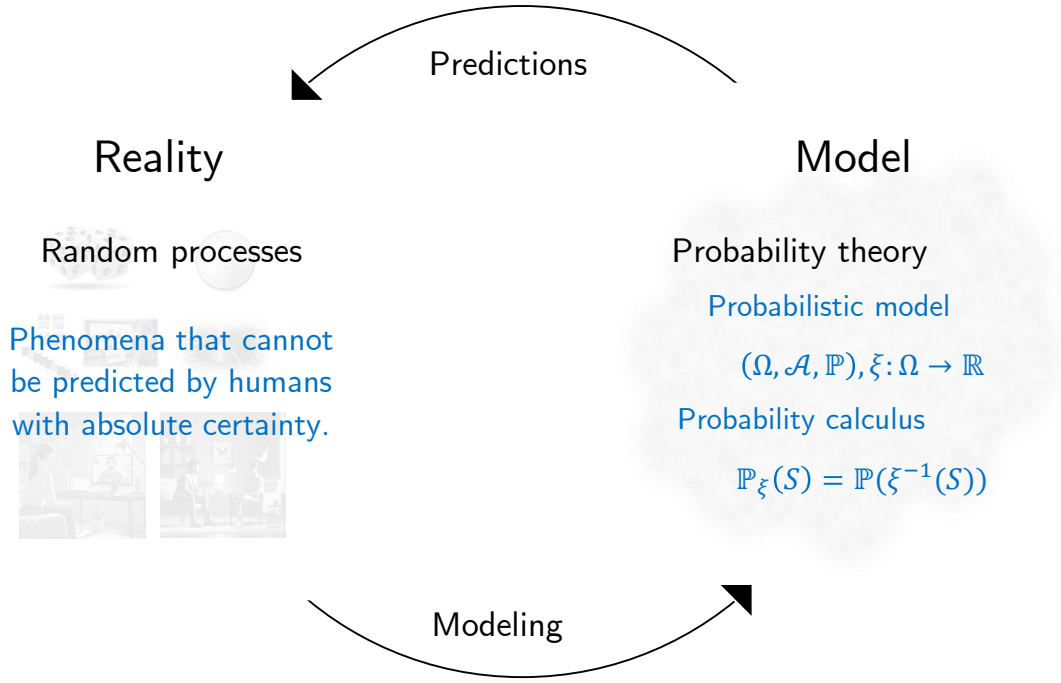


Figure 18.1 *Probability theory as a model of random processes.* The starting point of probability theory is the intention to draw logical-quantitative conclusions about a random process, that is, a phenomenon of reality subject to uncertainty. The representation of central aspects of the random process with the help of probability-theoretic terminology is called *modeling*. The probability-theoretic model itself then guarantees, in the sense of probability calculus, the correctness of logical-quantitative conclusions that can be used to predict aspects of the random process.

As a mathematical model of random processes, probability theory in particular allows reason-based, quantitative reasoning about random processes. This is reflected primarily in so-called *probability calculus*. Quantitative conclusions of probability calculus have, for example, the following form: If I assume that event A occurs with probability p_1 and event B occurs with probability p_2, then event C has probability p_3. The conclusion about the probability of C is logically and mathematically secured in the same sense in which, for example, it is logically and mathematically secured that $1 + 1 = 2$. Whether the assumptions about the probabilities of A and B correspond to the conditions of the

random process in reality, however, is not something probability theory makes statements about.

Probability theory itself uses the mathematical theory of sets and functions. At least since Kolmogoroff (1933), an axiomatic approach has prevailed: in probability theory itself, one does not ask what a probability is or to what extent the predictions of probability theory agree with reality, but instead tries to develop a self-consistent formal-mathematical system of ungrounded but intuitively plausible basic assumptions and their consequences. The starting point of this development is the *probability space model* of a random process, which will be introduced in Chapter 19. Indeed, beyond the formal-mathematical system of probability theory, mathematical-philosophical discussions continue to this day about exactly what is to be understood by the term "probability of an event". Roughly speaking, two somewhat opposing interpretations are predominant: on the one hand the so-called *frequentist interpretation* as a prototypical example of *realist interpretations*, and on the other hand the so-called *Bayesian interpretation* as a general form of so-called *evidential interpretations*. For a more detailed overview of different interpretations of the concept of probability, see Table 18.2 after Hájek (2019).

Table 18.2 Interpretations of probability after Hájek (2019). In its essential features, the *frequentist interpretation* corresponds to the *infinite frequentist realist interpretation* sketched in the table; in its essential features, the *Bayesian interpretation* corresponds to the *subjective evidential* and *epistemic evidential interpretations* sketched in the table.

Interpretation	Intuition	Field of application	Problems	References
Finite frequentist	The probability of a coin showing heads corresponds to the relative frequency with which the coin shows heads in a finite number of observations.	-	A single coin toss does not yield a meaningful prediction; probabilities change depending on the observation series and are therefore not uniquely defined.	Venn (1876)
Infinite frequentist	The probability of a coin showing heads corresponds to the relative frequency with which the coin shows heads in infinitely many observations.	Frequentist inference	There are no infinite observation series; for a finite number of coin tosses, the probability of an event is not defined.	Reichenbach (1949) von Mises (1951)

Interpretation	Intuition	Field of application	Problems	References
Propensity	The probability of a coin showing heads corresponds to its inherent physical tendency to land heads up in the gravitational field of the Earth.	Causal inference	The connection to relative frequencies or physical theories is not immediately clear.	Peirce (1957) Popper (1959) Gillies (2000)
Classical	In the absence of further information, the probabilities of a coin showing heads or tails are equal.	Games of chance, textbooks	Restriction to finite outcome spaces	Laplace (1814) Jaynes (1968)
Subjective	The probability of a coin showing heads corresponds to the subjective degree of uncertainty that an observer assigns to the outcome of a coin toss.	Bayesian inference	No principled restriction to axiomati-cally valid probabilities	De Finetti (1975)
Epistemic	The probability of a coin showing heads corresponds to the subjective degree of uncertainty that a rational observer assigns to the outcome of a coin toss.	Bayesian inference	The connection to relative frequencies or physical theories is not immediately clear	Ramsey (1926) Jeffreys (1939) Savage (1954)

According to the *frequentist interpretation*, the probability of an event is the idealized relative frequency with which an event tends to occur under the same external conditions. For example, the frequentist interpretation of the statement "With a probability of 1/6, the die will show a 2 on the next throw" is the following: "If one were to throw a die infinitely often and determine the relative frequency of the event that the die shows a 2, then this relative frequency would be equal to 1/6." Note that, in this interpretation, one de facto cannot empirically determine the probability of an event, since one cannot throw

a die infinitely often. Of course, in this interpretation one can estimate the probability empirically. Estimation procedures themselves, however, are not part of probability theory, but of *inference*.

According to the *Bayesian interpretation*, the probability of an event is the degree of certainty that an observer assigns to the occurrence of the event based on her subjective assessment of the situation. For example, the Bayesian interpretation of the statement "With a probability of 1/6, the die will show a two on the next throw" would then be something like the following: "Based on my own and transmitted experience with throwing a die, I am 16.6% certain that the die will show a two on the next throw."

In models of random processes that can in fact be repeated at least under similar circumstances, such as throwing a die, the difference between the frequentist and Bayesian interpretations is often rather subtle. However, as indicated above, there are many random processes that can be described with probabilities for which, because of their uniqueness, a frequentist interpretation is not appropriate. For example, statements of the form "The probability that the global heat records in 2023 are not attributable to climate change is less than 0.01" (cf. Philip et al. (2020)) make sense only under the Bayesian interpretation, because the weather records of the year 2023 are a unique, non-repeatable event.

Although, then, the interpretation of the concept of probability is by no means unambiguous, the formal definitions and calculation rules for probabilities do not differ. Both frequentist and Bayesian inference have an identical mathematical reference system and common foundation in probability theory. In essence, this fact is not so different from many other forms of mathematical modeling. For example, the derivative of a function can be interpreted as the velocity of an object or as the growth rate of a bacterial population. Although these phenomena differ intuitively very strongly with respect to reality, logical reasoning about both phenomena is identical in the domain of mathematics.

19 Probability spaces

A *probability space* is a very general formal-mathematical model of a random process. The central importance of this model for probability theory and probabilistic inference follows from the fact that the probability space model provides guidance for translating arbitrary random processes about which one wants to reason quantitatively into the formal-mathematical system of probability theory. At the same time, the probability space model and the concepts built on it guarantee that the mechanics of probability calculus lead to logically meaningful quantitative conclusions about random processes. In this chapter, we introduce the concept of a probability space (Section 19.1) and then use probability functions (Section 19.2) to give first examples of modeling random processes (Section 19.3).

19.1 Definition and first properties

We begin with the definition of the probability space model after Kolmogoroff (1933), which we then explain in its individual parts from a frequentist perspective.

Definition 19.1 (Probability space)**.** A *probability space* is a triple $(\Omega, \mathcal{A}, \mathbb{P})$, where

- Ω is an arbitrary nonempty set of *outcomes* ω and is called the *outcome set*,
- $\mathcal{A}$ is a set of subsets of Ω with the properties

  – $\Omega \in \mathcal{A}$,
  – for all $A \in \mathcal{A}$, also $A^c := \Omega \setminus A \in \mathcal{A}$,
  – from $A_1, A_2, ... \in \mathcal{A}$ it follows that also $\cup_{i=1}^{\infty} A_i \in \mathcal{A}$

  holds, is called a *σ-algebra on* Ω, and is called the *event system*,
- $\mathbb{P}$ is a mapping of the form $\mathbb{P} : \mathcal{A} \to [0, 1]$ with the properties

  – $\mathbb{P}(A) \geq 0$ for all $A \in \mathcal{A}$ (*non-negativity*),
  – $\mathbb{P}(\Omega) = 1$ (*normalization*),
  – $\mathbb{P}(\cup_{i=1}^{\infty} A_i) = \sum_{i=1}^{\infty} \mathbb{P}(A_i)$ for pairwise disjoint $A_i \in \mathcal{A}$ (σ-*additivity*)

  holds and is called a *probability measure*.

•

Outcome set and mechanics

We begin with explanations of the concept of the *outcome set* Ω and the implicit mechanics of the probability space model. To make the introduction easier, in what follows we first primarily consider *finite probability spaces*, in which the cardinality of Ω is not infinitely large. Thus let $|\Omega| < \infty$, so that Ω has only finitely many elements. For modeling the throw of a die, for example, one could define $\Omega := \{1, 2, 3, 4, 5, 6\}$.

Behind the formal definition of the probability space model are the following frequentist-influenced assumptions about its mechanics as a model of a random process. We first imagine sequential runs of a random process, for example the repeated throwing of a die. According to the assumptions of the probability space model, in each of these runs exactly one ω from Ω is *realized*, that is, selected as actually occurring. For example, if one throws a die and a two appears, one says that a two was realized. The probability with which an ω from Ω is realized in a run is described by the value $\mathbb{P}(\{\omega\}) \in [0, 1]$. If, for example, $\mathbb{P}(\{\omega\}) = 1$, then this ω is realized in every run of the random process; if $\mathbb{P}(\{\omega\}) = 0$, then this ω is realized in no run of the random process; and if $\mathbb{P}(\{\omega\}) = 1/2$, then ω is realized in about half of the runs of the random process. In the model of throwing a fair die, one usually assumes $\mathbb{P}(\{\omega\}) = 1/6$ for all $\omega \in \Omega$. Here, for example, a four could be realized in the first run, a one in the second run, a five in the third run, then perhaps a four again, and so on.

Events and event system

The concept of an *event* $A \in \mathcal{A}$ is best imagined as a conceptual summary of one or more outcomes. When throwing a die, possible events are, for example, "An even number of pips appears", that is, $\omega \in \{2, 4, 6\}$, "A number of pips greater than two appears", that is, $\omega \in \{3, 4, 5, 6\}$, or "A one or a five appears", that is, $\omega \in \{1, 5\}$. Note that, against the background of the mechanics of the probability space, the event "An even number of pips appears" occurs exactly when, in a run of the random process *throwing a die*, the realized ω is an element of the set $\{2, 4, 6\}$, for example when a four appears. The occurrence of the event "A number of pips greater than two appears" may also be read as "In a run of the random process *throwing a die*, an element of $\{3, 4, 5, 6\}$ is realized", that is, concretely, either a three, a four, a five, or a six appears. Of course, the outcomes $\omega \in \Omega$ themselves are also possible events, so that, for example, the following interpretations hold: the event "A one appears" corresponds to the realization $\omega \in \{1\}$, and the event "A six appears" corresponds to the realization $\omega \in \{6\}$. If one considers an outcome $\omega \in \Omega$ as an event in this context, it is called an *elementary event* and is written as the one-element set $\{\omega\}$.

Overall, this procedure for describing random events corresponds to the inherent goal of the definition of the probability space. Kolmogoroff (1933) writes that the actual objects of the subsequent considerations, the random events, have been defined as sets. This has the advantage that sets are mathematical objects with which one can work mathematically, so that an aspect of reality, a "random event", has been translated into the model domain of mathematics.

The sole purpose of the *event system* $\mathcal{A}$ is now to mathematically represent all events that can arise, based on a given outcome set, when an $\omega \in \Omega$ is selected. Thus there should be no events in reality that were not considered in advance in the probability space model of the random process. If this were the case, the model would be deficient, since it could

not assign probabilities to certain events occurring in reality. The event system $\mathcal{A}$ should therefore be the complete set of all possible events for a given Ω. The requirement that $\mathcal{A}$ should satisfy the so-called σ-algebra criteria for this purpose is intuitively motivated as follows.

- It should first be ensured that $\omega \in \Omega$ for an arbitrary ω, that is, that some outcome is realized, is one of the possible events. This corresponds to the property $\Omega \in \mathcal{A}$.
- For every event it should also be possible that this event does not occur. This corresponds to the property that from $A \in \mathcal{A}$ it should follow that $A^c := \Omega \setminus A$ is also in $\mathcal{A}$. This also implies in particular that $\emptyset = \Omega \setminus \Omega \in \mathcal{A}$. Thus one event is that no elementary event occurs, although this happens only with probability zero, $\mathbb{P}(\emptyset) = 0$. Thus at least one elementary event always occurs with certainty.
- Finally, the combination of events should always also be an event. In modeling the throw of a die, for example, in addition to the events "An even number appears" and "A number greater than two appears", the event "An even number appears and/or this number is greater than two" should also be an event. In its most general form, this corresponds to the requirement that from $A_1, A_2, ... \in \mathcal{A}$ it should follow that also $\cup_{i=1}^{\infty} A_i \in \mathcal{A}$.

Even if the concepts of the event system and the σ-algebra may seem somewhat abstract, their definition in the practical modeling of random processes is usually not a major challenge, since suitable event systems have long been known both for finite outcome sets and for infinite countable and uncountable outcome sets. Thus, for outcome sets with finite cardinality, the power set of the outcome set always satisfies the requirements of an event system and can always be used to formulate a model of a random process with a finite outcome set. This is the statement of the following theorem.

Theorem 19.1 (Event system for finite outcome set)**.** *Let $\Omega := \{\omega_1, \omega_2, ..., \omega_n\}$ with $n \in \mathbb{N}$ be a finite set. Then the power set $\mathcal{P}(\Omega)$ of Ω is a σ-algebra on Ω and thus a suitable event system in the probability space model.*

∘

Proof. The power set of Ω is the set of all subsets of Ω. We check the σ-algebra properties. First, Ω itself is one of the subsets of Ω, so the first σ-algebra property is satisfied. Now let A be a subset of Ω. Then $A^c = \Omega \setminus A$ is also a subset of Ω, and hence the second σ-algebra property is also satisfied. Finally, consider the union of k subsets $A_1, A_2, ..., A_k \subseteq \Omega$. Then $\cup_{i=1}^{k} A_i$ is the set of $\omega \in \Omega$ for which $\omega \in A_1$ and/or $\omega \in A_2$ … and/or $\omega \in A_k$. Since for all these ω it holds that $\omega \in \Omega$, also $\cup_{i=1}^{k} A_i$ is a subset of Ω, and therefore the third σ-algebra property is also satisfied. The power set thus satisfies the required properties of an event system. □

For uncountable outcome sets such as the real numbers $\mathbb{R}$ or the n-dimensional real space $\mathbb{R}^n$, the construction of a suitable event system is more complex, so in this respect we refer to the advanced literature for formal developments (cf. Meintrup & Schäffler (2005), Schmidt (2009)). However, with the so-called *Borel σ-algebra*, which goes back to Borel (1898), a set system is known that satisfies the requirements of a σ-algebra on uncountable outcome sets. We denote the Borel σ-algebra on $\mathbb{R}$ by $\mathcal{B}(\mathbb{R})$ and the Borel σ-algebra on $\mathbb{R}^n$ by $\mathcal{B}(\mathbb{R}^n)$. As sets of subsets of $\mathbb{R}$ and $\mathbb{R}^n$, respectively, $\mathcal{B}(\mathbb{R})$ and $\mathcal{B}(\mathbb{R}^n)$ contain all sets whose probability assigned by $\mathbb{P}$ one may be interested in. Intuitively, one may therefore think of the Borel σ-algebras $\mathcal{B}(\mathbb{R})$ and $\mathcal{B}(\mathbb{R}^n)$ as the power sets of $\mathbb{R}$ and $\mathbb{R}^n$, respectively,

even though this is formally incorrect. In fact, the Borel σ-algebra contains only subsets generated by countable set operations, not by uncountable ones.

Overall, this leads to the following procedure for selecting event systems depending on the outcome set Ω. If Ω is finite, one chooses as event system $\mathcal{A}$ the power set $\mathcal{P}(\Omega)$ of Ω. If Ω is given by $\mathbb{R}$, one chooses as event system $\mathcal{A}$ the Borel σ-algebra $\mathcal{B}(\mathbb{R})$. Finally, if Ω is given by $\mathbb{R}^n$, one chooses for $\mathcal{A}$ the Borel σ-algebra $\mathcal{B}(\mathbb{R}^n)$. In special cases and for very special outcome sets Ω, one may want to deviate from this procedure, but in general the three cases considered cover most applications.

Probability measure $\mathbb{P}$

With Ω and $\mathcal{A}$, which in tuple form $(\Omega, \mathcal{A})$ are also called a *measurable space*, we have so far examined more closely the *structural basis* of a probability space model. Many probability spaces, for example for random processes concerning real numbers, are identical with respect to their measurable space. The *probability measure* $\mathbb{P}$ now represents the probabilistic characteristics of a probability space model and thus forms the *functional basis* of such a model. In what follows, especially after introducing the concepts of random variables and random vectors, we will encounter many different probability measures. At this point, we first want to consider only general properties of probability measures.

With the definition

$$\mathbb{P} : \mathcal{A} \to [0, 1], A \mapsto \mathbb{P}(A) \tag{19.1}$$

it first follows that a probability measure is defined on a set of sets and assigns probabilities, that is, values in the interval $[0, 1]$, to the elements of this set, namely to the sets $A \in \mathcal{A}$. Of course, with $\{\omega\} \in \mathcal{A}$ for all $\omega \in \Omega$, $\mathbb{P}$ also assigns probabilities to the elementary events, but not only to them. We also emphasize that, by definition, the probability $\mathbb{P}(A)$ of an event $A \in \mathcal{A}$ is a number in the interval $[0, 1]$ and not, for example, a percentage or a ratio. In what follows, we will examine more closely the defining properties of *non-negativity*, *normalization*, and *σ-additivity* of $\mathbb{P}$.

The *non-negativity* $\mathbb{P}(A) \geq 0$ for all $A \in \mathcal{A}$ is of course implicit in the definition $[0, 1]$ of the target set of $\mathbb{P}$. In fact, the mapping form of $\mathbb{P}$ is an addition we have made to the formulation of Kolmogoroff (1933) in the interest of clarity. Formally, the form of the target set of $\mathbb{P}$ actually follows from the defining properties of non-negativity, normalization, and σ-additivity of $\mathbb{P}$.

The *normalization* $\mathbb{P}(\Omega) = 1$ corresponds to the fact that, in every run of a random process, the realized ω is certainly an element of Ω. Thus in every run of a random process an elementary event occurs and, depending on the nature of the measurable space, many others as well. In the model of throwing a die, for example, the outcome/elementary event realized in a run of the random process is an element of $\Omega := \{1, 2, 3, 4, 5, 6\}$ with probability 1. If the realized outcome is, for example, a one, then in addition to the event "A one appears", the events "An odd number appears", "A number less than three appears", "An odd number less than three appears", and many others also occur.

The *σ-additivity of the probability measure* $\mathbb{P}$, finally, forms the foundation of *probability calculus*, that is, the basis for calculating with probabilities. The σ-additivity of $\mathbb{P}$ makes it possible to calculate the probabilities of other events from already known event probabilities. Based on a definition of $\Omega, \mathcal{A}$, and $\mathbb{P}$, one can therefore calculate probabilities for all possible events of a probability space model. Whether these probabilities actually

have anything to do with real events concerning a random process in reality depends on whether the modeling is reasonably successful or not. However, the calculated probabilities are at least determined rationally, that is, according to the rules of reason, namely logic and mathematics. Overall, the probability model thus allows inferential thinking about random processes, that is, about phenomena subject to uncertainty.

Finally, we want to illustrate probability calculations based on the σ-additivity of $\mathbb{P}$ with two basic examples. The first example states that the probability that the ω realized in a run of a random process is not an element of the outcome set is equal to zero.

Theorem 19.2 (Probability of the impossible event)**.** *Let $(\Omega, \mathcal{A}, \mathbb{P})$ be a probability space. Then*

$$\mathbb{P}(\emptyset) = 0. \tag{19.2}$$

∘

Proof. For $i = 1, 2, \ldots$, let $A_i := \emptyset$. Then $A_1, A_2, \ldots$ is a sequence of disjoint events because $\emptyset \cap \emptyset = \emptyset$, and $\cup_{i=1}^{\infty} A_i = \emptyset$. With the σ-additivity of $\mathbb{P}$ it then follows that

$$\mathbb{P}(\emptyset) = \mathbb{P}\left(\cup_{i=1}^{\infty} A_i\right) = \sum_{i=1}^{\infty} \mathbb{P}\left(A_i\right) = \sum_{i=1}^{\infty} \mathbb{P}\left(\emptyset\right). \tag{19.3}$$

Thus the infinite addition of the number $\mathbb{P}(\emptyset) \in [0, 1]$ should again yield $\mathbb{P}(\emptyset)$. This is only possible if $\mathbb{P}(\emptyset) = 0$. □

Note that, intuitively, a possible inadequacy of the probability space as a model for random processes in reality can arise here: if, when playing dice, the die falls out of reach under the sofa, then an elementary event $\omega \notin \Omega$ has occurred, even though its modeled probability is equal to zero.

As a second example, we want to show that σ-additivity, which in the definition of the probability space is defined only for the union of infinitely many disjoint events, implies the σ-additivity of finitely many disjoint events as they often occur in applications.

Theorem 19.3 (σ-additivity for finite sequences of disjoint events)**.** *Let $(\Omega, \mathcal{A}, \mathbb{P})$ be a probability space and let $A_1, \ldots, A_n$ be a finite sequence of pairwise disjoint events. Then*

$$\mathbb{P}(\cup_{i=1}^{n} A_i) = \sum_{i=1}^{n} \mathbb{P}(A_i). \tag{19.4}$$

∘

Proof. We consider an infinite sequence of pairwise disjoint events $A_1, A_2, \ldots$, where for an $n \in \mathbb{N}$ it is to hold that $A_i := \emptyset$ for $i > n$. Then, with the σ-additivity of $\mathbb{P}$, it first follows that

$$\mathbb{P}\left(\cup_{i=1}^{n} A_i\right) = \mathbb{P}\left(\cup_{i=1}^{\infty} A_i\right) = \sum_{i=1}^{\infty} \mathbb{P}\left(A_i\right) = \sum_{i=1}^{n} \mathbb{P}\left(A_i\right) + \sum_{i=n+1}^{\infty} \mathbb{P}\left(A_i\right). \tag{19.5}$$

With $\mathbb{P}\left(A_i\right) = \mathbb{P}(\emptyset) = 0$ for $i = n+1, n+2, \ldots$, it follows directly that

$$\mathbb{P}\left(\cup_{i=1}^{n} A_i\right) = \sum_{i=1}^{n} \mathbb{P}\left(A_i\right) + 0 = \sum_{i=1}^{n} \mathbb{P}\left(A_i\right). \tag{19.6}$$

□

To prepare the concept of the joint probability of events, we finally consider one further important property of σ-algebras: in addition to being closed under countable unions, $\mathcal{A}$ is also closed under countable intersections. This is a direct consequence of the closure of $\mathcal{A}$ under complements and countable unions and is recorded in the following theorem.

Theorem 19.4 (Closure of σ-algebras under intersections)**.** *Let $\mathcal{A}$ be a σ-algebra on a set Ω. Then from $A, B \in \mathcal{A}$ it follows that also $A \cap B \in \mathcal{A}$. More generally, for $A_1, A_2, ... \in \mathcal{A}$, also $\cap_{i=1}^{\infty} A_i \in \mathcal{A}$.*

∘

Proof. We show the first statement by way of example. To this end, first note that by the first De Morgan rule and the properties of complements,

$$(A \cap B)^c = A^c \cup B^c \Leftrightarrow ((A \cap B)^c)^c = (A^c \cup B^c)^c \Leftrightarrow A \cap B = (A^c \cup B^c)^c . \tag{19.7}$$

Furthermore, with the defining properties of σ-algebras, for $A, B \in \mathcal{A}$ it holds that also $(A^c \cup B^c)^c \in \mathcal{A}$. For from the closure of $\mathcal{A}$ under complements it first follows that also $A^c \in \mathcal{A}$ and $B^c \in \mathcal{A}$. Further, from the closure of $\mathcal{A}$ under countable unions it also follows that $A^c \cup B^c \in \mathcal{A}$. Finally, with the closure of $\mathcal{A}$ under complements, $(A^c \cup B^c)^c \in \mathcal{A}$. Overall, then, from $A, B \in \mathcal{A}$ it follows that $A \cap B \in \mathcal{A}$. The generalization to countable intersections $A_1, A_2, ... \in \mathcal{A}$ follows analogously. □

According to Theorem 19.4, in addition to the probabilities $\mathbb{P}(A), \mathbb{P}(B)$ and $\mathbb{P}(A \cup B)$ with $A, B \in \mathcal{A}$, the probability $\mathbb{P}(A \cap B)$ is therefore also well-defined.

19.2 Probability functions

In this section, with *probability functions*, we want to learn about a way of specifying probability measures for probability spaces with finite outcome sets. In the sections on random variables and random vectors, we will encounter probability functions again under the name *probability mass functions*. We first define the concept of a probability function as follows.

Definition 19.2 (Probability function)**.** Let Ω be a finite set. Then a function $\pi : \Omega \to [0, 1]$ is called a *probability function* if

$$\sum_{\omega \in \Omega} \pi(\omega) = 1. \tag{19.8}$$

Furthermore, let $\mathbb{P}$ be a probability measure. Then the function defined by

$$\pi : \Omega \to [0, 1], \omega \mapsto \pi(\omega) := \mathbb{P}(\{\omega\}) \tag{19.9}$$

is called the *probability function of $\mathbb{P}$ on Ω*.

•

We note that because $\mathbb{P}$ is σ-additive by definition, in particular

$$\mathbb{P}(\Omega) = \mathbb{P}(\cup_{\omega \in \Omega} \{\omega\}) = \sum_{\omega \in \Omega} \mathbb{P}(\{\omega\}) = \sum_{\omega \in \Omega} \pi(\omega) = 1. \tag{19.10}$$

The following theorem provides the formal basis for constructing probability measures through probability functions. In particular, it states that for finite Ω, the probabilities of *all* possible events can be calculated from the probabilities of the elementary events $\pi(\omega)$.

Theorem 19.5. *Let $(\Omega, \mathcal{A}, \mathbb{P})$ be a probability space with finite outcome set, and let $\pi : \Omega \to [0,1]$ be a probability function. Then there exists a probability measure $\mathbb{P}$ on Ω with π as the probability function of $\mathbb{P}$. This probability measure is defined as*

$$\mathbb{P} : \mathcal{A} \to [0,1], A \mapsto \mathbb{P}(A) := \sum_{\omega \in A} \pi(\omega). \tag{19.11}$$

∘

Proof. We first check the probability measure properties of $\mathbb{P}$. Because $\pi(\omega) \in [0,1]$ for all $\omega \in \Omega$, it is always true that $\sum_{\omega \in A} \pi(\omega) \geq 0$, and hence non-negativity of $\mathbb{P}$ holds. Furthermore, as seen above, normalization of $\mathbb{P}$ follows directly from normalization of π. Now let $A_1, A_2, \ldots \in \mathcal{A}$. Then

$$\mathbb{P}\left(\cup_{i=1}^{\infty} A_i\right) = \sum_{\omega \in \cup_{i=1}^{\infty} A_i} \pi(\omega) = \sum_{i=1}^{\infty} \sum_{\omega \in A_i} \pi(\omega) = \sum_{i=1}^{\infty} \mathbb{P}(A_i) \tag{19.12}$$

and hence σ-additivity of $\mathbb{P}$. □

Thus, if for a given outcome set Ω one defines a function $\pi : \Omega \to [0,1]$, ensures that its function values $\pi(\omega)$ sum to 1 over all $\omega \in \Omega$, and then interprets the individual function value $\pi(\omega)$ as the probability $\mathbb{P}(\{\omega\})$ of the elementary event $\{\omega\} \in \mathcal{A}$, one has constructed a probability measure. Conversely, if for a finite outcome set Ω one defines a probability $\mathbb{P}(\{\omega\}) \in [0,1]$ for all elementary events $\{\omega\} \in \mathcal{A}$ and ensures that $\mathbb{P}(\Omega) = 1$, then one has of course implicitly also satisfied the requirements of a probability function. In this case, the σ-additivity of $\mathbb{P}$ can be used directly for the pairwise disjoint elementary events to calculate compound events. We illustrate this and the procedure using a probability function in Section 19.3 by means of examples.

19.3 Examples with finite outcome space

From what has been said so far, the following procedure for modeling a random process using a probability space $(\Omega, \mathcal{A}, \mathbb{P})$ can be summarized:

(1) In a first step, one considers a sensible definition of the outcome set Ω, that is, of the outcomes or elementary events that are to be realized in each run of the random process.

(2) In a second step, one then chooses a suitable event system. In the case of a finite outcome set, the power set $\mathcal{P}(\Omega)$ is suitable; in the case of the uncountable outcome set $\Omega := \mathbb{R}$, the Borel σ-algebra $\mathcal{B}(\mathbb{R})$ is suitable.

(3) Finally, one defines a probability measure $\mathbb{P}$ that represents the probabilities of the occurrence of all possible events. In the case of a finite outcome set, this is accomplished in particular by defining the probabilities of the elementary events. In what follows, we illustrate this procedure with examples. For the uncountable outcome set $\Omega := \mathbb{R}$, defining $\mathbb{P}$ with the help of *probability density functions* is suitable, as we will see later.

Throwing one die

We model throwing one die. It is certainly sensible to define the outcome set as $\Omega := \{1, 2, 3, 4, 5, 6\}$. However, the definition

$$\Omega := \{\cdot, \cdot\cdot, \cdot\cdot\cdot, \cdot\cdot\cdot\cdot, \cdot\cdot\cdot\cdot\cdot, \cdot\cdot\cdot\cdot\cdot\cdot\} \tag{19.13}$$

would also be possible in an equivalent way.

Because this is a finite outcome set, we choose the power set $\mathcal{P}(\Omega)$ as the σ-algebra $\mathcal{A}$. Then $\mathcal{A}$ automatically contains all possible events. The cardinality of $\mathcal{A} := \mathcal{P}(\Omega)$ is $|\mathcal{P}(\Omega)| = 2^{|\Omega|} = 2^6 = 64$. In Table 19.1 we list six of these 64 events in their verbal description and as a subset A of Ω.

Table 19.1 Selected events in the model of throwing a die.

Description	Set form
Any number of pips appears	$\omega \in A = \Omega$
No number of pips appears	$\omega \in A = \emptyset$
A number of pips greater than 4 appears	$\omega \in A = \{5, 6\}$
An even number of pips appears	$\omega \in A = \{2, 4, 6\}$
A six appears	$\omega \in A = \{6\}$
A one, a three, or a six appears	$\omega \in A = \{1, 3, 6\}$

This completes the definition of the measurable space $(\Omega, \mathcal{A})$ in the modeling of throwing a die.

As described in Section 19.2, a probability measure $\mathbb{P}$ can be defined by specifying $\mathbb{P}(\{\omega\})$ for all $\omega \in \Omega$. For the model of a fair die, one would choose

$$\mathbb{P}(\{\omega\}) := \frac{1}{|\Omega|} := 1/6 \text{ for all } \omega \in \Omega. \tag{19.14}$$

For a model of a biased die that favors throwing a six, one could, for example, define

$$\mathbb{P}(\{\omega\}) := \frac{1}{8} \text{ for } \omega \in \{1, 2, 3, 4, 5\} \text{ and } \mathbb{P}(\{6\}) := \frac{3}{8}. \tag{19.15}$$

In the case of the fair die, for example, the probability of the event "An even number of pips appears" is then obtained with the σ-additivity of $\mathbb{P}$ as

$$\mathbb{P}(\{2, 4, 6\}) = \mathbb{P}(\{2\} \cup \{4\} \cup \{6\}) = \mathbb{P}(\{2\}) + \mathbb{P}(\{4\}) + \mathbb{P}(\{6\}) = \frac{1}{6} + \frac{1}{6} + \frac{1}{6} = \frac{3}{6}. \tag{19.16}$$

In the case of the above model of a biased die, by contrast, the probability of the same event is

$$\mathbb{P}(\{2, 4, 6\}) = \mathbb{P}(\{2\} \cup \{4\} \cup \{6\}) = \mathbb{P}(\{2\}) + \mathbb{P}(\{4\}) + \mathbb{P}(\{6\}) = \frac{1}{8} + \frac{1}{8} + \frac{3}{8} = \frac{5}{8}. \tag{19.17}$$

The event considered has a higher probability in the model of the biased die than in the model of the fair die, which is intuitively sensible because six is an even number.

Simultaneous throwing of a blue and a red die

We now want to model the simultaneous throwing of a blue and a red die. For this purpose it is sensible to define the outcome set as

$$\Omega := \{(r, b) | r \in \{1, 2, 3, 4, 5, 6\}, b \in \{1, 2, 3, 4, 5, 6\}\} \tag{19.18}$$

with cardinality $|\Omega| = 36$, where r is to represent the number of pips on the red die and b the number of pips on the blue die.

Again, choosing the power set of Ω as the σ-algebra is suitable, so we again define $\mathcal{A} := \mathcal{P}(\Omega)$. The number of events possible in this model is $|\mathcal{A}| = 2^{|\Omega|} = 2^{36} = 68,719,476,736$. In Table 19.2 we list six of these events in their verbal description and as a subset A of Ω.

Table 19.2 Selected events in the model of throwing a red and a blue die.

Description	Set form
A three appears on the red die	$\omega \in A = \{(3,1), (3,2), (3,3), (3,4), (3,5), (3,6)\}$
A three appears on the blue die	$\omega \in A = \{(1,3), (2,3), (3,3), (4,3), (5,3), (6,3)\}$
A three appears on both dice	$\omega \in A = \{(3,3)\}$
Doubles appear	$\omega \in A = \{(1,1), (2,2), (3,3), (4,4), (5,5), (6,6)\}$
The sum of the numbers that appear is four	$\omega \in A = \{(1,3), (3,1), (2,2)\}$

The definition of the measurable space $(\Omega, \mathcal{A})$ is therefore complete. A probability measure $\mathbb{P}$ can again be specified by defining $\mathbb{P}(\{\omega\})$ for all $\omega \in \Omega$. For the model of two fair dice, one would choose

$$\mathbb{P}(\{\omega\}) := \frac{1}{|\Omega|} = \frac{1}{36} \text{ for all } \omega \in \Omega. \tag{19.19}$$

Under this probability measure, the probability of the event "The sum of the numbers that appear is four", for example, is obtained with the σ-additivity of $\mathbb{P}$ as

$$\begin{aligned}
\mathbb{P}\left(\{(1,3), (3,1), (2,2)\}\right) &= \mathbb{P}\left(\{(1,3)\} \cup \{(3,1)\} \cup \{(2,2)\}\right) \\
&= \mathbb{P}\left(\{(1,3)\}\right) + \mathbb{P}\left(\{(3,1)\}\right) + \mathbb{P}\left(\{(2,2)\}\right) \\
&= 1/36 + 1/36 + 1/36 \\
&= 1/12.
\end{aligned}$$

Tossing a coin

We model tossing a coin, one side of which shows heads and the other side tails. It is sensible to define the outcome set as $\Omega := \{H, T\}$, where H represents "heads" and T represents "tails". However, any other binary definition of Ω would also be possible, e.g. $\Omega := \{0, 1\}, \Omega := \{-1, 1\}$, or $\Omega := \{1, 2\}$.

The power set $\mathcal{A} := \mathcal{P}(\Omega)$ contains all possible events. In this case, we can easily list the entire set system $\mathcal{A}$, as shown in Table 19.3.

Table 19.3 Event system $\mathcal{A}$ in the model of tossing a coin.

Description	Set form
Neither heads nor tails	$\omega \in A = \emptyset$
Heads appears	$\omega \in A = \{H\}$
Tails appears	$\omega \in A = \{T\}$
Heads or tails appears	$\omega \in A = \{H, T\}$

The definition of the measurable space $(\Omega, \mathcal{A})$ is therefore complete.

A probability measure $\mathbb{P}$ can again be specified by defining $\mathbb{P}(\{\omega\})$ for all $\omega \in \Omega$. The normalization of Ω here implies in particular that

$$\mathbb{P}(\Omega) = 1 \Leftrightarrow \mathbb{P}(\{H\}) + \mathbb{P}(\{T\}) = 1 \Leftrightarrow \mathbb{P}(\{T\}) = 1 - \mathbb{P}(\{H\}). \tag{19.20}$$

Thus, when the probability of the elementary event $\{H\}$ is specified, the probability of the elementary event $\{T\}$ is immediately specified as well, and conversely, of course. For the model of a fair coin, one would choose $\mathbb{P}(\{H\}) = \mathbb{P}(\{T\}) := 1/2$. In this case, the probabilities of all possible events are

$$\mathbb{P}(\emptyset) = 0, \mathbb{P}(\{H\}) = 1/2, \mathbb{P}(\{T\}) = 1/2 \text{ and } \mathbb{P}(\{H, T\}) = 1. \tag{19.21}$$

Tossing a coin twice

We model tossing a coin twice. Based on the model of a single coin toss, it is sensible to define the outcome set as $\Omega := \{HH, HT, TH, TT\}$. The power set $\mathcal{A} := \mathcal{P}(\Omega)$ again contains all $2^{|\Omega|} = 2^4 = 16$ possible events. In the table below, we list four of them.

Table 19.4 Selected events in the model of tossing a coin twice.

Description	Set form
Heads appears on the first toss	$\omega \in A = \{HH, HT\}$
Heads appears on the second toss	$\omega \in A = \{HH, TH\}$
Heads does not appear	$\omega \in A = \{TT\}$
Tails appears at least once	$\omega \in A = \{HT, TH, TT\}$

The definition of the measurable space $(\Omega, \mathcal{A})$ is therefore complete. A probability measure $\mathbb{P}$ can again be specified by defining $\mathbb{P}(\{\omega\})$ for all $\omega \in \Omega$. For the model of tossing a fair coin twice, one would define

$$\mathbb{P}(\{HH\}) = \mathbb{P}(\{HT\}) = \mathbb{P}(\{TH\}) = \mathbb{P}(\{TT\}) := \frac{1}{4}. \tag{19.22}$$

Value of a BDI-II item

We model the value of a patient for the item "Sadness" of the English version of the Beck Depression Inventory II (Hautzinger et al. (2006)),

(0) I do not feel sad.
(1) I feel sad much of the time.
(2) I am sad all the time.

(3) I am so sad or unhappy that I can't stand it.

The random process under consideration is therefore a patient's response to this item. For modeling, the outcome set $\Omega := \{0, 1, 2, 3\}$ is then suitable. For a first patient, for example, the realization is $3 \in \Omega$, for a second patient the realization is $0 \in \Omega$, for a third patient $1 \in \Omega$, and so on. As the event system $\mathcal{A}$, the power set is again suitable. In the present case, it has cardinality

$$|\mathcal{P}(\Omega)| = 2^{|\Omega|} = 2^4 = 16 \tag{19.23}$$

and can easily be written down completely as shown in Table 19.5.

Table 19.5 Selected events in the model of responding to a BDI-II item.

Given response	Set form
None	$\emptyset$
"I do not feel sad"	$\{0\}$
"I feel sad much of the time"	$\{1\}$
"I am sad all the time"	$\{2\}$
"I am so sad or unhappy that I can't stand it"	$\{3\}$
"I do not feel sad" or "I feel sad much of the time"	$\{0, 1\}$
"I do not feel sad" or "I am sad all the time"	$\{0, 2\}$
"I do not feel sad" or "I am so sad or unhappy that I can't stand it"	$\{0, 3\}$
"I feel sad much of the time" or "I am sad all the time"	$\{1, 2\}$
"I feel sad much of the time" or "I am so sad or unhappy that I can't stand it"	$\{1, 3\}$
"I am sad all the time" or "I am so sad or unhappy that I can't stand it"	$\{2, 3\}$
Not "I am so sad or unhappy that I can't stand it"	$\{0, 1, 2\}$
Not "I am sad all the time"	$\{0, 1, 3\}$
Not "I feel sad much of the time"	$\{0, 2, 3\}$
Not "I do not feel sad"	$\{1, 2, 3\}$
Any response option	$\{0, 1, 2, 3\}$

Now suppose that the probabilities of the item responses are known as

$$\mathbb{P}(\{0\}) := \frac{2}{10}, \quad \mathbb{P}(\{1\}) := \frac{3}{10}, \quad \mathbb{P}(\{2\}) := \frac{4}{10}, \quad \mathbb{P}(\{3\}) := \frac{1}{10}. \tag{19.24}$$

Then these probabilities define, in the sense of a probability function,

$$\pi : \Omega \to [0, 1], \omega \mapsto \pi(\omega) =: \mathbb{P}(\{\omega\}) \tag{19.25}$$

a probability measure, and the definition of the probability space model $(\Omega, \mathcal{A}, \mathbb{P})$ for the scenario under consideration is complete. With the help of the σ-additivity of $\mathbb{P}$, for example, the probability of an item response greater than 1 can now be determined:

$$\mathbb{P}(\{2, 3\}) = \mathbb{P}(\{2\} \cup \{3\}) = \mathbb{P}(\{2\}) + \mathbb{P}(\{3\}) = \frac{4}{10} + \frac{1}{10} = \frac{5}{10} = \frac{1}{2} \tag{19.26}$$

Similarly, for example, the probability that a patient gives one of the two item responses "(0) I do not feel sad" or "(3) I am so sad or unhappy that I can't stand it" is

$$\mathbb{P}(\{0, 3\}) = \mathbb{P}(\{0\} \cup \{3\}) = \mathbb{P}(\{0\}) + \mathbb{P}(\{3\}) = \frac{2}{10} + \frac{1}{10} = \frac{3}{10}. \tag{19.27}$$

19.4 Literature notes

Andrey Kolmogorov's monograph "Grundbegriffe der Wahrscheinlichkeitsrechnung" (Kolmogoroff (1933)) symbolizes the foundation of modern probability calculus. In addition to the axiomatic introduction of the probability space model discussed here, Kolmogoroff (1933) considers many other aspects of probability calculus and thus offers a readable introduction to the whole field of probability theory. Of course, the approach formulated by Kolmogoroff (1933) is only a provisional end product of the long historical development of probability theory. After all, the development of the mathematical modeling of random processes by no means came to an end with Kolmogoroff (1933). Later works in the 20th century concerned in particular the interpretation of the concept of probability (cf. De Finetti (1975)) or introduced generalized quantitative measures of subjective uncertainty (cf. Walley (1991)). A current overview of the interpretation of the concept of probability and its formal foundations is given by Hájek (2019).

Self-control questions

1. Explain the concept of a random process.
2. State the definition of the concept of a σ-algebra.
3. State the definition of the concept of a probability measure.
4. State the definition of the concept of a probability space.
5. Explain the concept of the outcome set Ω.
6. Explain the tacit mechanics of the probability space model.
7. Explain the concept of an event $A \in \mathcal{A}$.
8. Explain the concept of the event system $\mathcal{A}$.
9. Which σ-algebra is sensibly chosen for a probability space with finite outcome set?
10. Explain the concept of the probability measure $\mathbb{P}$.
11. State the definition of the concept of a probability function.
12. Why is the concept of a probability function helpful when modeling a random process by a probability space with finite outcome set?
13. Explain the modeling of throwing a die using a probability space.
14. Explain the modeling of the simultaneous throwing of a red and a blue die using a probability space.

Solutions

1. A random process is a phenomenon of reality that cannot be predicted with absolute certainty or is associated with uncertainty for humans. See the introduction to probability theory.
2. See Definition 19.1 regarding $\mathcal{A}$.
3. See Definition 19.1 regarding $\mathbb{P}$.
4. See Definition 19.1.
5. The outcome set contains the elementary events possible in a run of the modeled random process; see the explanations of **Outcome set and mechanics**.
6. See the explanations of **Outcome set and mechanics**.
7. See the explanations of **Events and event system**.
8. See the explanations of **Events and event system**.
9. The power set of the outcome set; it contains all events that are possible in principle; see also Theorem 19.1.
10. See the explanations of **Probability measure** $\mathbb{P}$.
11. See Definition 19.2.
12. By defining the function values of a probability function, a probability measure can be specified; see also Section 19.3.
13. See the explanations of **Throwing one die**.
14. See the explanations of **Simultaneous throwing of a blue and a red die**.

20 Elementary probabilities

In this section, with the concepts of the *joint probability* of two events and the *conditional probability* of an event, we introduce two elementary forms of probabilities. Intuitively, the concept of joint probability refers to the probability of the "simultaneous" occurrence of two events A and B, and the concept of conditional probability refers to the probability of the occurrence of an event A "when one knows about the occurrence of another event B". If it is irrelevant for the probability of an event A whether or not an event B has occurred, then A and B are called *independent events*. Intuitively, independent events model the absence of mutual influences.

20.1 Joint probabilities

The concept of the joint probability of two events is defined as follows.

Definition 20.1 (Joint probability)**.** Let $(\Omega, \mathcal{A}, \mathbb{P})$ be a probability space and let $A, B \in \mathcal{A}$. Then

$$\mathbb{P}(A \cap B) \tag{20.1}$$

is called the *joint probability of A and B*.

•

As noted above, $\mathbb{P}(A \cap B)$ corresponds to the probability that the events A and B occur "simultaneously". This is best understood against the background of the mechanics of the probability space model. According to this mechanics, $\mathbb{P}(A \cap B)$ is precisely the probability that, in one run of a random process, an ω is realized for which both $\omega \in A$ and $\omega \in B$ hold.

Example

As a first example of a joint probability of two events, we again consider the probability space model of throwing a fair die. Let A be the event "An even number of pips appears", that is, $A := \{2, 4, 6\}$, and let B be the event "A number of pips greater than three appears", that is, $B := \{4, 5, 6\}$. Set-theoretically, we then have

$$A \cap B = \{2, 4, 6\} \cap \{4, 5, 6\} = \{4, 6\}. \tag{20.2}$$

The interpretation of $A \cap B = \{4, 6\}$ is exactly "An even number of pips appears *and* this number of pips is greater than three". Assuming fairness of the die, that is, for

$\mathbb{P}(\{4\}) = \mathbb{P}(\{6\}) := 1/6$, we can easily calculate the probability of this event using the σ-additivity of $\mathbb{P}$. We obtain

$$\begin{aligned}\mathbb{P}(A \cap B) &= \mathbb{P}(\{2,4,6\} \cap \{4,5,6\}) \\ &= \mathbb{P}(\{4,6\}) \\ &= \mathbb{P}(\{4\}) + \mathbb{P}(\{6\}) \\ &= \frac{1}{6} + \frac{1}{6} \\ &= \frac{1}{3}.\end{aligned} \tag{20.3}$$

When thinking about joint probabilities, it is of course important not to confuse the joint probability $\mathbb{P}(A \cap B)$ with the probability $\mathbb{P}(A \cup B)$ of the event $A \cup B$. Recall that the union of two sets $\cup$ corresponds to the *inclusive or*, that is, to an *and/or* (cf. Section 1.3 and Section 2.2). The event $A \cup B$ therefore corresponds to the event that event A and/or event B occurs. In particular, $\omega \in A \cup B$ is already satisfied if for the outcome of one run of a random process *only* $\omega \in A$ or *only* $\omega \in B$ holds. Concretely, for the events $A := \{2,4,6\}$ and $B := \{4,5,6\}$ from the die example above, we obtain

$$A \cup B = \{2,4,6\} \cup \{4,5,6\} = \{2,4,5,6\} \tag{20.4}$$

with the interpretation "An even number of pips appears and/or a number of pips greater than three appears". For the corresponding probability we obtain

$$\mathbb{P}(\{2,4,5,6\}) = \frac{2}{3}, \tag{20.5}$$

so in this case apparently $\mathbb{P}(A \cap B) \neq \mathbb{P}(A \cup B)$.

With the help of the following theorem, in this section we finally record a few useful properties for calculating with probabilities, which follow directly from the connection between set operations and the σ-additivity of probability measures. We visualize the corresponding statements in Figure 20.1 using Venn diagrams.

Theorem 20.1 (Further properties of probabilities)**.** *Let $(\Omega, \mathcal{A}, \mathbb{P})$ be a probability space and let $A, B \in \mathcal{A}$ be events. Then*

1. $\mathbb{P}(A^c) = 1 - \mathbb{P}(A)$.
2. $A \subset B \Rightarrow \mathbb{P}(A) \leq \mathbb{P}(B)$.
3. $\mathbb{P}(A \cap B^c) = \mathbb{P}(A) - \mathbb{P}(A \cap B)$
4. $\mathbb{P}(A \cup B) = \mathbb{P}(A) + \mathbb{P}(B) - \mathbb{P}(A \cap B)$.

∘

Proof. The second, third, and fourth statements of this theorem are based on elementary set-theoretic statements and the σ-additivity of $\mathbb{P}$. We do not prove these elementary set-theoretic statements here, but instead refer in each case to the intuition conveyed by the Venn diagrams in Figure 20.2.

For 1.: We first note that from $A^c := \Omega \setminus A$ it follows that $A^c \cup A = \Omega$ and that $A^c \cap A = \emptyset$. With normalization and σ-additivity of $\mathbb{P}$ it then follows that

$$\mathbb{P}(\Omega) = 1 \Leftrightarrow \mathbb{P}(A^c \cup A) = 1 \Leftrightarrow \mathbb{P}(A^c) + \mathbb{P}(A) = 1 \Leftrightarrow \mathbb{P}(A^c) = 1 - \mathbb{P}(A). \tag{20.6}$$

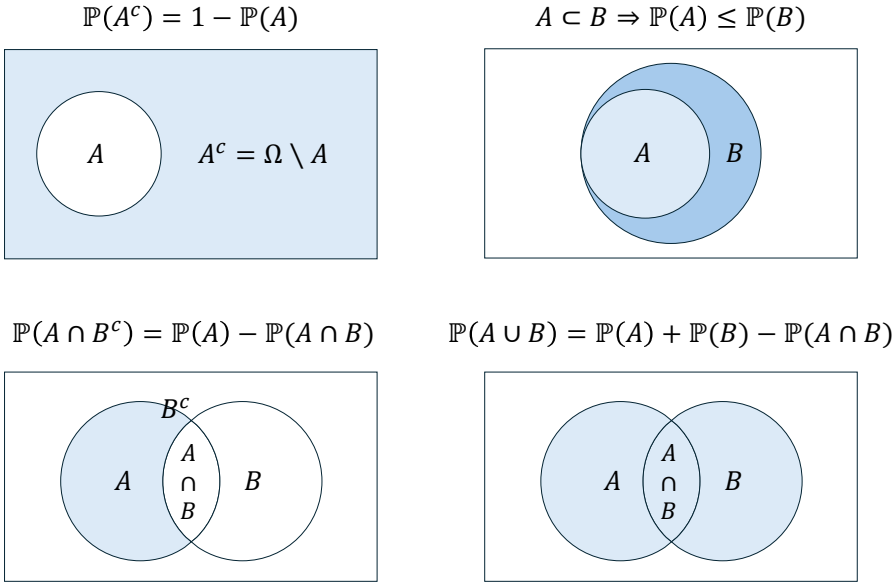


Figure 20.1 Venn diagrams for Theorem 20.1. The light-blue areas correspond to the probabilities of interest.

For 2.: First, we have (cf. Figure A)

$$A \subset B \Rightarrow B = A \cup (B \cap A^c) \text{ with } A \cap (B \cap A^c) = \emptyset. \tag{20.7}$$

With the σ-additivity of $\mathbb{P}$ it follows that

$$\mathbb{P}(B) = \mathbb{P}(A) + \mathbb{P}(B \cap A^c). \tag{20.8}$$

With $\mathbb{P}(B \cap A^c) \geq 0$ it follows that $\mathbb{P}(A) \leq \mathbb{P}(B)$.

For 3.: First, we have (cf. Figure B)

$$(A \cap B) \cap (A \cap B^c) = \emptyset \text{ and } A = (A \cap B) \cup (A \cap B^c). \tag{20.9}$$

With the σ-additivity of $\mathbb{P}$ it follows that

$$\mathbb{P}(A) = \mathbb{P}(A \cap B) + \mathbb{P}(A \cap B^c) \Leftrightarrow \mathbb{P}(A \cap B) = \mathbb{P}(A) - \mathbb{P}(A \cap B^c). \tag{20.10}$$

For 4.: First, we have (cf. Figure C)

$$B \cap (A \cap B^c) = \emptyset \text{ and } A \cup B = B \cup (A \cap B^c). \tag{20.11}$$

With the σ-additivity of $\mathbb{P}$ it follows that

$$\mathbb{P}(A \cup B) = \mathbb{P}(B) + \mathbb{P}(A \cap B^c). \tag{20.12}$$

With 3. it then follows that

$$\mathbb{P}(A \cup B) = \mathbb{P}(B) + \mathbb{P}(A) - \mathbb{P}(A \cap B). \tag{20.13}$$

□

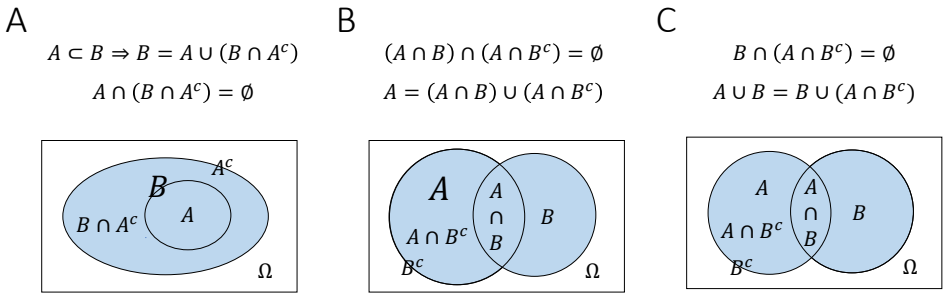


Figure 20.2 Venn diagrams for the proof of Theorem 20.1

20.2 Conditional probabilities

We now turn to the concept of conditional probability.

Definition 20.2 (Conditional probability)**.** Let $(\Omega, \mathcal{A}, \mathbb{P})$ be a probability space and let $A, B \in \mathcal{A}$ be events with $\mathbb{P}(B) > 0$. The *conditional probability of event A given event B* is defined as

$$\mathbb{P}(A|B) := \frac{\mathbb{P}(A \cap B)}{\mathbb{P}(B)}. \tag{20.14}$$

•

We note: The conditional probability $\mathbb{P}(A|B)$ of an event A given an event B is the joint probability $\mathbb{P}(A \cap B)$ of the events A and B, scaled by $1/\mathbb{P}(B)$. Thus, if in the formulation of a probabilistic model one specifies the joint probability $\mathbb{P}(A \cap B)$ as well as the probability $\mathbb{P}(B) > 0$ of the event B, then in particular one also specifies the conditional probability $\mathbb{P}(A|B)$ of event A given event B. We further point out that there is no reason to confuse the conditional probabilities

$$\mathbb{P}(A|B) = \frac{\mathbb{P}(A \cap B)}{\mathbb{P}(B)} \text{ and } \mathbb{P}(B|A) = \frac{\mathbb{P}(B \cap A)}{\mathbb{P}(A)} = \frac{\mathbb{P}(A \cap B)}{\mathbb{P}(A)} \tag{20.15}$$

(cf. Herzog & Ostwald (2013)). In particular, from $\mathbb{P}(A) \neq \mathbb{P}(B)$ it always follows directly that $\mathbb{P}(A|B) \neq \mathbb{P}(B|A)$. Finally, note that a generalization of the conditional probability in Definition 20.2 to the case $\mathbb{P}(B) = 0$ is possible, but technically involved. For this we refer to the advanced literature, e.g. Meintrup & Schäffler (2005) and Schmidt (2009).

Examples

Throwing one die

As a first example of a conditional probability, we again consider the model $(\Omega, \mathcal{A}, \mathbb{P})$ of the fair die. We want to calculate the conditional probability of the event "An even number of pips appears" given the event "A number greater than three appears". We have already seen above that the event "An even number of pips appears" corresponds to the subset $A := \{2, 4, 6\}$ of Ω, and that the event "A number greater than three appears" corresponds

to the subset $B := \{4, 5, 6\}$ of Ω. Furthermore, we have seen that, under the assumption that the modeled die is fair,

$$\mathbb{P}(\{2, 4, 6\}) = \mathbb{P}(\{2\}) + \mathbb{P}(\{4\}) + \mathbb{P}(\{6\}) = \frac{1}{6} + \frac{1}{6} + \frac{1}{6} = \frac{3}{6} \tag{20.16}$$

and that

$$\mathbb{P}(\{4, 5, 6\}) = \mathbb{P}(\{4\}) + \mathbb{P}(\{5\}) + \mathbb{P}(\{6\}) = \frac{1}{6} + \frac{1}{6} + \frac{1}{6} = \frac{3}{6}. \tag{20.17}$$

Finally, we had also seen that the event $A \cap B$, that is, the event "An even number of pips greater than three appears", has probability

$$\mathbb{P}(A \cap B) = \mathbb{P}(\{2, 4, 6\} \cap \{4, 5, 6\}) = \mathbb{P}(\{4, 6\}) = \mathbb{P}(\{4\}) + \mathbb{P}(\{6\}) = \frac{1}{6} + \frac{1}{6} = \frac{2}{6} \tag{20.18}$$

By the definition of conditional probability, we then directly obtain

$$\mathbb{P}(A|B) = \frac{\mathbb{P}(A \cap B)}{\mathbb{P}(B)} = \frac{\mathbb{P}(\{4, 6\})}{\mathbb{P}(\{4, 5, 6\})} = \frac{2}{6} \cdot \frac{6}{3} = \frac{2}{3}. \tag{20.19}$$

In this context, an interpretation of the conditional probability $\mathbb{P}(A|B)$ as a decrease in subjective uncertainty or as a gain in subjective information relative to the unconditional probability $\mathbb{P}(A)$ suggests itself: if one knows that a number of pips greater than three has appeared, that is, that the event $\omega \in B$ is present, then the probability that ω is an even number of pips is 2/3. If, by contrast, one does not know that the event $\omega \in B$ is present (and has no other information about ω), then the probability of an even number of pips appearing is only 1/2. Conditioning on the presence of an event therefore corresponds to including information and thus to the decrease of uncertainty in probability-theoretic models. This is the basis of Bayesian statistics. A similar interpretation is also suggested by the following, perhaps more everyday, example.

Value of a BDI-II item

We again consider the example of the value of a patient for the item "Sadness" of the English version of the BDI-II. Above, we assigned the probabilities

$$\mathbb{P}(\{0\}) := \frac{2}{10}, \quad \mathbb{P}(\{1\}) := \frac{3}{10}, \quad \mathbb{P}(\{2\}) := \frac{4}{10}, \quad \mathbb{P}(\{3\}) := \frac{1}{10}, \tag{20.20}$$

to the possible item responses (0), (1), (2), (3). We now assume that we know of a patient that she chose a value greater than (1), and ask for the probabilities that the patient chose value (2) or (3). To this end, we first consider the probability of the event "The patient chose a value greater than (1)". This obviously corresponds to the subset $B := \{2, 3\}$ of Ω. With σ-additivity, as seen above, the probability of this event is $\mathbb{P}(\{2, 3\}) = 1/2$. We therefore consider the events listed in Table 20.1.

Table 20.1 Events when answering a BDI-II item

Description	Set form
The patient chose a value greater than (1)	$\omega \in B := \{2, 3\}$
The patient chose response (2)	$\omega \in A_1 := \{2\}$
The patient chose response (3)	$\omega \in A_2 := \{3\}$

We obtain

$$\mathbb{P}(A_1|B) = \frac{\mathbb{P}(A_1 \cap B)}{\mathbb{P}(B)} = \frac{\mathbb{P}(\{2\} \cap \{2,3\})}{\mathbb{P}(\{2,3\})} = \frac{\mathbb{P}(\{2\})}{\mathbb{P}(\{2,3\})} = \frac{4}{10} \cdot \frac{2}{1} = \frac{8}{10} \tag{20.21}$$

and

$$\mathbb{P}(A_2|B) = \frac{\mathbb{P}(A_2 \cap B)}{\mathbb{P}(B)} = \frac{\mathbb{P}(\{3\} \cap \{2,3\})}{\mathbb{P}(\{2,3\})} = \frac{\mathbb{P}(\{3\})}{\mathbb{P}(\{2,3\})} = \frac{1}{10} \cdot \frac{2}{1} = \frac{2}{10}. \tag{20.22}$$

Note that

$$\mathbb{P}(A_1|B) + \mathbb{P}(A_2|B) = \frac{8}{10} + \frac{2}{10} = 1. \tag{20.23}$$

Based on the definition of the conditional probability of an event under the condition of the occurrence of a fixed event B, one can define a probability measure that gives the probabilities of *all* events of the event system under the condition of the occurrence of a fixed event B. This probability measure is called the *conditional probability given event* B.

Theorem 20.2 (Conditional probability)**.** *For a fixed $B \in \mathcal{A}$ with $\mathbb{P}(B) > 0$, let*

$$\mathbb{P}(\cdot|B) : \mathcal{A} \to [0,1], A \mapsto \mathbb{P}(A|B) := \frac{\mathbb{P}(A \cap B)}{\mathbb{P}(B)} \tag{20.24}$$

Then $\mathbb{P}(\cdot|B)$ is a probability measure and is called the conditional probability given event *B.*

∘

Proof. We verify the defining properties of a probability measure.

(1) $\mathbb{P}(A|B) \geq 0$ for all $A \in \mathcal{A}$

By the definition of conditional probability, $\mathbb{P}(A|B) = \mathbb{P}(A \cap B)/\mathbb{P}(B)$ with $\mathbb{P}(B) > 0$. With the theorem on closure of σ-algebras under intersections and the definition of $\mathbb{P}$, $\mathbb{P}(A \cap B) \geq 0$ holds, and hence the statement follows.

(2) $\mathbb{P}(\Omega|B) = 1$.

We have

$$\mathbb{P}(\Omega|B) = \frac{\mathbb{P}(\Omega \cap B)}{\mathbb{P}(B)} = \frac{\mathbb{P}(B)}{\mathbb{P}(B)} = 1. \tag{20.25}$$

(3) $\mathbb{P}\left(\cup_{i=1}^{\infty} A_i|B\right) = \sum_{i=1}^{\infty} \mathbb{P}(A_i|B)$ for pairwise disjoint $A_1, A_2, \ldots \in \mathcal{A}$.

With the associative and distributive laws of union and intersection, first

$$\left(\cup_{i=1}^{\infty} A_i\right) \cap B = (A_1 \cup A_2 \cup \cdots) \cap B = (A_1 \cap B) \cup (A_2 \cap B) \cup \cdots = \cup_{i=1}^{\infty} \left(A_i \cap B\right). \tag{20.26}$$

Furthermore, with $A_i \cap A_j = \emptyset$ and the associative and distributive laws of union and intersection, the $A_i \cap B$ for $i = 1, 2, \ldots$ are also pairwise disjoint, because

$$(A_i \cap B) \cap \left(A_j \cap B\right) = \left(A_i \cap A_j\right) \cap (B \cap B) = \emptyset \cap B = \emptyset. \tag{20.27}$$

With the σ-additivity of $\mathbb{P}$ it then follows that

$$\begin{aligned}
\mathbb{P}\left(\cup_{i=1}^{\infty} A_i | B\right) &= \frac{\mathbb{P}\left(\left(\cup_{i=1}^{\infty} A_i\right) \cap B\right)}{\mathbb{P}(B)} \\
&= \frac{\mathbb{P}\left(\cup_{i=1}^{\infty} \left(A_i \cap B\right)\right)}{\mathbb{P}(B)} \\
&= \frac{\sum_{i=1}^{\infty} \mathbb{P}\left(A_i \cap B\right)}{\mathbb{P}(B)} \\
&= \sum_{i=1}^{\infty} \frac{\mathbb{P}\left(A_i \cap B\right)}{\mathbb{P}(B)} \\
&= \sum_{i=1}^{\infty} \mathbb{P}(A_i | B).
\end{aligned} \tag{20.28}$$

□

Note that for the conditional probability $\mathbb{P}(\cdot|B)$, the calculation rules of probability theory apply to the events to the left of the bar. In particular, in contrast to $\mathbb{P}(\cdot \cap B)$, $\mathbb{P}(\cdot|B)$ defines a probability measure for all $A \in \mathcal{A}$. Thus, for example, for $0 < \mathbb{P}(B) < 1$, $\mathbb{P}(\Omega|B) = 1$, but $\mathbb{P}(\Omega \cap B) = \mathbb{P}(B) < 1$. One therefore also says that $\mathbb{P}(\cdot|B)$ is *normalized*, whereas $\mathbb{P}(\cdot \cap B)$ is not.

At the end of this section, we consider three technical consequences of the definition of conditional probability, which we formulate as theorems.

The first theorem concerns the relation between joint and conditional probabilities and reiterates how joint probabilities can be calculated from conditional and total probabilities.

Theorem 20.3 (Joint and conditional probabilities)**.** *Let $(\Omega, \mathcal{A}, \mathbb{P})$ be a probability space and let $A, B \in \mathcal{A}$ with $\mathbb{P}(A) > 0$ and $P(B) > 0$. Then*

$$\mathbb{P}(A \cap B) = \mathbb{P}(A|B)\mathbb{P}(B) = \mathbb{P}(B|A)\mathbb{P}(A). \tag{20.29}$$

∘

Proof. With the definition of the respective conditional probability, it follows directly that

$$\mathbb{P}(A|B) = \frac{\mathbb{P}(A \cap B)}{\mathbb{P}(B)} \Leftrightarrow \mathbb{P}(A \cap B) = \mathbb{P}(A|B)\mathbb{P}(B) \tag{20.30}$$

and

$$\mathbb{P}(B|A) = \frac{\mathbb{P}(B \cap A)}{\mathbb{P}(A)} \Leftrightarrow \mathbb{P}(A \cap B) = \mathbb{P}(B|A)\mathbb{P}(A). \tag{20.31}$$

□

Just as specifying $\mathbb{P}(A \cap B)$ and $\mathbb{P}(A)$ specifies the conditional probability $\mathbb{P}(B|A)$, specifying $\mathbb{P}(A)$ and $\mathbb{P}(B|A)$ therefore specifies the joint probability $\mathbb{P}(A \cap B)$.

The following so-called *law of total probability* states how unconditional, so-called *total probabilities* can be calculated based on joint probabilities.

Theorem 20.4 (Law of total probability)**.** *Let $(\Omega, \mathcal{A}, \mathbb{P})$ be a probability space and let $A_1, ..., A_k$ with $\mathbb{P}(A_i) > 0$ be a partition of Ω. Then for every $B \in \mathcal{A}$,*

$$\mathbb{P}(B) = \sum_{i=1}^{k} \mathbb{P}(B \cap A_i) = \sum_{i=1}^{k} \mathbb{P}(B|A_i)\mathbb{P}(A_i). \tag{20.32}$$

∘

Proof. For $i = 1, ..., k$, let $C_i := B \cap A_i$, so that $\cup_{i=1}^{k} C_i = B$ and $C_i \cap C_j = \emptyset$ for $1 \leq i, j \leq k, i \neq j$. We illustrate these definitions in Figure 20.3 using a Venn diagram.

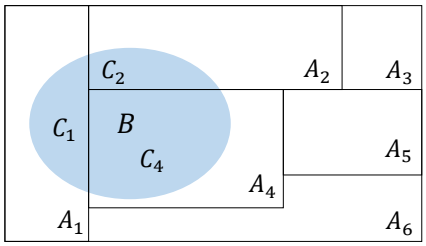


Figure 20.3 Venn diagram for the proof of Theorem 20.4.

Thus, with the definition of conditional probability for $\mathbb{P}(A_i) > 0$, we have

$$\mathbb{P}(B) = \sum_{i=1}^{k} \mathbb{P}(C_i) = \sum_{i=1}^{k} \mathbb{P}(B \cap A_i) = \sum_{i=1}^{k} \mathbb{P}(B|A_i)\mathbb{P}(A_i). \tag{20.33}$$

□

Intuitively, $\mathbb{P}(B)$ therefore corresponds to the weighted sum of the conditional probabilities $\mathbb{P}(B|A_i)$, where the weighting factors are precisely the unconditional probabilities $\mathbb{P}(A_i)$ for $i = 1, .., k$. The requirement $\mathbb{P}(A_i) > 0$ for all $i = 1, ..., k$ is necessary here to guarantee the representation of $\mathbb{P}(B \cap A_i)$ using Definition 20.2.

Example

As an example of representing the probability of an event using the law of total probability, we consider the probability space model of the fair die. For the outcome space $\Omega := \{1, 2, 3, 4, 5, 6\}$, we consider the event $B := \{2, 4, 6\}$ ("An even number appears") with probability $\mathbb{P}(B) = \frac{1}{2}$ as well as the partition $A_1 := \{1, 2\}$, $A_2 := \{3\}$, $A_3 := \{4, 5\}$, $A_4 := \{6\}$ of Ω. Then apparently

$$\begin{array}{rcrcrcl} \mathbb{P}(B \cap A_1) & = & \mathbb{P}(\{2,4,6\} \cap \{1,2\}) & = & \mathbb{P}(\{2\}) & = & \frac{1}{6} \\ \mathbb{P}(B \cap A_2) & = & \mathbb{P}(\{2,4,6\} \cap \{3\}) & = & \mathbb{P}(\emptyset) & = & 0 \\ \mathbb{P}(B \cap A_3) & = & \mathbb{P}(\{2,4,6\} \cap \{4,5\}) & = & \mathbb{P}(\{4\}) & = & \frac{1}{6} \\ \mathbb{P}(B \cap A_4) & = & \mathbb{P}(\{2,4,6\} \cap \{6\}) & = & \mathbb{P}(\{6\}) & = & \frac{1}{6} \end{array} \tag{20.34}$$

Thus we obtain

$$\begin{aligned} \sum_{i=1}^{4} \mathbb{P}(B \cap A_i) &= \mathbb{P}(B \cap A_1) + \mathbb{P}(B \cap A_2) + \mathbb{P}(B \cap A_3) + \mathbb{P}(B \cap A_4) \\ &= \frac{1}{6} + 0 + \frac{1}{6} + \frac{1}{6} \\ &= \frac{1}{2} \end{aligned} \tag{20.35}$$

and hence

$$\mathbb{P}(B) = \sum_{i=1}^{4} \mathbb{P}(B \cap A_i). \tag{20.36}$$

Note that the partition property of A_1, A_2, A_3, A_4 is essential for the validity of the theorem in this example. If, for example, $A_4 = \{4, 6\}$ and thus A_3 and A_4 were not disjoint, then the right-hand side of the law of total probability would yield a value greater than $\mathbb{P}(B)$. If, by contrast, the union of A_1, A_2, A_3, A_4 did not yield Ω, for example because $A_4 = \emptyset$, then conversely a value smaller than $\mathbb{P}(B)$ would result.

Finally, with *Bayes' theorem*, we consider a formula for the alternative calculation of conditional probabilities.

Theorem 20.5 (Bayes' theorem)**.** *Let $(\Omega, \mathcal{A}, \mathbb{P})$ be a probability space and let $A_1, ..., A_k$ be a partition of Ω with $\mathbb{P}(A_i) > 0$ for all $i = 1, ..., k$. If $\mathbb{P}(B) > 0$, then for every $i = 1, ..., k$,*

$$\mathbb{P}(A_i|B) = \frac{\mathbb{P}(B|A_i)\mathbb{P}(A_i)}{\sum_{i=1}^{k} \mathbb{P}(B|A_i)\mathbb{P}(A_i)}. \tag{20.37}$$

∘

Proof. With the definition of conditional probability and the law of total probability,

$$\mathbb{P}(A_i|B) = \frac{\mathbb{P}(A_i \cap B)}{\mathbb{P}(B)} = \frac{\mathbb{P}(B|A_i)\mathbb{P}(A_i)}{\mathbb{P}(B)} = \frac{\mathbb{P}(B|A_i)\mathbb{P}(A_i)}{\sum_{i=1}^{k} \mathbb{P}(B|A_i)\mathbb{P}(A_i)}. \tag{20.38}$$

□

Note that Bayes' theorem is independent of the frequentist or Bayesian interpretation of probability and merely makes a statement about calculating with conditional probabilities. In frequentist inference, however, Bayes' theorem is used rather rarely. In Bayesian inference, by contrast, Bayes' theorem is central. In this context, $\mathbb{P}(A_i)$ is then often called the *prior probability of event* A_i and $\mathbb{P}(A_i|B)$ the *posterior probability of event* A_i. As explained above, $\mathbb{P}(A_i|B)$ corresponds to the probability of A_i when one knows about the occurrence of B.

In applications, the results formulated here as the *law of total probability* (Theorem 20.4), the *theorem on joint and conditional probabilities* (Theorem 20.3), and *Bayes' theorem* (Theorem 20.5) are often used as the central calculation rules of probability theory. In practice, they are usually called the *summation rule*, *multiplication rule*, and *Bayes rule* (cf. Table 20.2).

Table 20.2 Central calculation rules of probability theory

Calculation rule	Probability space form
Multiplication rule	$\mathbb{P}(A \cap B) = \mathbb{P}(B\|A)\,\mathbb{P}(A)$
Summation rule	$\mathbb{P}(B) = \sum_{i=1}^{k} \mathbb{P}(B \cap A_i)$, if $\Omega = \cup_{i=1}^{k} A_i$ and $A_i \cap A_j = \emptyset$
Bayes rule	$\mathbb{P}(A\|B) = \frac{\mathbb{P}(B\|A)\mathbb{P}(A)}{\mathbb{P}(B)}$, if $\mathbb{P}(B) > 0$

20.3 Independent events

The independence of events serves to model the absence of mutual influences of events. Its definition states that the joint probability of two events should result from the product of the probabilities of the individual events. In this context, one also speaks of the *factorization of the joint probability* of the events. The meaning of this definition becomes clear in light of the concept of conditional probability in Theorem 20.6. We first consider the definition.

Definition 20.3 (Independent events)**.** Two events $A \in \mathcal{A}$ and $B \in \mathcal{A}$ are called *independent* if

$$\mathbb{P}(A \cap B) = \mathbb{P}(A)\mathbb{P}(B). \tag{20.39}$$

A set of events $\{A_i | i \in I\} \subset \mathcal{A}$ with arbitrary index set I is called *independent* if for every finite subset $J \subseteq I$,

$$\mathbb{P}\left(\cap_{j \in J} A_j\right) = \prod_{j \in J} \mathbb{P}(A_j). \tag{20.40}$$

•

Note that the independence of certain events can be assumed in the definition of a probabilistic model or can also follow from the definition of a probabilistic model. If two events are not independent, one also says that these events are *dependent*. The meaning of the product property under independence becomes clear, as we show below, in the context of conditional probabilities. First, we give an example that illustrates the relationship between independence and pairwise independence of events.

Example

Independent events are by definition always also pairwise independent. For example, for the set $\{A_i | i \in \{1, 2, 3\}\}$ of events and the possible subsets $J_1 := \{1, 2\}$, $J_2 := \{1, 3\}$, $J_3 := \{2, 3\}$, $J := \{1, 2, 3\}$ of $I = \{1, 2, 3\}$, the defining condition of independence of A_1, A_2, A_3 implies the statements

$$\begin{aligned} \mathbb{P}(A_1 \cap A_2) &= \mathbb{P}(A_1)\,\mathbb{P}(A_2) \\ \mathbb{P}(A_1 \cap A_3) &= \mathbb{P}(A_1)\,\mathbb{P}(A_3) \\ \mathbb{P}(A_2 \cap A_3) &= \mathbb{P}(A_2)\,\mathbb{P}(A_3) \\ \mathbb{P}(A_1 \cap A_2 \cap A_3) &= \mathbb{P}(A_1)\,\mathbb{P}(A_2)\,\mathbb{P}(A_2) \end{aligned} \tag{20.41}$$

Conversely, pairwise independence of events does not imply independence of events, as the following example shows (cf. DeGroot & Schervish (2012)). Consider the model of tossing a fair coin twice with outcome space

$$\Omega := \{HH, HT, TH, TT\} \tag{20.42}$$

with elementary event probabilities

$$\mathbb{P}\{HH\} = \mathbb{P}\{HT\} = \mathbb{P}\{TH\} = \mathbb{P}\{TT\} = \frac{1}{4} \tag{20.43}$$

Then the events listed in Table 20.3 are pairwise independent, but not independent.

Table 20.3 Coin-toss example for independence

Verbal description	Set form
Heads appears on the first toss	$A_1 := \{HH, HT\}$
Heads appears on the second toss	$A_2 := \{HH, TH\}$
The same outcome appears on both tosses	$A_3 := \{HH, TT\}$

Here, apparently,

$$\mathbb{P}(A_1) = \mathbb{P}(A_2) = \mathbb{P}(A_2) = \frac{1}{2}, \tag{20.44}$$

and

$$\begin{aligned}
\mathbb{P}(A_1 \cap A_2) = \mathbb{P}(\{HH\}) = \frac{1}{4} = \frac{1}{2} \cdot \frac{1}{2} = \mathbb{P}(A_1)\,\mathbb{P}(A_2)\,, \\
\mathbb{P}(A_1 \cap A_3) = \mathbb{P}(\{HH\}) = \frac{1}{4} = \frac{1}{2} \cdot \frac{1}{2} = \mathbb{P}(A_2)\,\mathbb{P}(A_3)\,, \\
\mathbb{P}(A_2 \cap A_3) = \mathbb{P}(\{HH\}) = \frac{1}{4} = \frac{1}{2} \cdot \frac{1}{2} = \mathbb{P}(A_1)\,\mathbb{P}(A_3)\,.
\end{aligned} \tag{20.45}$$

The events are therefore pairwise independent. However, it is also true that

$$\mathbb{P}(A_1 \cap A_2 \cap A_3) = \mathbb{P}(\{HH\}) = \frac{1}{4} \neq \frac{1}{8} = \frac{1}{2} \cdot \frac{1}{2} \cdot \frac{1}{2} = \mathbb{P}(A_1)\,\mathbb{P}(A_2)\,\mathbb{P}(A_3) \tag{20.46}$$

and the events A_1, A_2, A_3 are therefore not independent.

Finally, with the relation between independence and conditional probability, we explain the meaning of the product property under independence.

Theorem 20.6 (Conditional probability under independence)**.** *Let $(\Omega, \mathcal{A}, \mathbb{P})$ be a probability space and let $A, B \in \mathcal{A}$ be independent events with $\mathbb{P}(B) > 0$. Then*

$$\mathbb{P}(A|B) = \mathbb{P}(A). \tag{20.47}$$

∘

Proof. Under the assumptions of the theorem,

$$\mathbb{P}(A|B) = \frac{\mathbb{P}(A \cap B)}{\mathbb{P}(B)} = \frac{\mathbb{P}(A)\mathbb{P}(B)}{\mathbb{P}(B)} = \mathbb{P}(A). \tag{20.48}$$

□

Given independence of two events A and B, it is therefore irrelevant for the probability of event A whether B also occurs or not; the probability $\mathbb{P}(A)$ remains the same. Independence of events is thus modeled precisely as factorization of the joint probability of A and B so that $\mathbb{P}(A|B) = \mathbb{P}(A)$ follows. From the perspective of modeling subjective uncertainty through probabilities, the independence of two events means that knowing about the presence of one of the two events does not change the probability of the presence of the other event. Conversely, dependence of two events means that knowing about the presence of one of the two events changes the probability of the presence of the other event, that is, either increases or decreases it.

Example

We want to illustrate the relation between independence, conditional probability, and factorization of the joint probability using the model of the fair die. To this end, we consider the event $A := \{2, 4, 6\}$ ("An even number appears") and the event $B := \{3, 4, 5, 6\}$ ("A number greater than two appears"). Intuitively, the probability that an even number has appeared does not change if one knows that a number greater than two has appeared: if one has the information that a number greater than two has appeared, then three of six cases correspond to the event of an even number appearing; if one has the information, then two of four cases correspond to the event of an even number appearing. The relative proportion of cases corresponding to the event "An even number appears" therefore remains the same in view of the absence or presence of the information "A number greater than two appears".

Formally, we first establish the factorization of the joint probability of the two events, that is, of the event "An even number appears and this number is greater than two", into the product of the probabilities of the two events. Here, with the σ-additivity of $\mathbb{P}$,

$$\begin{aligned}
\mathbb{P}(A \cap B) &= \mathbb{P}\left(\{2, 4, 6\} \cap \{3, 4, 5, 6\}\right) \\
&= \mathbb{P}\left(\{4, 6\}\right) \\
&= \frac{2}{6} \\
&= \frac{1}{2} \cdot \frac{2}{3} \\
&= \mathbb{P}(\{2, 4, 6\}) \cdot \mathbb{P}(\{3, 4, 5, 6\}) \\
&= \mathbb{P}(A)\mathbb{P}(B)
\end{aligned} \tag{20.49}$$

Thus the joint probability of A and B can be written as the product of the probabilities of A and B. For the conditional probability of A given B, we further have

$$\begin{aligned}
\mathbb{P}(A|B) &= \frac{\mathbb{P}(A \cap B)}{\mathbb{P}(B)} \\
&= \frac{\mathbb{P}\left(\{4, 6\}\right)}{\mathbb{P}(\{3, 4, 5, 6\})} \\
&= \frac{1}{3} \cdot \frac{3}{2} \\
&= \frac{1}{2} \\
&= \mathbb{P}(\{2, 4, 6\}) \\
&= \mathbb{P}(A)
\end{aligned} \tag{20.50}$$

Thus in this example both Definition 20.3 and Theorem 20.6 are satisfied.

The property of events of being independent should not be confused with the possibility that events can be disjoint. In fact, the following theorem holds.

Theorem 20.7 (Disjointness and dependence of events)**.** *Let $A \in \mathcal{A}$ and $B \in \mathcal{A}$ be two disjoint events with $\mathbb{P}(A) > 0$ and $\mathbb{P}(B) > 0$. Then A and B are dependent events.*

∘

Proof. First, because of the disjointness of A and B,

$$\mathbb{P}(A \cap B) = \mathbb{P}(\emptyset) = 0 \tag{20.51}$$

Second,

$$\mathbb{P}(A)\mathbb{P}(B) > 0, \tag{20.52}$$

because by assumption both $\mathbb{P}(A) > 0$ and $\mathbb{P}(B) > 0$ hold. Thus

$$\mathbb{P}(A \cap B) \neq \mathbb{P}(A)\mathbb{P}(B) \tag{20.53}$$

and by Definition 20.3, A and B are therefore not independent, hence dependent. □

Example

We want to illustrate Theorem 20.7 using the example of the fair die. Let $A = \{2, 4, 6\}$ be the event "An even number appears" and $B = \{1, 3, 5\}$ the event "An odd number appears". Intuitively, the events are dependent, because the information that an even number has appeared reduces the subjective probability that an odd number has appeared to zero. Formally, on the one hand,

$$\mathbb{P}(A)\mathbb{P}(B) = \frac{1}{2} \cdot \frac{1}{2} = \frac{1}{4}, \tag{20.54}$$

but

$$\mathbb{P}(A \cap B) = \mathbb{P}\left(\{2, 4, 6\} \cap \{1, 3, 5\}\right) = \mathbb{P}\left(\emptyset\right) = 0. \tag{20.55}$$

On the other hand, it is also true that

$$\mathbb{P}(A|B) = \frac{\mathbb{P}(A \cap B)}{\mathbb{P}(B)} = \frac{\mathbb{P}(\emptyset)}{\mathbb{P}(B)} = \frac{0}{\mathbb{P}(B)} = 0 \neq = \frac{1}{2} = \mathbb{P}(\{2, 4, 6\}) = \mathbb{P}(A). \tag{20.56}$$

20.4 Literature notes

Many of the concepts introduced in this section are closely interwoven with the historical genesis of probability theory, so no individual references will be given. An introduction to the history of probability theory of the last two centuries is provided by Hald (1990), and an overview of more modern developments is given by Von Plato (1994). Bayes' theorem is generally traced back to Bayes (1763), even though it is not the actual main topic of this work.

Self-control questions

1. State the definition of the joint probability of two events.
2. Explain the intuitive meaning of the joint probability of two events.
3. State the theorem on further properties of probabilities.
4. State the definition of the conditional probability of an event.
5. State the theorem on conditional probability.
6. State the theorem on joint and conditional probabilities.
7. State the law of total probability.
8. State Bayes' theorem.
9. Reproduce the proof of Bayes' theorem.
10. State the definition of the independence of two events.
11. State the theorem on conditional probability under independence.
12. Reproduce the proof of the theorem on conditional probability under independence.
13. Explain the theorem on conditional probability under independence.

Solutions

1. See Definition 20.1.
2. See the remarks on Definition 20.1.
3. See Theorem 20.1.
4. See Definition 20.2.
5. See Theorem 20.2.
6. See Theorem 20.3.
7. See Theorem 20.4.
8. See Theorem 20.5.
9. See proof of Theorem 20.5.
10. See Definition 20.3.
11. See Theorem 20.6.
12. See proof of Theorem 20.6.
13. See remarks on Theorem 20.6.

21 Random variables

With the concept of a *random variable*, this chapter introduces the standard probabilistic model for a univariate data point. Central to this model is the possibility of specifying distributions of random variables by means of probability mass functions and probability density functions. Moreover, because random variables are constructed as maps of random outcomes, the study of the distributions of these transformed random outcomes leads directly to a core topic of frequentist inference.

21.1 Definition

We first sketch the construction of a random variable and its distribution using Figure 21.1. Let $(\Omega, \mathcal{A}, \mathbb{P})$ be a probability space and let

$$\xi : \Omega \to \mathcal{X}, \omega \mapsto \xi(\omega) \tag{21.1}$$

be a map. Furthermore, let $\mathcal{S}$ be a σ-algebra on the target set $\mathcal{X}$ of this map. For every $S \in \mathcal{S}$, let the preimage set of S be given by (cf. Definition 4.2)

$$\xi^{-1}(S) := \{\omega \in \Omega | \xi(\omega) \in S\}. \tag{21.2}$$

If now $\xi^{-1}(S) \in \mathcal{A}$ holds for all $S \in \mathcal{S}$, then the map ξ is called *measurable*. Thus, assume that ξ is measurable. Then each $S \in \mathcal{S}$ can be assigned the probability

$$\mathbb{P}_\xi : \mathcal{S} \to [0,1], S \mapsto \mathbb{P}_\xi(S) := \mathbb{P}\left(\xi^{-1}(S)\right) = \mathbb{P}\left(\{\omega \in \Omega | \xi(\omega) \in S\}\right). \tag{21.3}$$

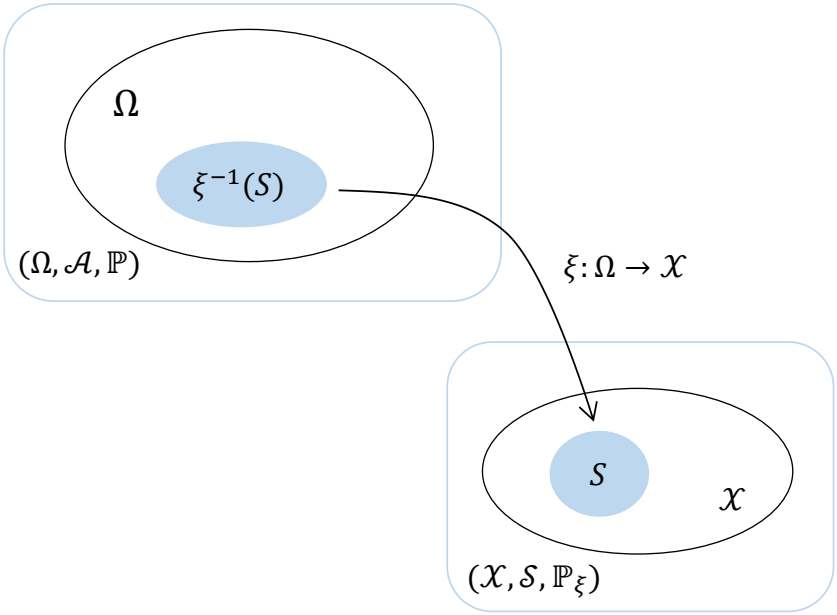
In this context, ξ is called a *random variable* and $\mathbb{P}_\xi$ is called the *image measure* or the *distribution* of ξ. Overall, starting from the probability space $(\Omega, \mathcal{A}, \mathbb{P})$, the random variable ξ has thus been used to construct the probability space $(\mathcal{X}, \mathcal{S}, \mathbb{P}_\xi)$. Formally, a random variable is therefore defined as follows.

Definition 21.1 (Random variable)**.** Let $(\Omega, \mathcal{A}, \mathbb{P})$ be a probability space and let $(\mathcal{X}, \mathcal{S})$ be a *measurable space*. A *random variable* is defined as a map $\xi : \Omega \to \mathcal{X}$ with the *measurability property*

$$\{\omega \in \Omega | \xi(\omega) \in S\} \in \mathcal{A} \text{ for all } S \in \mathcal{S}. \tag{21.4}$$

•

According to Definition 21.1, random variables are neither "random" nor "variables", but measurable maps. If one asks about the meaning of randomness for the values $\xi(\omega)$ of random variables, then the implicit frequentist mechanics of the probability space $(\Omega, \mathcal{A}, \mathbb{P})$ again provides the corresponding intuition: in each run of the modeled random process, an ω is realized according to $\mathbb{P}$ and is then mapped deterministically to $\xi(\omega)$. Accordingly, we define the concepts of the *outcome space* and the *realization* of a random variable.



$$\mathbb{P}\left(\xi^{-1}(S)\right) = \mathbb{P}(\{\omega \in \Omega | \xi(\omega) \in S\}) =: \mathbb{P}_\xi(S)$$

Figure 21.1 Construction of a random variable and a distribution.

Definition 21.2 (Realization of a random variable)**.** Let $(\Omega, \mathcal{A}, \mathbb{P})$ be a probability space, let $(\mathcal{X}, \mathcal{S})$ be a measurable space, and let $\xi : \Omega \to \mathcal{X}$ be a random variable. Then $\mathcal{X}$ is called the *outcome space of the random variable* ξ, and $\xi(\omega) \in \mathcal{X}$ is called a *realization of the random variable* ξ.

•

Example

Building on the example considered in Section 19.3, namely a probability space for modeling the simultaneous throw of a blue and a red die, we consider the sum of the numbers of pips as a first example of a random variable and its distribution. In Section 19.3 we saw that a sensible probability space model for the simultaneous throw of a blue and a red die is given by $(\Omega, \mathcal{A}, \mathbb{P})$ with

- $\Omega := \{(r, b) | r \in \mathbb{N}_6, b \in \mathbb{N}_6\}$,
- $\mathcal{A} := \mathcal{P}(\Omega)$ and
- $\mathbb{P} : \mathcal{A} \to [0, 1]$ with $\mathbb{P}(\{(r, b)\}) = \frac{1}{36}$ for all $(r, b) \in \Omega$.

The evaluation of the sum of the two numbers of pips is then sensibly described by the map

$$\xi : \Omega \to \mathcal{X}, (r, b) \mapsto \xi((r, b)) := r + b, \tag{21.5}$$

where evidently $\mathcal{X} := \{2, 3, ..., 12\}$ must hold. The outcome space of the random variable is therefore again finite, and $\mathcal{S} := \mathcal{P}(\mathcal{X})$ is a sensible σ-algebra on $\mathcal{X}$. With the help of the σ-additivity of $\mathbb{P}$, we can now compute the distribution $\mathbb{P}_\xi$ of ξ for all elementary

events $\{x\} \in \mathcal{S}$ and thereby, in particular, also verify the measurability of ξ, as shown in Table 21.1.

Table 21.1 Probability distribution of the sum of pips when throwing two dice

$\mathbb{P}_\xi(\{x\})$	$\mathbb{P}(\xi^{-1}(\{x\}))$	$\mathbb{P}(\{\omega \mid \xi(\omega) \in \{x\}\})$	
$\mathbb{P}_\xi(\{2\})$	$\mathbb{P}\left(\xi^{-1}(\{2\})\right)$	$\mathbb{P}\left(\{(1,1)\}\right)$	$\frac{1}{36}$
$\mathbb{P}_\xi(\{3\})$	$\mathbb{P}\left(\xi^{-1}(\{3\})\right)$	$\mathbb{P}\left(\{(1,2),(2,1)\}\right)$	$\frac{2}{36}$
$\mathbb{P}_\xi(\{4\})$	$\mathbb{P}\left(\xi^{-1}(\{4\})\right)$	$\mathbb{P}\left(\{(1,3),(3,1),(2,2)\}\right)$	$\frac{3}{36}$
$\mathbb{P}_\xi(\{5\})$	$\mathbb{P}\left(\xi^{-1}(\{5\})\right)$	$\mathbb{P}\left(\{(1,4),(4,1),(2,3),(3,2)\}\right)$	$\frac{4}{36}$
$\mathbb{P}_\xi(\{6\})$	$\mathbb{P}\left(\xi^{-1}(\{6\})\right)$	$\mathbb{P}\left(\{(1,5),(5,1),(2,4),(4,2),(3,3)\}\right)$	$\frac{5}{36}$
$\mathbb{P}_\xi(\{7\})$	$\mathbb{P}\left(\xi^{-1}(\{7\})\right)$	$\mathbb{P}\left(\{(1,6),(6,1),(2,5),(5,2),(3,4),(4,3)\}\right)$	$\frac{6}{36}$
$\mathbb{P}_\xi(\{8\})$	$\mathbb{P}\left(\xi^{-1}(\{8\})\right)$	$\mathbb{P}\left(\{(2,6),(6,2),(3,5),(5,3),(4,4)\}\right)$	$\frac{5}{36}$
$\mathbb{P}_\xi(\{9\})$	$\mathbb{P}\left(\xi^{-1}(\{9\})\right)$	$\mathbb{P}\left(\{(3,6),(6,3),(4,5),(5,4)\}\right)$	$\frac{4}{36}$
$\mathbb{P}_\xi(\{10\})$	$\mathbb{P}\left(\xi^{-1}(\{10\})\right)$	$\mathbb{P}\left(\{(4,6),(6,4),(5,5)\}\right)$	$\frac{3}{36}$
$\mathbb{P}_\xi(\{11\})$	$\mathbb{P}\left(\xi^{-1}(\{11\})\right)$	$\mathbb{P}\left(\{(5,6),(6,5)\}\right)$	$\frac{2}{36}$
$\mathbb{P}_\xi(\{12\})$	$\mathbb{P}\left(\xi^{-1}(\{12\})\right)$	$\mathbb{P}\left(\{(6,6)\}\right)$	$\frac{1}{36}$

The probabilities of the elementary events in $\mathcal{S}$ in turn allow us to compute arbitrary event probabilities with respect to the sum of the dice pips (cf. Section 19.2). Overall, based on $(\Omega, \mathcal{A}, \mathbb{P})$ and ξ, we have thus constructed another probability space model $(\mathcal{X}, \mathcal{S}, \mathbb{P}_\xi)$.

The following **R** code demonstrates how the construction of the example considered here can be simulated for one run of a random process using computer-based generation of random outcomes.

```
# probability space model formulation
Omega    = list()                                # outcome space initialization
idx      = 1                                     # outcome index initialization
for(r in 1:6){                                   # outcomes red die
    for(b in 1:6){                               # outcomes blue die
        Omega[[idx]] = c(r,b)                    # \omega \in \Omega
        idx          = idx + 1 }}                # outcome index update
K        = length(Omega)                         # cardinality of \Omega
pi       = rep(1/K,1,K)                          # probability function \pi

# random process run
omega    = Omega[[which(rmultinom(1,1,pi) == 1)]] # selection of \omega according to \mathbb{P}({\omega})

# realization of the random variable
xi_omega = sum(omega)                            # \xi(\omega)
```

```
omega     :  5 5
xi(omega) :  10
```

In the context of measuring randomly selected experimental units, random variables often serve as models of measurement processes. For example, consider the determination of the value of a psychological questionnaire (BDI-II) in randomly selected participants (Figure 21.2). This yields the following interpretation: the outcome space of the underlying probability space (Ω) is meant to represent the set of all eligible participants, and the selection of a participant from this space is the selection of an outcome ω, which is to occur with probability $\mathbb{P}(\{\omega\})$. If a particular property of this participant is then measured in an idealized way, this is a deterministic map to a measurement value $\xi(\omega)$ assigned to this participant in the space of measurement values $\mathcal{X}$. The measurement values themselves then follow a probability distribution that is determined by the underlying distribution and the type of measurement process.

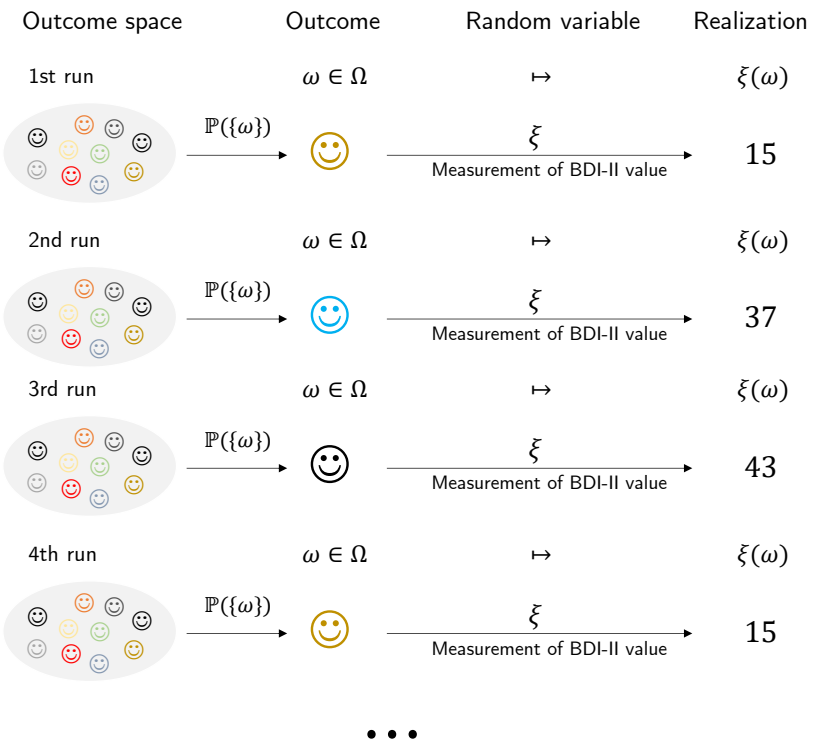


Figure 21.2 Random variable as a model of a measurement process.

Notation

The conventions for denoting probabilities and distributions associated with random variables take some getting used to, so we record them in the following definition.

Definition 21.3 (Notation for random variables)**.** Let $(\Omega, \mathcal{A}, \mathbb{P})$ and $(\mathcal{X}, \mathcal{S}, \mathbb{P}_\xi)$ be probability spaces and let $\xi : \Omega \to \mathcal{X}$ be a random variable. Then, with $S \in \mathcal{S}$ and $x \in \mathcal{X}$, the following notational conventions hold for events in $\mathcal{A}$:

$$
\begin{aligned}
\{\xi \in S\} &:= \{\omega \in \Omega | \xi(\omega) \in S\} \\
\{\xi = x\} &:= \{\omega \in \Omega | \xi(\omega) = x\} \\
\{\xi \leq x\} &:= \{\omega \in \Omega | \xi(\omega) \leq x\} \\
\{\xi < x\} &:= \{\omega \in \Omega | \xi(\omega) < x\} \\
\{\xi \geq x\} &:= \{\omega \in \Omega | \xi(\omega) \geq x\} \\
\{\xi > x\} &:= \{\omega \in \Omega | \xi(\omega) > x\}
\end{aligned}
$$

From these conventions follow the following conventions for probabilities of distributions:

$$\begin{aligned}
\mathbb{P}_\xi(\xi \in S) &= \mathbb{P}(\{\xi \in S\}) = \mathbb{P}(\{\omega \in \Omega | \xi(\omega) \in S\}) \\
\mathbb{P}_\xi(\xi = x) &= \mathbb{P}(\{\xi = x\}) = \mathbb{P}(\{\omega \in \Omega | \xi(\omega) = x\}) \\
\mathbb{P}_\xi(\xi \leq x) &= \mathbb{P}(\{\xi \leq x\}) = \mathbb{P}(\{\omega \in \Omega | \xi(\omega) \leq x\}) \\
\mathbb{P}_\xi(\xi < x) &= \mathbb{P}(\{\xi < x\}) = \mathbb{P}(\{\omega \in \Omega | \xi(\omega) < x\}) \\
\mathbb{P}_\xi(\xi \geq x) &= \mathbb{P}(\{\xi \geq x\}) = \mathbb{P}(\{\omega \in \Omega | \xi(\omega) \geq x\}) \\
\mathbb{P}_\xi(\xi > x) &= \mathbb{P}(\{\xi > x\}) = \mathbb{P}(\{\omega \in \Omega | \xi(\omega) > x\}) .
\end{aligned}$$

Often the subscript on distribution symbols is also omitted, and, for example, $\mathbb{P}_\xi(\xi \in S)$ is written simply as $\mathbb{P}(\xi \in S)$ as long as the context does not allow the two probability measures to be confused.

•

The omission of explicit function notation for functions determined uniquely by random variables, as sketched in 1 for $\mathbb{P}_\xi$ and $\mathbb{P}$, is a fundamental phenomenon of modern, application-oriented probability theory. In particular, functions of random variables are in most cases identified through their arguments. Thus, from "$\mathbb{P}(\xi \in S)$", an attentive reader should understand that $\mathbb{P}$ refers to the image measure of the random variable ξ, whereas in "$\mathbb{P}(\{\omega \in \Omega | \xi(\omega) \in S\})$" it refers to the probability measure on the measurable space $(\Omega, \mathcal{A})$. In computer-science applications, the identification of functions by means of their arguments, which also appears in the *probability mass functions*, *probability density functions*, and *cumulative distribution functions* discussed in the following sections, is also called *multiple dispatch*.

To conclude this section, we record in Theorem 21.1 that arithmetic operations with real-valued random variables again lead to real-valued random variables.

Theorem 21.1 (Arithmetic of real-valued random variables)**.** *Let $(\Omega, \mathcal{A}, \mathbb{P})$ be a probability space, let $(\mathbb{R}, \mathcal{B}(\mathbb{R}))$ be the real measurable space, let $\xi_1 : \Omega \to \mathbb{R}$ and $\xi_2 : \Omega \to \mathbb{R}$ be real-valued random variables, and let $c \in \mathbb{R}$ be a constant. Furthermore, let*

$$\begin{aligned}
\xi_1 + c : \Omega \to \mathbb{R}, \omega \mapsto (\xi_1 + c)(\omega) &:= \xi_1(\omega) + c \text{ for } c \in \mathbb{R} \\
c\xi_1 : \Omega \to \mathbb{R}, \omega \mapsto (c\xi_1)(\omega) &:= c\xi_1(\omega) \text{ for } c \in \mathbb{R} \\
\xi_1 + \xi_2 : \Omega \to \mathbb{R}, \omega \mapsto (\xi_1 + \xi_2)(\omega) &:= \xi_1(\omega) + \xi_2(\omega) \\
\xi_1\xi_2 : \Omega \to \mathbb{R}, \omega \mapsto (\xi_1\xi_2)(\omega) &:= \xi_1(\omega)\xi_2(\omega)
\end{aligned} \tag{21.6}$$

be the addition of a constant to a real-valued random variable, the multiplication of a real-valued random variable by a constant, the addition of two real-valued random variables, and the multiplication of two real-valued random variables, respectively. Then $\xi_1 + c$, $c\xi_1$, $\xi_1 + \xi_2$, and $\xi_1\xi_2$ are also real-valued random variables.

∘

For a proof of this theorem, we refer to the advanced literature, for example Hesse (2009). Intuitively, Theorem 21.1 says that adding a random quantity to a constant quantity, multiplying a random quantity by a constant, adding two random quantities, and multiplying two random quantities always again produces random quantities with their own distributions.

21.2 Probability mass functions

In this section, with *probability mass functions (PMFs)*, we introduce a tool for defining distributions of random variables with a *discrete* (more precisely *finite* or *countable*) outcome space. We illustrate the concept using three important examples: Bernoulli and binomial random variables as well as discrete uniform random variables. We first define the concept of a PMF as follows.

Definition 21.4 (Discrete random variable and probability mass function)**.** A random variable ξ is called *discrete* if its outcome space $\mathcal{X}$ is finite or countable and if a function of the form

$$p : \mathcal{X} \to \mathbb{R}_{\geq 0}, x \mapsto p(x) \tag{21.7}$$

exists such that

(1) $\sum_{x \in \mathcal{X}} p(x) = 1$
(2) $\mathbb{P}(\xi = x) = p(x)$ for all $x \in \mathcal{X}$.

Such a function p is called the *probability mass function (PMF)* of ξ.

•

Recall that a set is called *countable* if it can be mapped bijectively to $\mathbb{N}$. In German, PMFs are also called *counting densities*. In English, they are called *probability mass functions (PMFs)*; we follow this terminology here. Without proof, we record that every function $p : \mathcal{X} \to \mathbb{R}_{\geq 0}$ with the normalization property $\sum_{x \in \mathcal{X}} p(x) = 1$ can be interpreted as the PMF of a random variable.

Examples

With *Bernoulli random variables*, *binomial random variables*, and *discrete uniform random variables*, we consider three first examples of defining distributions by means of PMFs.

Definition 21.5 (Bernoulli random variable)**.** Let ξ be a random variable with outcome space $\mathcal{X} = \{0, 1\}$ and PMF

$$p : \mathcal{X} \to [0, 1], x \mapsto p(x) := \mu^x (1 - \mu)^{1-x} \text{ with } \mu \in [0, 1]. \tag{21.8}$$

Then we say that ξ follows a *Bernoulli distribution with parameter* $\mu \in [0, 1]$ and call ξ a *Bernoulli random variable*. We abbreviate this as $\xi \sim \text{Bern}(\mu)$. We denote the PMF of a Bernoulli random variable by

$$\text{Bern}(x; \mu) := \mu^x (1 - \mu)^{1-x}. \tag{21.9}$$

•

Bernoulli random variables can be used for probabilistic modeling whenever the phenomenon under consideration is binary and the possible values of the random variable can be mapped bijectively to $\{0, 1\}$. Note that the functional form of the Bernoulli random variable $\text{Bern}(x; \mu)$ makes sense only for $x \in \{0, 1\}$ and not, for example, for $x \in \{\text{H}, \text{T}\}$. However, if the possible outcomes of a coin toss are mapped to $\{0, 1\}$, for example by defining $0 := \text{H}$ and $1 := \text{T}$, then a Bernoulli random variable can certainly serve as a model of a coin toss.

The parameter $\mu \in [0, 1]$ of a Bernoulli random variable is the probability that the random variable ξ assumes the value 1; this can be seen from

$$\mathbb{P}(\xi = 1) = p(1) = \mu^1(1 - \mu)^{1-1} = \mu. \tag{21.10}$$

We visualize the PMFs of Bernoulli random variables for $\mu := 0.1, \mu := 0.5$, and $\mu := 0.7$ in Figure 21.3.

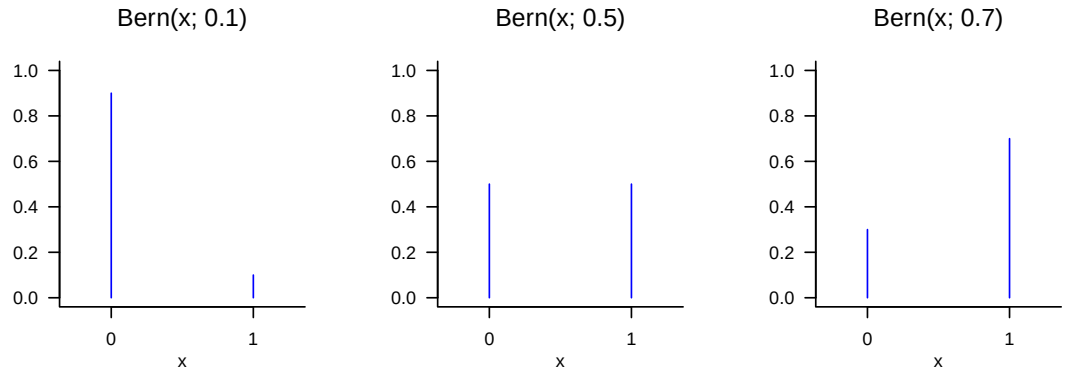


Figure 21.3 PMFs of Bernoulli random variables.

Definition 21.6 (Binomial random variable)**.** Let ξ be a random variable with outcome space $\mathcal{X} := \mathbb{N}_n^0$ and PMF

$$p : \mathcal{X} \to [0, 1], x \mapsto p(x) := \binom{n}{x} \mu^x (1 - \mu)^{n-x} \text{ for } \mu \in [0, 1]. \tag{21.11}$$

Then we say that ξ follows a *binomial distribution with parameters* $n \in \mathbb{N}$ *and* $\mu \in [0, 1]$ and call ξ a *binomial random variable*. We abbreviate this as $\xi \sim \text{Bin}(n, \mu)$. We denote the PMF of a binomial random variable by

$$\text{Bin}(x; n, \mu) := \binom{n}{x} \mu^x (1 - \mu)^{n-x}. \tag{21.12}$$

•

Without proof, we record that a binomial random variable can be used as a model of the sum of n independent and identically distributed Bernoulli random variables. In particular, $\text{Bin}(x; 1, \mu) = \text{Bern}(x; \mu)$ holds. Binomial random variables have the property that with n, one of their parameters determines not only the functional form of their PMF, but also their outcome space $\mathcal{X}$. We visualize the PMFs of binomial random variables for $(n, \mu) := (5, 0.1), (n, \mu) := (10, 0.5)$, and $(n, \mu) := (15, 0.7)$ in Figure 21.4.

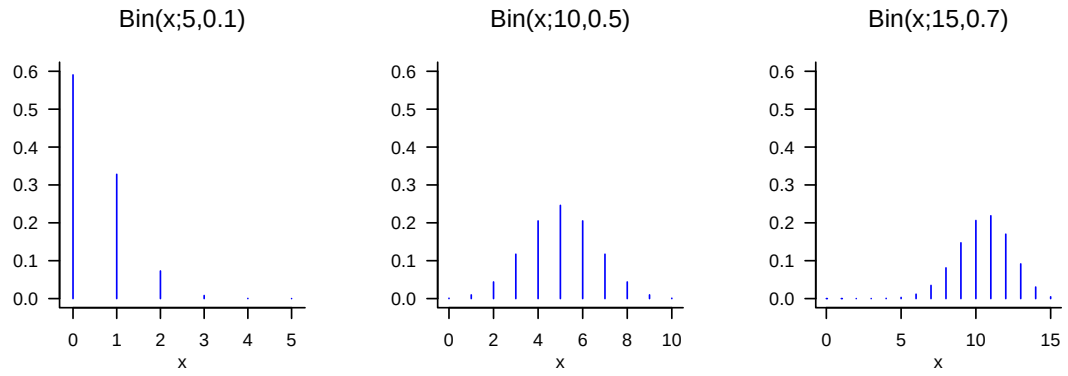


Figure 21.4 PMFs of binomial random variables.

Definition 21.7 (Discrete uniform random variable)**.** Let ξ be a discrete random variable with finite outcome space $\mathcal{X}$ and PMF

$$p : \mathcal{X} \to \mathbb{R}_{\geq 0}, x \mapsto p(x) := \frac{1}{|\mathcal{X}|}. \tag{21.13}$$

Then we say that ξ follows a *discrete uniform distribution* and call ξ a *discrete uniform random variable*. We abbreviate this as $\xi \sim U(|\mathcal{X}|)$. We denote the PMF of a discrete uniform random variable by

$$U(x; |\mathcal{X}|) := \frac{1}{|\mathcal{X}|}. \tag{21.14}$$

•

Discrete uniform random variables can evidently be used for probabilistic modeling whenever the possible discrete outcomes of the modeled phenomenon have the same probability. In the case of discrete uniform random variables, no additional parameter is needed to define their functional form once the outcome space has been specified. Evidently, for $\mathcal{X} := \{0, 1\}$,

$$U(x; |\mathcal{X}|) = \text{Bern}(x; 0.5) = \text{Bin}(x; 1, 0.5). \tag{21.15}$$

We visualize the PMFs of discrete uniform random variables for $\mathcal{X} := \{0, 1\}$, $\mathcal{X} := \{-3, -2, -1, 0, 1\}$, and $\mathcal{X} := \{-4, -3, -2, -1, 0, 1, 2, 3, 4\}$ in Figure 21.5. Note that Definition 21.7 does not formally require the elements of $\mathcal{X}$ to be sequential, so one can just as well define a discrete uniform random variable with outcome space $\mathcal{X} := \{1, 5, 7\}$ or even with a non-numeric outcome space $\mathcal{X} := \{a, b, x, y\}$.

21.3 Probability density functions

In this section, with *probability density functions (PDFs)*, we introduce a tool for defining distributions of random variables with a *continuous* (more precisely *uncountable*) outcome space. We first illustrate the concept using the most fundamental of all random variables, the normally distributed random variable. With the gamma random variable, the beta

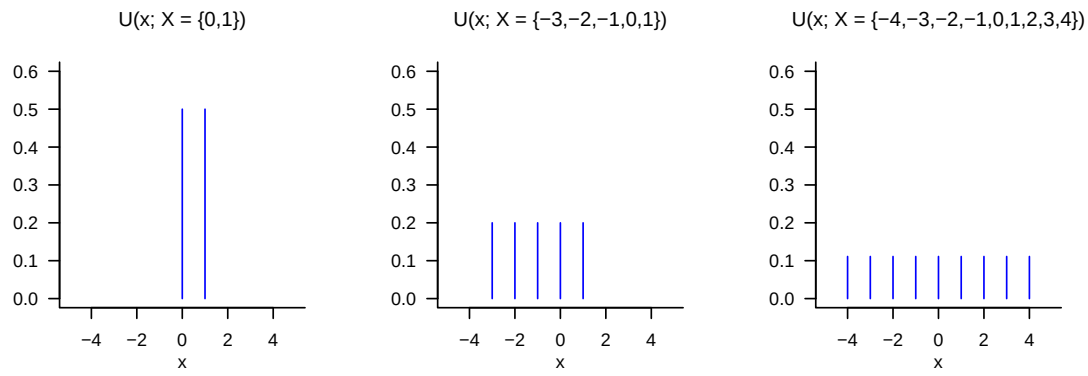


Figure 21.5 PMFs of discrete uniform random variables.

random variable, and the uniform random variable, we then consider three further examples of random variables that are used in many places in the model formulation of both frequentist and Bayesian inference. We define the concept of a PDF as follows.

Definition 21.8 (Continuous random variable and probability density function)**.** A random variable ξ is called *continuous* if $\mathbb{R}$ is the outcome space of ξ and if a function

$$p : \mathbb{R} \to \mathbb{R}_{\geq 0}, x \mapsto p(x) \tag{21.16}$$

exists such that

(1) $\int_{-\infty}^{\infty} p(x)\, dx = 1$ and
(2) $\mathbb{P}(\xi \in [a, b]) = \int_a^b p(x)\, dx$ for all $a, b \in \mathbb{R}$ with $a \leq b$.

Such a function p is called the *probability density function (PDF)* of ξ.

•

Without proof, we record that every function $p : \mathbb{R} \to \mathbb{R}_{\geq 0}$ whose improper integral satisfies $\int_{-\infty}^{\infty} p(x)\, dx = 1$, that is, every normalized such function, can be interpreted as the PDF of a random variable.

When working with PDFs, and in contrast to PMFs, one should always keep the *density property* of a PDF in mind: the values of a PDF are not probabilities but probability densities; probabilities are computed from PDFs by integration according to Definition 21.8 (2). As in the physical sense, the probability mass assigned to a real interval therefore arises only through "multiplication" by the corresponding "interval volume". Here one may also think of the approximation of the definite integral $\int_a^b p(x)\, dx$ by a Riemann sum term (cf. Definition 7.3). Intuitively, we therefore have

$$\text{(probability) mass} = \text{(probability) density} \cdot \text{(set) volume}, \tag{21.17}$$

where volume in the sense of Lebesgue measure refers to the width of the interval $[a, b]$. As in the physical analogy, the probability mass of an interval with no volume is zero,

$$\mathbb{P}(\xi = a) = \int_a^a p(x)\, dx = 0. \tag{21.18}$$

Furthermore, for sufficiently small intervals, PDFs can also assume values greater than 1, even though this cannot be the case for the probability that results from the corresponding integration. Finally, despite these technical details, the following intuition should be emphasized: if one considers the graphical representation of the PDF of a random variable and imagines a partition of $\mathbb{R}$ into equally sized intervals, then the random variable naturally has a higher probability of assuming values in an interval with a higher associated probability density than values in an interval with a relatively lower associated probability density.

Normally distributed random variables

As a first example of defining a continuous random variable by means of a PDF, we now introduce the most important random variable of probabilistic modeling: the normally distributed random variable.

Definition 21.9 (Normally distributed random variable)**.** Let ξ be a random variable with outcome space $\mathbb{R}$ and PDF

$$p : \mathbb{R} \to \mathbb{R}_{>0}, x \mapsto p(x) := \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{1}{2\sigma^2}(x-\mu)^2\right). \tag{21.19}$$

Then we say that ξ follows a *normal distribution with parameters* $\mu \in \mathbb{R}$ *and* $\sigma^2 > 0$ and call ξ a *normally distributed random variable*. We abbreviate this as $\xi \sim N(\mu, \sigma^2)$. We denote the PDF of a normally distributed random variable by

$$N(x; \mu, \sigma^2) := \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{1}{2\sigma^2}(x-\mu)^2\right). \tag{21.20}$$

A normally distributed random variable with $\mu = 0$ and $\sigma^2 = 1$ is called a *standard normally distributed random variable*.

•

We visualize the PDFs of normally distributed random variables for $(\mu, \sigma^2) := (0, 1)$, $(\mu, \sigma^2) := (-2.5, 10)$, and $(\mu, \sigma^2) := (3, 0.5)$ in Figure 21.6. This figure makes graphically clear that the PDFs of normally distributed random variables always have exactly one value of highest probability density, namely at the location of the parameter $\mu \in \mathbb{R}$. This follows from the fact that, because of the negative sign of the square of $x - \mu$ and the positivity of σ^2, the argument of the exponential function in the functional form of $N(x; \mu, \sigma^2)$ is always non-positive, and the exponential function on the non-positive real numbers assumes its maximum at $x = \mu$, that is, $x - \mu = 0$. The figure also makes graphically clear that the parameter $\sigma^2 > 0$ encodes the width of the PDF of a normally distributed random variable.

Further examples

With *gamma random variables*, *beta random variables*, and *uniform random variables*, we consider three further examples of defining distributions by means of PDFs.

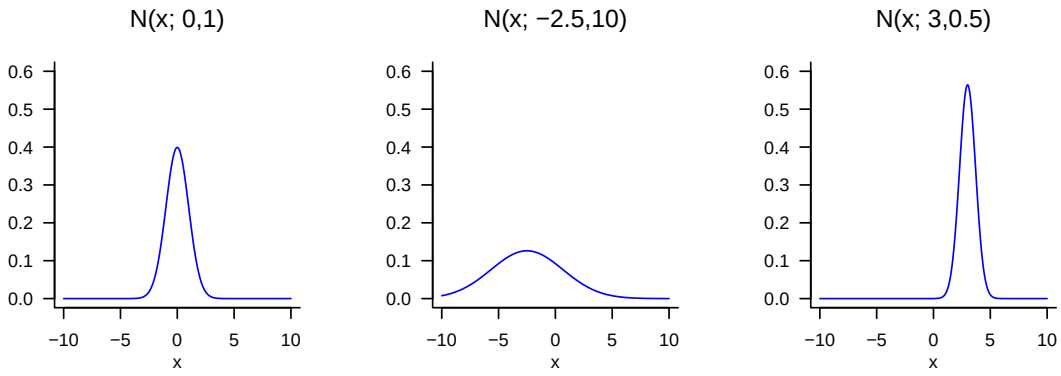


Figure 21.6 PDFs of normally distributed random variables.

Definition 21.10 (Gamma random variable)**.** Let ξ be a random variable with outcome space $\mathcal{X} := \mathbb{R}_{>0}$ and PDF

$$p : \mathbb{R}_{>0} \to \mathbb{R}_{>0}, x \mapsto p(x) := \frac{1}{\Gamma(\alpha)\beta^\alpha} x^{\alpha-1} \exp\left(-\frac{x}{\beta}\right), \tag{21.21}$$

where Γ denotes the gamma function. Then we say that ξ follows a *gamma distribution with shape parameter* $\alpha > 0$ *and scale parameter* $\beta > 0$ and call ξ a *gamma-distributed random variable*. We abbreviate this as $\xi \sim G(\alpha, \beta)$. We denote the PDF of a gamma-distributed random variable by

$$G(x; \alpha, \beta) := \frac{1}{\Gamma(\alpha)\beta^\alpha} x^{\alpha-1} \exp\left(-\frac{x}{\beta}\right). \tag{21.22}$$

•

The special gamma random variable with PDF $G\left(x; \frac{n}{2}, 2\right)$ is called the *chi-square* (χ^2) *distribution with* n *degrees of freedom*. We visualize the PDFs of gamma random variables for $(\alpha, \beta) := (1, 1)$, $(\alpha, \beta) := (2, 2)$, and $(\alpha, \beta) := (5, 1)$ in Figure 21.7.

Definition 21.11 (Beta random variable)**.** Let ξ be a random variable with outcome space $\mathcal{X} := [0, 1]$ and PDF

$$p : \mathcal{X} \to [0, 1], x \mapsto p(x) := \frac{\Gamma(\alpha + \beta)}{\Gamma(\alpha)\Gamma(\beta)} x^{\alpha-1}(1 - x)^{\beta-1} \text{ with } \alpha, \beta \in \mathbb{R}_{>0}, \tag{21.23}$$

where Γ denotes the gamma function. Then we say that ξ follows a *beta distribution* with parameters $\alpha > 0$ and $\beta > 0$, and call ξ a *beta random variable*. We abbreviate this as $\xi \sim \text{Beta}(\alpha, \beta)$. We denote the PDF of a beta random variable by

$$\text{Beta}(x; \alpha, \beta) := \frac{\Gamma(\alpha + \beta)}{\Gamma(\alpha)\Gamma(\beta)} x^{\alpha-1}(1 - x)^{\beta-1}. \tag{21.24}$$

•

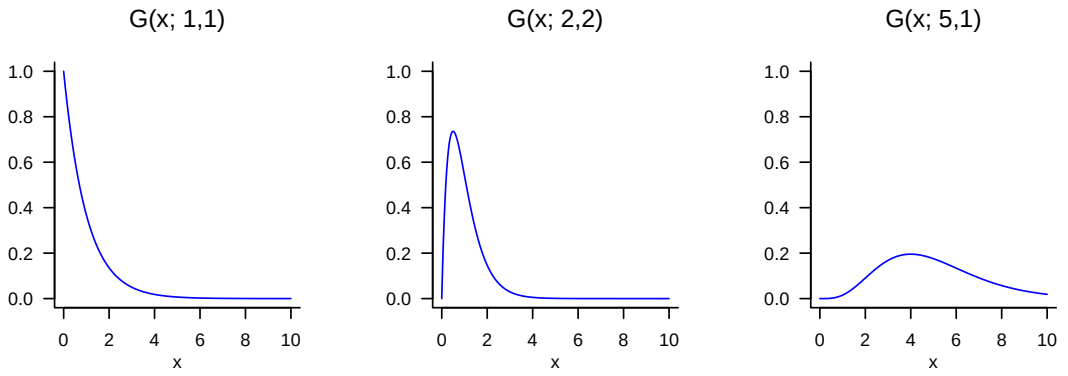


Figure 21.7 PDFs of gamma random variables.

Because the outcome space of a beta random variable is restricted to the interval $\mathcal{X} := [0, 1]$ (more precisely $\mathcal{X} :=]0, 1[$ for $\alpha < 1, \beta < 1$), a beta random variable is useful, among other things, for describing probabilities of probabilities, that is, values between 0 and 1. We visualize the PDFs of beta random variables for $(\alpha, \beta) := (1, 1)$, $(\alpha, \beta) := (3, 2)$, and $(\alpha, \beta) := (10, 5)$ in Figure 21.8.

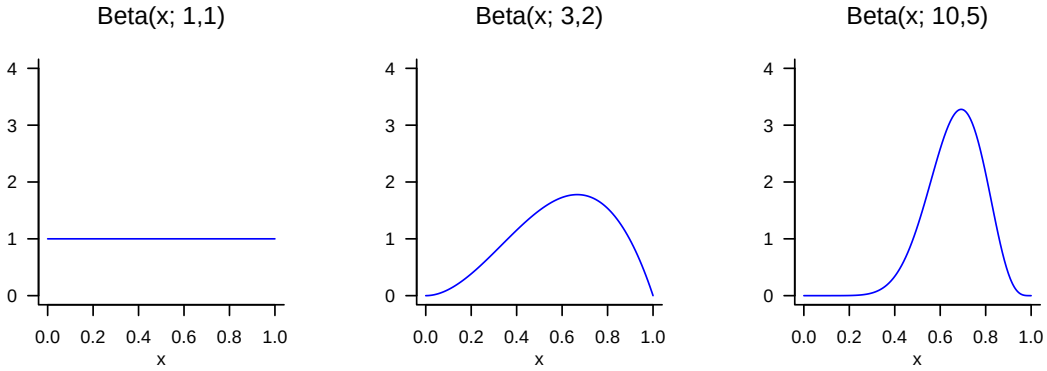


Figure 21.8 PDFs of beta random variables.

With uniform random variables, we finally consider the analogue of discrete uniform random variables for the case of continuous random variables.

Definition 21.12 (Uniform random variable)**.** Let ξ be a continuous random variable with outcome space $\mathbb{R}$ and PDF

$$p : \mathbb{R} \to \mathbb{R}_{\geq 0}, x \mapsto p(x) := \begin{cases} \frac{1}{b-a} & x \in [a, b] \\ 0 & x \notin [a, b] \end{cases}. \tag{21.25}$$

Then we say that ξ follows a *uniform distribution with parameters a and b* and call ξ a *uniform random variable*. We abbreviate this as $\xi \sim U(a, b)$. We denote the PDF of a

uniform random variable by

$$U(x; a, b) := \frac{1}{b-a}. \tag{21.26}$$

•

We visualize the PDFs of uniform random variables for $(a, b) := (0, 1)$, $(a, b) := (-3, 1)$, and $(a, b) := (-4, 4)$ in Figure 21.9.

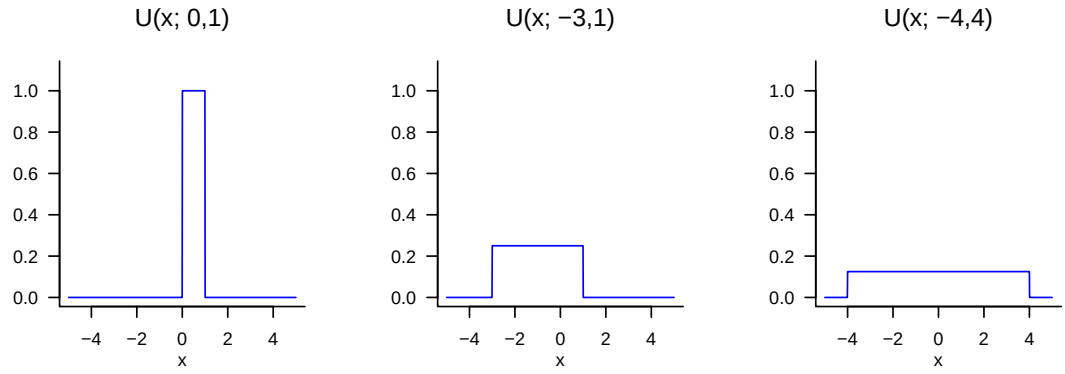


Figure 21.9 PDFs of uniform random variables.

21.4 Cumulative distribution functions

In the final section of this chapter, with *cumulative distribution functions (CDFs)*, we introduce another way of specifying the distributions of discrete or continuous random variables. In general, however, it is more common to do this with PMFs or PDFs. Nevertheless, CDFs are useful in many places because, for both discrete and continuous random variables, they establish a direct connection between values of the random variable and certain probabilities that does not require summation or integration in application. We first consider the general definition of CDFs for both discrete and continuous random variables and then turn to the CDFs of discrete and continuous random variables in detail. For an arbitrary random variable, we define the concept of a CDF as follows.

Definition 21.13 (Cumulative distribution function)**.** The *cumulative distribution function (CDF)* of a random variable ξ is defined as

$$P : \mathbb{R} \to [0, 1], x \mapsto P(x) := \mathbb{P}(\xi \leq x). \tag{21.27}$$

•

Note that $P(x)$ is defined for every $x \in \mathbb{R}$, even if for a given $x \in \mathbb{R}$ we have $x \notin \mathcal{X}$. With the help of CDFs, for example, *exceedance probabilities* and *interval probabilities* of random variables can be evaluated directly. This is the content of the following theorems.

Theorem 21.2 (Exceedance probability)**.** *Let ξ be a random variable with outcome space $\mathcal{X}$ and let P be its cumulative distribution function. Then, for the* exceedance probability *$\mathbb{P}(\xi > x)$,*

$$\mathbb{P}(\xi > x) = 1 - P(x) \text{ for all } x \in \mathcal{X}. \tag{21.28}$$

∘

Proof. The events $\{\xi > x\}$ and $\{\xi \leq x\}$ are disjoint, and

$$\Omega = \{\omega \in \Omega | \xi(\omega) > x\} \cup \{\omega \in \Omega | \xi(\omega) \leq x\} = \{\xi > x\} \cup \{\xi \leq x\}. \tag{21.29}$$

With the σ-additivity of $\mathbb{P}$ it follows that

$$\begin{aligned} & \mathbb{P}(\Omega) = 1 \\ \Leftrightarrow\ & \mathbb{P}(\{\xi > x\} \cup \{\xi \leq x\}) = 1 \\ \Leftrightarrow\ & \mathbb{P}(\{\xi > x\}) + \mathbb{P}(\{\xi \leq x\}) = 1 \\ \Leftrightarrow\ & \mathbb{P}(\{\xi > x\}) = 1 - \mathbb{P}(\{\xi \leq x\}) \\ \Leftrightarrow\ & \mathbb{P}(\{\xi > x\}) = 1 - P(x). \end{aligned} \tag{21.30}$$

□

Theorem 21.3 (Interval probabilities)**.** *Let ξ be a random variable with outcome space $\mathcal{X}$ and let P be its CDF. Then, for the* interval probability *$\mathbb{P}(\xi \in]x_1, x_2])$,*

$$\mathbb{P}(\xi \in]x_1, x_2]) = P(x_2) - P(x_1) \text{ for all } x_1, x_2 \in \mathcal{X} \text{ with } x_1 < x_2. \tag{21.31}$$

∘

Proof. We consider the events $\{\xi \leq x_1\}$, $\{x_1 < \xi \leq x_2\}$, and $\{\xi \leq x_2\}$, where

$$\{\xi \leq x_1\} \cap \{x_1 < \xi \leq x_2\} = \emptyset \text{ and } \{\xi \leq x_1\} \cup \{x_1 < \xi \leq x_2\} = \{\xi \leq x_2\}. \tag{21.32}$$

With the σ-additivity of $\mathbb{P}$ it then follows that

$$\begin{aligned} & \mathbb{P}(\{\xi \leq x_1\} \cup \{x_1 < \xi \leq x_2\}) = \mathbb{P}(\{\xi \leq x_2\}) \\ \Leftrightarrow\ & \mathbb{P}(\{\xi \leq x_1\}) + \mathbb{P}(\{x_1 < \xi \leq x_2\}) = \mathbb{P}(\{\xi \leq x_2\}) \\ \Leftrightarrow\ & \mathbb{P}(\{x_1 < \xi \leq x_2\}) = \mathbb{P}(\{\xi \leq x_2\}) - \mathbb{P}(\{\xi \leq x_1\}) \\ \Leftrightarrow\ & \mathbb{P}(\{x_1 < \xi \leq x_2\}) = P(x_2) - P(x_1) \\ \Leftrightarrow\ & \mathbb{P}(\xi \in]x_1, x_2]) = P(x_2) - P(x_1). \end{aligned} \tag{21.33}$$

□

The following theorem states three central properties of CDFs. The third property says that a CDF has no jumps when one approaches limit points from the right. In fact, the properties discussed are precisely the defining properties of CDFs; that is, every function P that satisfies the properties of Theorem 21.4 can be interpreted as the CDF of a random variable. For a proof of this fact, we refer to the advanced literature.

Theorem 21.4 (Properties of cumulative distribution functions)**.** *Let ξ be a random variable and let P be its cumulative distribution function. Then P has the following properties:*

(1) P is monotonically increasing*, that is, if $x_1 < x_2$, then $P(x_1) \leq P(x_2)$.*
(2) $\lim_{x \to -\infty} P(x) = 0$ and $\lim_{x \to \infty} P(x) = 1$.

(3) P is right-continuous, *that is, $P(x) = \lim_{y \to x, y > x} P(y)$ for all $x \in \mathbb{R}$.*

∘

Proof. We consider the properties in order.

(1) First, we record that for events $A \subset B$, $\mathbb{P}(A) \leq \mathbb{P}(B)$ holds. We then note that for $x_1 < x_2$,

$$\{\xi \leq x_1\} = \{\omega \in \Omega | \xi(\omega) \leq x_1\} \subset \{\omega \in \Omega | \xi(\omega) \leq x_2\} = \{\xi \leq x_2\}. \tag{21.34}$$

Thus,

$$\mathbb{P}(\{\xi \leq x_1\}) \leq \mathbb{P}\{\xi \leq x_2\} \Rightarrow P(x_1) \leq P(x_2). \tag{21.35}$$

(2) For a proof, we refer to the advanced literature.

(3) We define

$$P\left(x^+\right) = \lim_{y \to x, y > x} P(y). \tag{21.36}$$

Now let $y_1 > y_2 > \cdots$ be such that $\lim_{n \to \infty} y_n = x$. Then

$$\{\xi \leq x\} = \cap_{n=1}^{\infty} \{\xi \leq y_n\}. \tag{21.37}$$

Thus,

$$P(x) = \mathbb{P}(\{\xi \leq x\}) = \mathbb{P}(\cap_{n=1}^{\infty} \{\xi \leq y_n\}) = \lim_{n \to \infty} \mathbb{P}(\{\xi \leq y_n\}) = P\left(x^+\right), \tag{21.38}$$

where we leave the third equality unjustified.

□

CDFs of discrete random variables

Using Figure 21.10 and Figure 21.11, we want to make the above properties of CDFs of discrete random variables visually clear. One should always keep in mind that the value $P(x)$ of a CDF at the outcome value x of the random variable ξ corresponds to the probability $\mathbb{P}(\xi \leq x)$, that is, the probability that the random variable ξ assumes values less than or equal to x. If these probabilities are read off accordingly from the representations of the corresponding PMF, then the functional form of the CDFs follows intuitively in comparison with the corresponding PMFs. Furthermore, the following properties of the CDFs of discrete random variables also become intuitive: if $a < b$ and $\mathbb{P}(a < \xi < b) = 0$, then the CDF of ξ is horizontally constant on $]a, b[$. Furthermore, at every point x with $\mathbb{P}(\xi = x) > 0$, the CDF jumps by the amount $\mathbb{P}(\xi = x)$ and is therefore not left-continuous at that point. In general, the CDF of a discrete random variable with outcome space $\mathbb{N}_0$ is given by

$$P : \mathbb{R} \to [0, 1], x \mapsto P(x) := \sum_{k=0}^{\lfloor x \rfloor} \mathbb{P}(\xi = k), \tag{21.39}$$

where $\lfloor x \rfloor$ denotes the floor function.

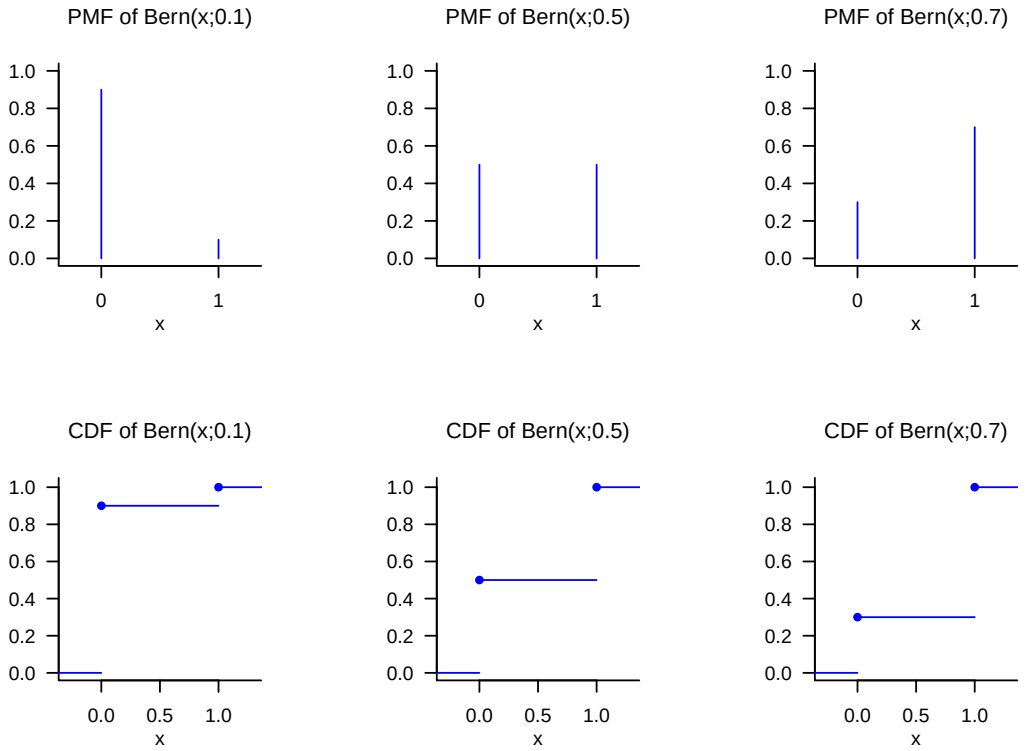


Figure 21.10 PMFs and CDFs of Bernoulli random variables.

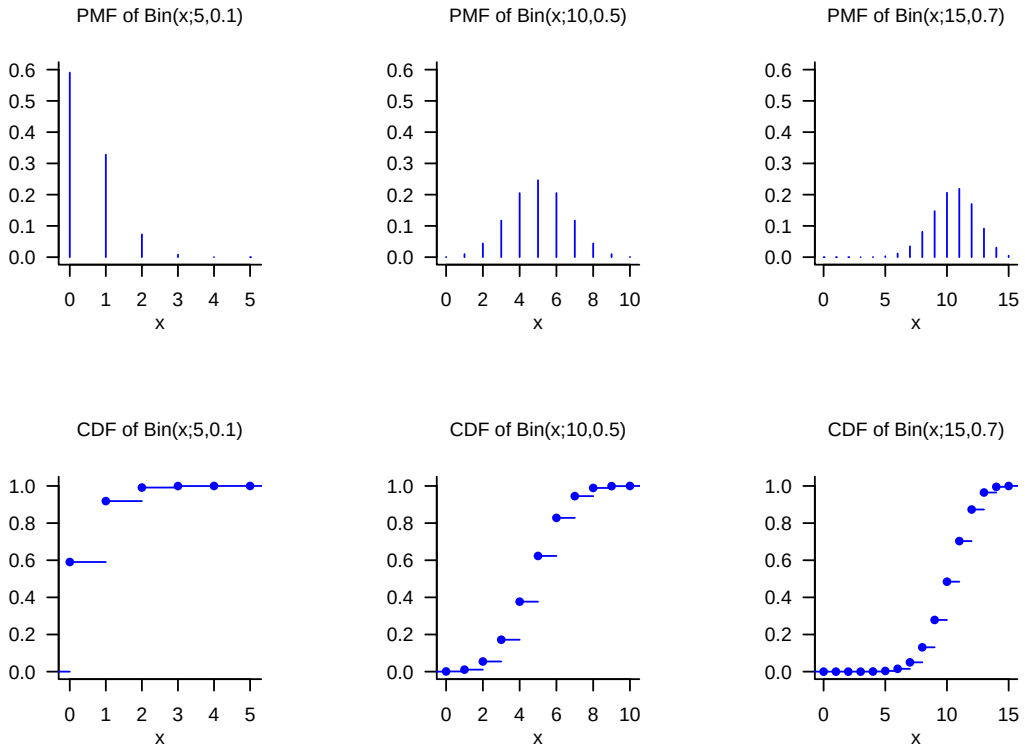


Figure 21.11 PMFs and CDFs of binomial random variables.

CDFs of continuous random variables

The CDFs of continuous random variables are analytically somewhat more accessible than the CDFs of discrete random variables because they have no discontinuities. We first have the following, perhaps somewhat surprising theorem.

Theorem 21.5 (Cumulative distribution functions of continuous random variables)**.** *Let ξ be a continuous random variable with PDF p and CDF P. Then*

$$P(x) = \int_{-\infty}^{x} p(s)\, ds \text{ and } p(x) = P'(x). \tag{21.40}$$

∘

Proof. First, we record that because $\mathbb{P}(\xi = x) = 0$ for all $x \in \mathbb{R}$, the CDF of ξ has no jumps and P is therefore continuous. From the definitions of PDF and CDF, it follows that P has the form of an antiderivative of p. That p is the derivative of P then follows directly from the first fundamental theorem of calculus (Theorem 7.4). □

For continuous random variables, the CDF of the random variable is therefore an antiderivative of the corresponding PDF, and conversely, the PDF is the derivative of the CDF. In the context of the *Radon-Nikodym theorem*, this insight is generalized to general measures (cf. Schmidt (2009)). CDFs of continuous random variables are also often called cumulative density functions.

As an example, consider the CDF of a normally distributed random variable. Let $\xi \sim N(\mu, \sigma^2)$. Then the PDF of ξ is known to be given by

$$p : \mathbb{R} \to \mathbb{R}_{>0}, x \mapsto p(x) := \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{1}{2\sigma^2}(x-\mu)^2\right). \tag{21.41}$$

For the CDF of ξ, it follows accordingly that

$$P : \mathbb{R} \to]0,1[, x \mapsto P(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \int_{-\infty}^{x} \exp\left(-\frac{1}{2\sigma^2}(s-\mu)^2\right) ds. \tag{21.42}$$

Interestingly, the defining integral of the CDF of a normally distributed random variable can be computed only numerically, not analytically. We visualize selected PDFs and CDFs of normally distributed random variables in Figure 21.12.

Inverse cumulative distribution function

We conclude this section with the concept of the *inverse cumulative distribution function*, a technical tool that is used especially in confidence intervals and hypothesis tests of frequentist inference to determine critical values. We define the inverse CDF as follows.

Definition 21.14 (Inverse cumulative distribution function)**.** Let ξ be a continuous random variable with CDF P. Then the function

$$P^{-1} :]0,1[\to \mathbb{R}, q \mapsto P^{-1}(q) := \{x \in \mathbb{R} | P(x) = q\} \tag{21.43}$$

is called the *inverse cumulative distribution function of* ξ.

•

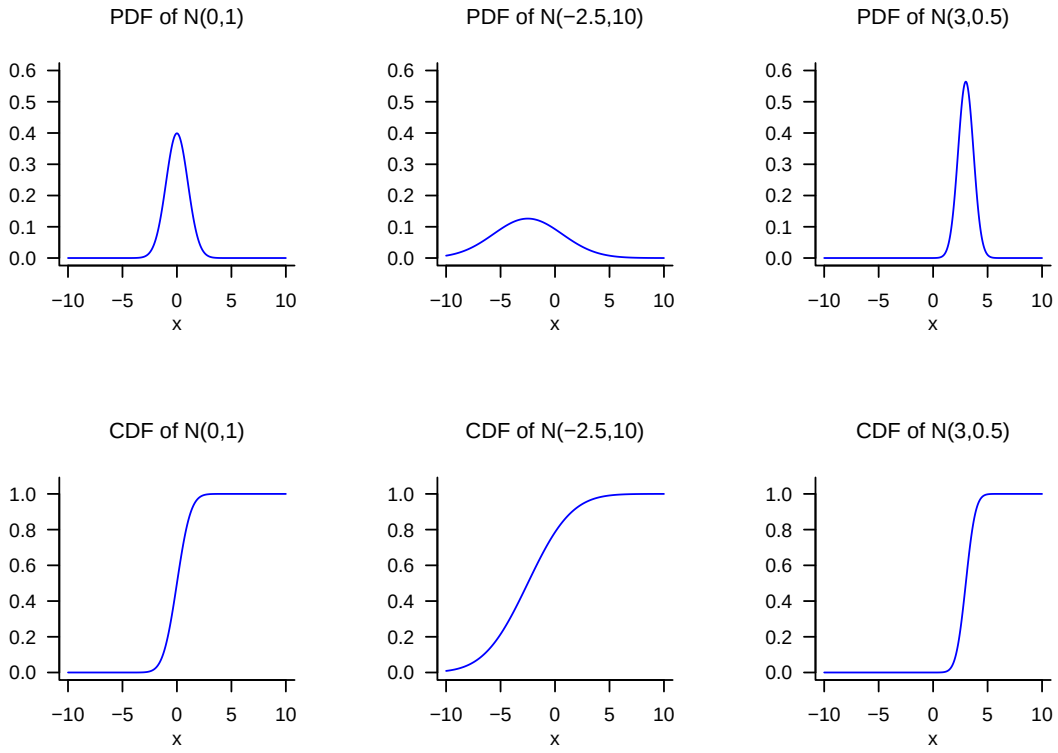


Figure 21.12 PDFs and CDFs of normally distributed random variables.

According to Definition 21.14, the function P^{-1} is evidently the inverse function of P, so that

$$P^{-1}(P(x)) = x. \tag{21.44}$$

Since it is well known that

$$P(x) = q \Leftrightarrow \mathbb{P}(\xi \leq x) = q \text{ for } q \in]0, 1[, \tag{21.45}$$

$P^{-1}(q)$ is therefore the value x of ξ such that $\mathbb{P}(\xi \leq x) = q$. We visualize the CDFs and inverse CDFs of normally distributed random variables in Figure 21.13. In the case of a normally distributed random variable $\xi \sim N(0, 1)$, for example,

$$P(1.645) = 0.950 \Leftrightarrow P^{-1}(0.950) = 1.645, \tag{21.46}$$

and

$$P(1.906) = 0.975 \Leftrightarrow P^{-1}(0.975) = 1.960. \tag{21.47}$$

21.5 Random outcomes and random variables

With the *outcomes* ω discussed in Chapter 19 and the values of *random variables* ξ discussed in this chapter, we have now encountered two concepts that can describe uncertainty about a usually numerical value of a random process. For example, the number of pips that a die shows in a single throw can be conceived as the realization of an outcome or

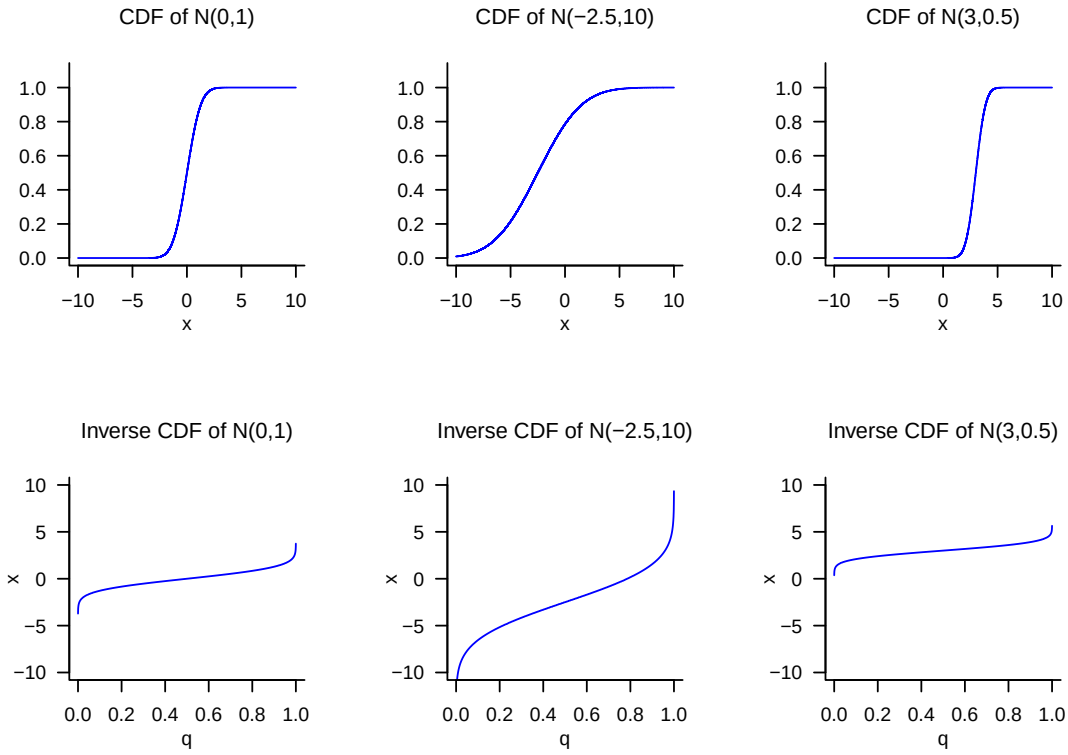


Figure 21.13 CDFs and inverse CDFs of normally distributed random variables.

of a random variable. In fact, there is no standardized answer to the question of whether one should model a random process only with a probability space or with the underlying probability space of a random variable and the probability space induced by both. In applications, the concept of the random variable is usually preferred and corresponding PMFs or PDFs are specified without specifying an underlying probability space or the mapping form of the random variable. The following formulation is typical, for example:

ξ is a normally distributed random variable with expectation parameter μ and variance parameter σ^2.

Implicitly, based on the definition of the normally distributed random variable (cf. Definition 21.9), this statement specifies for ξ the outcome space $\mathcal{X} := \mathbb{R}$, the σ-algebra $\mathcal{S} := \mathcal{B}(\mathbb{R})$, and the distribution $\mathbb{P}_\xi := N(\mu, \sigma^2)$, that is, it considers the "induced" probability space $(\mathbb{R}, \mathcal{B}(\mathbb{R}), N(\mu, \sigma^2))$. However, it remains unclear by which probability space and which exact mapping form $(\mathbb{R}, \mathcal{B}(\mathbb{R}), N(\mu, \sigma^2))$ was induced. This can always be supplemented by assuming that ξ is the identity function. Concretely, one could choose as the underlying probability space here $(\mathbb{R}, \mathcal{B}(\mathbb{R}), N(\mu, \sigma^2))$ and choose ξ as

$$\xi : \mathbb{R} \to \mathbb{R}, \omega \mapsto \xi(\omega) := \omega := x. \tag{21.48}$$

Since ξ then changes nothing about the outcome ω realized randomly in the context of the underlying probability space and merely renames it as x, the distribution of ξ here directly corresponds to the probability measure of the underlying probability space.

In general, one may note that modeling random processes by means of random variables therefore contains the elementary probability space model as a special case, but beyond this it opens up the possibility of formalizing and studying the transformation of probability

measures on different measurable spaces by means of random variables that differ from the identity map.

21.6 Literature

The genesis of the concept of the random variable is closely interwoven with the development of probability theory over the last three centuries, so no publication decisive for the concept can be named. The mathematical development of the concept of the normal distribution by Abraham De Moivre (1667-1754), Pierre Simon Laplace (1749-1827), Johann Carl Friedrich Gauss (1777-1855), and many others, its descriptive-statistical counterparts in empirical research of the nineteenth century, and its multivariate generalization in the late nineteenth century are presented in detail in Stigler (1986).

Self-control questions

1. State the definition of the concept of a random variable.
2. Explain the equation $\mathbb{P}_\xi(\xi = x) = \mathbb{P}(\{\xi = x\})$.
3. Explain the intuitive meaning of $\mathbb{P}(\xi = x)$.
4. State the definition of the concept of a probability mass function.
5. State the definition of the concept of a probability density function.
6. State the definition of the PDF of a normally distributed random variable.
7. State the definition of the concept of a cumulative distribution function.
8. Write the exceedance probability $\mathbb{P}(\xi > x)$ of a random variable using its CDF.
9. Write the interval probability $\mathbb{P}(\xi \in]x_1, x_2])$ of a random variable using its CDF.
10. Write the value $P(x)$ of the CDF of a continuous random variable using its PDF p.
11. Write the value $p(x)$ of the PDF of a continuous random variable using its CDF P.
12. State the definition of the concept of the inverse distribution function.

Solutions

1. See Definition 21.1.
2. The equation says that the probability value assigned by the distribution $\mathbb{P}_\xi$ of the random variable ξ to the event $\xi = x$ in $\mathcal{X}$ is, by definition, equal to the value assigned by the probability measure $\mathbb{P}$ to the preimage of the event $\xi = x$, that is, to the set $\{\xi = x\} = \{\omega \in \Omega | \xi(\omega) = x\}$ in $\mathcal{A}$. See

1.

1. Intuitively, $\mathbb{P}(\xi = x)$ is the probability that the random variable ξ assumes the value x.
2. See Definition 21.4.
3. See Definition 21.8.
4. See Definition 21.9.
5. See Definition 21.13.
6. See Theorem 21.2.
7. See Theorem 21.3.
8. See Theorem 21.5. We have
$$P(x) = \int_{-\infty}^{x} p(s)\, ds. \tag{21.49}$$
9. See Theorem 21.5. We have
$$p(x) = \frac{d}{dx} P(x) = P'(x). \tag{21.50}$$
10. See Definition 21.14.

References

Abbott, S. (2015). *Understanding Analysis*. Springer New York. https://doi.org/10.1007/978-1-4939-2712-8

Arens, T., Hettlich, F., Karpfinger, C., Kockelkorn, U., Lichtenegger, K., & Stachel, H. (2018). *Mathematik*. Springer Berlin Heidelberg. https://doi.org/10.1007/978-3-662-56741-8

Bärwolff, G. (2017). *Höhere Mathematik für Naturwissenschaftler und Ingenieure*. Springer Berlin Heidelberg. https://doi.org/10.1007/978-3-662-55022-9

Bayes, T. (1763). An essay towards solving a problem in the doctrine of chances. By the late Rev. Mr. Bayes, F. R. S. communicated by Mr. Price, in a letter to John Canton, A. M. F. R. S. *Philosophical Transactions of the Royal Society of London*, *53*, 370–418. https://doi.org/10.1098/rstl.1763.0053

Becker, R. A., Chambers, J. M., & Wilks, A. R. (1988). *The new S language: A programming environment for data analysis and graphics* (Reprint). Chapman & Hall.

Bertram, M. G., Sundin, J., Roche, D. G., Sánchez-Tójar, A., Thoré, E. S. J., & Brodin, T. (2023). Open science. *Current Biology*, *33*(15), R792–R797. https://doi.org/10.1016/j.cub.2023.05.036

Borel, É. (1898). *Leçons sur la théorie des fonctions*. Paris: Gauthier Villars.

Burden, R. L., Faires, J. D., & Burden, A. M. (2016). *Numerical analysis* (Tenth edition). Cengage Learning.

Cantor, G. (1874). Über eine Eigenschaft des Inbegriffes aller reellen algebraischen Zahlen. *Jahresbericht Der Deutschen Mathematiker-Vereinigung*, *1*.

Cantor, G. (1891). Über eine elementare Frage der Mannigfaltigkeitslehre. *Jahresbericht Der Deutschen Mathematiker-Vereinigung*, *1*, 75–78.

Cantor, G. (1895). Beiträge zur Begründung der transfiniten Mengenlehre. *Mathematische Annalen*, *46*(4), 481–512. https://doi.org/10.1007/BF02124929

Chiossi, S. G. (2021). *Essential Mathematics for Undergraduates: A Guided Approach to Algebra, Geometry, Topology and Analysis*. Springer International Publishing. https://doi.org/10.1007/978-3-030-87174-1

Cotton, R. (2013). *Learning R* (First Edition). O'Reilly.

Dahm, M. (2005). *Grundlagen der Mensch-Computer-Interaktion*. Pearson Studium.

De Finetti, B. (1975). *Theory of probability*. John Wiley & Sons.

DeGroot, M. H., & Schervish, M. J. (2012). *Probability and statistics* (4th ed). Addison-Wesley.

Deisenroth, M. P., Faisal, A. A., & Ong, C. S. (2020). *Mathematics for Machine Learning* (1st ed.). Cambridge University Press. https://doi.org/10.1017/9781108679930

Friston, K. (2005). A theory of cortical responses. *Philosophical Transactions of the Royal Society B: Biological Sciences*, *360*(1456), 815–836. https://doi.org/10.1098/rstb.2005.1622

Gillies, D. (2000). Varieties of Propensity. *The British Journal for the Philosophy of Science*, *51*(4), 807–835. https://doi.org/10.1093/bjps/51.4.807

Golub, G. H., & Van Loan, C. F. (2013). *Matrix computations* (Fourth edition). The Johns Hopkins University Press.

Gray, R. (1994). Georg Cantor and Transcendental Numbers. *The American Mathematical Monthly*, *101*(9), 819–832. https://doi.org/10.1080/00029890.1994.11997035

Hájek, A. (2019). Interpretations of probability. In E. N. Zalta (Ed.), *The Stanford encyclopedia of philosophy* (Fall 2019). Metaphysics Research Lab, Stanford University.

Hald, A. (1990). *A history of probability and statistics and their applications before 1750*. Wiley.

Hautzinger, M., Keller, F., & Kühner, C. (2006). *BDI-II Beck Depressions-Inventar*. Pearson.

Henze, N. (2018). *Stochastik für Einsteiger*. Springer Fachmedien Wiesbaden. https://doi.org/10.1007/978-3-658-22044-0

Herzog, S., & Ostwald, D. (2013). Sometimes Bayesian statistics are better. *Nature*, *494*(7435), 35–35. https://doi.org/10.1038/494035b

Hesse, C. (2009). *Wahrscheinlichkeitstheorie* (2nd ed.). Vieweg + Teubner.

Huber, P. J. (1981). *Robust statistics*. Wiley.

Hyndman, R. J., & Fan, Y. (1996). Sample Quantiles in Statistical Packages. *The American Statistician*, *50*(4), 361. https://doi.org/10.2307/2684934

Ihaka, R., & Gentleman, R. (1996). R: A Language for Data Analysis and Graphics. *Journal of Computational and Graphical Statistics*, *5*(3), 2999–2314.

Jaynes, E. (1968). Prior Probabilities. *IEEE Transactions on Systems Science and Cybernetics*, *4*(3), 227–241. https://doi.org/10.1109/TSSC.1968.300117

Jeffreys, H. (1939). *Theory Of Probability*. Oxford Classic Texts in the Physical Sciences.

Kolmogoroff, A. (1933). *Grundbegriffe der Wahrscheinlichkeitsrechnung*. Springer Berlin Heidelberg. https://doi.org/10.1007/978-3-642-49888-6

Laplace, P. S. (1814). *A philosophical essay on probabilities*.

Mardia, K. V., Kent, J. T., & Bibby, J. M. (1979). *Multivariate analysis*. Academic Press.

McGill, R., Tukey, J. W., & Larsen, W. A. (1978). Variations of Box Plots. *The American Statistician*, *32*(1), 12. https://doi.org/10.2307/2683468

Meintrup, D., & Schäffler, S. (2005). *Stochastik: Theorie und Anwendungen*. Springer.

Miedema, F. (2022). *Open Science: The Very Idea*. Springer Netherlands. https://doi.org/10.1007/978-94-024-2115-6

Newton, I. (1687). *Philosophiae Naturalis Principia Mathematica*. Royal Society.

Peirce, C. S. (1957). Notes on the Doctrine of Chances. In *Essays in the Philosophy of Science* (pp. 74–84). Bobbs-Merrill.

Philip, S., Kew, S., Van Oldenborgh, G. J., Otto, F., Vautard, R., Van Der Wiel, K., King, A., Lott, F., Arrighi, J., Singh, R., & Van Aalst, M. (2020). A protocol for probabilistic extreme event attribution analyses. *Advances in Statistical Climatology, Meteorology and Oceanography*, *6*(2), 177–203. https://doi.org/10.5194/ascmo-6-177-2020

Popper, K. R. (1959). The propensity interpreation of probability. *The British Journal for the Philosophy of Science*, *10*(37), 25–42. https://doi.org/10.1093/bjps/X.37.25

Ramsey, F. P. (1926). Truth and Probability. In *The Foundations of Mathematics and other Logical Essays, Chp VII* (pp. 156–198). Harcourt, Brace and Company.

Rat für Informationsinfrastrukturen. (2017). *Datenschutz und Foschungsdaten - Aktuelle Empfehlungen*.

Reichenbach, H. (1949). Philosophical Foundations of Probability. *Proceedings of the Berkeley Symposium on Mathematical Statistics and Probability*, 1–20.

Richter, T., & Wick, T. (2017). *Einführung in die Numerische Mathematik: Begriffe, Konzepte und zahlreiche Anwendungsbeispiele*. Springer Berlin Heidelberg. https://doi.org/10.1007/978-3-662-54178-4

Riley, J. (2017). *Understanding metadata: What is metadata, and what is it for*. National

Information Standards Organization.
Rosenblatt, B. (1997). The Digital Object Identifier: Solving the Dilemma of Copyright Protection Online. *Journal of Electronic Publishing*, *3*(2). https://doi.org/10.3998/3336451.0003.204
Savage, L. J. (1954). *The foundations of statistics* (pp. xv, 294). John Wiley & Sons.
Schmidt, K. D. (2009). *Maß und Wahrscheinlichkeit*. Springer.
Scott, D. W. (1979). *On optimal and data-based histograms*. 6.
Searle, S. (1982). *Matrix Algebra Useful for Statistics*. Wiley-Interscience.
Spivak, M. (2008). *Calculus* (4th ed.). Publish or Perish, Inc.
Stigler, S. M. (1986). *The history of statistics: The measurement of uncertainty before 1900*. Belknap Press of Harvard University Press.
Strang, G. (2009). *Introduction to Linear Algebra*. Cambridge University Press.
Sturges, H. A. (1926). The Choice of a Class Interval. *Journal of the American Statistical Association*, *21*(153), 65–66. https://doi.org/10.1080/01621459.1926.10502161
Toelch, U., & Ostwald, D. (2018). Digital open science—Teaching digital tools for reproducible and transparent research. *PLOS Biology*, *16*(7), e2006022. https://doi.org/10.1371/journal.pbio.2006022
Ulrich, H., Kock-Schoppenhauer, A.-K., Deppenwiese, N., Gött, R., Kern, J., Lablans, M., Majeed, R. W., Stöhr, M. R., Stausberg, J., Varghese, J., Dugas, M., & Ingenerf, J. (2022). Understanding the Nature of Metadata: Systematic Review. *Journal of Medical Internet Research*, *24*(1), e25440. https://doi.org/10.2196/25440
Unger, L. (2000). *Grundkurs Mathematik*.
Van Rooij, I., & Wareham, T. (2012). Intractability and approximation of optimization theories of cognition. *Journal of Mathematical Psychology*, *56*(4), 232–247. https://doi.org/10.1016/j.jmp.2012.05.002
Venn, J. (1876). *The logic of chance*. MacMillan.
von Mises, R. (1951). *Wahrscheinlichkeit, Statistik und Wahrheit*. Springer International Publishing. https://doi.org/10.1007/978-3-319-29251-9_9
Von Neumann, J. (1945). *First draft of a report on the EDVAC*. Moore School of Electrical Engineering, University of Pennsylvania. https://doi.org/10.5479/sil.538961.39088011475779
Von Plato, J. (1994). *Creating modern probability: Its mathematics, physics, and philosophy in historical perspective*. Cambridge University Press.
Walley, P. (1991). *Statistical reasoning with imprecise probabilities* (1st ed). Chapman and Hall.
Wickham, H. (2019). *Advanced R, Second Edition*. CRC Press.
Wilkinson, M. D., Dumontier, M., Aalbersberg, Ij. J., Appleton, G., Axton, M., Baak, A., Blomberg, N., Boiten, J.-W., da Silva Santos, L. B., Bourne, P. E., Bouwman, J., Brookes, A. J., Clark, T., Crosas, M., Dillo, I., Dumon, O., Edmunds, S., Evelo, C. T., Finkers, R., … Mons, B. (2016). The FAIR Guiding Principles for scientific data management and stewardship. *Scientific Data*, *3*(1), 160018. https://doi.org/10.1038/sdata.2016.18